4

ระบบผู้แนะนำอิงเนื้อหาหลายรูปแบบ
Multimodal Recommender System



บทนี้ต่อยอดบทที่ 3 โดยเพิ่มหลักฐานจากภาพ เสียง และวิดีโอ แล้วรวมกับข้อความเพื่อแนะนำภาพยนตร์เฉพาะบุคคล

เนื้อหาเริ่มจากการสกัดคุณลักษณะในยุคต่าง ๆ แล้วอธิบาย CNN และ Transformer อย่างละเอียด ก่อนเน้นการสร้างโปรไฟล์ผู้ใช้และเทคนิค Fusion

เนื้อหา

วัตถุประสงค์ 2
4.1 ทำไมต้องหลายรูปแบบ 4
4.2 การสกัดคุณลักษณะ 13
4.3 CNN 29
4.4 Transformer 54
4.5 โปรไฟล์ผู้ใช้ 89
4.6 Fusion 116
4.7 การประเมินผล 138
4.8 ระบบจริงและข้อจำกัด 143
4.9 โปรแกรมและ Lab 146
4.10 บทวิเคราะห์ผู้เขียน 149
สรุปท้ายบท 151
แบบฝึกหัด 152
อภิธานศัพท์ 154
ภาคผนวก 4ก 156

วัตถุประสงค์สำหรับผู้อ่านประจำบท

เมื่ออ่านบทนี้จบแล้ว ผู้อ่านควรทำสิ่งต่อไปนี้ได้

- **อธิบาย** ว่าข้อความ ภาพ เสียง และวิดีโอให้หลักฐานเกี่ยวกับภาพยนตร์ต่างกันอย่างไร และเมื่อใดการใช้เรื่องย่ออย่างเดียวจึงไม่พอ
- **อธิบายวิวัฒนาการของการสกัดคุณลักษณะ (Feature Extraction)** สามยุค ตั้งแต่คุณลักษณะที่ออกแบบด้วยมือ การเรียนรู้เชิงลึก จนถึง Transformer และแบบจำลองหลายรูปแบบ
- **คำนวณด้วยมือ** การทำงานของนิวรอน ReLU การลดตามเกรเดียนต์ และการฝัง (Embedding) จากตัวอย่างขนาดเล็ก
- **อธิบายแนวคิดและคำนวณขั้นตอนของ CNN** ได้แก่ คอนโวลูชันหนึ่งมิติและสองมิติ การเติมขอบ ระยะก้าว Pooling และระยะมองเห็น
- **อธิบายแนวคิดและคำนวณขั้นตอนของ Transformer** ได้แก่ Self-Attention หลายหัว การฝังตำแหน่ง Layer Normalization เครือข่ายป้อนไปข้างหน้า และ Pooling
- **เลือกเทคนิคและไลบรารี Python** ของ CNN และ Transformer ให้เหมาะกับข้อความ ภาพ เสียง และวิดีโอ
- **สร้างโปรไฟล์ผู้ใช้** จากเวกเตอร์ของเทคนิค CNN และ Transformer ด้วยวิธีค่าเฉลี่ย ถ่วงด้วยคะแนน Rocchio ลดน้ำหนักตามเวลา หลายจุดสนใจ และแบบ Attention
- **เลือกและคำนวณเทคนิค Fusion** ได้แก่ Late Fusion การปรับมาตราคะแนน การรวมด้วยอันดับ Early Fusion Gate การจัดแนวแบบ CLIP และ VBPR
- **อธิบายที่มาของไฮเปอร์พารามิเตอร์** ที่สำคัญ และ **ประเมินผล** ระบบหลายรูปแบบด้วยตัวชี้วัดของบทที่ 2

บทนี้ต่อยอดจากบทที่ 3 ซึ่งสร้างระบบผู้แนะนำจากข้อความด้วย TF-IDF, BM25 และ Static Embedding ชุดข้อมูลยังคงเป็น MovieMini-12 ผู้ใช้หลักยังคงเป็น u1 ผู้ชอบ i1 Toy Story 5 และ i2 Inside Out 2 และรายการผู้สมัครยังคงเป็นเจ็ดเรื่องเดิม ผลของบทนี้จึงเทียบกับบทที่ 3 ได้โดยตรง

เนื้อหาเรียงจากง่ายไปยากเป็นสี่ช่วง

- **ช่วงที่ 1 พื้นฐาน (หัวข้อ 4.1–4.2)** เหตุผลของการใช้หลายสื่อ ข้อมูลของบท และวิวัฒนาการของการสกัดคุณลักษณะ พร้อมหน่วยพื้นฐานของการเรียนรู้ ได้แก่ นิวรอน การลดตามเกรเดียนต์ และการฝัง
- **ช่วงที่ 2 ตัวเข้ารหัสสองเทคนิค (หัวข้อ 4.3–4.4)** CNN และ Transformer ตั้งแต่แนวคิด ทฤษฎี และขั้นตอนวิธี จนถึงการประยุกต์กับข้อความ ภาพ เสียง และวิดีโอ
- **ช่วงที่ 3 หัวใจของบท (หัวข้อ 4.5–4.7)** สร้างโปรไฟล์ผู้ใช้ รวมหลักฐานด้วย Fusion และประเมินผล
- **ช่วงที่ 4 มองภาพใหญ่ (หัวข้อ 4.8–4.10)** ระบบจริง ไฮเปอร์พารามิเตอร์ โปรแกรมประกอบบท และบทวิเคราะห์ของผู้เขียน

ผู้อ่านที่ต้องการภาพรวมก่อนอาจอ่านเฉพาะกล่อง "อธิบายแบบเข้าใจง่าย" และตัวอย่างการคำนวณข้อแรกของแต่ละหัวข้อ แล้วจึงกลับมาอ่านรายละเอียด ความรู้ที่ควรมีก่อนอ่านสรุปไว้ในตารางที่ 4.1

ตารางที่ 4.1 ความรู้พื้นฐานที่ควรมีก่อนอ่านบทนี้

ความรู้	ใช้ในหัวข้อ	ทบทวนได้ที่
ผลคูณจุด ความยาวเวกเตอร์ และความคล้ายโคไซน์	4.1–4.7	บทที่ 3 หัวข้อ 3.3

ความรู้	ใช้ในหัวข้อ	ทบทวนได้ที่
TF-IDF และการปรับความยาวเวกเตอร์ (L^2 Normalization)	4.1, 4.5	บทที่ 3 หัวข้อ 3.5
Static Word Embedding และเวกเตอร์ 12 มิติของเรื่อง	4.3, 4.4	บทที่ 3 หัวข้อ 3.7
โปรไฟล์ผู้ใช้แบบค่าเฉลี่ย ถ่วงคะแนน และ Rocchio	4.5	บทที่ 3 หัวข้อ 3.9
HR@K, MRR, nDCG@K และการกันออกทีละรายการ	4.7	บทที่ 2 หัวข้อ 2.4–2.8
การคูณเมทริกซ์ ฟังก์ชันเลขชี้กำลัง และอนุพันธ์อย่างง่าย	4.2–4.6	ภาคผนวกของหนังสือ

4.1 ทำไมต้องใช้ข้อมูลหลายรูปแบบ

หัวข้อนี้ตอบคำถามแรกของบทว่าเหตุใดเรื่องย่ออย่างเดียวจึงไม่พอ แล้วแนะนำขั้นตอนการทำงานของระบบ ชุดข้อมูล MovieMini-12 และข้อตกลงที่ใช้ตลอดบท ดังสรุปในภาพที่ 4.52 แสดงว่าข้อความ ภาพ เสียง และวิดีโอให้หลักฐานคนละด้านก่อนนำมารวมกัน



ภาพที่ 4.52 ภาพรวมเหตุผลที่ต้องใช้ข้อมูลหลายสื่อ

4.1.1 สิ่งที่เรื่องย่อบอกไม่ได้

ระบบในบทที่ 3 อ่านเพียงข้อความ คือแนวและเรื่องย่อ แล้วแปลงเป็นเวกเตอร์ด้วย TF-IDF, BM25 หรือ Static Embedding วิธีนี้ได้ผลดีเมื่อถ้อยคำในเรื่องย่อสะท้อนสิ่งที่ผู้ชมรู้สึกจริง

แต่ประสบการณ์การดูภาพยนตร์ไม่ได้มีแค่เนื้อเรื่อง ผู้ชมยังรับรู้ **โทนสีของภาพ จังหวะของเพลงประกอบ** และ **ความเร็วของการตัดต่อ** ด้วย สิ่งเหล่านี้แทบไม่ปรากฏในเรื่องย่อ

ตัวอย่างที่เห็นชัดคือ i7 Ballerina กับ i8 Furiosa: A Mad Max Saga เรื่องย่อของสองเรื่องแทบไม่มีคำร่วมกัน แต่ทั้งสองเรื่องเป็นภาพยนตร์แอ็กชันจังหวะเร็วเหมือนกัน ตารางที่ 4.2 เทียบความคล้ายของสองเรื่องนี้เมื่อแทนภาพยนตร์ด้วยข้อมูลต่างชนิดกัน โดยวัดด้วยความคล้ายโคไซน์ในสมการ (4.1)

$$\cos(\mathbf{a},\mathbf{b}) = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\,\|\mathbf{b}\|} = \frac{\sum_{k=1}^{d} a_k b_k}{\sqrt{\sum_{k=1}^{d} a_k^2}\sqrt{\sum_{k=1}^{d} b_k^2}} \tag{4.1}$$

โดยที่

- $\mathbf{a}, \mathbf{b}$ เวกเตอร์ของรายการสองรายการ
- a_k, b_k ค่ามิติที่ k ของเวกเตอร์ทั้งสอง
- d จำนวนมิติของเวกเตอร์
- $\|\,\mathbf{a}\,\|$ ความยาว (L^2 Norm) ของเวกเตอร์ $\mathbf{a}$

- $\cos(\mathbf{a}, \mathbf{b})$ ความคล้ายโคไซน์ มีค่าระหว่าง -1 ถึง 1 ค่ายิ่งมากยิ่งคล้าย

ตารางที่ 4.2 ความคล้ายของ i7 Ballerina กับ i8 Furiosa เมื่อใช้ข้อมูลหกแบบ

ข้อมูลที่ใช้แทนภาพยนตร์	ที่มาของเวกเตอร์	โคไซน์ i7–i8
TF-IDF ของเรื่องย่อ	บทที่ 3 หัวข้อ 3.5	0.0000
ข้อความ เทคนิค CNN (Static Embedding)	ตารางที่ 4.12	0.4493
ข้อความ เทคนิค Transformer	ตารางที่ 4.15	0.4486
ภาพโปสเตอร์ เทคนิค Transformer	ตารางที่ 4.16	−0.5187
เสียงเพลงประกอบ เทคนิค Transformer	ตารางที่ 4.16	0.9642
ตัวอย่างภาพยนตร์ เทคนิค Transformer	ตารางที่ 4.16	0.9968

ที่มา: ผู้เขียนคำนวณ เวกเตอร์ภาพ เสียง และวิดีโอผ่านการเตรียมตามหัวข้อ 4.5.1 แล้ว

อ่านตารางที่ 4.2 จากบนลงล่างได้สามข้อ

- **ข้อความแบบนับคำ** มองไม่เห็นความคล้ายเลย (0.0000) เพราะสองเรื่องไม่มีคำร่วมกัน
- **ข้อความแบบความหมาย** ทั้งเทคนิค CNN (0.4493) และ Transformer (0.4486) เห็นความคล้ายระดับปานกลาง เพราะทั้งสองเรื่องมีความหมายด้านอาชญากรรมและแอ็กชันใกล้กัน
- **เสียงและวิดีโอ** ให้โคไซน์สูงถึง 0.9642 และ 0.9968 เพราะทั้งสองเรื่องมีเพลงหนักและตัดต่อเร็วเหมือนกัน ส่วนภาพโปสเตอร์ให้ค่าติดลบ (−0.5187) เพราะโทนสีของโปสเตอร์ต่างกันมาก

ข้อสังเกตนี้คือเหตุผลหลักของบท **แต่ละสื่อเห็นความคล้ายคนละด้าน** ระบบที่ใช้หลายสื่อจึงมีโอกาสจับความชอบของผู้ใช้ได้ครบกว่า แต่ก็ต้องตัดสินใจว่าจะเชื่อสื่อใดมากน้อยเพียงใด ซึ่งเป็นเรื่องของ Fusion ในหัวข้อ 4.6

ตัวอย่างการคำนวณ 4.1 โคไซน์ข้อความของ i7 กับ i8 จากเทคนิค Transformer

โจทย์ จงคำนวณโคไซน์ระหว่างเวกเตอร์ข้อความ 12 มิติของ i7 Ballerina และ i8 Furiosa เพื่อยืนยันค่าในตารางที่ 4.2

สมการที่ใช้ สมการ (4.1)

ข้อมูลที่ใช้ เวกเตอร์ข้อความเทคนิค Transformer จากตารางที่ 4.15 ซึ่งปรับความยาวเป็น 1 แล้ว

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- ชื่อเรื่องและแนวมาจากตารางที่ 4.4: i7 Ballerina (2025) แนว Action, Thriller และ i8 Furiosa: A Mad Max Saga (2024) แนว Action, Sci-Fi ทั้งสองเรื่องมีแนวร่วมกันเพียง Action
- เวกเตอร์ข้อความ 12 มิติสร้างจาก เรื่องย่อในตารางที่ 4.6 ด้วยขั้นตอนเดียวกับบทที่ 3 หัวข้อ 3.7 คือ ตัดคำและคำหยุด ⟶ เปิดคลังแนวคิด 12 มิติ ⟶ เฉลี่ยถ่วงด้วย IDF ⟶ ปรับความยาวเป็น 1
- เรื่องย่อ i7 เหลือ 20 โทเคนหลังตัดคำหยุด และพบในคลังแนวคิดครบทุกโทเคน คำกลุ่มอาชญากรรม เช่น assassin, killers (2 ครั้ง), criminal, cult, murdered, revenge, violent และ underworld จึงดันมิติ crime ให้สูงที่สุด

- เรื่องย่อ i8 มี 21 โทเคน พบในคลังแนวคิด 18 โทเคน (fertile, place, becomes ไม่มีในคลัง จึงถูกข้าม) คำกลุ่มการเอาชีวิตรอด เช่น survivor, warlord, horde, fight, survival, road และ chase ดันมิติ action
- เทคนิค Transformer ให้ค่าต่างจาก Static Embedding เฉพาะคำหลายความหมาย ใน i8 คำว่า chase อยู่ในบริบท "a road chase across the wasteland" จึงถูกอ่านเป็น "การไล่ล่า" (action 1) แทนค่าเฉลี่ยสองความหมาย (action 0.5, ambition 0.5) ส่วน i7 ไม่มีคำหลายความหมาย เวกเตอร์สองเทคนิคจึงเท่ากัน

ขั้นที่ 1 หามิติที่ทั้งสองเรื่องมีค่าไม่เป็นศูนย์

- มิติที่ไม่เป็นศูนย์พร้อมกันมี 3 มิติ คือ crime, action, family
- มิติอื่นมีอย่างน้อยหนึ่งเรื่องเป็น 0 ผลคูณจึงเป็น 0 ไม่ต้องคำนวณ

ขั้นที่ 2 คูณทีละมิติแล้วบวกกัน

- crime: $0.9065 \times 0.0449 = 0.0407$
- action: $0.4121 \times 0.9755 = 0.4020$
- family: $0.0769 \times 0.0772 = 0.0059$

$$\mathbf{a} \cdot \mathbf{b} = 0.0407 + 0.4020 + 0.0059 = 0.4486$$

ขั้นที่ 3 หารด้วยผลคูณของความยาว

- เวกเตอร์ทั้งสองปรับความยาวเป็น 1 แล้ว จึงได้ $\| \mathbf{a} \| \| \mathbf{b} \| = 1$
- โคไซน์จึงเท่ากับผลคูณจุด คือ 0.4486

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

ก. ที่มาของเวกเตอร์ i7 จากเรื่องย่อ (สมการ 3.21 และ 3.22 ของบทที่ 3)

- ค่า IDF คำนวณจากเรื่องย่อทั้ง 12 เรื่อง: idf = ln((1 + 12)/(1 + df)) + 1 คำที่พบเรื่องเดียว (df = 1) ได้ 2.8718 คำที่พบสองเรื่อง (df = 2) ได้ 2.4663
- ตัวส่วน (ผลรวม IDF ของ 20 โทเคน): 17 × 2.8718 + 3 × 2.4663 = 56.2196

มิติ	โทเคนที่มีน้ำหนักในมิตินี้	ตัวเศษ = Σ idf × น้ำหนัก	ผลรวม
crime	assassin, killers ×2, criminal, cult, murdered, revenge, underworld (น้ำหนัก 1) · violent (1, idf 2.4663) · trained, school, hunts, hotels, clubs (0.5) · secret (0.5, idf 2.4663)	8×2.8718 + 2.4663 + 5×0.5×2.8718 + 0.5×2.4663	33.8534
action	assassin, killers ×2, revenge, ballerina, trained, hunts, cuts, path (0.5) · violent, rival (0.5, idf 2.4663)	9×0.5×2.8718 + 2×0.5×2.4663	15.3894
family	father (1)	1×2.8718	2.8718
music	ballerina (0.5)	0.5×2.8718	1.4359
mind	secret (0.5, idf 2.4663)	0.5×2.4663	1.2332

- หารด้วยตัวส่วน: crime = 33.8534/56.2196 = 0.6022, action = 0.2737, family = 0.0511, music = 0.0255, mind = 0.0219
- ความยาว = $\sqrt{0.6022^2 + 0.2737^2 + 0.0511^2 + 0.0255^2 + 0.0219^2}$ = 0.6643 แล้วหารทุกมิติด้วย 0.6643 ได้ crime 0.9065, action 0.4121, family 0.0769, music 0.0384, mind 0.0330 ตรงกับแถว i7 ของตารางที่ 4.15

ข. ที่มาของเวกเตอร์ i8 (เทคนิค Transformer อ่าน chase เป็นการไล่ล่า)

- ตัวส่วน: 16 โทเคนที่ idf = 2.8718 และ chase, child ที่ idf = 2.4663 รวม 16 × 2.8718 + 2 × 2.4663 = 50.8815
- ตัวเศษมิติ action = 31.1844 (คำแอ็กชันเกือบทุกคำ เช่น survivor 1, warlord 1, horde 1, fight 1, survival 1 และ chase 1) ส่วน society = 5.7436 (ruled 1, warlord 0.5, freedom 0.5) space = 2.8718 (wasteland 0.5 สองครั้ง) family = 2.4663 (child 1) crime = 1.4359 (snatched 0.5)
- หารด้วยตัวส่วนแล้วปรับความยาว (ความยาวก่อนปรับ = 0.6283) ได้ action 0.9755, society 0.1797, space 0.0898, family 0.0772, crime 0.0449 ตรงกับแถว i8 ของตารางที่ 4.15

ค. ผลคูณจุดทีละมิติ

มิติ	i7	i8	i7 × i8
mind	0.0330	0.0000	0.0000
space	0.0000	0.0898	0.0000
crime	0.9065	0.0449	0.0407
action	0.4121	0.9755	0.4020
family	0.0769	0.0772	0.0059
music	0.0384	0.0000	0.0000
society	0.0000	0.1797	0.0000
รวม			**0.4486**

- มิติ mind, music มีค่าเฉพาะใน i7 และ space, society มีค่าเฉพาะใน i8 ผลคูณของมิติเหล่านี้จึงเป็น 0 เหลือสามมิติที่มีค่าทั้งคู่ คือ crime, action และ family

ตีความ

ค่าเกือบทั้งหมดมาจากมิติ action (0.4020) เพราะทั้งสองเรื่องเป็นแอ็กชัน แต่เรื่องหนึ่งเน้นอาชญากรรม (crime) อีกเรื่องเน้นการเอาชีวิตรอด โคไซน์จึงอยู่ระดับกลาง ไม่สูงเท่าเสียงหรือวิดีโอ

นิยาม 4.1 รูปแบบข้อมูล (Modality) และระบบผู้แนะนำหลายรูปแบบ

รูปแบบข้อมูล (Modality) คือชนิดของสัญญาณที่มีโครงสร้างต่างกัน ข้อความเป็นลำดับคำ ภาพเป็นตารางพิกเซล เสียงเป็นลำดับแอมพลิจูดตามเวลา และวิดีโอเป็นลำดับภาพตามเวลา

ระบบผู้แนะนำหลายรูปแบบ (Multimodal Recommender System)

คือระบบที่ใช้หลักฐานจากรูปแบบข้อมูลอย่างน้อยสองรูปแบบประกอบกันในการให้คะแนนและจัดอันดับรายการ (Baltrušaitis et al., 2019; Deldjoo et al., 2020)

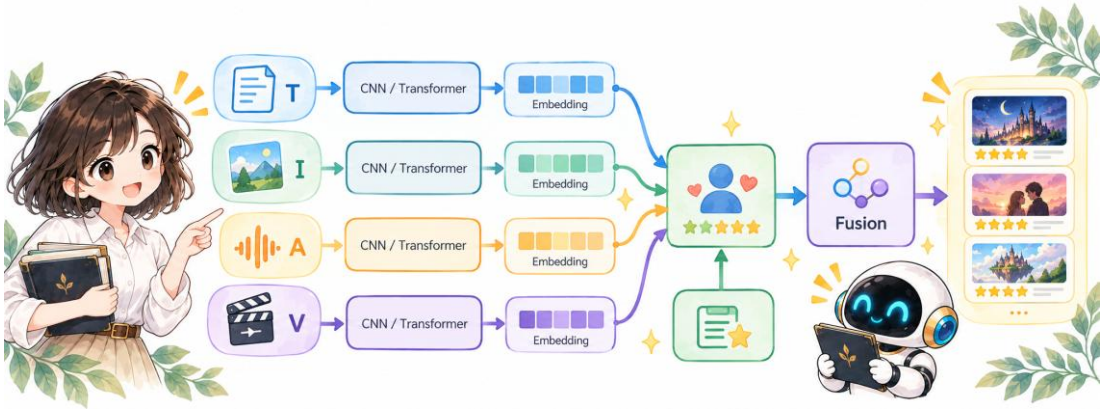
อธิบายแบบเข้าใจง่าย ดูภาพยนตร์ด้วยประสาทสัมผัสครบ

ลองนึกถึงการเลือกร้านอาหาร ถ้าอ่านเพียงรายชื่อเมนู (ข้อความ) เราตัดสินใจได้ระดับหนึ่ง แต่ถ้าได้เห็นรูปอาหาร (ภาพ) ได้ยินเสียงบรรยากาศในร้าน (เสียง) และดูคลิปรีวิว (วิดีโอ) เราจะตัดสินใจได้มั่นใจขึ้น

ระบบหลายรูปแบบทำแบบเดียวกัน คือไม่เชื่อหลักฐานชิ้นเดียว แต่ฟังความเห็นจากหลายแหล่งแล้วจึงสรุป

4.1.2 ขั้นตอนการทำงานของระบบและแผนที่ของบท

ระบบหลายรูปแบบในบทนี้ทำงานเป็นหกขั้นดังภาพที่ 4.1 ผู้อ่านควรจำภาพนี้ไว้ เพราะทุกหัวข้อหลังจากนี้คือรายละเอียดของกล่องใดกล่องหนึ่งในภาพ



ภาพที่ 4.1 ขั้นตอนการทำงานของระบบผู้แนะนำหลายรูปแบบในบทนี้

ภาพที่ 4.1 อ่านจากซ้ายไปขวาได้ดังนี้

- **ขั้นที่ 1 ข้อมูลนำเข้า (Input)** ภาพยนตร์แต่ละเรื่องมีสื่อสี่ชนิด คือเรื่องย่อ (T) โปสเตอร์ (I) คลิปเสียง (A) และตัวอย่างภาพยนตร์ (V)
- **ขั้นที่ 2 ตัวเข้ารหัส (Encoder)** แปลงข้อมูลนำเข้าเป็นเวกเตอร์ บทนี้ใช้สองเทคนิค คือ CNN (หัวข้อ 4.3) และ Transformer (หัวข้อ 4.4)
- **ขั้นที่ 3 เวกเตอร์ฝัง (Embedding)** ผลของขั้นที่ 2 คือ **ข้อมูลที่ได้จากการคำนวณจากเทคนิค** CNN หรือ Transformer สื่อละหนึ่งเวกเตอร์ต่อเรื่อง
- **ขั้นที่ 4 โปรไฟล์ผู้ใช้รายสื่อ** รวมเวกเตอร์ของเรื่องที่ผู้ใช้เคยให้คะแนนเป็นโปรไฟล์ของแต่ละสื่อ (หัวข้อ 4.5)
- **ขั้นที่ 5 Fusion** รวมหลักฐานจากสี่สื่อเป็นคะแนนเดียว (หัวข้อ 4.6)
- **ขั้นที่ 6 Top-K** จัดอันดับรายการผู้สมัครแล้วเสนอ K เรื่องแรก จากนั้นจึงประเมินผล (หัวข้อ 4.7)

ตารางที่ 4.3 สรุปว่าแต่ละหัวข้อรับอะไรเข้า และส่งอะไรต่อให้หัวข้อถัดไป

ตารางที่ 4.3 แผนที่ของบท: สิ่งที่แต่ละหัวข้อรับเข้าและส่งออก

หัวข้อ	รับเข้า	ส่งออก
4.2 การสกัดคุณลักษณะ	ข้อมูลนำเข้าของสี่สื่อ	แนวคิดของตัวเข้ารหัส นิวรอน การฝัง
4.3 CNN	เทนเซอร์ของแต่ละสื่อ	เวกเตอร์รายการจากเทคนิค CNN
4.4 Transformer	ลำดับโทเคนหรือแพตช์	เวกเตอร์รายการจากเทคนิค Transformer
4.5 โปรไฟล์ผู้ใช้	เวกเตอร์รายการ + คะแนนผู้ใช้	โปรไฟล์และคะแนนรายสื่อ

หัวข้อ	รับเข้า	ส่งออก
4.6 Fusion	คะแนนรายสื่อ	คะแนนรวมและ Top-K
4.7 ประเมินผล	Top-K ของทุกผู้ใช้	HR@3, MRR, nDCG@3

4.1.3 ข้อมูล MovieMini-12 และข้อตกลงของบท

MovieMini-12 ในบทนี้คือชุดเดียวกับบทที่ 3 มีภาพยนตร์ 12 เรื่องและผู้ใช้ 8 คน รหัสรายการ i1–i12 เป็น**กุญแจเชื่อม**ทุกตาราง ถ้าจับคู่รหัสผิด ระบบอาจนำเรื่องย่อของเรื่องหนึ่งไปรวมกับเสียงของอีกเรื่องโดยไม่มีข้อความแจ้งเตือนใด ๆ ตารางที่ 4.4 แสดงบัญชีภาพยนตร์ และตารางที่ 4.5 แสดงคะแนนของผู้ใช้ ตารางที่ 4.6 แสดงเรื่องย่อซึ่งเป็นที่มาของเวกเตอร์ข้อความ และตารางที่ 4.7 สรุปว่าตัวเลขในตัวอย่างการคำนวณแต่ละข้อมาจากตารางใดและผ่านขั้นตอนใด

ตารางที่ 4.4 บัญชีภาพยนตร์ของ MovieMini-12

รหัส	ชื่อเรื่อง (ปี)	แนว	รหัส	ชื่อเรื่อง (ปี)	แนว
i1	Toy Story 5 (2026)	Animation, Family	i7	Ballerina (2025)	Action, Thriller
i2	Inside Out 2 (2024)	Animation, Family	i8	Furiosa: A Mad Max Saga (2024)	Action, Sci-Fi
i3	The Wild Robot (2024)	Animation, Sci-Fi	i9	Conclave (2024)	Drama, Thriller
i4	It Ends with Us (2024)	Drama, Romance	i10	Dune: Part Two (2024)	Action, Sci-Fi
i5	Wicked (2024)	Fantasy, Musical	i11	Mickey 17 (2025)	Sci-Fi, Thriller
i6	Bridget Jones: Mad About the Boy (2025)	Drama, Romance	i12	Project Hail Mary (2026)	Drama, Sci-Fi

ที่มา: ชุดข้อมูล MovieMini-12 ชุดเดียวกับบทที่ 3 หัวข้อ 3.2

ตารางที่ 4.5 คะแนนของผู้ใช้ 8 คน จัดกลุ่มตามความชอบ

ผู้ใช้	ชอบ (4–5)	ไม่ชอบ (1–2)	กลาง (3)	ยังไม่เคยให้คะแนน
u1	i1 (5), i2 (5), i3 (4)	i4 (1), i6 (1)	–	i5, i7, i8, i9, i10, i11, i12
u2	i4 (5), i5 (4), i6 (5)	i1 (2)	–	i2, i3, i7, i8, i9, i10, i11, i12
u3	i10 (5), i11 (5), i12 (4)	i7 (1)	–	i1, i2, i3, i4, i5, i6, i8, i9
u4	i1 (4), i7 (5), i8 (5)	i5 (1)	–	i2, i3, i4, i6, i9, i10, i11, i12
u5	i3 (4), i9 (5)	i10 (2)	i6 (3)	i1, i2, i4, i5, i7, i8, i11, i12
u6	i2 (5), i5 (4)	–	i11 (3)	i1, i3, i4, i6, i7, i8, i9, i10, i12
u7	i11 (4), i12 (5)	i8 (2)	i3 (3)	i1, i2, i4, i5, i6, i7, i9, i10
u8	i4 (4), i6 (5)	i2 (2)	i11 (3)	i1, i3, i5, i7, i8, i9, i10, i12

ที่มา: เมทริกซ์คะแนนของ MovieMini-12 (บทที่ 3 หัวข้อ 3.2) รวม 32 คะแนน ตัวเลขในวงเล็บคือคะแนน 1–5 ช่อง "ยังไม่เคยให้คะแนน" ไม่ใช่คะแนนศูนย์ ผู้ใช้อาจยังไม่เคยเห็นเรื่องนั้นเลย และเป็นแหล่งของรายการผู้สมัคร

ผู้ใช้หลัก u1 ชอบ i1 Toy Story 5, i2 Inside Out 2 และ i3 The Wild Robot และไม่ชอบ i4 It Ends with Us กับ i6 Bridget Jones: Mad About the Boy เมื่อตัดห้าเรื่องที่ u1 ให้คะแนนแล้วออก จะเหลือ

รายการผู้สมัคร (Candidate Items) เจ็ดเรื่อง คือ i5, i7, i8, i9, i10, i11 และ i12 ทุกตัวอย่างการจัดอันดับของ u1 ในบทนี้ใช้เจ็ดเรื่องนี้

เวกเตอร์ข้อความทุกชุดในบทนี้สร้างจาก เรื่องย่อ ของภาพยนตร์ทั้ง 12 เรื่องในตารางที่ 4.6 ซึ่งเป็นชุดเดียวกับบทที่ 3 คำในเรื่องย่อจึงเป็น "ที่มา" ของตัวเลขแต่ละมิติ เช่น Toy Story 5 มีคำ toy และ toys รวม 5 ครั้ง มิติ toys จึงสูงที่สุด ส่วน Ballerina มีคำกลุ่มอาชญากรรมเกือบทั้งเรื่อง มิติ crime จึงสูงที่สุด

ตารางที่ 4.6 เรื่องย่อ (Synopsis) ของภาพยนตร์ทั้ง 12 เรื่อง

รหัส	ชื่อเรื่อง	เรื่องย่อ
i1	Toy Story 5	A cowboy toy and a space ranger toy lead the toys of a girl bedroom against a glowing tablet that steals every playtime, and the toys must prove to the girl that friendship and imagination still matter more than a screen, so the toys win the girl back.
i2	Inside Out 2	Inside the mind of a teenage girl, five emotions meet an anxious new emotion that seizes control, while each night a dream replays a memory, a dream of friendship and a dream of the future, until a frightening dream blurs belief, memory and self.
i3	The Wild Robot	A shipwrecked robot boots up on a wild island where animals fear the machine, until she raises an orphaned gosling as her own child, learns the language of the wild world, and gives up her programming to protect the family and the island she now calls home.
i4	It Ends with Us	A florist falls in love with a charming surgeon, but when his love turns violent she must choose between the boy she remembers from her first love in a small town and her own safety, so that the cycle of violence ends with her and never reaches her daughter.
i5	Wicked	A green-skinned woman with a gift for magic joins a popular friend to chase a dream of meeting the wizard, sharing music, a rival and a first love, until ambition and a dream of a better land pull the two friends apart and the whole city brands her wicked.
i6	Bridget Jones: Mad About the Boy	A widowed mother of two children keeps talking to the memory of her late husband, until a younger man and a kind teacher bring love into her life again, and she discovers that a new love does not erase a memory, so both of them can live under one roof.
i7	Ballerina	A ballerina trained as an assassin by a secret school of killers hunts the criminal cult that murdered her father, and she cuts a violent path of revenge through hotels, clubs and rival killers of the underworld.
i8	Furiosa: A Mad Max Saga	Snatched from a fertile place as a child, a survivor crosses a scorched wasteland ruled by a warlord and his biker horde, and a road chase across the wasteland becomes a fight for fuel, water, freedom and survival.
i9	Conclave	After the death of the pope, a cardinal must run the secret election of the next pope behind the sealed doors of a chapel, where ambition, a hidden secret of each candidate and opposing factions turn a holy vote into a thriller, until one final secret about the chosen candidate changes the church.
i10	Dune: Part Two	On a desert planet, a young heir of a fallen dynasty allies with the desert tribes, adopts their ways, rides the giant sand worms and leads a war of rebellion to topple an empire, while a prophecy of the desert world and his visions of the future weigh on him.
i11	Mickey 17	A disposable worker on an icy planet dies on every dangerous mission and comes back as a fresh printed copy, until a copy that should be dead returns alive, and two copies of one man must hide from the colony leader, the alien creatures of the planet and the fear of losing his own memory.

รหัส	ชื่อเรื่อง	เรื่องย่อ
i12	Project Hail Mary	A lone science teacher wakes on a spaceship far from Earth with no memory of his mission, finds out that a microbe is dimming the sun, and travels through space with an alien friend from another star to save humanity, while space and time carry him ever farther from home.

ที่มา: เรื่องย่อชุดเดียวกับบทที่ 3 เขียนขึ้นใหม่เพื่อการสอน ไม่ใช่เรื่องย่อทางการของภาพยนตร์ หลังตัดคำและคำหยุดเหลือ 297 โทเคน จากคำต่างกัน 242 คำ

ตัวเลขในตัวอย่างการคำนวณของบทนี้มาจากข้อมูลต้นทางเพียงสามตาราง คือบัญชีภาพยนตร์ คะแนนของผู้ใช้ และเรื่องย่อ รวมกับเวกเตอร์ภาพ เสียง และวิดีโอที่จำลองขึ้นเพื่อการสอน ตารางที่ 4.7 สรุปเส้นทางจากข้อมูลต้นทางจนถึงตัวเลขที่ใช้ และทุกตัวอย่างการคำนวณมีส่วน "ที่มาของข้อมูล" และ "ขั้นตอนโดยละเอียด" กำกับไว้ เพื่อให้ผู้อ่านย้อนกลับไปตรวจตัวเลขทุกตัวได้

ตารางที่ 4.7 เส้นทางของข้อมูล: จากตารางต้นทางสู่ตัวเลขในตัวอย่างการคำนวณ

ข้อมูลต้นทาง	สิ่งที่ได้	ขั้นตอนการได้มา	ใช้ในตัวอย่างการคำนวณ
ตารางที่ 4.4 บัญชีภาพยนตร์ (รหัส ชื่อเรื่อง ปี แนว)	รหัส i1–i12 ซึ่งเป็นกุญแจเชื่อมทุกตาราง และแนวที่ใช้ตีความผล	อ่านตรงจากตาราง	ทุกตัวอย่าง
ตารางที่ 4.5 คะแนนของผู้ใช้ 8 คน	เรื่องที่ชอบ (4–5) เรื่องที่ไม่ชอบ (1–2) รายการผู้สมัคร (ยังไม่เคยให้คะแนน) และ 20 กรณีทดสอบ	จัดกลุ่มตามช่วงคะแนน ผู้สมัคร = 12 เรื่อง − เรื่องที่ให้คะแนนแล้ว	4.4, 4.25–4.38, 4.40, 4.41
ตารางที่ 4.6 เรื่องย่อ	เวกเตอร์ข้อความ 12 มิติ (ตารางที่ 4.12 และ 4.15) และโทเคนตัวอย่าง เช่น toy, friendship, screen	ตัดคำและคำหยุด ⟶ คลังแนวคิด 12 มิติ ⟶ เฉลี่ยถ่วงด้วย IDF ⟶ ปรับ L^2 (บทที่ 3 หัวข้อ 3.7) เทคนิค Transformer เลือกความหมายของคำหลายความหมายตามบริบท	4.1, 4.5, 4.12, 4.14–4.17, 4.21, 4.23, 4.25–4.28
โปสเตอร์ คลิปเสียง และตัวอย่างภาพยนตร์ (รูปแบบในตารางที่ 4.8)	เวกเตอร์ภาพ เสียง วิดีโอ สื่อละ 3 มิติ (ตารางที่ 4.13 และ 4.16)	ค่าจำลองของตัวเข้ารหัสที่กำหนดให้สอดคล้องกับแนว เช่น แอนิเมชันมี V3 = 1 แล้วลบค่าเฉลี่ยของคลังและปรับ L^2 (หัวข้อ 4.5.1)	4.3, 4.4, 4.24, 4.29–4.31, 4.37, 4.40
ตารางที่ 4.19 คะแนนรายสื่อของ u1	ข้อมูลนำเข้าของ Fusion	ผลของโปรไฟล์ค่าเฉลี่ยในหัวข้อ 4.5	4.32–4.38
ค่าสมมติเพื่อการสอน	น้ำหนักของนิวรอน ตัวกรองเมทริกซ์ฉาย วันที่ให้คะแนน และพารามิเตอร์ของ VBPR	กำหนดขึ้นให้คำนวณด้วยมือได้ และระบุไว้ในส่วนที่มาของข้อมูลของแต่ละตัวอย่าง	4.2–4.13, 4.15–4.22, 4.28, 4.31, 4.39, 4.40

ที่มา: ผู้เขียนสรุปจากหัวข้อ 4.1–4.7

ประเด็นสำคัญ ข้อตกลงที่ใช้ตลอดบท

- ลำดับสื่อใช้ T, I, A, V (ข้อความ ภาพ เสียง วิดีโอ) เสมอ ทั้งในตาราง สมการ และน้ำหนัก
- เวกเตอร์ของรายการ i ในสื่อ m เขียนเป็น $\mathbf{v}_i^m$ และโปรไฟล์ของผู้ใช้ u ในสื่อ m เขียนเป็น $\mathbf{p}_u^m$ โดย $m \in \{T, I, A, V\}$
- แยก **คะแนนที่ผู้ใช้ให้ (Rating)** คือค่า 1–5 ออกจาก **คะแนนของระบบ (Score)** คือค่าที่ระบบคำนวณเพื่อจัดอันดับ ซึ่งติดลบได้
- เรื่องที่ "ชอบ" คือเรื่องที่ได้คะแนน 4–5 และเรื่องที่ "ไม่ชอบ" คือเรื่องที่ได้ 1–2
- คะแนนเสมอกันเรียงตามเลขรหัสจากน้อยไปมาก และ Top-K หมายถึงไม่เกิน K เรื่อง บทนี้ใช้ $K = 3$
- ระหว่างคำนวณใช้ค่าละเอียดเต็ม แสดงผลทศนิยมสี่ตำแหน่ง ถ้าแทนค่าด้วยค่าที่ปัดแล้ว ผลอาจต่างกันที่หลักสุดท้าย
- ทุกตัวอย่างการคำนวณเรียงเป็น โจทย์ → สมการที่ใช้ → ข้อมูลที่ใช้ → ขั้นตอน → ตีความ และระบุที่มาของตัวเลขที่ยกมาจากตารางหรือตัวอย่างอื่นทุกครั้ง

โปรแกรม 4.1 อ่านข้อมูลกลางของบทจากไฟล์ mm4.py (ภาคผนวก 4ก ส่วนที่ 1) แล้วพิมพ์รายการผู้สมัครของ u1 และรูปร่างของเวกเตอร์แต่ละสื่อ ทุกโปรแกรมในบทนี้เรียกใช้ไฟล์เดียวกันนี้

โปรแกรม 4.1 โหลดข้อมูลกลางของ MovieMini-12

```
from mm4 import ITEMS, R, vectors


V = vectors('TF')                    # เวกเตอร์เทคนิค Transformer
cand = [i for i in ITEMS if i not in R['u1']]
print('ผู้สมัครของ u1:', cand)
for m in ['T', 'I', 'A', 'V']:
    print(m, 'มิติ =', len(V[m]['i1']))
```

ผลลัพธ์

```
ผู้สมัครของ u1: ['i5', 'i7', 'i8', 'i9', 'i10', 'i11', 'i12']
T มิติ = 12
I มิติ = 3
A มิติ = 3
V มิติ = 3
```

4.2 การสกัดคุณลักษณะ (Feature Extraction) ในยุคต่าง ๆ

ก่อนรู้จัก CNN และ Transformer ผู้อ่านควรเห็นภาพใหญ่ก่อนว่าตัวเข้ารหัสทั้งสองแบบเกิดขึ้นเพื่อแก้ปัญหาอะไร หัวข้อนี้อธิบายความหมายของคุณลักษณะ วิวัฒนาการของการสกัดคุณลักษณะสามยุค และหน่วยพื้นฐานที่ทำให้เครื่อง "เรียนรู้" คุณลักษณะได้เอง

4.2.1 คุณลักษณะคืออะไร และทำไมต้องสกัด

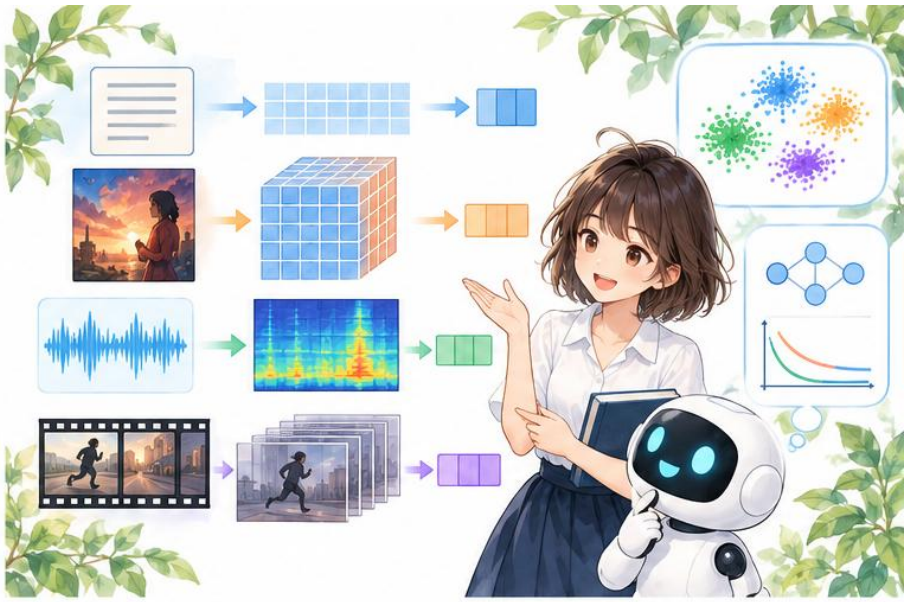
นิยาม 4.2 คุณลักษณะ (Feature) และการสกัดคุณลักษณะ (Feature Extraction)

คุณลักษณะ (Feature) คือตัวเลขชุดหนึ่งที่สรุปสิ่งสำคัญของข้อมูลเพื่อใช้เปรียบเทียบหรือตัดสินใจ เช่น ตัวเลขที่บอกว่าโปสเตอร์เป็นภาพการ์ตูนมากเพียงใด

การสกัดคุณลักษณะ (Feature Extraction) คือกระบวนการแปลงข้อมูลนำเข้าซึ่งมีตัวเลขจำนวนมาก ให้เป็นเวกเตอร์สั้นที่เก็บเฉพาะสาระที่จำเป็น

ตัวแทน (Representation) หรือ **เวกเตอร์ฝัง (Embedding)** คือผลลัพธ์ของการสกัดคุณลักษณะ ซึ่งเป็นเวกเตอร์ยาวคงที่ที่นำไปวัดความคล้ายได้

ข้อมูลนำเข้าของแต่ละสื่อมีขนาดใหญ่มาก ตารางที่ 4.8 สรุปรูปแบบที่ MovieMini เก็บไว้สำหรับภาพยนตร์หนึ่งเรื่อง ตัวเลขขนาดคำนวณจากรูปร่างของข้อมูล ไม่ใช่ขนาดไฟล์หลังบีบอัด ดังสรุปในภาพที่ 4.53 สรุปการเปลี่ยนจากคุณลักษณะที่ออกแบบด้วยมือไปสู่ CNN และ Transformer



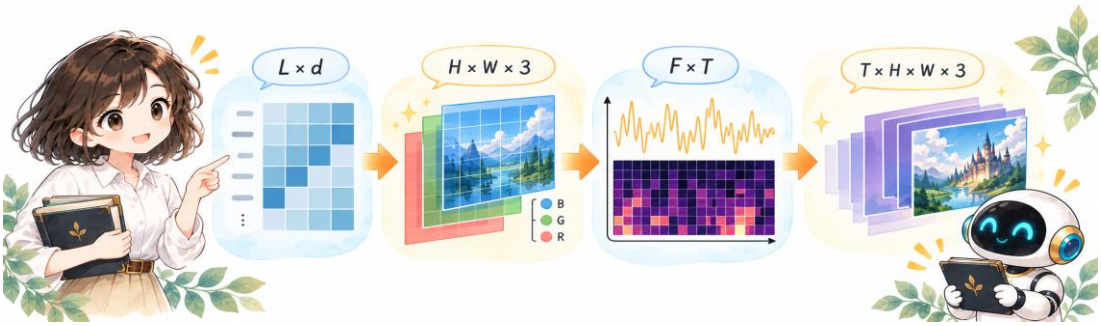
ภาพที่ 4.53 วิวัฒนาการของการสกัดคุณลักษณะ

ตารางที่ 4.8 ขนาดของข้อมูลนำเข้าทั้งสี่สื่อต่อภาพยนตร์หนึ่งเรื่อง

สื่อ	สิ่งที่เก็บ	รูปร่างของเทนเซอร์	จำนวนตัวเลข
ข้อความ (T)	เรื่องย่อ 60 โทเคน	60 × 768	46,080
ภาพ (I)	โปสเตอร์ 224 × 224 พิกเซล สี RGB	224 × 224 × 3	150,528

สื่อ	สิ่งที่เก็บ	รูปร่างของเทนเซอร์	จำนวนตัวเลข
เสียง (A)	คลิปเพลงประกอบ 10.24 วินาที	Log-Mel 128 × 1,024	131,072
วิดีโอ (V)	ตัวอย่างภาพยนตร์ เลือก 16 เฟรม	16 × 224 × 224 × 3	2,408,448

ภาพที่ 4.2 แสดงรูปร่างของข้อมูลทั้งสี่ก่อนเข้าตัวเข้ารหัส ข้อความเป็นตารางสองมิติที่แต่ละแถวคือโทเคนหนึ่งตัว ภาพเป็นตารางสามมิติซ้อนกันสามชั้นตามช่องสี R, G, B เสียงเริ่มจากคลื่นหนึ่งมิติแล้วแปลงเป็นตาราง ความถี่ × เวลา และวิดีโอคือภาพหลายภาพเรียงตามเวลา



ภาพที่ 4.2 รูปร่างของข้อมูลนำเข้าทั้งสี่สื่อก่อนเข้าตัวเข้ารหัส

ระบบนำตัวเลขเหล่านี้ไปเทียบกันตรง ๆ ไม่ได้ เพราะทั้งช้าและไม่มีความหมาย เช่น พิกเซลมุมซ้ายบนของโปสเตอร์สองใบไม่จำเป็นต้องเป็นวัตถุเดียวกัน การสกัดคุณลักษณะจึงต้องย่อข้อมูลให้เหลือเวกเตอร์สั้น ขนาดของการย่อวัดได้ด้วยสมการ (4.2) และ (4.3)

$$N_{in} = \prod_{k=1}^{K} n_k \tag{4.2}$$

โดยที่

- K จำนวนแกนของเทนเซอร์ เช่น ภาพมี 3 แกน (สูง กว้าง ช่องสี) วิดีโอมี 4 แกน
- n_k ขนาดของแกนที่ k เช่น 224, 224 และ 3
- $\prod$ ผลคูณของทุกพจน์ ทำนองเดียวกับที่ $\sum$ คือผลบวกของทุกพจน์
- N_{in} จำนวนตัวเลขทั้งหมดของข้อมูลนำเข้าหนึ่งชิ้น

$$\rho = \frac{N_{in}}{d} \tag{4.3}$$

โดยที่

- d จำนวนมิติของเวกเตอร์ที่ตัวเข้ารหัสคืน เช่น 768
- ρ อัตราการย่อ (Compression Ratio) บอกว่าตัวเลขนำเข้ากี่ค่าถูกสรุปเหลือหนึ่งค่า

ตัวอย่างการคำนวณ 4.2 นับตัวเลขนำเข้าและอัตราการย่อของโปสเตอร์และตัวอย่างภาพยนตร์

โจทย์ ถ้าตัวเข้ารหัสคืนเวกเตอร์ $d = 768$ มิติ
จงหาจำนวนตัวเลขนำเข้าและอัตราการย่อของโปสเตอร์และตัวอย่างภาพยนตร์

สมการที่ใช้ สมการ (4.2) และ (4.3)

ข้อมูลที่ใช้ รูปร่างของเทนเซอร์จากตารางที่ 4.8

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- รูปร่างของเทนเซอร์มาจากตารางที่ 4.8 ซึ่งกำหนดรูปแบบข้อมูลนำเข้าของภาพยนตร์ทุกเรื่องใน MovieMini-12 ให้เหมือนกัน เช่น โปสเตอร์ของ i1 Toy Story 5 ถึง i12 Project Hail Mary ถูกย่อเป็น 224 × 224 พิกเซล 3 ช่องสี
- ตัวอย่างภาพยนตร์ใช้ 16 เฟรม ซึ่งเลือกห่างเท่ากันจากตัวอย่างยาว 2 นาที (ที่มาของ 16 เฟรมแสดงในตัวอย่างการคำนวณ 4.13 ขั้นที่ 4)
- ค่า d = 768 คือขนาดเวกเตอร์ของตัวเข้ารหัสมาตรฐานอย่าง ViT-B/16 และ BERT-base (หัวข้อ 4.2.5)

ขั้นที่ 1 นับตัวเลขของโปสเตอร์ (3 แกน)

- หนึ่งช่องสีมี $224 \times 224 = 50{,}176$ พิกเซล
- สามช่องสีจึงมี $50{,}176 \times 3 = 150{,}528$ ค่า

ขั้นที่ 2 นับตัวเลขของตัวอย่างภาพยนตร์ (4 แกน)

- หนึ่งเฟรมมี 150,528 ค่าเท่ากับโปสเตอร์ (จากขั้นที่ 1)
- 16 เฟรมจึงมี $16 \times 150{,}528 = 2{,}408{,}448$ ค่า

ขั้นที่ 3 หาอัตราการย่อ

$$\rho_I = \frac{150{,}528}{768} = 196, \qquad \rho_V = \frac{2{,}408{,}448}{768} = 3{,}136$$

- เวกเตอร์ของโปสเตอร์เหลือเพียง $768/150{,}528 \approx 0.51\%$ ของข้อมูลนำเข้า
- เวกเตอร์ของตัวอย่างภาพยนตร์เหลือเพียง $768/2{,}408{,}448 \approx 0.03\%$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

คูณขนาดของทุกแกนตามสมการ (4.2) ทีละขั้น

- โปสเตอร์: 224 × 224 = 50,176 (จำนวนพิกเซลในหนึ่งช่องสี) แล้ว × 3 ช่องสี (R, G, B) = 150,528
- ตัวอย่างภาพยนตร์: แต่ละเฟรมมีรูปร่างเท่าโปสเตอร์ คือ 150,528 ค่า แล้ว × 16 เฟรม = 2,408,448
- เรื่องย่อ (เพื่อเทียบ): 60 โทเคน × 768 = 46,080 และเสียง: 128 × 1,024 = 131,072

หารด้วย d = 768 ตามสมการ (4.3)

สื่อ	Nin	ρ = Nin / 768	สัดส่วน d / Nin
ข้อความ	46,080	60	1.67%
ภาพ	150,528	196	0.51%
เสียง	131,072	170.67	0.59%
วิดีโอ	2,408,448	3,136	0.03%

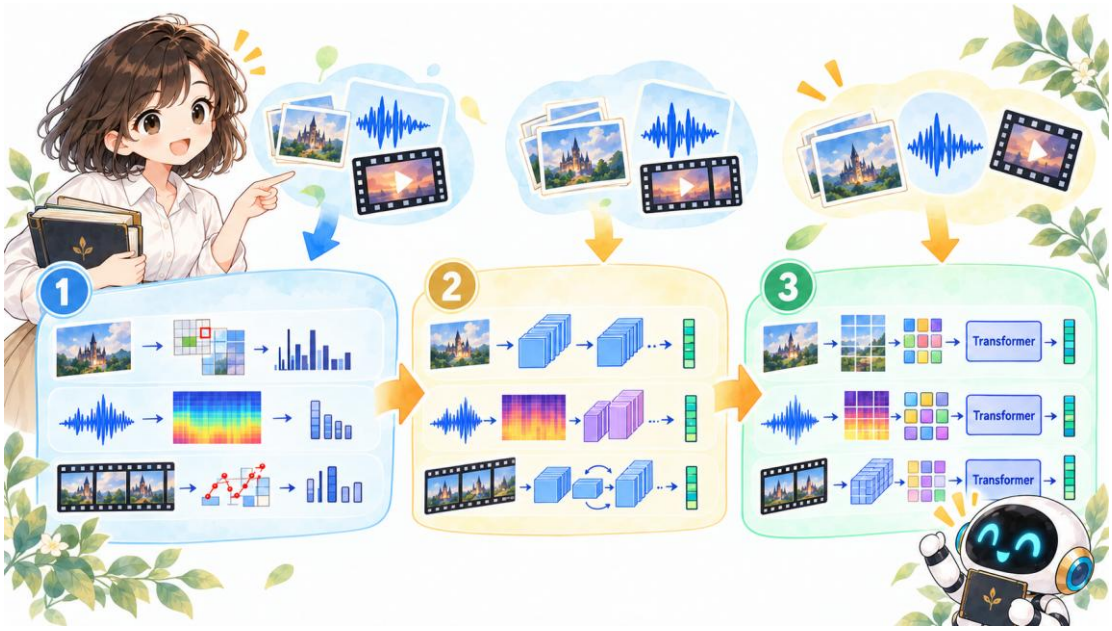
- ตรวจคำตอบ: 196 × 768 = 150,528 และ 3,136 × 768 = 2,408,448 พอดี ตัวเลขทั้งสองจึงเป็นจำนวนเต็ม

ตีความ

ตัวเข้ารหัสสรุปตัวเลข 196 ค่าของโปสเตอร์ให้เหลือหนึ่งค่า และสรุปวิดีโอถึง 3,136 ค่าให้เหลือหนึ่งค่า การย่อนี้ไม่ใช่การบีบอัดไฟล์ เพราะสร้างภาพกลับคืนจากเวกเตอร์ไม่ได้ แต่เป็นการ **เก็บเฉพาะสิ่งที่จำเป็นต่อการเปรียบเทียบ**

4.2.2 สามยุคของการสกัดคุณลักษณะ

วิธีย่อข้อมูลให้เหลือเวกเตอร์สั้นพัฒนามาเป็นสามยุค ภาพที่ 4.3 สรุปพัฒนาการของภาพ เสียง และวิดีโอ แยกตามประเภทข้อมูลนำเข้าและยุคของเทคนิค พร้อมตารางเปรียบเทียบด้านล่างของภาพ



ภาพที่ 4.3 สามยุคของการสกัดคุณลักษณะจากภาพ เสียง และวิดีโอ

ที่มา: ผู้เขียนสรุปจาก Lowe (2004), Krizhevsky et al. (2012), Dosovitskiy et al. (2021) และ Radford et al. (2021)

ภาพที่ 4.3 อ่านจากซ้ายไปขวาได้ดังนี้

- **ยุคที่ 1 คุณลักษณะออกแบบด้วยมือ (ค.ศ. 1990–2011)** ผู้เชี่ยวชาญกำหนดสูตรเองว่าจะวัดอะไร เช่น ฮิสโทแกรมของทิศขอบภาพ (SIFT, HOG) สัมประสิทธิ์ทางความถี่ของเสียง (MFCC) หรือเส้นทางการเคลื่อนที่ในวิดีโอ ผลลัพธ์เรียกว่า **ตัวบรรยาย (Descriptor)** ข้อดีคืออธิบายง่ายและใช้ข้อมูลน้อย แต่จับความหมายระดับสูงได้จำกัด
- **ยุคที่ 2 การเรียนรู้เชิงลึกเฉพาะโดเมน (ค.ศ. 2012–2020)** โครงข่ายประสาทเรียนรู้คุณลักษณะเองจากข้อมูลนำเข้าทั้งก้อน เช่น CNN บนพิกเซล (AlexNet, ResNet) CNN บนสเปกโทรแกรม และ 3D CNN บนวิดีโอ ผลลัพธ์คือ **แผนที่คุณลักษณะ (Feature Map)** แล้วรวมเป็นเวกเตอร์ฝัง
- **ยุคที่ 3 Transformer และแบบจำลองพื้นฐานหลายรูปแบบ (ค.ศ. 2021–ปัจจุบัน)** แบ่งข้อมูลเป็นโทเคนหรือแพตช์ แล้วให้ทุกชิ้นปรึกษากันด้วย Attention เช่น ViT, AST, Video Transformer และ CLIP ซึ่งวางภาพกับข้อความไว้ใน **ปริภูมิร่วม (Shared Embedding Space)** เดียวกัน

ระหว่างยุคมีลูกศรสามอันกำกับว่า ยุคหลังใช้ **ข้อมูลมากขึ้น การคำนวณมากขึ้น** และได้ **ความเข้าใจเชิงความหมายสูงขึ้น** ข้อความก็มีพัฒนาการคู่ขนานกัน คือจากการนับคำ (TF-IDF ในบทที่ 3) สู่ Static Embedding และ CNN บนคำ แล้วจึงถึง Transformer อย่าง BERT ตารางที่ 4.9 สรุปสามยุคครบทั้งสี่สื่อ

ตารางที่ 4.9 สรุปสามยุคของการสกัดคุณลักษณะสำหรับสี่สื่อ

สื่อ	ยุคที่ 1 ออกแบบด้วยมือ	ยุคที่ 2 เรียนรู้เชิงลึก (CNN)	ยุคที่ 3 Transformer
ข้อความ	Bag-of-Words, TF-IDF, BM25	Word2Vec, GloVe + Text CNN	BERT, Sentence-BERT, MiniLM

สื่อ	ยุคที่ 1 ออกแบบด้วยมือ	ยุคที่ 2 เรียนรู้เชิงลึก (CNN)	ยุคที่ 3 Transformer
ภาพ	SIFT, HOG, LBP, GIST	AlexNet, VGG, ResNet, EfficientNet	ViT, Swin, DINOv2, CLIP
เสียง	MFCC, Chroma, Spectral Centroid	CNN บนสเปกโทรแกรม, VGGish, PANNs	AST, wav2vec 2.0, HuBERT, CLAP
วิดีโอ	STIP, HOF, Dense Trajectories	2D CNN รายเฟรม, C3D, I3D, SlowFast	TimeSformer, ViViT, VideoMAE
ผลลัพธ์	ตัวบรรยาย (Descriptor)	แผนที่คุณลักษณะ → เวกเตอร์ฝัง	เวกเตอร์ตามบริบท / ปริภูมิร่วม

ที่มา: ผู้เขียนสรุปจาก Baltrušaitis et al. (2019) และ Deldjoo et al. (2020)

อธิบายแบบเข้าใจง่าย จากสูตรอาหารที่เขียนเองสู่ลิ้นที่ฝึกมา

- ยุคที่ 1 เหมือนพ่อครัวที่ชิมอาหารด้วย **รายการตรวจที่เขียนไว้ล่วงหน้า** เช่น เค็มเท่าไร หวานเท่าไร ใช้ง่ายแต่ตรวจได้เฉพาะสิ่งที่นึกออก
- ยุคที่ 2 เหมือนพ่อครัวที่ **ชิมอาหารมาหลายหมื่นจาน** จนรู้เองว่ารสใดสำคัญ แต่เชี่ยวชาญเฉพาะอาหารประเภทเดียว
- ยุคที่ 3 เหมือนนักวิจารณ์ที่ **ได้ชิม ดู และฟังพร้อมกัน** และเชื่อมคำบรรยายกับรสชาติได้ จึงพูดถึงอาหารข้ามประเภทได้ แต่ต้องฝึกนานและใช้ทรัพยากรมาก

บทนี้เน้นยุคที่ 2 และยุคที่ 3 เพราะเป็นเทคนิคที่ใช้สร้างเวกเตอร์ของรายการในระบบผู้แนะนำปัจจุบัน ส่วนการนับคำของยุคที่ 1 ผู้อ่านได้เรียนแล้วในบทที่ 3 ก่อนเข้าสู่ CNN และ Transformer หัวข้อ 4.2.3–4.2.5 จะอธิบายหน่วยพื้นฐานที่ทั้งสองเทคนิคใช้ร่วมกัน คือนิวรอน การลดตามเกรเดียนต์ และการฝัง

4.2.3 นิวรอน: หน่วยพื้นฐานของการเรียนรู้คุณลักษณะ

ทั้ง CNN และ Transformer สร้างจากหน่วยเล็กที่สุดชนิดเดียวกัน คือ **นิวรอน (Neuron)** นิวรอนหนึ่งตัวรับตัวเลขหลายค่า คูณแต่ละค่าด้วยน้ำหนัก บวกกัน บวกค่าเอนเอียง แล้วส่งผ่านฟังก์ชันกระตุ้น ดังสมการ (4.4) และ (4.5)

$$z = \sum_{k=1}^{n} w_k \, x_k + b = \mathbf{w} \cdot \mathbf{x} + b \tag{4.4}$$

โดยที่

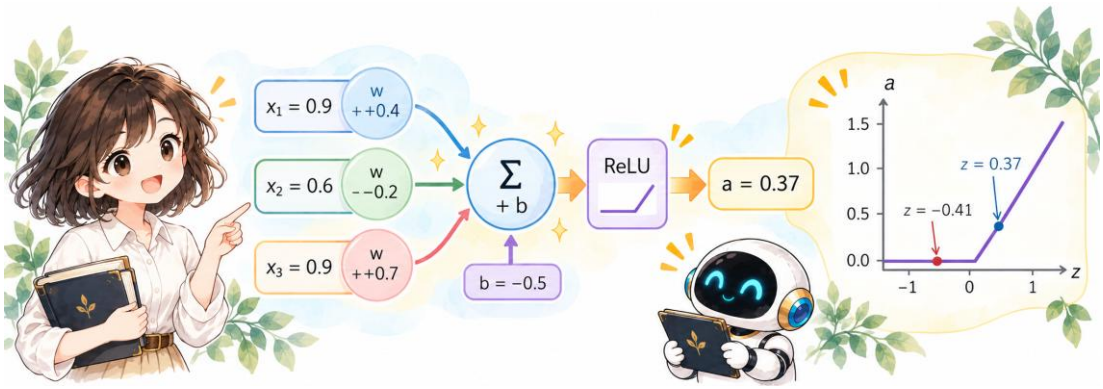
- x_k ค่านำเข้าตัวที่ k จากทั้งหมด n ค่า
- w_k น้ำหนัก (Weight) ของค่านำเข้าตัวที่ k บอกว่าค่านั้นสำคัญและมีทิศทางอย่างไร
- b ค่าเอนเอียง (Bias) ใช้เลื่อนเกณฑ์การตอบสนอง
- z ผลรวมถ่วงน้ำหนัก ก่อนผ่านฟังก์ชันกระตุ้น

$$a = \mathrm{ReLU}(z) = \max(0, z) \tag{4.5}$$

โดยที่

- a ค่าส่งออกของนิวรอน (Activation)
- ReLU ฟังก์ชันกระตุ้นแบบ Rectified Linear Unit ตัดค่าลบเป็น 0 และส่งค่าบวกผ่านตามเดิม

ภาพที่ 4.4 แสดงนิวรอนที่รับเวกเตอร์ภาพสามมิติของ i1 Toy Story 5 จากตารางที่ 4.16 ฝั่งซ้ายคือการคูณและบวกตามสมการ (4.4) ฝั่งขวาคือกราฟของ ReLU ที่ส่งค่าบวกผ่านแต่ตัดค่าลบทิ้ง



ภาพที่ 4.4 นิวรอนหนึ่งตัวกับฟังก์ชัน ReLU

ตัวอย่างการคำนวณ 4.3 นิวรอนตรวจจับโปสเตอร์แบบแอนิเมชัน

โจทย์ นิวรอนมีน้ำหนัก $\mathbf{w} = (0.4, -0.2, 0.7)$ และ $b = -0.5$ จงหาค่าส่งออกเมื่อป้อนเวกเตอร์ภาพของ i1 Toy Story 5 และ i7 Ballerina

สมการที่ใช้ สมการ (4.4) และ (4.5)

ข้อมูลที่ใช้ เวกเตอร์ภาพเทคนิค Transformer จากตารางที่ 4.16: i1 = (0.9, 0.6, 0.9) และ i7 = (0.2, 0.3, 0.1) น้ำหนักเป็นค่าสมมติเพื่อการสอน

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- เวกเตอร์ภาพมาจากตารางที่ 4.16 (เทคนิค Transformer) แถว i1 Toy Story 5 = (0.9, 0.6, 0.9) และแถว i7 Ballerina = (0.2, 0.3, 0.1) ค่าเหล่านี้ยังไม่ผ่านการลบค่าเฉลี่ยของหัวข้อ 4.5.1
- เลือกสองเรื่องนี้เพราะอยู่คนละขั้วของบัญชีภาพยนตร์ในตารางที่ 4.4: i1 เป็นแอนิเมชันที่ u1 ให้ 5 คะแนน (ตารางที่ 4.5) ส่วน i7 เป็นแอ็กชันโทนมืดที่ u1 ยังไม่เคยให้คะแนน
- มิติที่ 3 ของภาพ (I3) ถูกกำหนดให้สูงในแอนิเมชันทั้งสามเรื่อง (i1 = 0.9, i2 = 0.9, i3 = 0.8) และต่ำในเรื่องอื่น (0.1–0.5) น้ำหนัก w3 = 0.7 จึงทำให้นิวรอนตัวนี้ "ไว" ต่อโปสเตอร์แอนิเมชัน
- น้ำหนัก w = (0.4, −0.2, 0.7) และค่าเอนเอียง b = −0.5 เป็นค่าสมมติเพื่อการสอน

ขั้นที่ 1 คำนวณ z ของ i1

- คูณทีละมิติ: $0.4 \times 0.9 = 0.36,\ -0.2 \times 0.6 = -0.12,\ 0.7 \times 0.9 = 0.63$
- รวมแล้วบวกค่าเอนเอียง: $0.36 - 0.12 + 0.63 - 0.5 = 0.37$

ขั้นที่ 2 ผ่าน ReLU

- $a = \max(0,\ 0.37) = 0.37$ นิวรอน "ตื่นตัว"

ขั้นที่ 3 ทำซ้ำกับ i7

- $z = 0.4 \times 0.2 - 0.2 \times 0.3 + 0.7 \times 0.1 - 0.5 = 0.08 - 0.06 + 0.07 - 0.5 = -0.41$
- $a = \max(0,\ -0.41) = 0$ นิวรอน "เงียบ"

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

ทำซ้ำการคำนวณกับภาพยนตร์ทั้ง 12 เรื่องในตารางที่ 4.16

- สูตรเดียวกันทุกแถว: z = 0.4 × I1 − 0.2 × I2 + 0.7 × I3 − 0.5 และ a = max(0, z)
- ตัวอย่าง i2 Inside Out 2: 0.4 × 0.8 − 0.2 × 0.8 + 0.7 × 0.9 − 0.5 = 0.32 − 0.16 + 0.63 − 0.5 = 0.29

รายการ	เวกเตอร์ภาพ (I1, I2, I3)	z	a = ReLU(z)
i1 Toy Story 5	(0.9, 0.6, 0.9)	0.37	0.37
i2 Inside Out 2	(0.8, 0.8, 0.9)	0.29	0.29
i3 The Wild Robot	(0.7, 0.5, 0.8)	0.24	0.24
i4 It Ends with Us	(0.5, 0.7, 0.2)	−0.30	0.00
i5 Wicked	(0.7, 0.6, 0.5)	0.01	0.01
i6 Bridget Jones: Mad About the Boy	(0.6, 0.8, 0.3)	−0.21	0.00
i7 Ballerina	(0.2, 0.3, 0.1)	−0.41	0.00
i8 Furiosa: A Mad Max Saga	(0.6, 0.9, 0.3)	−0.23	0.00
i9 Conclave	(0.4, 0.3, 0.1)	−0.33	0.00

i10 Dune: Part Two	(0.5, 0.8, 0.1)	−0.39	0.00
i11 Mickey 17	(0.3, 0.2, 0.1)	−0.35	0.00
i12 Project Hail Mary	(0.3, 0.3, 0.1)	−0.37	0.00

- นิวรอนตื่นตัวชัดเจนเพียงสามเรื่อง คือ i1, i2 และ i3 (a = 0.24–0.37)
  ซึ่งเป็นแอนิเมชันทั้งหมดและเป็นสามเรื่องที่ u1 ชอบพอดี i5 Wicked ได้ 0.01 ซึ่งแทบเป็นศูนย์
  ส่วนเรื่องอื่นได้ z ติดลบ ReLU จึงให้ 0

ตีความ

น้ำหนักบวกขนาดใหญ่ของมิติที่ 3 (0.7) ทำให้นิวรอนตอบสนองต่อเรื่องที่มีค่ามิติที่ 3 สูงอย่าง i1 แต่เงียบต่อ i7 นี่คือความหมายของ "การเรียนรู้คุณลักษณะ" คือการปรับน้ำหนักให้นิวรอนตอบสนองต่อลวดลายที่มีประโยชน์โดยไม่ต้องมีคนเขียนกฎเอง

อธิบายแบบเข้าใจง่าย นิวรอนคือกรรมการให้คะแนนหนึ่งคน

นิวรอนเหมือนกรรมการที่ดูหลักฐานหลายข้อ แล้วให้น้ำหนักแต่ละข้อตามความเห็นของตน (น้ำหนัก w_k)
ถ้าคะแนนรวมไม่ถึงเกณฑ์ (ค่าเอนเอียง) กรรมการจะ "ไม่ยกป้าย" (ReLU ให้ 0)

โครงข่ายประสาทคือคณะกรรมการหลายพันคนที่ซ้อนกันหลายชั้น ชั้นแรกดูลวดลายง่าย
ชั้นถัดไปนำความเห็นของชั้นก่อนมารวมเป็นลวดลายที่ซับซ้อนขึ้น

ที่มาของไฮเปอร์พารามิเตอร์ ฟังก์ชันกระตุ้น: ทำไมนิยมใช้ ReLU

- **ที่มา** ReLU ถูกนำมาใช้แพร่หลายหลัง Nair and Hinton (2010) และ AlexNet (Krizhevsky et al., 2012) เพราะคำนวณง่ายและเกรเดียนต์ไม่หดหายเมื่อค่าเป็นบวก
- **ผลเมื่อเปลี่ยน** Sigmoid ให้ค่าระหว่าง 0–1 แต่เกรเดียนต์เล็กลงมากเมื่อค่าใหญ่
  ทำให้โครงข่ายลึกเรียนรู้ช้า ส่วน GELU ซึ่งโค้งนุ่มกว่า ReLU นิยมใช้ใน Transformer (Hendrycks and Gimpel, 2016)
- **วิธีเลือก** ใช้ตามสถาปัตยกรรมต้นแบบ คือ ReLU สำหรับ CNN และ GELU สำหรับ Transformer

โปรแกรม 4.2 นิวรอนหนึ่งตัวกับ ReLU

```
import numpy as np
from mm4 import load


I = load('TF')['I']                 # เวกเตอร์ภาพ มิติ 1-3
w = np.array([0.4, -0.2, 0.7])
b = -0.5
for i in ['i1', 'i7']:
    z = w @ I[i] + b                # สมการผลรวมถ่วงน้ำหนัก
    a = max(0.0, z)                 # ReLU
    print(i, 'z = %.2f  a = %.2f' % (z, a))
```

ผลลัพธ์

```
i1 z = 0.37  a = 0.37
i7 z = -0.41  a = 0.00
```

4.2.4 ค่าสูญเสียและการลดตามเกรเดียนต์

นิวรอนในตัวอย่างการคำนวณ 4.3 ใช้น้ำหนักที่ผู้เขียนกำหนดเอง ในระบบจริงน้ำหนักได้จาก **การฝึก (Training)** การฝึกมีสองส่วน คือวัดว่าทายผิดมากเพียงใดด้วย **ค่าสูญเสีย (Loss)** แล้วปรับน้ำหนักไปทางที่ทำให้ค่าสูญเสียลดลงด้วย **การลดตามเกรเดียนต์ (Gradient Descent)**

$$L(w) = (\hat{y} - y)^2 \tag{4.6}$$

โดยที่

- $\hat{y}$ ค่าที่แบบจำลองทาย ซึ่งขึ้นกับน้ำหนัก w
- y ค่าเป้าหมายที่ถูกต้อง
- $L(w)$ ค่าสูญเสียแบบกำลังสองของความผิดพลาด

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w}(w_t) \tag{4.7}$$

โดยที่

- w_t น้ำหนักในรอบที่ t
- $\frac{\partial L}{\partial w}$ เกรเดียนต์ (ความชัน) ของค่าสูญเสียเทียบกับน้ำหนัก บอกว่าค่าสูญเสียเพิ่มขึ้นทางใด
- η อัตราการเรียนรู้ (Learning Rate) คือขนาดของก้าว
- w_{t+1} น้ำหนักใหม่ ซึ่งก้าวไปทางตรงข้ามกับความชัน

ตัวอย่างการคำนวณ 4.4 ปรับน้ำหนักหนึ่งตัวด้วยการลดตามเกรเดียนต์

โจทย์ นิวรอนมีค่านำเข้า $x = 0.9$ ทาย $\hat{y} = wx$ และเป้าหมาย $y = 1$ เริ่มจาก $w_0 = 0$ ใช้ $\eta = 0.5$ จงคำนวณการปรับน้ำหนักสามรอบ

สมการที่ใช้ สมการ (4.6) และ (4.7)

ข้อมูลที่ใช้ ค่าสมมติเพื่อการสอน

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- ค่านำเข้า x = 0.9 เลือกให้เท่ากับมิติที่ 1 ของเวกเตอร์ภาพ i1 Toy Story 5 ในตารางที่ 4.16 เพื่อให้ตัวเลขเชื่อมกับชุดข้อมูลของบท
- เป้าหมาย y = 1 แทนคำตอบ "ชอบ" เพราะ u1 ให้ i1 คะแนน 5 ซึ่งอยู่ในช่วงชอบ (4–5) ตามตารางที่ 4.5

- น้ำหนักเริ่มต้น w0 = 0 และอัตราการเรียนรู้ η = 0.5 เป็นค่าสมมติ ส่วนผลของ η อื่นแสดงในภาพที่ 4.5

ขั้นที่ 1 เขียนค่าสูญเสียและเกรเดียนต์

$$L(w) = (0.9w - 1)^2, \qquad \frac{\partial L}{\partial w} = 2(0.9w - 1) \times 0.9$$

- ใช้กฎลูกโซ่ (Chain Rule): อนุพันธ์ของ $(\cdot)^2$ คือ $2(\cdot)$ แล้วคูณด้วยอนุพันธ์ของ $0.9w - 1$ ซึ่งเท่ากับ 0.9

ขั้นที่ 2 คำนวณทีละรอบ

- รอบที่ 1: $w = 0.0000$, $L = 1.0000$, เกรเดียนต์ = -1.8000 จึงได้ $w_{new} = 0.0000 - 0.5 \times (-1.8000) = 0.9000$
- รอบที่ 2: $w = 0.9000$, $L = 0.0361$, เกรเดียนต์ = -0.3420 จึงได้ $w_{new} = 0.9000 - 0.5 \times (-0.3420) = 1.0710$
- รอบที่ 3: $w = 1.0710$, $L = 0.0013$, เกรเดียนต์ = -0.0650 จึงได้ $w_{new} = 1.0710 - 0.5 \times (-0.0650) = 1.1035$

ขั้นที่ 3 เทียบกับคำตอบจริง

- คำตอบที่ทำให้ $L = 0$ คือ $w = 1/0.9 = 1.1111$
- หลังสามรอบได้ $w = 1.1035$ เข้าใกล้คำตอบแล้ว

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

หาอนุพันธ์ทีละชั้นด้วยกฎลูกโซ่

- ให้ e = ŷ − y = 0.9w − 1 จะได้ L = e² ดังนั้น dL/de = 2e และ de/dw = 0.9
- คูณกัน: dL/dw = 2e × 0.9 = 1.8 × (0.9w − 1)

ตารางคำนวณสามรอบ (wใหม่ = w − 0.5 × เกรเดียนต์)

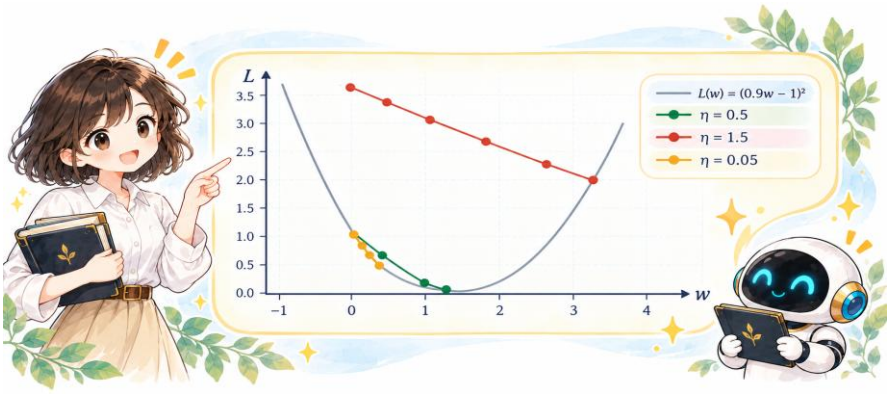
รอบ	w	ŷ = 0.9w	e = ŷ − 1	L = e²	dL/dw = 1.8e	wใหม่
1	0.0000	0.0000	−1.0000	1.0000	−1.8000	0.9000
2	0.9000	0.8100	−0.1900	0.0361	−0.3420	1.0710
3	1.0710	0.9639	−0.0361	0.0013	−0.0650	1.1035

- สังเกตว่าเกรเดียนต์ลดจาก −1.8000 เหลือ −0.3420 และ −0.0650 เพราะ e เข้าใกล้ 0 ก้าวของน้ำหนักจึงสั้นลงเองโดยไม่ต้องปรับ η
- คำตอบที่แท้จริงคือ w = 1/0.9 = 1.1111 หลังสามรอบ w = 1.1035 ห่างเพียง 0.0076

ตีความ

ทุกรอบน้ำหนักขยับไปทางที่ความชันชี้ลง และก้าวสั้นลงเองเมื่อใกล้จุดต่ำสุดเพราะความชันเล็กลง ระบบจริงทำแบบเดียวกันกับน้ำหนักหลายล้านตัวพร้อมกัน โดยคำนวณเกรเดียนต์ของทุกน้ำหนักด้วย **การแพร่ย้อนกลับ (Backpropagation)** ซึ่งก็คือการใช้กฎลูกโซ่ย้อนจากชั้นสุดท้ายไปชั้นแรก (Rumelhart et al., 1986)

ภาพที่ 4.5 เทียบอัตราการเรียนรู้สามค่าบนค่าสูญเสียเดียวกัน ค่า $\eta = 0.5$ ลงถึงก้นหุบในไม่กี่ก้าว ค่า $\eta = 1.5$ ก้าวยาวเกินจนกระโดดข้ามไปมาและค่าสูญเสียเพิ่มขึ้น ส่วนค่า $\eta = 0.05$ ปลอดภัยแต่ช้ามาก



ภาพที่ 4.5 ผลของอัตราการเรียนรู้สามค่าบนค่าสูญเสียเดียวกัน

ที่มาของไฮเปอร์พารามิเตอร์ อัตราการเรียนรู้ η มาจากไหน

- **ที่มา** ค่าเริ่มต้นที่นิยมมาจากงานต้นแบบ เช่น 0.1 สำหรับ SGD ของ ResNet (He et al., 2016) 0.001 สำหรับ Adam (Kingma and Ba, 2015) และ 2×10^{-5} ถึง 5×10^{-5} เมื่อปรับแบบจำลองที่ฝึกไว้แล้วอย่าง BERT (Devlin et al., 2019)
- **ผลเมื่อเปลี่ยน** ค่าใหญ่เกินทำให้ค่าสูญเสียแกว่งหรือพุ่งขึ้น ค่าเล็กเกินทำให้ฝึกช้าและอาจติดอยู่ที่จุดที่ไม่ดี
- **วิธีเลือก** ทดลองหลายค่าแบบลอการิทึม เช่น $10^{-5}, 10^{-4}, 10^{-3}$ แล้วเลือกค่าที่ค่าสูญเสียบนชุดตรวจสอบต่ำสุด และมักใช้การค่อย ๆ เพิ่มช่วงแรก (Warm-up) แล้วลดลงภายหลัง

อธิบายแบบเข้าใจง่าย เดินลงเขาในหมอก

การลดตามเกรเดียนต์เหมือนคนเดินลงเขาในหมอกหนาที่มองไม่เห็นหุบเขา
เขารู้เพียงว่าพื้นใต้เท้าลาดไปทางไหน จึงก้าวไปทางที่ลาดลงทีละก้าว
อัตราการเรียนรู้คือ **ความยาวของก้าว** ก้าวสั้นเกินก็ถึงช้า ก้าวยาวเกินก็อาจกระโดดข้ามหุบไปขึ้นอีกฝั่ง

โปรแกรม 4.3 การลดตามเกรเดียนต์ของน้ำหนักหนึ่งตัว

```
x, y = 0.9, 1.0
for eta in [0.5, 1.5, 0.05]:
    w = 0.0
    for t in range(3):
        grad = 2 * (w * x - y) * x      # ความชันของ (wx - y)^2
        w = w - eta * grad               # ก้าวสวนความชัน
    print('eta = %.2f  w หลัง 3 รอบ = %.4f  L = %.4f'
          % (eta, w, (w * x - y) ** 2))
```

ผลลัพธ์

```
eta = 0.50  w หลัง 3 รอบ = 1.1035  L = 0.0000
eta = 1.50  w หลัง 3 รอบ = 4.3602  L = 8.5510
eta = 0.05  w หลัง 3 รอบ = 0.2487  L = 0.6024
```

4.2.5 การฝัง (Embedding): เปลี่ยนสัญลักษณ์เป็นเวกเตอร์

นิวรอนรับได้เฉพาะตัวเลข แต่ข้อมูลหลายชนิดเป็น **สัญลักษณ์** เช่น คำ รหัสภาพยนตร์ หรือรหัสผู้ใช้ ถ้าใช้เลขลำดับในคลังศัพท์ตรง ๆ เช่น toy = 1, robot = 2 ระบบจะเข้าใจผิดว่า robot "มากกว่า" toy สองเท่า

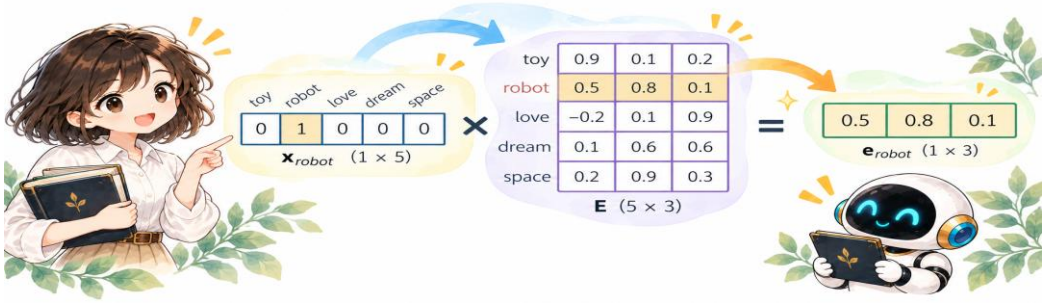
วิธีที่ถูกคือแปลงสัญลักษณ์เป็นเวกเตอร์หนึ่งร้อน (One-hot Vector) แล้วคูณกับ **ตารางฝัง (Embedding Matrix)** ดังสมการ (4.8)

$$\mathbf{e}_w = \mathbf{x}_w \mathbf{E} = \mathbf{E}[\mathrm{id}(w), :] \tag{4.8}$$

โดยที่

- $\mathbf{x}_w$ เวกเตอร์หนึ่งร้อนของสัญลักษณ์ w ยาวเท่าขนาดคลังศัพท์ $|V|$ มีค่า 1 ที่ตำแหน่งของ w ที่เหลือเป็น 0
- $\mathbf{E}$ ตารางฝังขนาด $|V| \times d$ แต่ละแถวคือเวกเตอร์ของหนึ่งสัญลักษณ์
- $\mathrm{id}(w)$ เลขแถวของสัญลักษณ์ w
- $\mathbf{e}_w$ เวกเตอร์ฝังของ w ขนาด d มิติ ซึ่งก็คือแถวที่ $\mathrm{id}(w)$ ของ $\mathbf{E}$

ภาพที่ 4.6 แสดงคลังศัพท์ห้าคำและตารางฝังสามมิติ การคูณเวกเตอร์หนึ่งร้อนของ robot กับตารางฝังให้ผลเท่ากับการ **หยิบแถวที่ 2** ของตารางออกมาตรง ๆ



ภาพที่ 4.6 เวกเตอร์หนึ่งร้อนคูณตารางฝังเท่ากับการหยิบแถวของคำนั้น

ตัวอย่างการคำนวณ 4.5 เปิดแถวของตารางฝังสำหรับคำว่า robot

โจทย์ คลังศัพท์มีห้าคำ คือ toy, robot, love, dream, space จงหาเวกเตอร์ฝังของ robot ด้วยสมการ (4.8) และหาโคไซน์ระหว่าง toy กับ robot

สมการที่ใช้ สมการ (4.8) และ (4.1)

ข้อมูลที่ใช้ ตารางฝัง $\mathbf{E}$ ขนาด 5×3 ในภาพที่ 4.6 (ค่าสมมติเพื่อการสอน)

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- คลังศัพท์ห้าคำเลือกจากเรื่องย่อในตารางที่ 4.6: toy (พบใน i1 สองครั้ง), robot (i3), love (i4 สามครั้ง, i5 หนึ่งครั้ง, i6 สองครั้ง), dream (i2 สี่ครั้ง, i5 สองครั้ง) และ space (i1 หนึ่งครั้ง, i12 สองครั้ง)
- ลำดับในคลังศัพท์ (toy = 1, robot = 2, ...) เป็นเพียงเลขแถว ไม่มีความหมายเชิงปริมาณ จึงต้องเปลี่ยนเป็นเวกเตอร์หนึ่งร้อนก่อน
- ค่าในตารางฝัง E ขนาด 5×3 เป็นค่าสมมติในภาพที่ 4.6 ไม่ใช่คลังแนวคิด 12 มิติของบทที่ 3

ขั้นที่ 1 สร้างเวกเตอร์หนึ่งร้อน

- robot อยู่ลำดับที่ 2 จึงได้ $\mathbf{x}_{robot} = (0,1,0,0,0)$

ขั้นที่ 2 คูณกับตารางฝังทีละคอลัมน์

- คอลัมน์ 1: $0(0.9) + 1(0.5) + 0(-0.2) + 0(0.1) + 0(0.2) = 0.5$
- คอลัมน์ 2: $1 \times 0.8 = 0.8$ และคอลัมน์ 3: $1 \times 0.1 = 0.1$

- จึงได้ $\mathbf{e}_{robot} = (0.5,0.8,0.1)$ ซึ่งเท่ากับแถวที่ 2 พอดี

ขั้นที่ 3 วัดความคล้ายกับ toy = (0.9, 0.1, 0.2)

- ผลคูณจุด = $0.9(0.5) + 0.1(0.8) + 0.2(0.1) = 0.55$
- ความยาว $\| \mathbf{e}_{toy} \| = \sqrt{0.86} = 0.9274$ และ $\| \mathbf{e}_{robot} \| = \sqrt{0.90} = 0.9487$
- โคไซน์ = $0.55/(0.9274 \times 0.9487) = 0.6252$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

คูณเวกเตอร์หนึ่งร้อนกับตารางฝังทีละคอลัมน์

แถว (คำ)	x	คอลัมน์ 1	คอลัมน์ 2	คอลัมน์ 3
1 toy	0	0 × 0.9 = 0	0 × 0.1 = 0	0 × 0.2 = 0
2 robot	1	1 × 0.5 = 0.5	1 × 0.8 = 0.8	1 × 0.1 = 0.1
3 love	0	0 × (−0.2) = 0	0 × 0.1 = 0	0 × 0.9 = 0
4 dream	0	0 × 0.1 = 0	0 × 0.6 = 0	0 × 0.6 = 0
5 space	0	0 × 0.2 = 0	0 × 0.9 = 0	0 × 0.3 = 0
ผลรวม		0.5	0.8	0.1

โคไซน์ระหว่าง toy กับ robot

- ผลคูณจุด: 0.9 × 0.5 + 0.1 × 0.8 + 0.2 × 0.1 = 0.45 + 0.08 + 0.02 = 0.55
- $\|\text{toy}\|^2$ = 0.81 + 0.01 + 0.04 = 0.86 จึงได้ $\|\text{toy}\|$ = 0.9274 และ $\|\text{robot}\|^2$ = 0.25 + 0.64 + 0.01 = 0.90 จึงได้ $\|\text{robot}\|$ = 0.9487
- โคไซน์ = 0.55 / (0.9274 × 0.9487) = 0.55 / 0.8798 = 0.6252

ตีความ

ระบบจริงไม่คูณเมทริกซ์ใหญ่จริง แต่ **เปิดแถว (Lookup)** ตามเลขรหัสซึ่งเร็วกว่ามาก
ค่าในตารางเริ่มจากค่าสุ่ม แล้วถูกปรับด้วยการลดตามเกรเดียนต์ในหัวข้อ 4.2.4
จนคำที่ใช้ในบริบทคล้ายกันมีเวกเตอร์ชี้ทิศใกล้กัน

แนวคิดเดียวกันนี้ใช้กับสัญลักษณ์ทุกชนิดในบทนี้ ได้แก่ **รหัสรายการและรหัสผู้ใช้** ใน VBPR (หัวข้อ 4.6.10) **แพตช์ของภาพและเสียง** ใน Transformer ซึ่งใช้การฉายเชิงเส้นแทนการเปิดแถว (หัวข้อ 4.4.7) และ **ตำแหน่ง** ของโทเคน (หัวข้อ 4.4.4)

ที่มาของไฮเปอร์พารามิเตอร์ ขนาดเวกเตอร์ฝัง d มาจากไหน

- **ที่มา** กำหนดโดยสถาปัตยกรรมของตัวเข้ารหัส เช่น BERT-base และ ViT-B/16 ใช้ 768 มิติ (Devlin et al., 2019; Dosovitskiy et al., 2021) ResNet-50 ให้ 2,048 มิติ และ MiniLM ให้ 384 มิติ
- **ผลเมื่อเปลี่ยน** d ใหญ่เก็บรายละเอียดได้มากกว่า แต่ใช้หน่วยความจำและเวลาค้นหามากขึ้นตามสัดส่วน d เล็กเร็วและประหยัด แต่ความหมายที่ต่างกันเล็กน้อยอาจถูกรวมเป็นทิศเดียวกัน
- **วิธีเลือก** ใช้ d ตามตัวเข้ารหัสที่เลือก แล้วจึงฉายลงมิติที่เล็กลงถ้าต้องการลดต้นทุน โดยเทียบตัวชี้วัดของบทที่ 2 บนชุดตรวจสอบ
- **ในบทนี้** ใช้ 12 มิติสำหรับข้อความ และ 3 มิติสำหรับภาพ เสียง และวิดีโอ เพื่อให้คำนวณด้วยมือได้

โปรแกรม 4.4 เปิดแถวของตารางฝัง

```
import numpy as np

vocab = ['toy', 'robot', 'love', 'dream', 'space']
E = np.array([[0.9, 0.1, 0.2], [0.5, 0.8, 0.1],
              [-0.2, 0.1, 0.9], [0.1, 0.6, 0.6],
              [0.2, 0.9, 0.3]])
x = np.zeros(len(vocab))
x[vocab.index('robot')] = 1          # เวกเตอร์หนึ่งร้อน
print('x @ E      =', x @ E)
print('E[id]      =', E[vocab.index('robot')])
a, b = E[0], E[1]
cos = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print('cos(toy, robot) = %.4f' % cos)
```

ผลลัพธ์

```
x @ E      = [0.5 0.8 0.1]
E[id]      = [0.5 0.8 0.1]
cos(toy, robot) = 0.6252
```

4.2.6 ตัวเข้ารหัส (Encoder) และเวกเตอร์ที่ใช้ในบทนี้

เมื่อนำนิวรอนจำนวนมากมาเรียงเป็นชั้นและซ้อนกันหลายชั้น จะได้ **ตัวเข้ารหัส (Encoder)** ซึ่งเป็นฟังก์ชันที่มีพารามิเตอร์ รับข้อมูลนำเข้าแล้วคืนเวกเตอร์สั้นที่เก็บเฉพาะความหมายสำคัญ ทุกเวกเตอร์ของรายการในบทนี้เป็นผลลัพธ์ของสมการ (4.9)

$$\mathbf{v} = f_\theta(x), \qquad \mathbf{v} \in \mathbb{R}^d \tag{4.9}$$

โดยที่

- x ข้อมูลนำเข้าของรายการหนึ่งชิ้น เช่น เรื่องย่อหรือโปสเตอร์ ในรูปเทนเซอร์ตามตารางที่ 4.8
- f_θ ตัวเข้ารหัส เช่น CNN หรือ Transformer ซึ่งทำงานตามค่าพารามิเตอร์ θ
- θ พารามิเตอร์ทั้งหมด (น้ำหนักและค่าเอนเอียง) ซึ่งได้จากการฝึกตามหัวข้อ 4.2.4
- $\mathbf{v}$ เวกเตอร์ฝังของรายการขนาด d มิติ

ในทางปฏิบัติ ผู้พัฒนาระบบผู้แนะนำแทบไม่ฝึกตัวเข้ารหัสเองตั้งแต่ต้น แต่ใช้ **แบบจำลองที่ฝึกไว้แล้ว (Pretrained Model)** บนข้อมูลขนาดใหญ่ เช่น ResNet ที่ฝึกบน ImageNet หรือ BERT

ที่ฝึกบนข้อความหลายพันล้านคำ แล้วนำส่วนท้ายสุดก่อนชั้นจำแนกมาใช้เป็นเวกเตอร์ของรายการ วิธีนี้เรียกว่า **การถ่ายโอนการเรียนรู้ (Transfer Learning)**

เวกเตอร์จริงยาวหลายร้อยถึงหลายพันมิติ ซึ่งยาวเกินกว่าจะคำนวณด้วยมือ บทนี้จึงใช้**ข้อมูลที่ได้จากการคำนวณจากเทคนิค** CNN และ Transformer ในขนาดย่อสำหรับการสอน ดังนี้

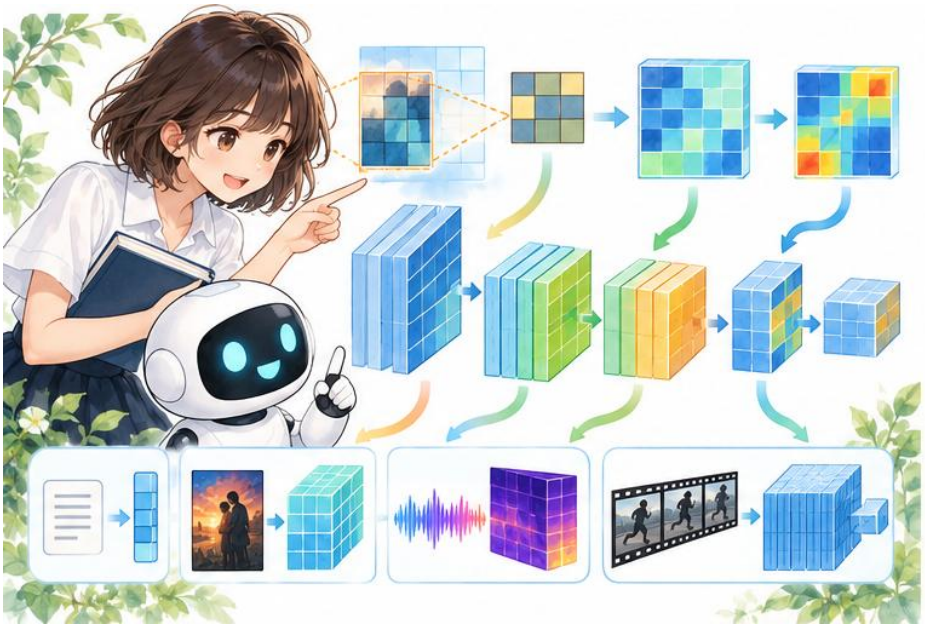
- **ข้อความ** ใช้ 12 มิติ เรียงตามคลังแนวคิดเดียวกับ Static Embedding ของบทที่ 3 หัวข้อ 3.7 คือ technology, mind, space, romance, crime, action, family, music, fantasy, toys, society และ ambition
- **ภาพ เสียง และวิดีโอ** ใช้สื่อละ 3 มิติ เรียกว่า **มิติที่ 1, 2 และ 3** ค่าแต่ละมิติเป็นผลลัพธ์ของตัวเข้ารหัส ไม่ได้ตั้งชื่อตามความหมายใด
- **สองเทคนิค** เวกเตอร์จากเทคนิค CNN อยู่ในหัวข้อ 4.3.7 และจากเทคนิค Transformer อยู่ในหัวข้อ 4.4.8 ทั้งสองชุดใช้รายการชุดเดียวกัน จึงเทียบผลกันได้

ข้อควรระวัง เวกเตอร์ของบทนี้เป็นข้อมูลเพื่อการสอน

- ค่าทุกช่องถูกกำหนดให้มีลักษณะสมเหตุสมผล เช่น แอนิเมชันสามเรื่อง (i1–i3) มีค่ามิติที่ 3 ของวิดีโอเท่ากับ 1 แต่ไม่ได้ผ่านตัวเข้ารหัสจริงของภาพยนตร์เหล่านั้น
- ขั้นตอนการคำนวณทั้งหมดตั้งแต่การเตรียมเวกเตอร์ การสร้างโปรไฟล์ จนถึง Fusion และการประเมินผล เป็นขั้นตอนเดียวกับระบบจริงทุกประการ ต่างกันเพียงความยาวของเวกเตอร์
- ห้ามนำตัวเลขชุดนี้ไปสรุปว่าผู้ชมจริงชอบภาพยนตร์ลักษณะใด

4.3 โครงข่ายประสาทแบบคอนโวลูชัน (CNN)

โครงข่ายประสาทแบบคอนโวลูชัน (Convolutional Neural Network: CNN) เป็นตัวเข้ารหัสหลักของยุคที่ 2 ในหัวข้อ 4.2.2 เริ่มจากงานรู้จำตัวเลขเขียนมือ (LeCun et al., 1998) และเป็นที่รู้จักทั่วไปหลัง AlexNet ชนะการแข่งขัน ImageNet (Krizhevsky et al., 2012) หัวข้อนี้อธิบายแนวคิด ทฤษฎี และขั้นตอนวิธีของ CNN ทีละชิ้น แล้วจึงสรุปการประยุกต์กับข้อความ ภาพ เสียง และวิดีโอ ปิดท้ายด้วยเวกเตอร์รายการจากเทคนิค CNN ที่ใช้สร้างโปรไฟล์ในหัวข้อ 4.5 ดังสรุปในภาพที่ 4.54 แสดงลำดับจากข้อมูลดิบผ่านคอนโวลูชัน การกระตุ้น และการรวมค่าเป็นเวกเตอร์



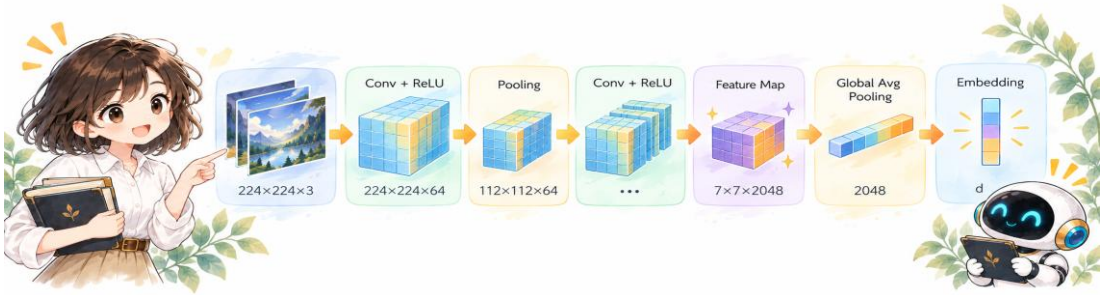
ภาพที่ 4.54 สายงานการสกัดคุณลักษณะด้วย CNN

4.3.1 แนวคิดของ CNN

ลองนึกถึงการให้นิวรอนหนึ่งตัวในหัวข้อ 4.2.3 ดูโปสเตอร์ทั้งใบพร้อมกัน นิวรอนตัวนั้นต้องมีน้ำหนักเท่ากับจำนวนตัวเลขนำเข้า คือ 150,528 ตัวสำหรับโปสเตอร์ 224 × 224 × 3 ถ้าชั้นแรกมีนิวรอน 1,000 ตัว น้ำหนักจะเกิน 150 ล้านตัว ซึ่งทั้งช้าและเรียนรู้ยากเมื่อข้อมูลน้อย CNN แก้ปัญหานี้ด้วยสามแนวคิด

- **มองทีละบริเวณ (Local Connectivity)** นิวรอนแต่ละตัวดูเฉพาะหน้าต่างเล็ก เช่น 3 × 3 พิกเซล หรือ 3 คำติดกัน เพราะลวดลายสำคัญ เช่น ขอบ มุม หรือวลี อยู่ในบริเวณแคบ
- **ใช้น้ำหนักร่วมกัน (Weight Sharing)** ใช้ **ตัวกรอง (Filter หรือ Kernel)** ชุดเดียวเลื่อนไปทั่วข้อมูล ลวดลายเดียวกันจึงถูกตรวจพบได้ทุกตำแหน่ง และจำนวนน้ำหนักไม่ขึ้นกับขนาดภาพ
- **ซ้อนเป็นลำดับชั้น (Hierarchy)** ชั้นต้นเห็นลวดลายง่าย ชั้นถัดไปนำผลของชั้นก่อนมารวมเป็นลวดลายที่ซับซ้อนขึ้นเรื่อย ๆ

ภาพที่ 4.7 แสดงโครงสร้างทั่วไปของ CNN สำหรับภาพ ข้อมูลผ่านชั้นคอนโวลูชันกับ ReLU และ Pooling สลับกันหลายรอบ ขนาดกว้างยาวเล็กลงแต่จำนวนช่องสัญญาณเพิ่มขึ้น จนเหลือแผนที่คุณลักษณะ 7 × 7 × 2,048 แล้วจึงเฉลี่ยทั้งแผนที่เป็นเวกเตอร์เดียว



ภาพที่ 4.7 โครงสร้างทั่วไปของ CNN จากภาพสู่เวกเตอร์ฝัง

ผลของสองแนวคิดแรกต่อจำนวนพารามิเตอร์คำนวณได้จากสมการ (4.10) และ (4.11)

$$P_{FC} = (N_{in} + 1) \times N_{out} \tag{4.10}$$

โดยที่

- N_{in} จำนวนตัวเลขนำเข้า ตามสมการ (4.2)
- N_{out} จำนวนนิวรอนในชั้น
- $+1$ ค่าเอนเอียงของนิวรอนแต่ละตัว
- P_{FC} จำนวนพารามิเตอร์ของชั้นเชื่อมต่อเต็ม (Fully Connected)

$$P_{conv} = (k_h \times k_w \times C_{in} + 1) \times C_{out} \tag{4.11}$$

โดยที่

- k_h, k_w ความสูงและความกว้างของตัวกรอง เช่น 3 × 3
- C_{in} จำนวนช่องสัญญาณนำเข้า เช่น 3 ช่องสี
- C_{out} จำนวนตัวกรอง ซึ่งเท่ากับจำนวนช่องสัญญาณส่งออก
- P_{conv} จำนวนพารามิเตอร์ของชั้นคอนโวลูชัน ซึ่งไม่ขึ้นกับขนาดภาพ

ตัวอย่างการคำนวณ 4.6 เทียบจำนวนพารามิเตอร์ของชั้นเชื่อมต่อเต็มกับชั้นคอนโวลูชัน

โจทย์ ชั้นแรกรับโปสเตอร์ 224 × 224 × 3 และส่งออก 64 ช่อง
จงเทียบจำนวนพารามิเตอร์ระหว่างชั้นเชื่อมต่อเต็ม 64 นิวรอน กับชั้นคอนโวลูชันตัวกรอง 3 × 3 จำนวน 64 ตัว

สมการที่ใช้ สมการ (4.10) และ (4.11)

ข้อมูลที่ใช้ ขนาดภาพจากตารางที่ 4.8 ค่า 3 × 3 และ 64 ตัวกรองตามชั้นแรกของ VGG (Simonyan and Zisserman, 2015)

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- ขนาดภาพ 224 × 224 × 3 มาจากรูปแบบโปสเตอร์ของ MovieMini-12 ในตารางที่ 4.8
- ตัวกรอง 3 × 3 จำนวน 64 ตัวมาจากชั้นแรกของ VGG (Simonyan and Zisserman, 2015) และใช้ 64 นิวรอนในชั้นเชื่อมต่อเต็มเพื่อให้ส่งออก 64 ค่าเท่ากัน

ขั้นที่ 1 ชั้นเชื่อมต่อเต็ม

- $N_{in} = 150{,}528$ และ $N_{out} = 64$
- $P_{FC} = (150{,}528 + 1) \times 64 = 9{,}633{,}856$ ตัว

ขั้นที่ 2 ชั้นคอนโวลูชัน

- ตัวกรองหนึ่งตัวมี $3 \times 3 \times 3 = 27$ น้ำหนัก บวกค่าเอนเอียง 1 เป็น 28
- $P_{conv} = 28 \times 64 = 1{,}792$ ตัว

ขั้นที่ 3 เทียบอัตราส่วน

- $9{,}633{,}856/1{,}792 \approx 5{,}376$ เท่า

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

ชั้นเชื่อมต่อเต็ม: นิวรอนทุกตัวต่อกับทุกพิกเซล

- นิวรอนหนึ่งตัวมีน้ำหนัก 150,528 ตัว (หนึ่งตัวต่อหนึ่งค่านำเข้า) บวกค่าเอนเอียง 1 ตัว = 150,529 พารามิเตอร์
- 64 นิวรอน: 150,529 × 64 = 9,633,856 พารามิเตอร์

ชั้นคอนโวลูชัน: ตัวกรองหนึ่งตัวใช้ซ้ำทุกตำแหน่ง

- ตัวกรองหนึ่งตัวมีน้ำหนัก 3 × 3 × 3 = 27 ตัว (สูง × กว้าง × ช่องสี) บวกค่าเอนเอียง 1 = 28
- 64 ตัวกรอง: 28 × 64 = 1,792 พารามิเตอร์ ไม่ว่าภาพจะมีขนาดเท่าใด

เทียบอัตราส่วน

- 9,633,856 / 1,792 = 5,376 เท่า (หารลงตัวพอดี เพราะ 150,529 / 28 = 5,376.04 และ 64 ตัดกันทั้งเศษและส่วน)

ตีความ

การมองทีละบริเวณและใช้น้ำหนักร่วมกันลดพารามิเตอร์ลงหลายพันเท่า
ทั้งที่ชั้นคอนโวลูชันยังส่งออกแผนที่คุณลักษณะ 64 ช่องเท่าเดิม CNN จึงเรียนรู้จากข้อมูลจำนวนจำกัดได้ดีกว่า

อธิบายแบบเข้าใจง่าย CNN คือการส่องไฟฉายไล่ดูโปสเตอร์

ลองนึกถึงการดูโปสเตอร์ในห้องมืดด้วยไฟฉายดวงเล็ก เราเห็นทีละบริเวณ แต่ถ้าส่องไล่จนทั่ว เราก็รู้ว่ามีขอบ มีสี และมีวัตถุอะไรอยู่ที่ใด

ตัวกรองหนึ่งตัวคือไฟฉายที่ "ไวต่อลวดลายหนึ่งแบบ" เช่น ขอบแนวตั้ง CNN ใช้ไฟฉายหลายสิบดวงพร้อมกัน แล้วส่งสิ่งที่เห็นต่อให้ไฟฉายชั้นถัดไปซึ่งมองบริเวณกว้างขึ้น

4.3.2 คอนโวลูชันหนึ่งมิติ การเติมขอบ และระยะก้าว

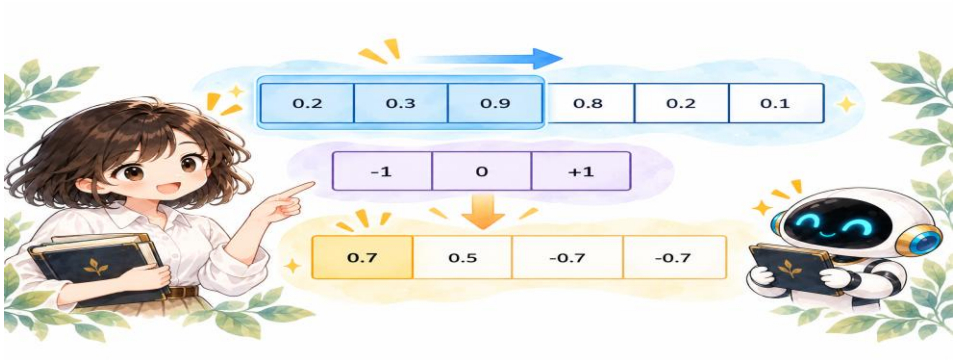
เริ่มจากกรณีง่ายที่สุด คือสัญญาณหนึ่งมิติ เช่น แถวพิกเซลแถวเดียว หรือค่าตามเวลาของเสียง คอนโวลูชันวางตัวกรองยาว k ช่องทาบบนสัญญาณ คูณทีละช่องแล้วบวกกัน จากนั้นเลื่อนไปทีละ s ช่อง ดังสมการ (4.12)

$$y_t = \sum_{j=0}^{k-1} w_j \; x_{t \cdot s+j} + b \tag{4.12}$$

โดยที่

- x สัญญาณนำเข้า (หลังเติมขอบถ้ามี)
- w_j น้ำหนักช่องที่ j ของตัวกรอง
- k ความยาวของตัวกรอง (Kernel Size)
- s ระยะก้าว (Stride) คือจำนวนช่องที่เลื่อนแต่ละครั้ง
- b ค่าเอนเอียงของตัวกรอง
- y_t ค่าส่งออกตำแหน่งที่ t เริ่มนับจาก 0

สมการ (4.12) ไม่กลับด้านตัวกรองก่อนคูณ ในทางคณิตศาสตร์จึงเรียกว่า **สหสัมพันธ์ไขว้ (Cross-correlation)** แต่ทุกไลบรารีเรียกว่าคอนโวลูชัน เพราะน้ำหนักถูกเรียนรู้อยู่แล้ว การกลับด้านจึงไม่เปลี่ยนความสามารถของแบบจำลอง ภาพที่ 4.8 แสดงตัวกรอง $(-1, 0, +1)$ เลื่อนไปบนแถวพิกเซลหกช่อง



ภาพที่ 4.8 การเลื่อนตัวกรองหนึ่งมิติไปบนแถวพิกเซล

ตัวอย่างการคำนวณ 4.7 คอนโวลูชันหนึ่งมิติตรวจจับขอบ

โจทย์ แถวพิกเซล $\mathbf{x} = (0.2,0.3,0.9,0.8,0.2,0.1)$ ตัวกรอง $\mathbf{w} = (-1,0,1)$ ค่าเอนเอียง $b = 0$ ระยะก้าว $s = 1$ ไม่เติมขอบ จงหาผลลัพธ์

สมการที่ใช้ สมการ (4.12)

ข้อมูลที่ใช้ ค่าในภาพที่ 4.8 (ค่าสมมติ)

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- แถวพิกเซล x = (0.2, 0.3, 0.9, 0.8, 0.2, 0.1) เป็นค่าสมมติในภาพที่ 4.8 แทนความสว่างของพิกเซลหกช่องที่เรียงกันบนโปสเตอร์หนึ่งแถว ช่วงกลาง (0.9, 0.8) คือบริเวณสว่าง
- ตัวกรอง (−1, 0, +1) คือ "ค่าทางขวาลบค่าทางซ้าย" ซึ่งเป็นตัวตรวจจับขอบแบบคลาสสิก ใน CNN จริงค่านี้ได้จากการฝึก

ขั้นที่ 1 ทาบตัวกรองที่ตำแหน่ง t = 0

- ทาบบน $(0.2,0.3,0.9)$: $(-1)(0.2) + 0(0.3) + 1(0.9) = 0.7$

ขั้นที่ 2 เลื่อนทีละช่องจนสุดแถว

- $t = 1$ ทาบบน $(0.3,0.9,0.8)$: $-0.3 + 0.8 = 0.5$
- $t = 2$ ทาบบน $(0.9,0.8,0.2)$: $-0.9 + 0.2 = -0.7$
- $t = 3$ ทาบบน $(0.8,0.2,0.1)$: $-0.8 + 0.1 = -0.7$

ขั้นที่ 3 รวมผล

- $\mathbf{y} = (0.7,0.5,-0.7,-0.7)$ ยาว 4 ช่อง สั้นกว่าข้อมูลนำเข้า 2 ช่อง

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

ทาบตัวกรองทุกตำแหน่ง (s = 1, ไม่เติมขอบ)

- ตำแหน่งสุดท้ายที่ทาบได้คือ t = 3 เพราะหน้าต่างยาว 3 ช่องต้องอยู่ในข้อมูล 6 ช่อง (ช่องที่ 3, 4, 5)

ตำแหน่ง	หน้าต่าง	คูณทีละช่องแล้วบวก	y_t
t = 0	x0–x2 = (0.2, 0.3, 0.9)	(−1)(0.2) + 0(0.3) + 1(0.9)	0.7
t = 1	x1–x3 = (0.3, 0.9, 0.8)	(−1)(0.3) + 0(0.9) + 1(0.8)	0.5
t = 2	x2–x4 = (0.9, 0.8, 0.2)	(−1)(0.9) + 0(0.8) + 1(0.2)	−0.7
t = 3	x3–x5 = (0.8, 0.2, 0.1)	(−1)(0.8) + 0(0.2) + 1(0.1)	−0.7

- ค่าบวก (0.7, 0.5) อยู่ตรงที่พิกเซลเปลี่ยนจากมืดไปสว่าง ค่าลบ (−0.7, −0.7) อยู่ตรงที่เปลี่ยนจากสว่างกลับมืด

ตีความ

ตัวกรอง $(-1,0,+1)$ คำนวณ "ค่าด้านขวาลบค่าด้านซ้าย" จึงให้ค่าบวกเมื่อพิกเซลสว่างขึ้น (ขอบจากมืดไปสว่าง) และให้ค่าลบเมื่อพิกเซลมืดลง ตัวกรองนี้จึงเป็น **ตัวตรวจจับขอบ** แบบหนึ่ง ใน CNN จริงค่าของตัวกรองไม่ได้มีคนกำหนด แต่ได้จากการฝึก

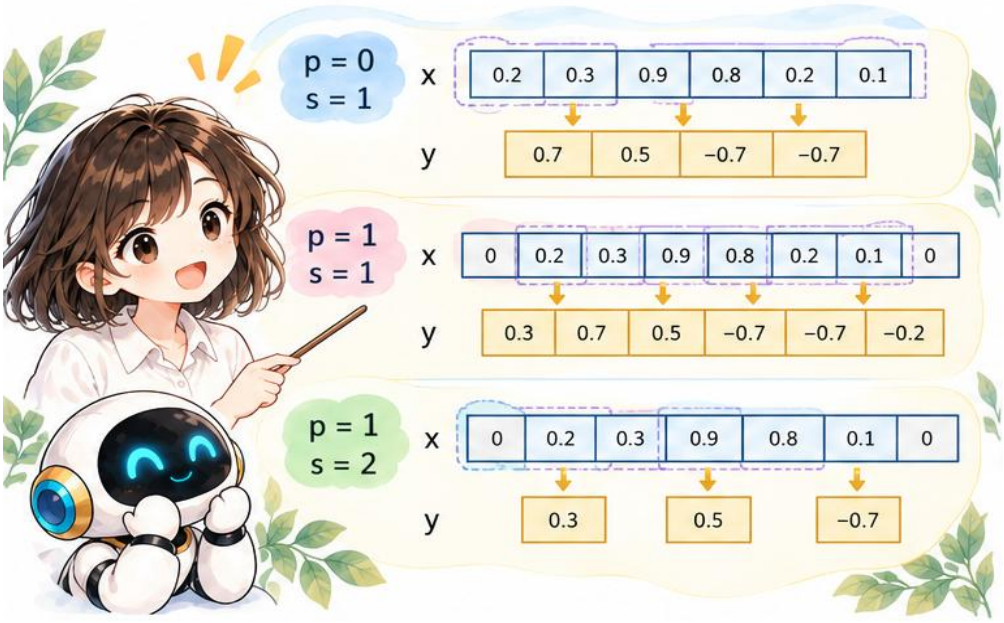
ผลลัพธ์ในตัวอย่างการคำนวณ 4.7 สั้นลงสองช่อง ถ้าต้องการให้ยาวเท่าเดิมต้อง **เติมขอบ (Padding)** ด้วยค่า 0 ข้างละ p ช่อง และถ้าต้องการย่อให้สั้นลงเร็วขึ้นต้องเพิ่ม **ระยะก้าว (Stride)** ความยาวผลลัพธ์ทั่วไปคำนวณได้จากสมการ (4.13)

$$L_{out} = \left\lfloor \frac{L_{in}+2p-k}{s} \right\rfloor + 1 \quad (4.13)$$

โดยที่

- L_{in} ความยาวข้อมูลนำเข้า
- p จำนวนศูนย์ที่เติมแต่ละข้าง (Padding)
- k ความยาวตัวกรอง
- s ระยะก้าว
- $\lfloor \cdot \rfloor$ การปัดเศษลง
- L_{out} ความยาวผลลัพธ์

ภาพที่ 4.9 แสดงสามกรณีบนแถวพิกเซลเดิม ได้แก่ ไม่เติมขอบ เติมขอบให้ยาวเท่าเดิม (Same Padding) และเติมขอบพร้อมก้าวทีละ 2 ช่อง



ภาพที่ 4.9 ผลของการเติมขอบ p และระยะก้าว s ต่อความยาวผลลัพธ์

ตัวอย่างการคำนวณ 4.8 ความยาวผลลัพธ์เมื่อเปลี่ยนการเติมขอบและระยะก้าว

โจทย์ แถวพิกเซลยาว $L_{in} = 6$ ตัวกรอง $k = 3$ จงหาความยาวผลลัพธ์ทั้งสามกรณีในภาพที่ 4.9

สมการที่ใช้ สมการ (4.13)

ข้อมูลที่ใช้ ค่าจากตัวอย่างการคำนวณ 4.7

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- ใช้แถวพิกเซลยาว 6 ช่องและตัวกรองยาว 3 ช่องชุดเดียวกับตัวอย่างการคำนวณ 4.7 ค่า p และ s ทั้งสามชุดตรงกับสามแถวในภาพที่ 4.9

ขั้นที่ 1 ไม่เติมขอบ p = 0, s = 1

- $\lfloor (6 + 0 - 3)/1 \rfloor + 1 = 4$

ขั้นที่ 2 Same Padding p = 1, s = 1

- $\lfloor (6 + 2 - 3)/1 \rfloor + 1 = 6$ เท่าความยาวเดิม

ขั้นที่ 3 p = 1, s = 2

- $\lfloor (6 + 2 - 3)/2 \rfloor + 1 = \lfloor 2.5 \rfloor + 1 = 3$ สั้นลงครึ่งหนึ่ง

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

แทนค่าในสมการ (4.13) และนับตำแหน่งที่ทาบได้จริง

กรณี	ความยาวหลังเติมขอบ	แทนค่า	Lout	ตำแหน่งเริ่มของหน้าต่าง
p = 0, s = 1	6	$\lfloor (6 + 0 - 3)/1 \rfloor + 1 = 3 + 1$	4	ช่อง 0, 1, 2, 3
p = 1, s = 1	8	$\lfloor (6 + 2 - 3)/1 \rfloor + 1 = 5 + 1$	6	ช่อง 0, 1, 2, 3, 4, 5
p = 1, s = 2	8	$\lfloor (6 + 2 - 3)/2 \rfloor + 1 = \lfloor 2.5 \rfloor + 1$	3	ช่อง 0, 2, 4

- กรณี p = 1, s = 1 แถวหลังเติมขอบคือ (0, 0.2, 0.3, 0.9, 0.8, 0.2, 0.1, 0) ผลที่ได้คือ (0.3, 0.7, 0.5, −0.7, −0.7, −0.2) ตรงกับผลของโปรแกรม 4.5
- กรณี s = 2 หน้าต่างที่เริ่มช่อง 6 จะต้องใช้ช่อง 6, 7, 8 แต่แถวมีเพียงช่อง 0–7 จึงทาบไม่ได้ การปัดเศษลง $\lfloor 2.5 \rfloor = 2$ คือการตัดหน้าต่างที่ไม่ครบนี้ทิ้ง

ตีความ

การเติมขอบช่วยให้ขอบภาพถูกมองครบเท่าบริเวณกลางภาพ ส่วนระยะก้าว 2 เป็นวิธีย่อขนาดที่ใช้แทน Pooling ได้ ทั้งสองค่าเป็นไฮเปอร์พารามิเตอร์ที่ผู้ออกแบบกำหนด

ที่มาของไฮเปอร์พารามิเตอร์ ขนาดตัวกรอง การเติมขอบ และระยะก้าวมาจากไหน

- **ขนาดตัวกรอง k** ค่า 3 ได้รับความนิยมตั้งแต่ VGG (Simonyan and Zisserman, 2015) ซึ่งแสดงว่าการซ้อนตัวกรอง 3 × 3 หลายชั้นดีกว่าตัวกรองใหญ่ชั้นเดียว (ดูหัวข้อ 4.3.5) ชั้นแรกของ ResNet ใช้ 7 × 7 เพื่อย่อภาพเร็ว ส่วน Text CNN นิยม 3, 4, 5 คำ (Kim, 2014)
- **การเติมขอบ p** นิยม Same Padding คือ $p = (k-1)/2$ เช่น $p = 1$ เมื่อ $k = 3$ เพื่อให้ขนาดไม่หดทุกชั้น
- **ระยะก้าว s** ใช้ 1 ในชั้นทั่วไป และใช้ 2 เมื่อต้องการย่อขนาดลงครึ่งหนึ่ง
- **วิธีเลือก** ใช้ค่าตามสถาปัตยกรรมมาตรฐานก่อน แล้วจึงปรับเมื่อข้อมูลมีลักษณะต่างจากเดิมมาก เช่น สเปกโทรแกรมที่ยาวมากตามแกนเวลา

โปรแกรม 4.5 คอนโวลูชันหนึ่งมิติ การเติมขอบ และระยะก้าว

```
import numpy as np

def conv1d(x, w, b=0.0, p=0, s=1):
    x = np.pad(x, p)                    # เติมศูนย์ข้างละ p ช่อง
    k = len(w)
    n = (len(x) - k) // s + 1           # สมการความยาวผลลัพธ์
    return np.array([w @ x[t * s:t * s + k] + b
                     for t in range(n)])

x = np.array([0.2, 0.3, 0.9, 0.8, 0.2, 0.1])
w = np.array([-1.0, 0.0, 1.0])
for p, s in [(0, 1), (1, 1), (1, 2)]:
    print('p=%d s=%d ->' % (p, s), np.round(conv1d(x, w, 0, p, s), 2))
```

ผลลัพธ์

```
p=0 s=1 -> [ 0.7  0.5 -0.7 -0.7]
p=1 s=1 -> [ 0.3  0.7  0.5 -0.7 -0.7 -0.2]
p=1 s=2 -> [ 0.3  0.5 -0.7]
```

4.3.3 ReLU และ Pooling

หลังคอนโวลูชัน CNN มีอีกสองขั้นที่ทำซ้ำทุกชั้น ขั้นแรกคือ **ReLU** ตามสมการ (4.5) ซึ่งตัดค่าลบทิ้ง ทำให้แผนที่คุณลักษณะเหลือเฉพาะตำแหน่งที่ "พบลวดลาย" ขั้นที่สองคือ **Pooling** ซึ่งสรุปค่าในหน้าต่างเล็กให้เหลือค่าเดียว เพื่อย่อขนาดและทำให้ผลไม่อ่อนไหวต่อการเลื่อนตำแหน่งเล็กน้อย

$$\mathrm{MaxPool}(\mathbf{y})_t = \max_{0 \le j < q} y_{t \cdot s + j}, \qquad \mathrm{AvgPool}(\mathbf{y})_t = \frac{1}{q}\sum_{j=0}^{q-1} y_{t \cdot s + j} \tag{4.14}$$

โดยที่

- q ขนาดหน้าต่างของ Pooling
- s ระยะก้าวของ Pooling มักเท่ากับ q
- $\mathrm{MaxPool}$ เลือกค่ามากที่สุดในหน้าต่าง เก็บสัญญาณที่เด่นที่สุด
- $\mathrm{AvgPool}$ หาค่าเฉลี่ยในหน้าต่าง เก็บภาพรวมของบริเวณ

$$\mathrm{GAP}(\mathbf{Y})_c = \frac{1}{H\,W}\sum_{r=1}^{H}\sum_{u=1}^{W} Y_{r,u,c} \tag{4.15}$$

โดยที่

- $\mathbf{Y}$ แผนที่คุณลักษณะขนาด $H \times W \times C$

- $\mathrm{GAP}(\mathbf{Y})_c$ ค่าเฉลี่ยของช่องสัญญาณที่ c ทั้งแผนที่ (Global Average Pooling) ผลรวมทุกช่องได้เวกเตอร์ยาว C

ตัวอย่างการคำนวณ 4.9 ReLU และ Pooling บนผลของคอนโวลูชันหนึ่งมิติ

โจทย์ นำผลลัพธ์ $\mathbf{y} = (0.7, 0.5, -0.7, -0.7)$ จากตัวอย่างการคำนวณ 4.7 ผ่าน ReLU แล้วทำ Max Pooling และ Average Pooling ขนาด $q = 2$ ระยะก้าว $s = 2$ และหา Global Average Pooling

สมการที่ใช้ สมการ (4.5), (4.14) และ (4.15)

ข้อมูลที่ใช้ ผลลัพธ์ของตัวอย่างการคำนวณ 4.7

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- ค่า y = (0.7, 0.5, −0.7, −0.7) คือผลลัพธ์ของตัวอย่างการคำนวณ 4.7 ซึ่งมาจากแถวพิกเซลของโปสเตอร์ในภาพที่ 4.8 จึงเป็นข้อมูลต่อเนื่องกัน ไม่ใช่ค่าใหม่
- ขนาดหน้าต่าง q = 2 และระยะก้าว s = 2 ตามค่ามาตรฐานของ AlexNet และ VGG

ขั้นที่ 1 ReLU

- $\max(0, 0.7) = 0.7$, $\max(0, 0.5) = 0.5$, $\max(0, -0.7) = 0$ และ $\max(0, -0.7) = 0$
- จึงได้ $\mathbf{a} = (0.7, 0.5, 0, 0)$

ขั้นที่ 2 แบ่งหน้าต่างละสองค่า

- หน้าต่างที่ 1 คือ $(0.7, 0.5)$ และหน้าต่างที่ 2 คือ $(0, 0)$

ขั้นที่ 3 Max และ Average Pooling

- Max Pooling: $\big(\max(0.7, 0.5), \max(0, 0)\big) = (0.7, 0)$
- Average Pooling: $\big((0.7 + 0.5)/2, (0 + 0)/2\big) = (0.6, 0)$

ขั้นที่ 4 Global Average Pooling

- $(0.7 + 0.5 + 0 + 0)/4 = 0.3$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

ขั้น	ตำแหน่ง 0	ตำแหน่ง 1	ตำแหน่ง 2	ตำแหน่ง 3
y (จากตัวอย่าง 4.7)	0.7	0.5	−0.7	−0.7
ReLU = max(0, y)	0.7	0.5	0	0
หน้าต่าง Pooling	หน้าต่างที่ 1	(ต่อ)	หน้าต่างที่ 2	(ต่อ)

- Max Pooling: หน้าต่างที่ 1 = max(0.7, 0.5) = 0.7 และหน้าต่างที่ 2 = max(0, 0) = 0 ได้ (0.7, 0)
- Average Pooling: (0.7 + 0.5)/2 = 0.6 และ (0 + 0)/2 = 0 ได้ (0.6, 0)
- Global Average Pooling: รวมทั้งแถว (0.7 + 0.5 + 0 + 0) = 1.2 หารด้วย 4 ตำแหน่ง = 0.3
- ความยาวผลลัพธ์ของ Pooling ใช้สมการ (4.13) เช่นกัน: $\lfloor (4 - 2)/2 \rfloor + 1 = 2$

ตีความ

Max Pooling บอกว่า "ครึ่งแรกของแถวมีขอบมืดไปสว่างที่ชัดที่สุดเท่ากับ 0.7" โดยไม่สนว่าอยู่ช่องใด ส่วน Global Average Pooling บอกว่า "ทั้งแถวมีขอบแบบนี้โดยเฉลี่ยเท่าใด" ซึ่งเป็นวิธีที่ CNN สมัยใหม่ใช้เปลี่ยนแผนที่คุณลักษณะเป็นเวกเตอร์ (Lin et al., 2014)

ที่มาของไฮเปอร์พารามิเตอร์ ขนาดหน้าต่างของ Pooling มาจากไหน

- **ที่มา** Max Pooling ขนาด 2 × 2 ระยะก้าว 2 ใช้ใน AlexNet และ VGG เพราะย่อขนาดลงครึ่งหนึ่งพอดีโดยไม่ทิ้งข้อมูลมากเกินไป
- **ผลเมื่อเปลี่ยน** หน้าต่างใหญ่ย่อเร็วแต่ทิ้งตำแหน่งมาก หน้าต่างเล็กเก็บรายละเอียดแต่แผนที่ยังใหญ่
- **วิธีเลือก** ใช้ 2 × 2 ระหว่างชั้น และใช้ Global Average Pooling เพียงครั้งเดียวที่ปลายโครงข่ายเพื่อได้เวกเตอร์ยาวคงที่ไม่ว่าภาพจะมีขนาดเท่าใด

4.3.4 คอนโวลูชันสองมิติและหลายช่องสัญญาณ

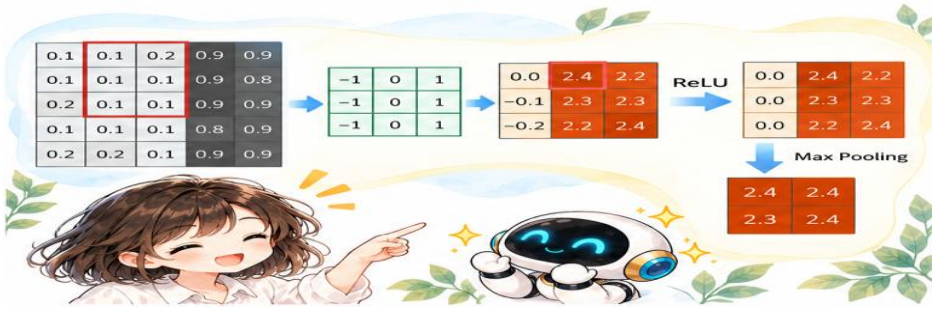
ภาพมีสองมิติ คือแถวและคอลัมน์ ตัวกรองจึงเป็นตารางเล็ก เช่น 3 × 3 และเลื่อนทั้งแนวนอนและแนวตั้ง ถ้าภาพมีหลายช่องสัญญาณ เช่น R, G, B ตัวกรองหนึ่งตัวจะมีความลึกเท่าจำนวนช่อง แล้วรวมผลของทุกช่องเป็นค่าเดียว ดังสมการ (4.16)

$$Y_{r,u} = \sum_{c=1}^{C_{in}} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} K_{i,j,c}\ X_{r+i,u+j,c} + b \qquad (4.16)$$

โดยที่

- $\mathbf{X}$ ภาพหรือแผนที่คุณลักษณะนำเข้าขนาด $H \times W \times C_{in}$
- $\mathbf{K}$ ตัวกรองขนาด $k \times k \times C_{in}$
- $Y_{r,u}$ ค่าส่งออกที่แถว r คอลัมน์ u ของตัวกรองหนึ่งตัว
- C_{in} จำนวนช่องสัญญาณนำเข้า

ภาพที่ 4.10 แสดงทุกขั้นบนภาพขาวดำขนาด 5 × 5 ซึ่งครึ่งซ้ายมืดและครึ่งขวาสว่าง ตัวกรอง 3 × 3 เป็นตัวตรวจจับขอบแนวตั้งแบบเดียวกับตัวอย่างการคำนวณ 4.7 แต่ซ้อนสามแถว



ภาพที่ 4.10 คอนโวลูชันสองมิติ ReLU และ Max Pooling บนภาพ 5 × 5

ตัวอย่างการคำนวณ 4.10 คอนโวลูชันสองมิติบนภาพ 5 × 5

โจทย์ จงคำนวณค่าส่งออกตำแหน่งแรก $(r,u) = (0,0)$ และตำแหน่ง $(0,1)$ ของคอนโวลูชันในภาพที่ 4.10 แล้วหาขนาดผลลัพธ์ทั้งหมด

สมการที่ใช้ สมการ (4.16) และ (4.13)

ข้อมูลที่ใช้ ภาพ 5 × 5 และตัวกรองในภาพที่ 4.10 ตัวกรองทุกแถวเป็น $(-1,0,1)$ ค่าเอนเอียง 0

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- ภาพขาวดำ 5 × 5 ในภาพที่ 4.10 เป็นค่าสมมติแทนมุมหนึ่งของโปสเตอร์ที่ครึ่งซ้ายมืด (ค่า 0.1–0.2) และครึ่งขวาสว่าง (ค่า 0.8–0.9) เช่น ขอบระหว่างฉากหลังกับตัวละคร
- ตัวกรองทุกแถวเป็น (−1, 0, +1) คือตัวตรวจจับขอบของตัวอย่างการคำนวณ 4.7 ที่ซ้อนกันสามแถว

ขั้นที่ 1 ทาบตัวกรองที่มุมซ้ายบน (0, 0)

- หน้าต่าง 3 × 3 คือแถว (0.1, 0.1, 0.2), (0.1, 0.1, 0.1), (0.2, 0.1, 0.1)
- แต่ละแถวให้ "ขวาลบซ้าย": $0.2 - 0.1 = 0.1$, $0.1 - 0.1 = 0$, $0.1 - 0.2 = -0.1$
- รวมสามแถว: $0.1 + 0 - 0.1 = 0.0$

ขั้นที่ 2 เลื่อนไปขวาหนึ่งช่อง (0, 1)

- หน้าต่างคือแถว (0.1, 0.2, 0.9), (0.1, 0.1, 0.9), (0.1, 0.1, 0.9)
- ขวาลบซ้าย: $0.8 + 0.8 + 0.8 = 2.4$ ตรงกับขอบมืดไปสว่างพอดี

ขั้นที่ 3 ขนาดผลลัพธ์

- แต่ละแกน $\lfloor (5 + 0 - 3)/1 \rfloor + 1 = 3$ จึงได้ 3 × 3 ดังภาพ
- ผลครบคือแถว (0.0, 2.4, 2.2), (−0.1, 2.3, 2.3), (−0.2, 2.2, 2.4)

ขั้นที่ 4 ReLU และ Max Pooling 2 × 2 ระยะก้าว 1

- ReLU เปลี่ยนค่าลบ −0.1 และ −0.2 เป็น 0
- Max Pooling ได้ 2 × 2 คือ (2.4, 2.4) และ (2.3, 2.4)

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

คำนวณครบทั้ง 9 ตำแหน่ง (แต่ละแถวของหน้าต่างให้ "คอลัมน์ขวา − คอลัมน์ซ้าย")

ตำแหน่ง (r, u)	สามแถวของหน้าต่าง	Y(r, u)
(0, 0)	(0.2 − 0.1) + (0.1 − 0.1) + (0.1 − 0.2)	0.0
(0, 1)	(0.9 − 0.1) + (0.9 − 0.1) + (0.9 − 0.1)	2.4
(0, 2)	(0.9 − 0.2) + (0.8 − 0.1) + (0.9 − 0.1)	2.2
(1, 0)	(0.1 − 0.1) + (0.1 − 0.2) + (0.1 − 0.1)	−0.1
(1, 1)	(0.9 − 0.1) + (0.9 − 0.1) + (0.8 − 0.1)	2.3
(1, 2)	(0.8 − 0.1) + (0.9 − 0.1) + (0.9 − 0.1)	2.3
(2, 0)	(0.1 − 0.2) + (0.1 − 0.1) + (0.1 − 0.2)	−0.2
(2, 1)	(0.9 − 0.1) + (0.8 − 0.1) + (0.9 − 0.2)	2.2
(2, 2)	(0.9 − 0.1) + (0.9 − 0.1) + (0.9 − 0.1)	2.4

- จัดเป็นตาราง 3 × 3: (0.0, 2.4, 2.2), (−0.1, 2.3, 2.3), (−0.2, 2.2, 2.4)
- ReLU เปลี่ยน −0.1, −0.2 เป็น 0 แล้ว Max Pooling 2 × 2 ระยะก้าว 1 มีหน้าต่าง 4 ตำแหน่ง: บนซ้าย max(0, 2.4, 0, 2.3) = 2.4, บนขวา max(2.4, 2.2, 2.3, 2.3) = 2.4, ล่างซ้าย max(0, 2.3, 0, 2.2) = 2.3, ล่างขวา max(2.3, 2.3, 2.2, 2.4) = 2.4

ตีความ

ค่าสูงทั้งหมดอยู่ในคอลัมน์ที่ 2–3 ซึ่งตรงกับเส้นแบ่งมืดสว่างของภาพ แผนที่คุณลักษณะจึงบอกทั้ง "พบอะไร" (ขอบแนวตั้ง) และ "พบที่ใด" (กลางภาพค่อนขวา)

ชั้นจริงมีตัวกรองหลายสิบถึงหลายร้อยตัว แต่ละตัวให้แผนที่หนึ่งช่อง ผลรวมของทุกตัวจึงเป็นเทนเซอร์ $H_{out} \times W_{out} \times C_{out}$ และจำนวนพารามิเตอร์ของชั้นคำนวณด้วยสมการ (4.11) ในหัวข้อ 4.3.1

โปรแกรม 4.6 คอนโวลูชันสองมิติ ReLU และ Max Pooling

```
import numpy as np

X = np.array([[.1, .1, .2, .9, .9], [.1, .1, .1, .9, .8],
              [.2, .1, .1, .9, .9], [.1, .1, .1, .8, .9],
              [.2, .2, .1, .9, .9]])
K = np.array([[-1, 0, 1]] * 3)            # ตัวกรองขอบแนวตั้ง
n = X.shape[0] - K.shape[0] + 1            # 5 - 3 + 1 = 3
Y = np.array([[np.sum(X[r:r + 3, u:u + 3] * K)
               for u in range(n)] for r in range(n)])
A = np.maximum(Y, 0)                       # ReLU
P = np.array([[A[r:r + 2, u:u + 2].max() for u in range(2)]
              for r in range(2)])          # Max Pooling 2x2
print(np.round(Y, 1))
print(P)
```

ผลลัพธ์

```
[[ 0.   2.4  2.2]
 [-0.1  2.3  2.3]
 [-0.2  2.2  2.4]]
[[2.4 2.4]
 [2.3 2.4]]
```

4.3.5 ซ้อนหลายชั้น: ระยะมองเห็นและสถาปัตยกรรม CNN

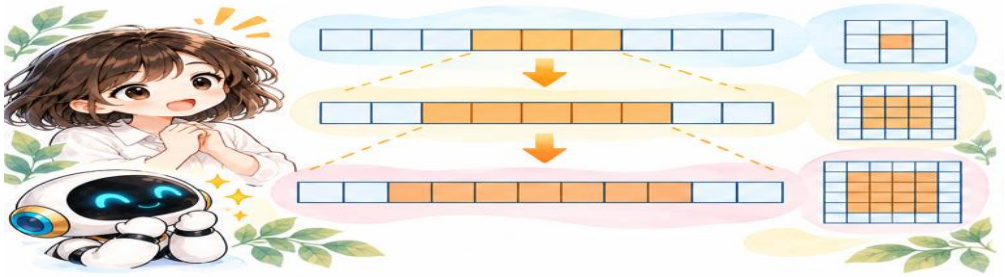
ตัวกรอง 3 × 3 หนึ่งชั้นเห็นภาพเพียง 3 × 3 พิกเซล แต่เมื่อซ้อนชั้นที่สองบนผลของชั้นแรก นิวรอนในชั้นที่สองจะเห็นบริเวณที่กว้างขึ้น บริเวณของภาพนำเข้าที่นิวรอนหนึ่งตัวได้รับผลเรียกว่า **ระยะมองเห็น (Receptive Field)** เมื่อทุกชั้นใช้ระยะก้าว 1 คำนวณได้จากสมการ (4.17)

$$r_L = 1 + L\left(k - 1\right) \tag{4.17}$$

โดยที่

- L จำนวนชั้นคอนโวลูชันที่ซ้อนกัน
- k ขนาดตัวกรองของทุกชั้น
- r_L ความกว้างของระยะมองเห็นหลังชั้นที่ L (หน่วยพิกเซล)

ภาพที่ 4.11 แสดงระยะมองเห็นตามแนวเดียวเมื่อซ้อนตัวกรอง 3 × 3 สามชั้น ระยะมองเห็นขยายจาก 3 เป็น 5 และ 7 พิกเซล



ภาพที่ 4.11 ระยะมองเห็นที่ขยายขึ้นเมื่อซ้อนตัวกรอง 3 × 3 สามชั้น

ตัวอย่างการคำนวณ 4.11 สามชั้นของ 3 × 3 เทียบกับหนึ่งชั้นของ 7 × 7

โจทย์ จงหาระยะมองเห็นของตัวกรอง 3 × 3 สามชั้น แล้วเทียบจำนวนน้ำหนักกับตัวกรอง 7 × 7 ชั้นเดียว เมื่อทุกชั้นมี C ช่องสัญญาณทั้งเข้าและออก (ไม่นับค่าเอนเอียง)

สมการที่ใช้ สมการ (4.17) และ (4.11)

ข้อมูลที่ใช้ แนวคิดจาก Simonyan and Zisserman (2015)

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- แนวคิดและตัวเลขมาจาก VGG (Simonyan and Zisserman, 2015) ค่า C = 64 เท่ากับจำนวนช่องสัญญาณของชั้นแรกในตัวอย่างการคำนวณ 4.6

ขั้นที่ 1 ระยะมองเห็น

- $r_3 = 1 + 3(3-1) = 7$ เท่ากับตัวกรอง 7 × 7 หนึ่งชั้น

ขั้นที่ 2 จำนวนน้ำหนัก

- สามชั้นของ 3 × 3: $3 \times (3 \times 3 \times C \times C) = 27C^2$
- หนึ่งชั้นของ 7 × 7: $7 \times 7 \times C \times C = 49C^2$
- เมื่อ $C = 64$ ได้ 110,592 เทียบกับ 200,704 ตัว

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

ระยะมองเห็นทีละชั้น (k = 3)

หลังชั้นที่	r = 1 + L(k − 1)	ระยะมองเห็น
1	1 + 1 × 2	3 พิกเซล
2	1 + 2 × 2	5 พิกเซล
3	1 + 3 × 2	7 พิกเซล

จำนวนน้ำหนักเมื่อ C = 64 (C^2 = 4,096)

- สามชั้นของ 3 × 3: ชั้นละ 3 × 3 × 64 × 64 = 36,864 รวมสามชั้น 3 × 36,864 = 110,592
- หนึ่งชั้นของ 7 × 7: 7 × 7 × 64 × 64 = 49 × 4,096 = 200,704
- อัตราส่วน 110,592 / 200,704 = 0.551 หรือใช้น้ำหนักเพียง 55.1%

ตีความ

สามชั้นเล็กมองเห็นกว้างเท่าหนึ่งชั้นใหญ่ แต่ใช้น้ำหนักน้อยกว่าเกือบครึ่ง และมี ReLU คั่นสามครั้งจึงแทนลวดลายที่ซับซ้อนกว่า นี่คือเหตุผลที่ CNN สมัยใหม่นิยมตัวกรองขนาด 3

ระยะมองเห็นที่ขยายช้า ๆ คือ **ข้อจำกัดสำคัญของ CNN** ถ้าความหมายของข้อมูลต้องอาศัยส่วนที่อยู่ไกลกัน เช่น คำต้นประโยคกับท้ายประโยค CNN ต้องซ้อนชั้นจำนวนมากกว่าจะเห็นทั้งสองส่วนพร้อมกัน ข้อจำกัดนี้นำไปสู่ Transformer ในหัวข้อ 4.4

สถาปัตยกรรมที่สำคัญของ CNN มีพัฒนาการดังตารางที่ 4.10 จุดเปลี่ยนที่ควรจำคือ **ทางลัด (Residual Connection)** ของ ResNet ซึ่งบวกข้อมูลนำเข้าของบล็อกกลับเข้าไปในผลลัพธ์ $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$ ทำให้ฝึกโครงข่ายลึกนับร้อยชั้นได้ และเป็นชิ้นส่วนเดียวกับที่ Transformer นำไปใช้ในหัวข้อ 4.4.5

ตารางที่ 4.10 สถาปัตยกรรม CNN ที่สำคัญ

สถาปัตยกรรม	ปี	แนวคิดเด่น	ขนาดเวกเตอร์ท้าย
LeNet-5	1998	คอนโวลูชันสลับ Pooling สำหรับตัวเลขเขียนมือ	84
AlexNet	2012	ReLU, Dropout, ฝึกบน GPU ชนะ ImageNet	4,096
VGG-16	2015	ซ้อนตัวกรอง 3×3 อย่างเดียว ลึก 16 ชั้น	4,096
ResNet-50	2016	ทางลัด $F(\mathbf{x}) + \mathbf{x}$ ลึก 50 ชั้น	2,048
EfficientNet-B0	2019	ปรับความลึก ความกว้าง ความละเอียดพร้อมกัน	1,280

ที่มา: LeCun et al. (1998), Krizhevsky et al. (2012), Simonyan and Zisserman (2015), He et al. (2016), Tan and Le (2019)

ขั้นตอนวิธี 4.1 สกัดเวกเตอร์ของรายการด้วย CNN ที่ฝึกไว้แล้ว

1. เลือกแบบจำลองที่ฝึกไว้แล้วให้ตรงกับสื่อ เช่น ResNet-50 สำหรับภาพ (ตารางที่ 4.11)
2. เตรียมข้อมูลนำเข้าให้ตรงกับตอนฝึก เช่น ย่อภาพเป็น 224×224 แล้วปรับค่าพิกเซลด้วยค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานของ ImageNet
3. ส่งข้อมูลผ่านชั้นคอนโวลูชันทั้งหมด แต่ **ตัดชั้นจำแนกสุดท้ายทิ้ง**
4. ทำ Global Average Pooling ตามสมการ (4.15) ได้เวกเตอร์ยาวคงที่ เช่น 2,048 มิติ
5. บันทึกเวกเตอร์ของทุกรายการไว้ล่วงหน้า (งานออฟไลน์ในหัวข้อ 4.8.1) แล้วจึงนำไปสร้างโปรไฟล์และให้คะแนน

4.3.6 การประยุกต์ CNN กับข้อความ ภาพ เสียง และวิดีโอ

แนวคิดของหัวข้อ 4.3.1–4.3.5 ใช้ได้กับทุกสื่อ สิ่งที่ต่างกันมีเพียงสองข้อ คือ **ต้องจัดข้อมูลให้เป็นเทนเซอร์รูปร่างใด** และ **ตัวกรองเลื่อนไปตามแกนใด** ตารางที่ 4.11 สรุปเทคนิค CNN ที่แนะนำสำหรับแต่ละสื่อพร้อมไลบรารี Python

ตารางที่ 4.11 สรุปการประยุกต์ CNN กับสี่สื่อและไลบรารี Python

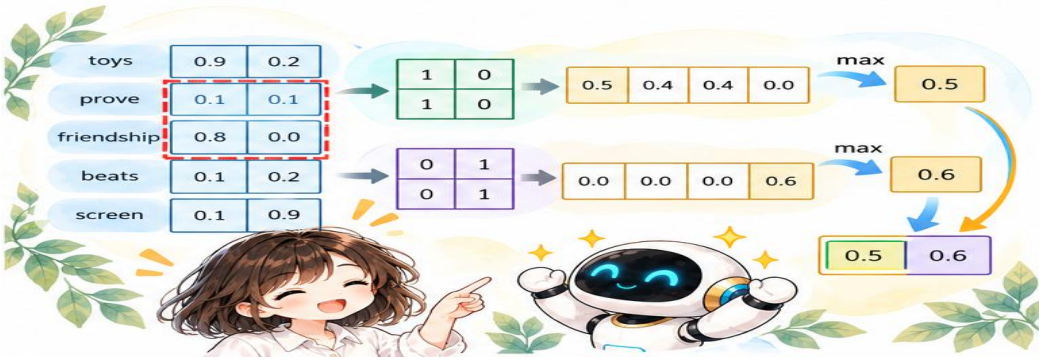
สื่อ	จัดข้อมูลเป็น	คอนโวลูชัน	แบบจำลองที่นิยม	ไลบรารี Python
ข้อความ	ลำดับเวกเตอร์คำ $L \times d$	1 มิติ ตามแกนคำ	Text CNN (Kim, 2014)	torch.nn.Conv1d, keras.layers.Conv1D, gensim
ภาพ	พิกเซล $H \times W \times 3$	2 มิติ	ResNet, EfficientNet	torchvision.models, timm, keras.applications
เสียง	Log-Mel Spectrogram $F \times T$	2 มิติ บนความถี่ × เวลา	VGGish, PANNs CNN14	librosa, torchaudio, panns_inference
วิดีโอ	ชุดเฟรม $T \times H \times W \times 3$	2 มิติรายเฟรม หรือ 3 มิติ	ResNet รายเฟรม, C3D, I3D, R(2+1)D, SlowFast	torchvision.models.video, pytorchvideo, decord

ที่มา: Kim (2014), He et al. (2016), Hershey et al. (2017), Kong et al. (2020), Tran et al. (2015), Carreira and Zisserman (2017), Feichtenhofer et al. (2019)

รายละเอียดโดยสังเขปของแต่ละสื่อมีดังนี้

- **ข้อความ** เรียงเวกเตอร์คำ (เช่น Static Embedding ของบทที่ 3) เป็นตาราง $L \times d$ แล้วเลื่อนตัวกรองกว้าง h คำตามแกนคำ ตัวกรองแต่ละตัวตรวจจับ **วลี** หนึ่งแบบ จากนั้นทำ Max Pooling ตลอดประโยค (Max-over-time Pooling) จึงได้หนึ่งค่าต่อตัวกรอง
- **ภาพ** ใช้คอนโวลูชันสองมิติตามหัวข้อ 4.3.4 ซ้อนหลายสิบชั้น แล้วทำ Global Average Pooling นิยมใช้ ResNet หรือ EfficientNet ที่ฝึกบน ImageNet
- **เสียง** แปลงคลื่นเสียงเป็น **Log-Mel Spectrogram** ซึ่งเป็นตารางความถี่ × เวลา แล้วปฏิบัติต่อตารางนี้เหมือนภาพขาวดำหนึ่งช่อง CNN จึงตรวจจับลวดลาย เช่น เสียงกลองที่เป็นเส้นแนวตั้ง หรือเสียงร้องที่เป็นแถบแนวนอน
- **วิดีโอ** มีสองแนวทาง แนวทางแรกส่งแต่ละเฟรมผ่าน CNN ภาพแล้วเฉลี่ยเวกเตอร์ตามเวลา ซึ่งง่ายแต่ไม่เห็นการเคลื่อนไหว แนวทางที่สองใช้ **3D CNN** ซึ่งตัวกรองมีแกนเวลาด้วย เช่น $3 \times 3 \times 3$ จึงตรวจจับการเคลื่อนไหวได้โดยตรง

ภาพที่ 4.12 แสดง Text CNN ขนาดจิ๋วบนประโยคห้าคำ เวกเตอร์คำมีสองมิติ และมีตัวกรองกว้างสองคำสองตัว ตัวกรองแรก $\mathbf{F}_1$ ไวต่อมิติแรกของคำ ตัวกรองที่สอง $\mathbf{F}_2$ ไวต่อมิติที่สอง



ภาพที่ 4.12 Text CNN สองตัวกรองกับ Max-over-time Pooling

ตัวอย่างการคำนวณ 4.12 Text CNN ขนาดจิ๋ว

โจทย์ ประโยค "toys prove friendship beats screen" มีเวกเตอร์คำสองมิติตามภาพที่ 4.12 ตัวกรอง $\mathbf{F}_1$ มีน้ำหนักคอลัมน์ซ้ายเป็น 1 ตัวกรอง $\mathbf{F}_2$ มีน้ำหนักคอลัมน์ขวาเป็น 1 ทั้งสองมีค่าเอนเอียง $b = -0.5$ จงหาเวกเตอร์ของประโยค

สมการที่ใช้ สมการ (4.12), (4.5) และ (4.14)

ข้อมูลที่ใช้ เวกเตอร์คำและตัวกรองในภาพที่ 4.12 (ค่าสมมติ)

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- ประโยค "toys prove friendship beats screen" สรุปใจความของเรื่องย่อ i1 Toy Story 5 ในตารางที่ 4.6 ซึ่งมีคำ toys (3 ครั้ง), prove, friendship และ screen ส่วน beats เป็นคำที่ใช้สรุปความว่ามิตรภาพชนะหน้าจอ
- เวกเตอร์คำสองมิติในภาพที่ 4.12 เป็นค่าสมมติ มิติที่ 1 แทนความเป็น "ของเล่นและมิตรภาพ" มิติที่ 2 แทนความเป็น "เทคโนโลยีและหน้าจอ" toys และ friendship จึงมีมิติที่ 1 สูง ส่วน screen มีมิติที่ 2 สูง

ขั้นที่ 1 ตัวกรอง F1 บนคู่คำติดกันสี่คู่

- toys–prove: $0.9 + 0.1 - 0.5 = 0.5$
- prove–friendship: $0.1 + 0.8 - 0.5 = 0.4$
- friendship–beats: $0.8 + 0.1 - 0.5 = 0.4$
- beats–screen: $0.1 + 0.1 - 0.5 = -0.3 \rightarrow$ ReLU = 0

ขั้นที่ 2 ตัวกรอง F2

- สามคู่แรกได้ $0.3 - 0.5$, $0.1 - 0.5$ และ $0.2 - 0.5$ ซึ่งติดลบ ReLU จึงให้ 0
- beats–screen: $0.2 + 0.9 - 0.5 = 0.6$

ขั้นที่ 3 Max-over-time Pooling

- $h_1 = \max(0.5,0.4,0.4,0) = 0.5$ และ $h_2 = \max(0,0,0,0.6) = 0.6$
- เวกเตอร์ของประโยคคือ $\mathbf{h} = (0.5,0.6)$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

คำนวณครบทุกคู่คำติดกันของทั้งสองตัวกรอง

- ตัวกรอง F1 รวมมิติที่ 1 ของสองคำ: F1 = x1(คำแรก) + x1(คำที่สอง) − 0.5 ส่วน F2 รวมมิติที่ 2: F2 = x2(คำแรก) + x2(คำที่สอง) − 0.5
- เวกเตอร์คำจากภาพที่ 4.12: toys = (0.9, 0.2), prove = (0.1, 0.1), friendship = (0.8, 0.0), beats = (0.1, 0.2), screen = (0.1, 0.9)

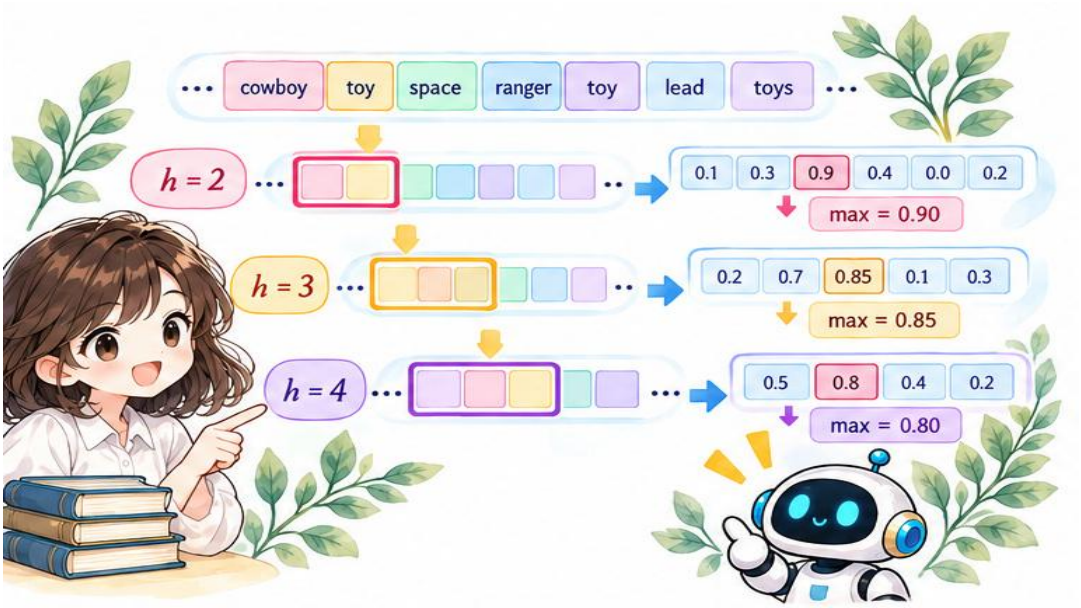
คู่คำ	F1 ก่อน ReLU	F1	F2 ก่อน ReLU	F2
toys–prove	0.9 + 0.1 − 0.5 = 0.5	0.5	0.2 + 0.1 − 0.5 = −0.2	0.0
prove–friendship	0.1 + 0.8 − 0.5 = 0.4	0.4	0.1 + 0.0 − 0.5 = −0.4	0.0
friendship–beats	0.8 + 0.1 − 0.5 = 0.4	0.4	0.0 + 0.2 − 0.5 = −0.3	0.0
beats–screen	0.1 + 0.1 − 0.5 = −0.3	0.0	0.2 + 0.9 − 0.5 = 0.6	0.6

- Max-over-time: h1 = max(0.5, 0.4, 0.4, 0) = 0.5 และ h2 = max(0, 0, 0, 0.6) = 0.6 จึงได้ h = (0.5, 0.6)
- ตีความคู่คำ: F1 ตอบสนองสูงสุดที่ toys–prove เพราะ toys มีมิติแรกสูง ส่วน F2 ตอบสนองเฉพาะ beats–screen เพราะ screen มีมิติที่สองสูง ซึ่งตรงกับสองขั้วของเรื่องย่อ i1 คือของเล่นกับหน้าจอ

ตีความ

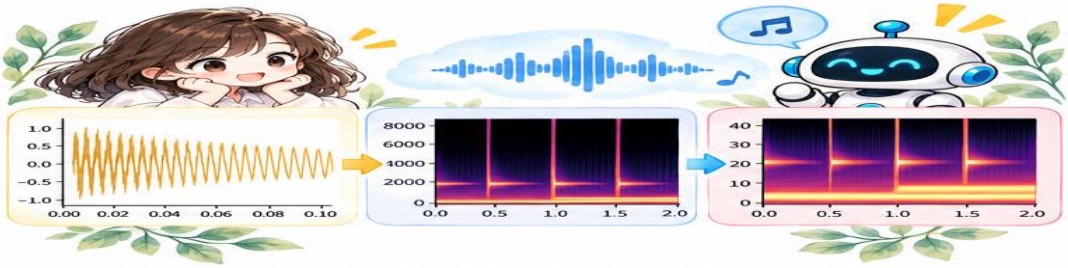
แต่ละมิติของ $\mathbf{h}$ ตอบคำถามว่า "ประโยคนี้มีวลีแบบที่ตัวกรองนั้นชอบหรือไม่" โดยไม่สนตำแหน่ง ข้อจำกัดคือตัวกรองเห็นเพียงสองคำติดกัน ถ้าความหมายของคำขึ้นกับคำที่อยู่ไกลออกไป Text CNN จะมองไม่เห็น

ภาพที่ 4.13 ขยายแนวคิดเดียวกันไปสู่เรื่องย่อของ i1 Toy Story 5 โดยใช้ตัวกรองสามขนาด คือกว้าง 2, 3 และ 4 คำ แต่ละขนาดจับวลียาวต่างกัน แล้วต่อผลของทุกตัวกรองเข้าด้วยกันก่อนฉายเป็นเวกเตอร์ของเรื่อง



ภาพที่ 4.13 Text CNN สามขนาดหน้าต่างบนเรื่องย่อของ i1 Toy Story 5

สำหรับเสียง ภาพที่ 4.14 แสดงการแปลงคลื่นเสียงเป็นสเปกโทรแกรมและ Log-Mel Spectrogram ส่วนวิดีโอ ภาพที่ 4.15 แสดงการเลือก 16 เฟรมห่างเท่ากันจากตัวอย่างภาพยนตร์ยาว 2 นาที



ภาพที่ 4.14 คลื่นเสียง สเปกโทรแกรม และ Log-Mel ของคลิปตัวอย่าง



ภาพที่ 4.15 ตำแหน่งของ 16 เฟรมที่เลือกจาก 2,880 เฟรม

การแปลงข้อมูลของเสียงและวิดีโอก่อนเข้า CNN ใช้สองสมการ คือจำนวนเฟรมเวลาของสเปกโทรแกรมในสมการ (4.18) และระยะห่างของเฟรมวิดีโอที่เลือกในสมการ (4.19)

$$T = 1 + \left\lfloor \frac{N - n_{win}}{n_{hop}} \right\rfloor \tag{4.18}$$

โดยที่

- N จำนวนจุดของคลื่นเสียง เท่ากับ ความยาว (วินาที) × อัตราสุ่ม
- n_{win} ความยาวหน้าต่างวิเคราะห์ (จุด)
- n_{hop} ระยะเลื่อนของหน้าต่าง (จุด)

- T จำนวนเฟรมเวลาของสเปกโทรแกรม

$$\Delta = \frac{F_{total}}{T_v}, \qquad t_j = \frac{\Delta}{2} + j\,\Delta \tag{4.19}$$

โดยที่

- F_{total} จำนวนเฟรมทั้งหมดของวิดีโอ
- T_v จำนวนเฟรมที่เลือก
- Δ ระยะห่างระหว่างเฟรมที่เลือก
- t_j ตำแหน่งของเฟรมที่เลือกลำดับที่ $j = 0,1,\ldots,T_v - 1$

ตัวอย่างการคำนวณ 4.13 ติดตามรูปร่างของข้อมูลผ่าน CNN ของสี่สื่อ

โจทย์ จงติดตามรูปร่างของเทนเซอร์ตั้งแต่ข้อมูลนำเข้าจนได้เวกเตอร์ของรายการหนึ่งเรื่องในแต่ละสื่อ

สมการที่ใช้ สมการ (4.13), (4.15), (4.18) และ (4.19)

ข้อมูลที่ใช้ รูปแบบข้อมูลจากตารางที่ 4.8 และสถาปัตยกรรมในตารางที่ 4.11

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- รูปแบบข้อมูลนำเข้าทั้งสี่สื่อมาจากตารางที่ 4.8 ซึ่งใช้เหมือนกันทุกเรื่องใน MovieMini-12: เรื่องย่อ 60 โทเคน โปสเตอร์ 224 × 224 × 3 คลิปเสียง 10.24 วินาที และตัวอย่างภาพยนตร์ 2 นาที
- สถาปัตยกรรมของแต่ละสื่อมาจากตารางที่ 4.11 (Text CNN, ResNet-50, PANNs CNN14 และ ResNet รายเฟรม) อัตราสุ่มเสียง 16,000 Hz และ 24 เฟรมต่อวินาทีเป็นค่ามาตรฐานของไฟล์ทั่วไป

ขั้นที่ 1 ข้อความ: Text CNN

- เรื่องย่อ 60 โทเคน แต่ละโทเคนเป็นเวกเตอร์ 300 มิติ ได้ตาราง 60 × 300
- ตัวกรองกว้าง $k = 3$ จำนวน 100 ตัว ไม่เติมขอบ: ความยาว $\lfloor (60-3)/1 \rfloor + 1 = 58$ จึงได้ 58 × 100
- Max-over-time Pooling เหลือ 100 ค่า คือเวกเตอร์ของเรื่องย่อ

ขั้นที่ 2 ภาพ: ResNet-50

- โปสเตอร์ 224 × 224 × 3 ผ่านชั้นที่ย่อขนาดลงครึ่งหนึ่งห้าครั้ง: $224/2^5 = 7$
- ได้แผนที่คุณลักษณะ 7 × 7 × 2,048 แล้ว Global Average Pooling เหลือ 2,048 ค่า

ขั้นที่ 3 เสียง: CNN บน Log-Mel

- คลิป 10.24 วินาทีที่ 16,000 Hz มี $N = 163{,}840$ จุด หน้าต่าง 25 ms = 400 จุด เลื่อน 10 ms = 160 จุด
- $T = 1 + \lfloor (163{,}840 - 400)/160 \rfloor = 1{,}022$ เฟรม แล้วเติมเป็น 1,024 เฟรม ได้ตาราง 128 × 1,024 × 1
- CNN แบบ PANNs CNN14 ย่อตารางด้วย Pooling แล้วเฉลี่ย ได้เวกเตอร์ 2,048 ค่า

ขั้นที่ 4 วิดีโอ: CNN รายเฟรม

- ตัวอย่างภาพยนตร์ 2 นาทีที่ 24 เฟรมต่อวินาทีมี $F_{total} = 2{,}880$ เฟรม เลือก $T_v = 16$ เฟรม
- $\Delta = 2{,}880/16 = 180$ เฟรม ตำแหน่งแรก $t_0 = 90$ ตามด้วย 270, 450, ..., 2,790 (ภาพที่ 4.15)
- ส่งแต่ละเฟรมผ่าน ResNet-50 ได้ 16 × 2,048 แล้วเฉลี่ยตามเวลาเหลือ 2,048 ค่า

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

สื่อ	การคำนวณ	ผล
ข้อความ	Lout = ⌊(60 − 3)/1⌋ + 1 = 57 + 1	58 × 100 ⟶ Max ⟶ 100
ภาพ	224 ÷ 2 ÷ 2 ÷ 2 ÷ 2 ÷ 2 = 224/32 = 7	7 × 7 × 2,048 ⟶ GAP ⟶ 2,048

เสียง: จุด	10.24 วินาที × 16,000 จุด/วินาที	N = 163,840
เสียง: หน้าต่าง	25 ms × 16 = 400 จุด, 10 ms × 16 = 160 จุด	nwin = 400, nhop = 160
เสียง: เฟรม	$1 + \lfloor (163{,}840 - 400)/160 \rfloor = 1 + \lfloor 1{,}021.5 \rfloor$	1,022 ⟶ เติมเป็น 1,024
วิดีโอ: เฟรมทั้งหมด	2 นาที × 60 วินาที × 24 เฟรม	Ftotal = 2,880
วิดีโอ: ระยะห่าง	2,880 / 16	$\Delta = 180$
วิดีโอ: ตำแหน่ง	$t_j = 90 + 180j$ สำหรับ $j = 0, \ldots, 15$	90, 270, 450, ..., 2,790
วิดีโอ: เวกเตอร์	16 เฟรม × 2,048 ⟶ เฉลี่ยตามเวลา	2,048

- ตรวจตำแหน่งสุดท้าย: $j = 15$ ได้ $90 + 180 \times 15 = 2{,}790$ ซึ่งน้อยกว่า 2,880 จึงอยู่ในตัวอย่างภาพยนตร์
- ใน PANNs ความยาวกรอบเวลาที่ไม่ครบ 1,024 ถูกเติมด้วย 0 ให้ทุกคลิปมีขนาดเท่ากัน

ตีความ

ไม่ว่าข้อมูลนำเข้าจะมีรูปร่างอย่างไร ปลายทางของทุกสื่อคือ **เวกเตอร์ยาวคงที่หนึ่งเส้นต่อหนึ่งรายการ** ซึ่งเป็นสิ่งที่หัวข้อ 4.5 ต้องการ ส่วนที่ต่างกันมีเพียงการเตรียมข้อมูลก่อนเข้าตัวเข้ารหัส

ไลบรารี Python ที่ใช้ได้ แนวทางเรียกใช้ CNN ที่ฝึกไว้แล้วด้วย Python

โค้ดต่อไปนี้เป็นแนวทางสำหรับระบบจริง ต้องติดตั้งไลบรารีและดาวน์โหลดน้ำหนักของแบบจำลองก่อน จึงไม่ได้รันในหนังสือ

```
# ภาพ: ResNet-50 ตัดชั้นจำแนกทิ้ง ได้เวกเตอร์ 2048 มิติ
import torch, torchvision
w = torchvision.models.ResNet50_Weights.DEFAULT
net = torchvision.models.resnet50(weights=w)
net.fc = torch.nn.Identity()
x = w.transforms()(poster).unsqueeze(0)   # poster = PIL.Image
v_img = net.eval()(x)                     # (1, 2048)


# เสียง: Log-Mel แล้วส่งเข้า CNN ของ PANNs
import librosa
y, sr = librosa.load("clip.wav", sr=16000)
mel = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=400,
                                     hop_length=160, n_mels=128)
logmel = librosa.power_to_db(mel)         # (128, T)


# วิดีโอ: 3D CNN ของ torchvision
from torchvision.models.video import r3d_18, R3D_18_Weights
net3d = r3d_18(weights=R3D_18_Weights.DEFAULT)
net3d.fc = torch.nn.Identity()            # เวกเตอร์ 512 มิติ
```

อธิบายแบบเข้าใจง่าย สี่สื่อ หนึ่งวิธีคิด

ให้มองทุกสื่อเป็น "ตาราง" ที่มีแกนต่างกัน ข้อความคือตาราง คำ × มิติ ภาพคือตาราง แถว × คอลัมน์ × สี เสียงคือตาราง ความถี่ × เวลา และวิดีโอคือ เวลา × แถว × คอลัมน์ × สี

CNN ทำสิ่งเดียวกันกับทุกตาราง คือเลื่อนตัวกรองหาลวดลายเฉพาะที่ ตัด ReLU ย่อด้วย Pooling แล้วเฉลี่ยเป็นเวกเตอร์

4.3.7 เวกเตอร์รายการจากเทคนิค CNN

หัวข้อนี้กำหนดเวกเตอร์ของภาพยนตร์ทั้ง 12 เรื่องที่ได้จากเทคนิค CNN ในขนาดย่อสำหรับการสอน เวกเตอร์ชุดนี้คือ **ข้อมูลที่ได้จากการคำนวณจากเทคนิค CNN** ซึ่งหัวข้อ 4.5 จะนำไปสร้างโปรไฟล์ผู้ใช้

สำหรับข้อความ Text CNN ทำงานบนเวกเตอร์คำแบบ Static ซึ่งให้คำหนึ่งคำมีความหมายเดียวทุกบริบท และตัวกรองเห็นเพียงคำข้างเคียง บทนี้จึงใช้ Static Embedding 12 มิติของบทที่ 3 หัวข้อ 3.7 เป็นเวกเตอร์ข้อความของเทคนิค CNN ดังตารางที่ 4.12

ตารางที่ 4.12 เวกเตอร์ข้อความ 12 มิติจากเทคนิค CNN

รหัส	tech	mind	spac	rom	crim	act	fam	mus	fant	toys	soc	amb
i1	.2212	.1327	.0822	·	.0885	.1327	.3482	·	.0885	.8787	·	·
i2	·	.9641	·	.0266	·	.0391	.2001	·	·	·	·	.1679
i3	.3654	.1741	.0523	·	·	.1218	.7835	·	.4525	·	·	·
i4	·	.1940	·	.8027	.3051	.0555	.4528	·	·	·	.1294	·
i5	·	.1189	·	.3029	·	.1189	.4649	.2076	.6229	·	.2076	.4356
i6	·	.1369	·	.6710	·	·	.7270	·	·	·	.0498	·
i7	·	.0330	·	·	.9065	.4121	.0769	.0384	·	·	·	·
i8	·	·	.0933	·	.0466	.9728	.0801	·	·	·	.1865	.0401
i9	·	.1355	·	·	.1734	.0758	·	·	.0379	·	.9473	.2167
i10	·	.2055	.4034	·	·	.5857	.0229	·	.1826	·	.6390	.0990
i11	.6062	.2146	.4942	.0727	.0606	.4418	.0467	·	.0606	·	.3637	·
i12	.2656	.2486	.8730	.0181	·	.1215	.2690	·	·	.0912	.0987	·

ที่มา: Static Embedding ของบทที่ 3 หัวข้อ 3.7 ปรับความยาวเป็น 1 แล้ว ละเลขศูนย์หน้าจุด จุด · แทนค่า 0

สำหรับภาพ เสียง และวิดีโอ ตารางที่ 4.13 แสดงเวกเตอร์สามมิติของแต่ละสื่อ เขียนเป็น I1–I3 (ภาพ) A1–A3 (เสียง) และ V1–V3 (วิดีโอ) ค่าเหล่านี้เป็นผลลัพธ์ของตัวเข้ารหัสในขนาดย่อ ไม่ได้ตั้งชื่อตามความหมาย ผู้อ่านจึงอ่านว่า "มิติที่ 1, 2 และ 3" ของแต่ละสื่อ

ตารางที่ 4.13 เวกเตอร์ภาพ เสียง และวิดีโอสามมิติจากเทคนิค CNN

รหัส	I1	I2	I3	A1	A2	A3	V1	V2	V3
i1	0.90	0.60	0.77	0.50	0.65	0.19	0.57	0.50	1.00
i2	0.80	0.80	0.77	0.50	0.80	0.26	0.52	0.42	1.00
i3	0.70	0.50	0.69	0.30	0.70	0.41	0.47	0.35	1.00
i4	0.50	0.70	0.24	0.30	0.70	0.34	0.47	0.35	0.00

รหัส	I1	I2	I3	A1	A2	A3	V1	V2	V3
i5	0.70	0.60	0.47	0.40	0.80	0.19	0.42	0.42	0.00
i6	0.60	0.80	0.32	0.20	0.70	0.19	0.37	0.27	0.00
i7	0.20	0.30	0.17	0.90	0.40	0.71	0.72	0.80	0.00
i8	0.60	0.90	0.32	1.00	0.45	0.64	0.77	0.87	0.00
i9	0.40	0.30	0.17	0.30	0.55	0.56	0.42	0.42	0.00
i10	0.50	0.80	0.17	0.80	0.45	0.71	0.62	0.57	0.00
i11	0.30	0.20	0.17	0.60	0.45	0.64	0.57	0.50	0.00
i12	0.30	0.30	0.17	0.50	0.60	0.64	0.52	0.42	0.00

ที่มา: ค่าจำลองเพื่อการสอน (ไฟล์ mm4.py ในภาคผนวก 4ก) I = ภาพ, A = เสียง, V = วิดีโอ

ตัวอย่างการคำนวณ 4.14 ความคล้ายข้อความของ Toy Story 5 กับ Inside Out 2 จากเทคนิค CNN

โจทย์ จงหาโคไซน์ระหว่างเวกเตอร์ข้อความของ i1 Toy Story 5 และ i2 Inside Out 2 จากเทคนิค CNN

สมการที่ใช้ สมการ (4.1)

ข้อมูลที่ใช้ ตารางที่ 4.12

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- i1 Toy Story 5 และ i2 Inside Out 2 เป็นแอนิเมชันครอบครัวทั้งคู่ (ตารางที่ 4.4) และ u1 ให้ 5 คะแนนทั้งสองเรื่อง (ตารางที่ 4.5)
- เวกเตอร์ข้อความเทคนิค CNN ในตารางที่ 4.12 คือ Static Embedding ของบทที่ 3 ซึ่งสร้างจากเรื่องย่อในตารางที่ 4.6
- เรื่องย่อ i1 มี toy และ toys รวม 5 ครั้ง มิติ toys จึงสูงที่สุด (.8787) และมี girl (3 ครั้ง) bedroom, friendship ช่วยมิติ family (.3482) ส่วนเรื่องย่อ i2 มี mind, memory (2 ครั้ง) และ dream (4 ครั้ง) มิติ mind จึงสูงที่สุด (.9641)
- คำว่า dream ใน Static Embedding ได้ค่าเฉลี่ยสองความหมาย (mind 0.5, ambition 0.5) i2 จึงมี ambition = .1679 ด้วย

ขั้นที่ 1 หามิติที่ไม่เป็นศูนย์พร้อมกัน

- มี 3 มิติ คือ mind, action, family

ขั้นที่ 2 ผลคูณจุด

- mind: $0.1327 \times 0.9641 = 0.1279$
- action: $0.1327 \times 0.0391 = 0.0052$
- family: $0.3482 \times 0.2001 = 0.0697$

$$\mathbf{a} \cdot \mathbf{b} = 0.1279 + 0.0052 + 0.0697 = 0.2028$$

ขั้นที่ 3 หารด้วยความยาว

- ตารางปรับความยาวเป็น 1 แล้ว ($\| \mathbf{a} \| = 1.0000, \| \mathbf{b} \| = 1.0000$) โคไซน์จึงเท่ากับ 0.2028

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

คูณครบทั้ง 12 มิติ

มิติ	i1	i2	i1 × i2
tech	0.2212	0.0000	0.0000
mind	0.1327	0.9641	0.1279
space	0.0822	0.0000	0.0000
romance	0.0000	0.0266	0.0000
crime	0.0885	0.0000	0.0000
action	0.1327	0.0391	0.0052
family	0.3482	0.2001	0.0697
music	0.0000	0.0000	0.0000
fantasy	0.0885	0.0000	0.0000

toys	0.8787	0.0000	0.0000
society	0.0000	0.0000	0.0000
ambition	0.0000	0.1679	0.0000
รวม			**0.2028**

- มีเพียง mind (0.1279), action (0.0052) และ family (0.0697) ที่ไม่เป็นศูนย์ ผลรวมคือ 0.2028
- ตรวจความยาว: ผลรวมกำลังสองของแถว i1 = 0.9999 และของแถว i2 = 1.0000 (ใกล้ 1 ตามการปัดทศนิยม) โคไซน์จึงเท่ากับผลคูณจุด

ตีความ

สองเรื่องเป็นแอนิเมชันครอบครัวเหมือนกัน แต่โคไซน์ข้อความเพียง 0.2028 เพราะ i1 เด่นที่มิติ toys ส่วน i2 เด่นที่มิติ mind ข้อความจึงมองว่าสองเรื่องนี้ต่างกันพอสมควร ในหัวข้อ 4.5
ผู้อ่านจะเห็นว่าภาพและวิดีโอช่วยให้ระบบรู้ว่าสองเรื่องนี้ใกล้กันในด้านรูปแบบการนำเสนอ

แนวทางเลือกใช้ จุดเด่น ข้อจำกัด และเมื่อใดควรเลือก CNN

- **จุดเด่น** พารามิเตอร์น้อย ฝึกเร็ว ใช้ข้อมูลน้อยกว่า Transformer ทำงานบนอุปกรณ์ขนาดเล็กได้ และมีแบบจำลองที่ฝึกไว้แล้วให้เลือกมาก
- **ข้อจำกัด** เห็นเฉพาะบริเวณใกล้ ความสัมพันธ์ระยะไกลต้องอาศัยการซ้อนชั้นจำนวนมาก และคำหนึ่งคำมีความหมายเดียวเมื่อใช้เวกเตอร์คำแบบ Static
- **ควรเลือกเมื่อ** ข้อมูลฝึกน้อย ต้องการความเร็วสูง หรือคุณลักษณะสำคัญเป็นลวดลายเฉพาะที่ เช่น สีพื้นผิว จังหวะเสียงสั้น ๆ

โปรแกรม 4.7 โคไซน์ของเรื่องที่ u1 ชอบจากเทคนิค CNN

```
from mm4 import vectors

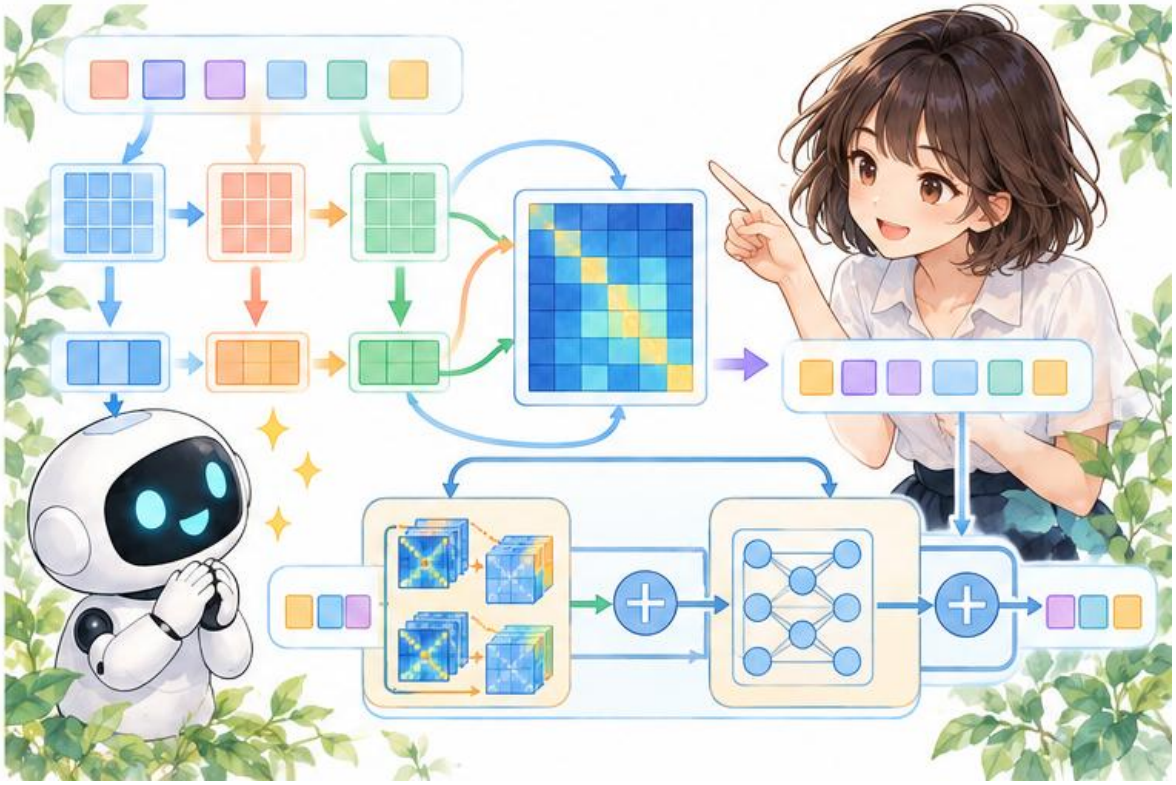

V = vectors('CNN')          # ลบค่าเฉลี่ยสี่อ I, A, V แล้วปรับ L2
liked = ['i1', 'i2', 'i3']
for m in ['T', 'I', 'A', 'V']:
    sims = ['%s-%s %.4f' % (a, b, V[m][a] @ V[m][b])
            for k, a in enumerate(liked) for b in liked[k + 1:]]
    print(m, ' | '.join(sims))
```

ผลลัพธ์

```
T i1-i2 0.2028 | i1-i3 0.4373 | i2-i3 0.3294
I i1-i2 0.9127 | i1-i3 0.9340 | i2-i3 0.7968
A i1-i2 0.8194 | i1-i3 0.3320 | i2-i3 0.4839
V i1-i2 0.9921 | i1-i3 0.9719 | i2-i3 0.9937
```

4.4 Transformer

Transformer เสนอโดย Vaswani et al. (2017) สำหรับงานแปลภาษา แล้วกลายเป็นตัวเข้ารหัสหลักของยุคที่ 3 ในหัวข้อ 4.2.2 ทั้งข้อความ (BERT) ภาพ (ViT) เสียง (AST) และวิดีโอ (Video Transformer) หัวข้อนี้อธิบายแนวคิด ทฤษฎี และขั้นตอนวิธีของ Transformer ทีละชิ้น แล้วสรุปการประยุกต์กับสี่สื่อ ปิดท้ายด้วยเวกเตอร์รายการจากเทคนิค Transformer และการเปรียบเทียบกับ CNN ดังสรุปในภาพที่ 4.55 แสดงการฝังข้อมูล Self-Attention บล็อกตัวเข้ารหัส และการรวมเป็นเวกเตอร์เดียว



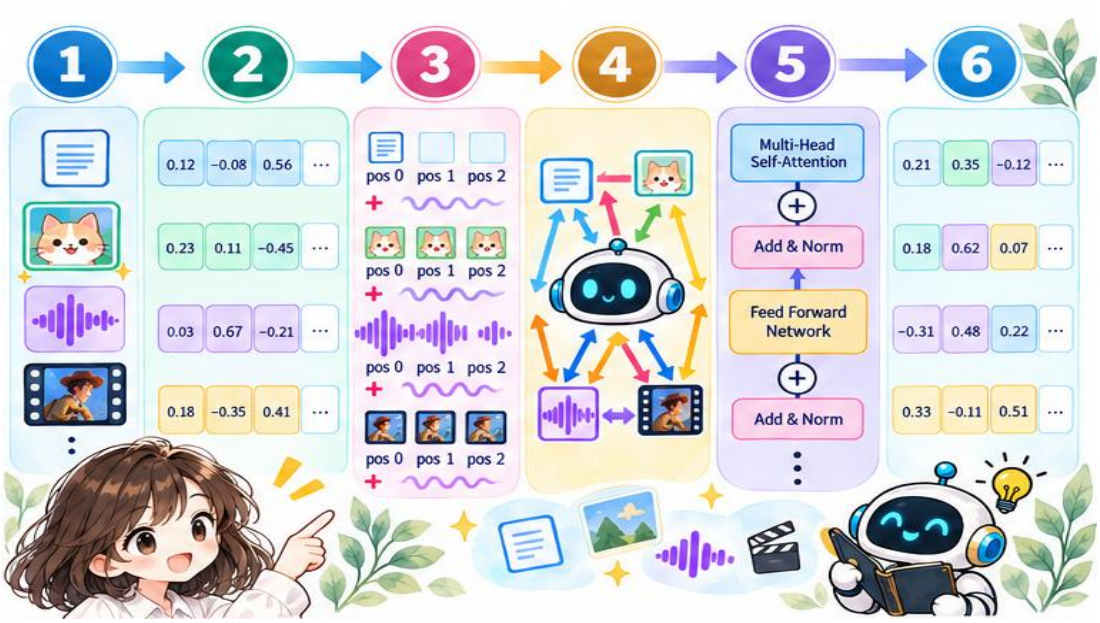
ภาพที่ 4.55 สายงานการเข้ารหัสด้วย Transformer

4.4.1 แนวคิดของ Transformer

หัวข้อ 4.3.5 สรุปว่า CNN เห็นเฉพาะบริเวณใกล้ และต้องซ้อนหลายชั้นกว่าจะเห็นส่วนที่อยู่ไกลกัน Transformer เปลี่ยนวิธีคิดเป็นสามข้อ

- **ทุกสื่อคือลำดับของชิ้น (Tokens)** ข้อความแบ่งเป็นคำย่อย ภาพแบ่งเป็นแพตช์ เสียงแบ่งเป็นแพตช์ของสเปกโทรแกรม และวิดีโอแบ่งเป็นก้อนของเวลา × พื้นที่ แต่ละชิ้นถูกฝังเป็นเวกเตอร์ตามหัวข้อ 4.2.5
- **ทุกชิ้นปรึกษากันได้ตั้งแต่ชั้นแรก** ด้วยกลไก **Self-Attention** แต่ละชิ้นเลือกเองว่าจะดึงข้อมูลจากชิ้นใดมากน้อยเพียงใด ไม่ว่าจะอยู่ใกล้หรือไกล
- **ซ้อนบล็อกเดียวกันหลายชั้น** แต่ละบล็อกมี Self-Attention ตามด้วยเครือข่ายป้อนไปข้างหน้า พร้อมทางลัดและการปรับมาตรฐาน

ภาพที่ 4.16 สรุปขั้นตอนทั้งหกของ Transformer ซึ่งใช้ได้กับข้อมูลทุกสื่อ ได้แก่ แบ่งเป็นหน่วยย่อย แปลงเป็นเวกเตอร์ เพิ่มตำแหน่ง Self-Attention ซ้อนบล็อก และได้ตัวแทนที่เข้าใจบริบท หัวข้อ 4.4.2–4.4.6 อธิบายทีละขั้น



ภาพที่ 4.16 ภาพรวมหกขั้นของ Transformer สำหรับข้อมูลทุกสื่อ

อธิบายแบบเข้าใจง่าย Transformer คือห้องประชุมที่ทุกคนถามกันได้

CNN เหมือนการส่งต่อข่าวแบบกระซิบ แต่ละคนคุยได้เฉพาะคนข้างตัว
กว่าข่าวจากหัวแถวจะถึงท้ายแถวต้องผ่านหลายทอด

Transformer เหมือนห้องประชุมที่ทุกคนยกมือถามใครก็ได้ในห้องโดยตรง
แต่ละคนจึงรู้บริบทของทั้งห้องตั้งแต่รอบแรก
ราคาที่ต้องจ่ายคือจำนวนคู่สนทนาเพิ่มเป็นกำลังสองของจำนวนคน

4.4.2 Self-Attention ทีละขั้น

ภาพที่ 4.17 แสดงแนวคิดของ Self-Attention บนเรื่องย่อ โทเคน playtime "มองหา" โทเคนอื่นในประโยค ให้คะแนนความเกี่ยวข้อง แล้วดึงข้อมูลจากโทเคนที่เกี่ยวข้องมากมารวมเป็นความหมายตามบริบทของตน



ภาพที่ 4.17 แนวคิด Self-Attention: playtime ถามโทเคนอื่นในเรื่องย่อ

แต่ละโทเคนเล่นสามบทบาทพร้อมกัน ซึ่งสร้างจากเวกเตอร์ของโทเคนเดียวกันด้วยเมทริกซ์ฉายสามตัว ดังสมการ (4.20)

- Query (Q) คือ "คำถาม" ว่าโทเคนนี้กำลังมองหาข้อมูลแบบใด
- Key (K) คือ "ป้ายชื่อ" ที่บอกว่าโทเคนนี้มีข้อมูลแบบใดให้
- Value (V) คือ "เนื้อหา" ที่จะถูกส่งต่อเมื่อได้รับเลือก

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \tag{4.20}$$

โดยที่

- $\mathbf{X}$ เมทริกซ์ของโทเคนขนาด $n \times d$ แต่ละแถวคือเวกเตอร์ของหนึ่งโทเคน
- $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ เมทริกซ์ฉายขนาด $d \times d_k$ ซึ่งเรียนรู้จากการฝึก
- $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ เมทริกซ์ Query, Key และ Value ขนาด $n \times d_k$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{4.21}$$

โดยที่

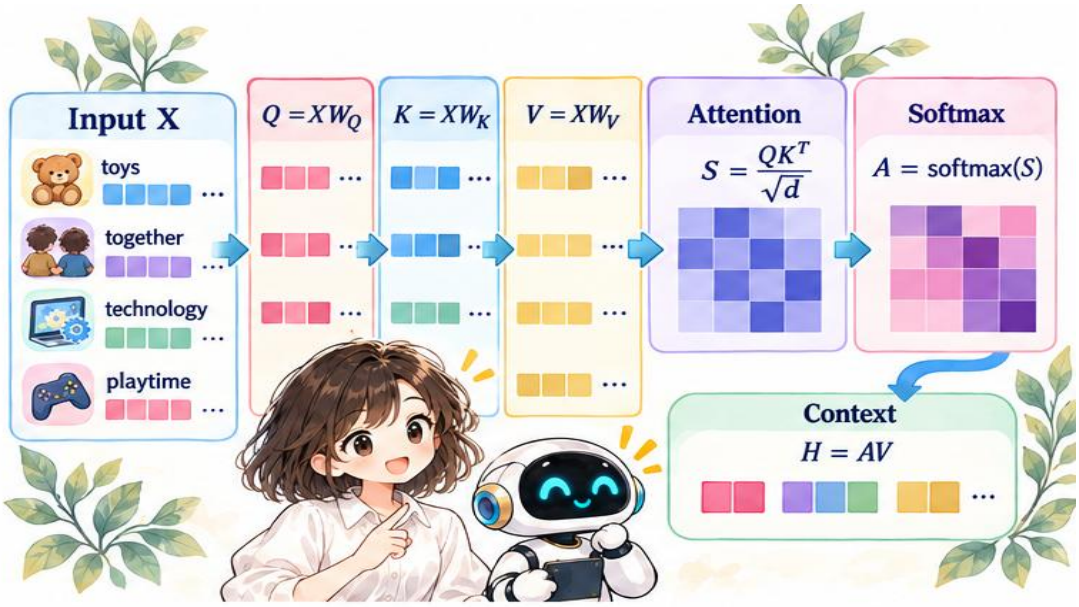
- $\mathbf{Q}\mathbf{K}^\top$ ตารางคะแนน $n \times n$ ช่อง (i, j) คือผลคูณจุดของ Query ของโทเคน i กับ Key ของโทเคน j
- $\sqrt{d_k}$ ตัวหารเพื่อไม่ให้คะแนนโตเกินไปเมื่อมิติมาก
- softmax แปลงคะแนนแต่ละแถวเป็นน้ำหนักบวกที่รวมกันได้ 1 ตามสมการ (4.22)
- $\text{Attention}(\cdot)$ ผลลัพธ์ $n \times d_k$ แต่ละแถวคือค่าเฉลี่ยถ่วงน้ำหนักของ Value

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{l=1}^{n} \exp(s_{il})}, \qquad \mathbf{h}_i = \sum_{j=1}^{n} \alpha_{ij}\, \mathbf{v}_j \tag{4.22}$$

โดยที่

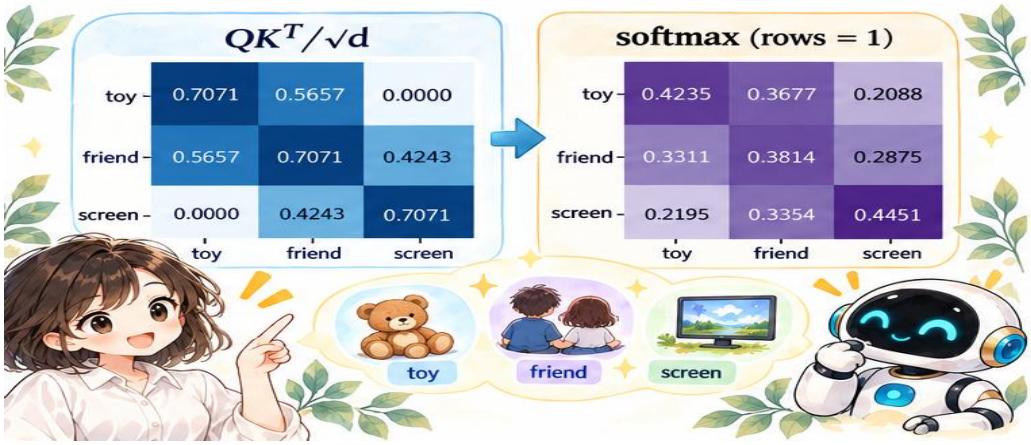
- s_{ij} คะแนนของคู่ (i, j) หลังหารด้วย $\sqrt{d_k}$
- α_{ij} น้ำหนัก Attention ที่โทเคน i ให้โทเคน j
- $\mathbf{h}_i$ เวกเตอร์ตามบริบทของโทเคน i

ภาพที่ 4.18 สรุปห้าขั้นของสมการ (4.20)–(4.22) คือ รับเวกเตอร์ของโทเคน สร้าง Q, K, V คำนวณคะแนน แปลงเป็นน้ำหนัก และรวม Value



ภาพที่ 4.18 Self-Attention ด้วย Query, Key และ Value

เริ่มจากตัวอย่างอุ่นเครื่องที่ง่ายที่สุด คือให้เมทริกซ์ฉายทั้งสามเป็นเมทริกซ์เอกลักษณ์ ($\mathbf{W} = \mathbf{I}$) ดังนั้น Q, K และ V เท่ากับเวกเตอร์ของโทเคนเอง ภาพที่ 4.19 แสดงคะแนนและน้ำหนักของสามโทเคน toy, friend และ screen



ภาพที่ 4.19 คะแนนและน้ำหนักของ toy, friend, screen เมื่อ W = I

ตัวอย่างการคำนวณ 4.15 Self-Attention เมื่อเมทริกซ์ฉายเป็นเมทริกซ์เอกลักษณ์

โจทย์ โทเคนสามตัวมีเวกเตอร์สองมิติ toy = (1, 0), friend = (0.8, 0.6), screen = (0, 1) ให้ $\mathbf{W}_Q = \mathbf{W}_K = \mathbf{W}_V = \mathbf{I}$ และ $d_k = 2$ จงหาเวกเตอร์ตามบริบทของ toy

สมการที่ใช้ สมการ (4.20), (4.21) และ (4.22)

ข้อมูลที่ใช้ เวกเตอร์สมมติเพื่อการสอน ผลทั้งตารางอยู่ในภาพที่ 4.19

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- สามโทเคน toy, friend และ screen แทนสามแนวคิดหลักของเรื่องย่อ i1 Toy Story 5 ในตารางที่ 4.6 คือ ของเล่น (toy, toys), มิตรภาพ (friendship) และหน้าจอ (screen, tablet) ซึ่งเป็นแกนของเรื่อง "ของเล่นต้องชนะหน้าจอด้วยมิตรภาพ"
- เวกเตอร์สองมิติเป็นค่าสมมติ มิติที่ 1 แทนความเป็น "ของเล่น" มิติที่ 2 แทนความเป็น "หน้าจอ" toy = (1, 0) อยู่ขั้วของเล่น screen = (0, 1) อยู่ขั้วหน้าจอ และ friend = (0.8, 0.6) อยู่ใกล้ขั้วของเล่นมากกว่า ทุกเวกเตอร์มีความยาว 1 (เช่น $0.8^2 + 0.6^2 = 1$)
- เมื่อ WQ = WK = WV = I ผลคูณกับเมทริกซ์เอกลักษณ์ไม่เปลี่ยนเวกเตอร์ Q, K และ V ของแต่ละโทเคนจึงเท่ากับเวกเตอร์ของโทเคนนั้นเอง

ขั้นที่ 1 คะแนนของ toy กับทุกโทเคน

- toy·toy = $1(1) + 0(0) = 1$, toy·friend = $1(0.8) + 0(0.6) = 0.8$, toy·screen = 0
- หารด้วย $\sqrt{2} = 1.4142$: $s = (0.7071, 0.5657, 0.0000)$

ขั้นที่ 2 softmax

- $e^{0.7071} = 2.0281$, $e^{0.5657} = 1.7607$, $e^0 = 1$
- ผลรวม = 4.7888 จึงได้ $\alpha = (0.4235, 0.3677, 0.2088)$ รวมกันได้ 1

ขั้นที่ 3 รวม Value ด้วยน้ำหนัก

$$\mathbf{h}_{toy} = 0.4235(1,0) + 0.3677(0.8,0.6) + 0.2088(0,1) = (0.7176,\ 0.4294)$$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

แถวของ toy ครบทุกขั้น

โทเคน j	v_j	s = q·k / √2	e^s	α	α × v_j
toy	(1.0, 0.0)	1.0 / 1.4142 = 0.7071	2.0281	0.4235	(0.4235, 0.0000)
friend	(0.8, 0.6)	0.8 / 1.4142 = 0.5657	1.7607	0.3677	(0.2941, 0.2206)
screen	(0.0, 1.0)	0.0 / 1.4142 = 0.0000	1.0000	0.2088	(0.0000, 0.2088)

- ผลรวม e^s = 2.0281 + 1.7607 + 1.0000 = 4.7888 แล้วหารแต่ละค่าด้วยผลรวมได้ **α**
- รวมคอลัมน์สุดท้าย: มิติที่ 1 = 0.4235 + 0.2941 + 0.0000 = 0.7176 และมิติที่ 2 = 0.0000 + 0.2206 + 0.2088 = 0.4294

ทำซ้ำกับ friend และ screen (ตารางน้ำหนักเต็มของภาพที่ 4.19)

ผู้ถาม	α ต่อ toy	α ต่อ friend	α ต่อ screen	h (ผลลัพธ์)
toy	0.4235	0.3677	0.2088	(0.7176, 0.4294)
friend	0.3311	0.3814	0.2875	(0.6363, 0.5163)
screen	0.2195	0.3354	0.4451	(0.4878, 0.6464)

- ทุกแถวรวมกันได้ 1 และ friend ให้น้ำหนักกับตัวเองสูงสุด เพราะ friend·friend = 1 มากกว่า friend·toy = 0.8 และ friend·screen = 0.6

ตีความ

toy ให้น้ำหนักกับตัวเองมากที่สุด (0.4235) รองลงมาคือ friend ซึ่งชี้ทิศใกล้กัน ผลลัพธ์ $\mathbf{h}_{toy}$ จึงเอียงไปทาง friend เล็กน้อย กล่าวคือความหมายของ toy ถูก "ปรับตามบริบท" ว่าอยู่ในประโยคเดียวกับ friend

ในแบบจำลองจริง เมทริกซ์ฉายทั้งสามต่างกันและได้จากการฝึก ทำให้ "สิ่งที่ถาม" ต่างจาก "สิ่งที่ให้" ภาพที่ 4.20 แสดงผลของตัวอย่างถัดไป ซึ่งใช้โทเคนชุดเดิมแต่เมทริกซ์ฉายต่างกัน

ตัวอย่างการคำนวณ 4.16 Self-Attention เมื่อ Q, K, V ต่างกัน

โจทย์ ใช้โทเคนชุดเดิมกับ $\mathbf{W}_Q = \begin{bmatrix} 1.0 & 0.5 \\ 0.0 & 1.0 \end{bmatrix}$, $\mathbf{W}_K = \begin{bmatrix} 1.0 & 0.0 \\ 0.5 & 1.0 \end{bmatrix}$ และ $\mathbf{W}_V = \begin{bmatrix} 0.5 & 0.0 \\ 0.0 & 1.0 \end{bmatrix}$ จงหาเวกเตอร์ตามบริบทของ toy

สมการที่ใช้ สมการ (4.20), (4.21) และ (4.22)

ข้อมูลที่ใช้ เวกเตอร์โทเคนจากตัวอย่างการคำนวณ 4.15 เมทริกซ์ฉายเป็นค่าสมมติ

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- ใช้สามโทเคนและเวกเตอร์ชุดเดิมของตัวอย่างการคำนวณ 4.15 ซึ่งแทนแนวคิดหลักของเรื่องย่อ i1 Toy Story 5
- เมทริกซ์ฉาย WQ, WK และ WV เป็นค่าสมมติ ในแบบจำลองจริงได้จากการฝึก

ขั้นที่ 1 สร้าง Q, K, V ตามสมการ (4.20)

- $\mathbf{q}_{toy} = (1,0)\mathbf{W}_Q = (1.0,0.5)$
- Key: toy = $(1.0,0.0)$, friend = $(1.1,0.6)$, screen = $(0.5,1.0)$
- Value: toy = $(0.5,0.0)$, friend = $(0.4,0.6)$, screen = $(0.0,1.0)$

ขั้นที่ 2 คะแนนและน้ำหนัก

- $\mathbf{q}_{toy} \cdot \mathbf{k}$: $1(1) + 0.5(0) = 1,\ 1(1.1) + 0.5(0.6) = 1.4,\ 1(0.5) + 0.5(1) = 1.0$
- หารด้วย $\sqrt{2}$: $(0.7071,0.9899,0.7071)$
- softmax: $\alpha = (0.3006,0.3988,0.3006)$

ขั้นที่ 3 รวม Value

$$\mathbf{h}_{toy} = 0.3006(0.5,0) + 0.3988(0.4,0.6) + 0.3006(0,1) = (0.3098,\ 0.5399)$$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

สร้าง Q, K, V ของทุกโทเคน (เวกเตอร์แถว × เมทริกซ์)

- ตัวอย่าง friend: Q = (0.8, 0.6) WQ = (0.8 × 1 + 0.6 × 0, 0.8 × 0.5 + 0.6 × 1) = (0.80, 1.00) และ K = (0.8 × 1 + 0.6 × 0.5, 0.8 × 0 + 0.6 × 1) = (1.10, 0.60)

โทเคน	Q = X WQ	K = X WK	V = X WV
toy	(1.00, 0.50)	(1.00, 0.00)	(0.50, 0.00)

friend	(0.80, 1.00)	(1.10, 0.60)	(0.40, 0.60)
screen	(0.00, 1.00)	(0.50, 1.00)	(0.00, 1.00)

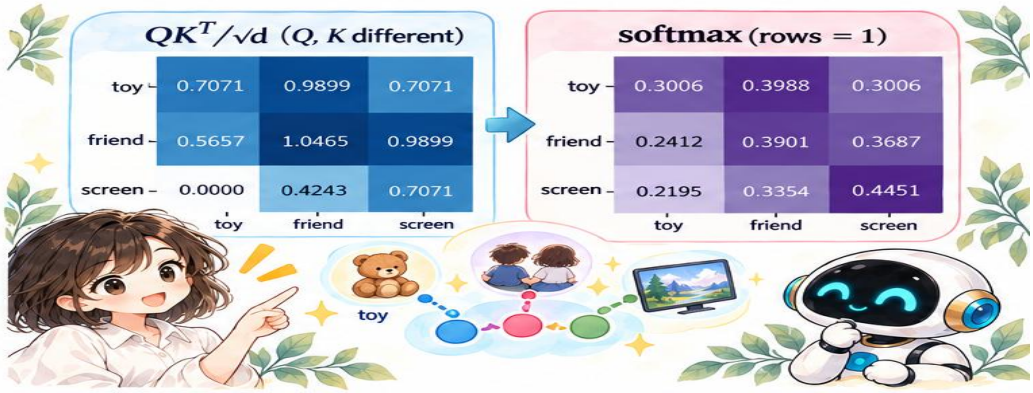
คะแนนและน้ำหนักของแถว toy

คู่	q_toy · k_j	÷ √2	e^s	α
toy–toy	1.00	0.7071	2.0281	0.3006
toy–friend	1.40	0.9899	2.6911	0.3988
toy–screen	1.00	0.7071	2.0281	0.3006

- ผลรวม e^s = 6.7473 · ผลลัพธ์ h(toy) = 0.3006 × (0.5, 0) + 0.3988 × (0.4, 0.6) + 0.3006 × (0, 1) = (0.3098, 0.5399)
- แม้ toy กับ friend จะไม่ได้คล้ายกันที่สุดในเวกเตอร์เดิม แต่ Key ของ friend (1.10, 0.60) ชี้ไปทาง Query ของ toy (1.00, 0.50) มากที่สุด น้ำหนักจึงย้ายไปที่ friend

ตีความ

เมื่อเมทริกซ์ฉายเปลี่ยน toy หันไปให้น้ำหนัก friend มากที่สุด (0.3988) แทนตัวเอง แสดงว่า **การฝึกเมทริกซ์ฉายคือการสอนว่าโทเคนควรสนใจโทเคนใด** ไม่ใช่เพียงวัดความคล้ายของเวกเตอร์เดิม



ภาพที่ 4.20 คะแนนและน้ำหนัก Attention เมื่อ Q, K, V ต่างกัน

ที่มาของไฮเปอร์พารามิเตอร์ ทำไมต้องหารด้วย $\sqrt{d_k}$

- **ที่มา** Vaswani et al. (2017) แสดงว่าถ้าแต่ละมิติของ $\mathbf{q}$ และ $\mathbf{k}$ มีค่าเฉลี่ย 0 และความแปรปรวน 1 ผลคูณจุดของ d_k มิติจะมีความแปรปรวน d_k การหารด้วย $\sqrt{d_k}$ จึงดึงความแปรปรวนกลับเป็น 1
- **ผลเมื่อไม่หาร** เมื่อ $d_k = 64$ คะแนนอาจต่างกันหลายสิบ softmax จะให้น้ำหนักเกือบทั้งหมดกับโทเคนเดียว และเกรเดียนต์ของโทเคนอื่นเกือบเป็นศูนย์ ทำให้ฝึกยาก
- **ในบทนี้** $d_k = 2$ จึงหารด้วย $\sqrt{2} = 1.4142$

โปรแกรม 4.8 Self-Attention ของสามโทเคน

```
import numpy as np
```

```
def softmax(S):
    E = np.exp(S - S.max(axis=1, keepdims=True))
    return E / E.sum(axis=1, keepdims=True)

X = np.array([[1.0, 0.0], [0.8, 0.6], [0.0, 1.0]])  # toy friend screen
WQ = np.array([[1.0, 0.5], [0.0, 1.0]])
WK = np.array([[1.0, 0.0], [0.5, 1.0]])
WV = np.array([[0.5, 0.0], [0.0, 1.0]])
for name, (q, k, v) in {'W = I': (np.eye(2),) * 3,
                        'W ต่างกัน': (WQ, WK, WV)}.items():
    Q, K, V = X @ q, X @ k, X @ v
    A = softmax(Q @ K.T / np.sqrt(2))     # น้ำหนัก Attention
    H = A @ V                              # เวกเตอร์ตามบริบท
    print(name, 'alpha(toy) =', np.round(A[0], 4),
          'h(toy) =', np.round(H[0], 4))
```

ผลลัพธ์

```
W = I alpha(toy) = [0.4235 0.3677 0.2088] h(toy) = [0.7176 0.4294]
W ต่างกัน alpha(toy) = [0.3006 0.3988 0.3006] h(toy) = [0.3098 0.5399]
```

4.4.3 หลายหัว (Multi-Head Attention)

Self-Attention หนึ่งชุดให้แต่ละโทเคนมีมุมมองเดียวว่าควรสนใจใคร แต่ความสัมพันธ์ในข้อมูลจริงมีหลายแบบพร้อมกัน เช่น ความสัมพันธ์ทางไวยากรณ์ ความสัมพันธ์ทางความหมาย หรือในภาพคือสีกับรูปทรง **Multi-Head Attention** จึงทำ Attention h ชุดขนานกัน แต่ละชุดมีเมทริกซ์ฉายของตนเอง แล้วต่อผลลัพธ์เข้าด้วยกัน ดังสมการ (4.23)

$$\mathrm{MHA}(\mathbf{X}) = \left[\mathbf{H}^{(1)} \parallel \mathbf{H}^{(2)} \parallel \cdots \parallel \mathbf{H}^{(h)}\right]\mathbf{W}_O, \qquad d_k = \frac{d}{h} \tag{4.23}$$

โดยที่

- $\mathbf{H}^{(r)}$ ผลของ Attention หัวที่ r ตามสมการ (4.21) ขนาด $n \times d_k$
- $\parallel$ การต่อเวกเตอร์ตามแนวมิติ (Concatenation)
- $\mathbf{W}_O$ เมทริกซ์ฉายผลรวมขนาด $d \times d$ ให้กลับเป็นมิติเดิม
- h จำนวนหัว
- d_k มิติของแต่ละหัว

ตัวอย่างการคำนวณ 4.17 สองหัวมองโทเคน toy ต่างกัน

โจทย์ หัวที่ 1 คือ Attention ในตัวอย่างการคำนวณ 4.15 หัวที่ 2 ใช้ $\mathbf{W}_Q = \mathbf{W}_K = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$ (สนใจเฉพาะมิติที่ 2) และ $\mathbf{W}_V = \mathbf{I}$ จงหาผลของ toy หลังต่อสองหัวแล้วฉายด้วย $\mathbf{W}_O$ ที่เฉลี่ยสองหัว

สมการที่ใช้ สมการ (4.21) และ (4.23)

ข้อมูลที่ใช้ ผลของหัวที่ 1 จากตัวอย่างการคำนวณ 4.15 เมทริกซ์ของหัวที่ 2 และ $\mathbf{W}_O$ เป็นค่าสมมติ

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- หัวที่ 1 นำผลมาจากตัวอย่างการคำนวณ 4.15 ซึ่งใช้สามโทเคนจากเรื่องย่อ i1 Toy Story 5
- หัวที่ 2 ใช้ WQ = WK = [[0, 0], [0, 1]] คือดูเฉพาะมิติที่ 2 (ความเป็นหน้าจอ) และ WO เฉลี่ยมิติคู่กันของสองหัว เป็นค่าสมมติ

ขั้นที่ 1 หัวที่ 1

- จากตัวอย่างการคำนวณ 4.15: $\mathbf{h}_{toy}^{(1)} = (0.7176, 0.4294)$

ขั้นที่ 2 หัวที่ 2

- Query ของ toy คือ $(1,0)\mathbf{W}_Q = (0,0)$ คะแนนกับทุกโทเคนจึงเป็น 0
- softmax ของ $(0,0,0)$ ได้น้ำหนักเท่ากัน $(1/3, 1/3, 1/3)$
- $\mathbf{h}_{toy}^{(2)} = 1/3\,[(1,0) + (0.8,0.6) + (0,1)] = (0.6000, 0.5333)$

ขั้นที่ 3 ต่อสองหัวแล้วฉาย

- ต่อกันได้เวกเตอร์ 4 มิติ $(0.7176, 0.4294, 0.6000, 0.5333)$
- $\mathbf{W}_O$ เฉลี่ยมิติคู่กัน: $(0.6588, 0.4814)$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

หัวที่ 2 ทีละขั้น

- Query ของ toy = (1, 0) × WQ = (1 × 0 + 0 × 0, 1 × 0 + 0 × 1) = (0, 0) เพราะ toy ไม่มีมิติที่ 2 เลย
- Key: toy = (0, 0), friend = (0, 0.6), screen = (0, 1) ผลคูณจุดกับ Query (0, 0) จึงเป็น 0 ทั้งสามคู่
- softmax(0, 0, 0): $e^0 = 1$ ทุกตัว ผลรวม 3 จึงได้ α = (1/3, 1/3, 1/3) = (0.3333, 0.3333, 0.3333)
- WV = I จึงได้ V = เวกเตอร์เดิม: h = (1/3)[(1, 0) + (0.8, 0.6) + (0, 1)] = (1/3)(1.8, 1.6) = (0.6000, 0.5333)

ต่อสองหัวแล้วฉายด้วย WO

- ต่อ: (h1 ‖ h2) = (0.7176, 0.4294, 0.6000, 0.5333)
- WO เฉลี่ยมิติที่ 1 ของสองหัว: (0.7176 + 0.6000)/2 = 0.6588 และมิติที่ 2: (0.4294 + 0.5333)/2 = 0.4814 (ค่าที่ปัดแล้วอาจต่างที่หลักสุดท้าย)

ตีความ

หัวที่ 1 มองว่า toy ใกล้ friend ส่วนหัวที่ 2 ซึ่งดูเฉพาะมิติที่ 2 "ไม่มีความเห็น" จึงเฉลี่ยทุกโทเคนเท่ากัน ระบบจริงมี 12 หัว แต่ละหัวเรียนรู้ความสัมพันธ์คนละแบบ แล้ว $\mathbf{W}_O$ ผสมมุมมองทั้งหมดเข้าด้วยกัน

ที่มาของไฮเปอร์พารามิเตอร์ จำนวนหัว h และมิติ d มาจากไหน

- **ที่มา** BERT-base และ ViT-B ใช้ $d = 768$, $h = 12$ จึงได้ $d_k = 64$ ส่วน BERT-large ใช้ $d = 1{,}024$, $h = 16$ (Devlin et al., 2019; Dosovitskiy et al., 2021)
- **ผลเมื่อเปลี่ยน** เพิ่ม h โดยคง d ทำให้แต่ละหัวมีมิติน้อยลง จำนวนพารามิเตอร์ของชั้น Attention คงที่ประมาณ $4d^2$ (สี่เมทริกซ์ $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$) หัวมากเกินไปทำให้แต่ละหัวมีข้อมูลน้อยเกินไป
- **ตัวอย่างตัวเลข** $d = 768$: $4 \times 768^2 = 2{,}359{,}296$ พารามิเตอร์ต่อชั้น ไม่รวมค่าเอนเอียง
- **วิธีเลือก** ใช้ค่าตามแบบจำลองที่ฝึกไว้แล้ว เพราะเปลี่ยน h หรือ d แล้วต้องฝึกใหม่ทั้งหมด

4.4.4 การฝังตำแหน่ง (Positional Encoding)

Self-Attention มีจุดอ่อนหนึ่งข้อ คือ **ไม่รู้ลำดับ** ถ้าสลับตำแหน่งโทเคนในตัวอย่างการคำนวณ 4.15 น้ำหนักแต่ละคู่จะเหมือนเดิม แต่ "toys beat the screen" กับ "the screen beat toys" มีความหมายต่างกัน Transformer จึงบวก **เวกเตอร์ตำแหน่ง** เข้าไปในเวกเตอร์ของโทเคนก่อนเข้าบล็อกแรก Vaswani et al. (2017) ใช้คลื่นไซน์และโคไซน์ความถี่ต่างกันตามสมการ (4.24)

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \tag{4.24}$$

โดยที่

- pos ตำแหน่งของโทเคนในลำดับ เริ่มจาก 0
- i ลำดับของคู่มิติ $i = 0, 1, \ldots, d/2 - 1$
- d จำนวนมิติของเวกเตอร์
- 10000 ค่าคงที่กำหนดช่วงความยาวคลื่นจาก 2π ถึง $10000 \cdot 2\pi$

ตัวอย่างการคำนวณ 4.18 เวกเตอร์ตำแหน่งสี่มิติ

โจทย์ จงหาเวกเตอร์ตำแหน่งของ $pos = 1$ และ $pos = 2$ เมื่อ $d = 4$ แล้วบวกกับเวกเตอร์โทเคน $(0.5,0.5,0.5,0.5)$ ที่ตำแหน่ง 1

สมการที่ใช้ สมการ (4.24)

ข้อมูลที่ใช้ ค่าสมมติเพื่อการสอน

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- เวกเตอร์โทเคน (0.5, 0.5, 0.5, 0.5) และ d = 4 เป็นค่าสมมติเพื่อให้คำนวณด้วยมือได้ ค่าคงที่ 10,000 มาจาก Vaswani et al. (2017)
- มุมของ sin และ cos ใช้หน่วยเรเดียน (ไม่ใช่องศา) ถ้าใช้เครื่องคิดเลขต้องตั้งโหมด RAD

ขั้นที่ 1 หาความถี่ของแต่ละคู่มิติ

- คู่ $i = 0$: ตัวหาร $10000^{0/4} = 1$
- คู่ $i = 1$: ตัวหาร $10000^{2/4} = 100$

ขั้นที่ 2 ตำแหน่ง pos = 1

- $(\sin 1, \cos 1, \sin 0.01, \cos 0.01) = (0.8415,0.5403,0.0100,1.0000)$

ขั้นที่ 3 ตำแหน่ง pos = 2

- $(\sin 2, \cos 2, \sin 0.02, \cos 0.02) = (0.9093, -0.4161,0.0200,0.9998)$

ขั้นที่ 4 บวกเข้ากับเวกเตอร์โทเคน

- $(0.5,0.5,0.5,0.5) + PE_1 = (1.3415,1.0403,0.5100,1.5000)$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

หาตัวหารของแต่ละคู่มิติ

- คู่ i = 0 (มิติที่ 0, 1): 10000^(0/4) = 10000^{0} = 1 มุมจึงเท่ากับ pos
- คู่ i = 1 (มิติที่ 2, 3): 10000^(2/4) = 10000^0.5 = 100 มุมจึงเท่ากับ pos/100

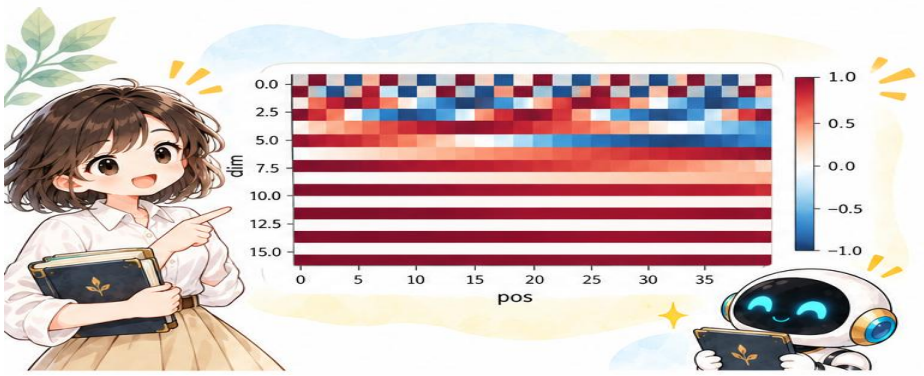
pos	มุมคู่แรก	มุมคู่ที่สอง	PE = (sin, cos, sin, cos)
0	0 rad	0.00 rad	(0.0000, 1.0000, 0.0000, 1.0000)
1	1 rad	0.01 rad	(0.8415, 0.5403, 0.0100, 1.0000)
2	2 rad	0.02 rad	(0.9093, −0.4161, 0.0200, 0.9998)

- บวกที่ pos = 1: (0.5 + 0.8415, 0.5 + 0.5403, 0.5 + 0.0100, 0.5 + 1.0000) = (1.3415, 1.0403, 0.5100, 1.5000)
- คู่แรกเปลี่ยนมากเมื่อ pos เพิ่มทีละ 1 (sin จาก 0 เป็น 0.8415 แล้ว 0.9093) ส่วนคู่ที่สองเปลี่ยนเพียงทีละประมาณ 0.01

ตีความ

คู่มิติแรกเปลี่ยนเร็วตามตำแหน่ง ส่วนคู่หลังเปลี่ยนช้า เหมือนเข็มวินาทีกับเข็มชั่วโมงของนาฬิกา ทุกตำแหน่งจึงมีลายเซ็นไม่ซ้ำกัน คำเดียวกันที่อยู่คนละตำแหน่งจึงมีเวกเตอร์ต่างกันเล็กน้อย

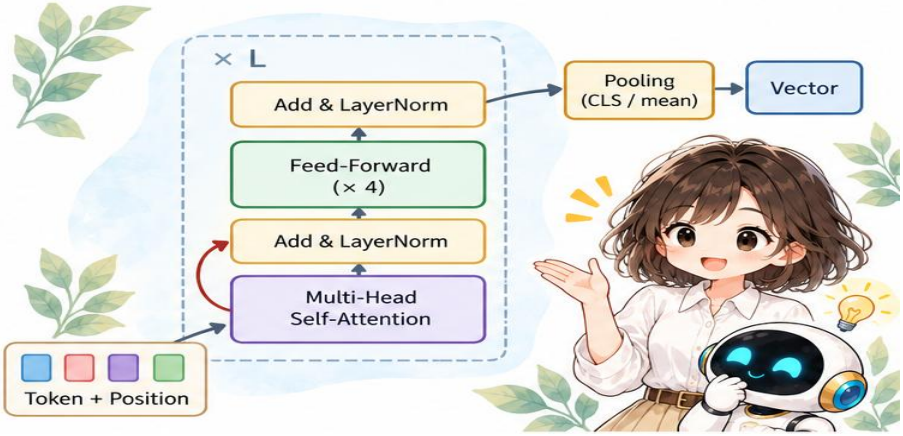
ภาพที่ 4.21 แสดงลวดลายของเวกเตอร์ตำแหน่ง 16 มิติสำหรับ 40 ตำแหน่ง มิติบนเปลี่ยนสีเร็ว มิติล่างเปลี่ยนช้า แบบจำลองสมัยใหม่หลายตัว เช่น BERT และ ViT ใช้ **ตำแหน่งที่เรียนรู้ได้ (Learned Position Embedding)** แทน ซึ่งก็คือตารางฝังตามหัวข้อ 4.2.5 ที่ใช้เลขตำแหน่งเป็นรหัส



ภาพที่ 4.21 ลวดลายการฝังตำแหน่งแบบไซน์–โคไซน์

4.4.5 บล็อก Transformer Encoder: Add & Norm และ FFN

Self-Attention เป็นเพียงส่วนหนึ่งของบล็อก ภาพที่ 4.22 แสดงโครงสร้างหนึ่งบล็อกของ Transformer Encoder ซึ่งซ้อนกัน L ชั้น แต่ละบล็อกมีสองชั้นย่อย คือ Multi-Head Self-Attention และ **เครือข่ายป้อนไปข้างหน้า (Feed-Forward Network: FFN)** ทั้งสองชั้นย่อยมีทางลัดและ Layer Normalization กำกับ เรียกรวมว่า **Add & Norm**



ภาพที่ 4.22 โครงสร้าง Transformer Encoder หนึ่งชั้น (ซ้อน L ชั้น)

$$\mathbf{z} = \mathrm{LN}\big(\mathbf{x} + \mathrm{MHA}(\mathbf{x})\big), \qquad \mathbf{h} = \mathrm{LN}\big(\mathbf{z} + \mathrm{FFN}(\mathbf{z})\big) \tag{4.25}$$

โดยที่

- $\mathbf{x}$ เวกเตอร์ของโทเคนก่อนเข้าบล็อก
- $\mathbf{x} + \mathrm{MHA}(\mathbf{x})$ ทางลัด (Residual Connection) แบบเดียวกับ ResNet ในหัวข้อ 4.3.5
- LN Layer Normalization ตามสมการ (4.26)
- $\mathbf{h}$ เวกเตอร์ของโทเคนหลังผ่านบล็อก

$$\mathrm{LN}(\mathbf{r}) = \gamma \odot \frac{\mathbf{r}-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta, \qquad \mu = \frac{1}{d}\sum_k r_k\,, \quad \sigma^2 = \frac{1}{d}\sum_k (r_k-\mu)^2 \tag{4.26}$$

โดยที่

- $\mathbf{r}$ เวกเตอร์ของโทเคนหนึ่งตัว
- μ, σ ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานของทุกมิติในเวกเตอร์นั้น

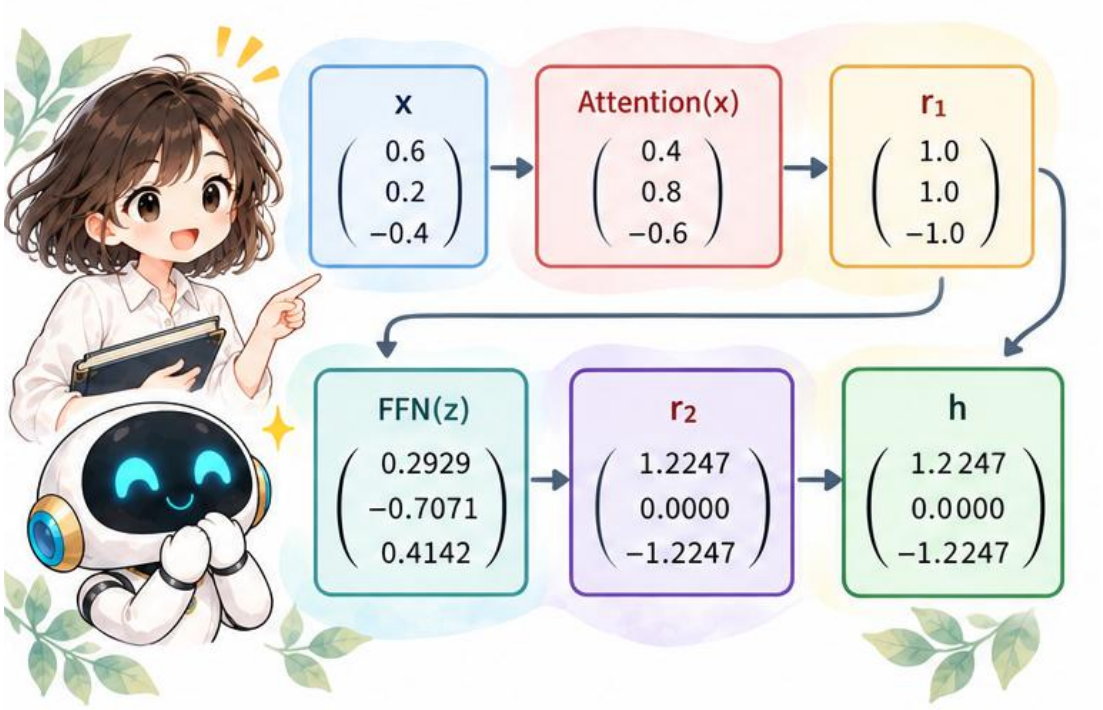
- γ, β พารามิเตอร์ปรับขนาดและเลื่อนที่เรียนรู้ได้ เริ่มที่ 1 และ 0
- ϵ ค่าบวกเล็กมาก เช่น 10^{-12} กันการหารด้วยศูนย์
- $\odot$ การคูณทีละมิติ

$$\mathrm{FFN}(\mathbf{z}) = \mathbf{W}_2\,\mathrm{ReLU}(\mathbf{W}_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2 \tag{4.27}$$

โดยที่

- $\mathbf{W}_1$ เมทริกซ์ขยายมิติจาก d เป็น d_{ff} ปกติ $d_{ff} = 4d$
- $\mathbf{W}_2$ เมทริกซ์ย่อมิติกลับเป็น d
- ReLU ฟังก์ชันกระตุ้น (BERT และ ViT ใช้ GELU ซึ่งคล้ายกัน)

ภาพที่ 4.23 ติดตามตัวเลขของโทเคนหนึ่งตัวผ่านทุกชั้นย่อยของบล็อก โดยใช้เวกเตอร์สามมิติเพื่อให้คำนวณด้วยมือได้



ภาพที่ 4.23 ตัวเลขของโทเคนหนึ่งตัวผ่านหนึ่งบล็อกของ Transformer Encoder

ตัวอย่างการคำนวณ 4.19 Add & Norm ของโทเคนหนึ่งตัว

โจทย์ โทเคนมีเวกเตอร์ $\mathbf{x} = (0.6, 0.2, -0.4)$ และ Attention ให้ $(0.4, 0.8, -0.6)$ ต่อมา FFN ให้ $(0.2929, -0.7071, 0.4142)$ จงคำนวณ Add & Norm ทั้งสองครั้ง ($\gamma = 1$, $\beta = 0$, ละ ϵ)

สมการที่ใช้ สมการ (4.25) และ (4.26)

ข้อมูลที่ใช้ ตัวเลขในภาพที่ 4.23 (ค่าสมมติ)

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- เวกเตอร์โทเคน x, ผลของ Attention และผลของ FFN เป็นค่าสมมติสามมิติในภาพที่ 4.23 เลือกให้ผลรวมหลังทางลัดเป็นเลขกลม (1, 1, −1) และ (1, 0, −1) เพื่อให้เห็นขั้นตอนชัด
- ค่าเริ่มต้น γ = 1 และ β = 0 ตรงกับค่าตั้งต้นของ Layer Normalization ในไลบรารีจริง

ขั้นที่ 1 ทางลัดครั้งที่ 1

- $\mathbf{r}_1 = \mathbf{x} + \mathrm{Attention} = (0.6 + 0.4,\ 0.2 + 0.8,\ -0.4 - 0.6) = (1.0, 1.0, -1.0)$

ขั้นที่ 2 Layer Normalization ครั้งที่ 1

- $\mu = (1 + 1 - 1)/3 = 0.3333$
- $\sigma^2 = \left[(0.6667)^2 + (0.6667)^2 + (-1.3333)^2\right]/3 = 0.8889$ จึงได้ $\sigma = 0.9428$
- $\mathbf{z} = (\mathbf{r}_1 - \mu)/\sigma = (0.7071, 0.7071, -1.4142)$

ขั้นที่ 3 ทางลัดครั้งที่ 2

- $\mathbf{r}_2 = \mathbf{z} + \text{FFN}(\mathbf{z}) = (0.7071 + 0.2929,\ 0.7071 - 0.7071,\ -1.4142 + 0.4142) = (1.0, 0, -1.0)$

ขั้นที่ 4 Layer Normalization ครั้งที่ 2

- $\mu = 0,\ \sigma = \sqrt{2/3} = 0.8165$
- $\mathbf{h} = (1.2247, 0.0000, -1.2247)$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

Layer Normalization ครั้งที่ 1 ทีละมิติ (r1 = (1.0, 1.0, −1.0))

มิติ	r	r − μ	(r − μ)²	(r − μ)/σ
1	1.0	0.6667	0.4444	0.7071
2	1.0	0.6667	0.4444	0.7071
3	−1.0	−1.3333	1.7778	−1.4142
รวม/เฉลี่ย	**μ** = 0.3333	0	**σ²** = 2.6667/3 = 0.8889	**σ** = 0.9428

Layer Normalization ครั้งที่ 2 (r2 = (1.0, 0.0, −1.0))

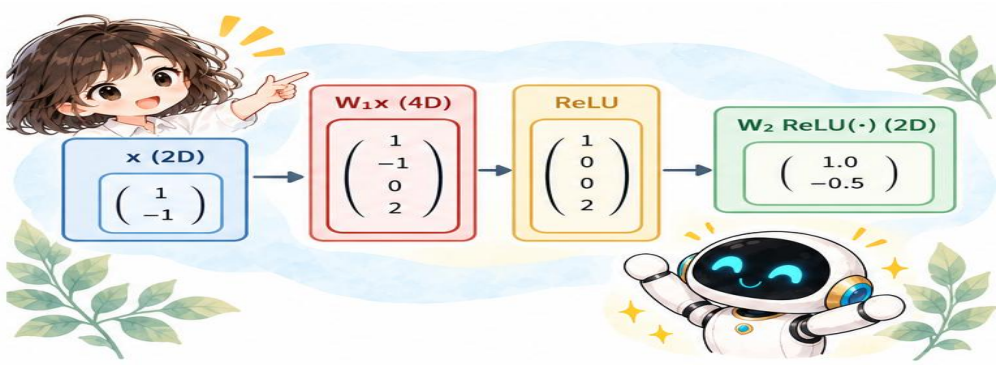
มิติ	r	r − μ	(r − μ)²	(r − μ)/σ
1	1.0	1.0	1.0	1.2247
2	0.0	0.0	0.0	0.0000
3	−1.0	−1.0	1.0	−1.2247
รวม/เฉลี่ย	**μ** = 0	0	**σ²** = 2/3 = 0.6667	**σ** = 0.8165

- ตรวจผล: ค่าเฉลี่ยของ h = (1.2247 + 0 − 1.2247)/3 = 0 และความแปรปรวน = (1.5 + 0 + 1.5)/3 = 1 ตามที่ Layer Normalization รับประกัน

ตีความ

ทางลัดทำให้ข้อมูลเดิม $\mathbf{x}$ ไม่หายไปไหน ชั้นย่อยเพียง "เพิ่มเติม" ส่วนที่เรียนรู้ ส่วน Layer Normalization ทำให้ทุกเวกเตอร์มีค่าเฉลี่ย 0 และส่วนเบี่ยงเบน 1 เสมอ ค่าจึงไม่ระเบิดหรือหดหายเมื่อซ้อนหลายสิบชั้น

ส่วน FFN ทำงานกับโทเคนทีละตัวโดยไม่ดูโทเคนอื่น หน้าที่ของมันคือแปลงข้อมูลที่ Attention รวบรวมมาให้เป็นคุณลักษณะใหม่ ภาพที่ 4.24 แสดงสามขั้นของ FFN ขนาดจิ๋วที่ขยายจาก 2 เป็น 4 มิติแล้วย่อกลับ



ภาพที่ 4.24 การทำงานสามขั้นของ FFN ขนาดจิ๋ว

ตัวอย่างการคำนวณ 4.20 FFN ขยาย 2 → 4 แล้วย่อกลับ

โจทย์ ให้ $\mathbf{z} = (1, -1)$, $\mathbf{W}_1 = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 1.0 \\ 1.0 & -1.0 \end{bmatrix}$, $\mathbf{W}_2 = \begin{bmatrix} 0.2 & 0.0 & 0.0 & 0.4 \\ 0.5 & 0.0 & 0.0 & -0.5 \end{bmatrix}$

และค่าเอนเอียงเป็นศูนย์ จงหา $\mathrm{FFN}(\mathbf{z})$

สมการที่ใช้ สมการ (4.27)

ข้อมูลที่ใช้ ค่าในภาพที่ 4.24 (ค่าสมมติ)

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- z = (1, −1) และเมทริกซ์ W1, W2 เป็นค่าสมมติในภาพที่ 4.24 ขนาด 2 ⟶ 4 ⟶ 2 ย่อมาจากอัตรา dff = 4d ของ Transformer จริง (768 ⟶ 3,072 ⟶ 768)

ขั้นที่ 1 ขยายมิติ

- $\mathbf{W}_1\mathbf{z}$: แถวที่ 1–4 ได้ $1, -1, 1-1=0$ และ $1+1=2$ จึงได้ $(1, -1, 0, 2)$

ขั้นที่ 2 ReLU

- ตัดค่าลบ: $(1, 0, 0, 2)$

ขั้นที่ 3 ย่อกลับ

- แถวที่ 1: $0.2(1) + 0.4(2) = 1.0$
- แถวที่ 2: $0.5(1) - 0.5(2) = -0.5$
- จึงได้ $(1.0, -0.5)$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

แถวของ W1	คูณกับ z = (1, −1)	ก่อน ReLU	หลัง ReLU
(1, 0)	1 × 1 + 0 × (−1)	1	1
(0, 1)	0 × 1 + 1 × (−1)	−1	0
(1, 1)	1 × 1 + 1 × (−1)	0	0
(1, −1)	1 × 1 + (−1) × (−1)	2	2

- แถวที่ 1 ของ W2 = (0.2, 0, 0, 0.4): 0.2 × 1 + 0 × 0 + 0 × 0 + 0.4 × 2 = 0.2 + 0.8 = 1.0
- แถวที่ 2 ของ W2 = (0.5, 0, 0, −0.5): 0.5 × 1 + 0 + 0 + (−0.5) × 2 = 0.5 − 1.0 = −0.5
- ReLU ตัดสองช่องกลางทิ้ง ผลจึงขึ้นกับคุณลักษณะที่ 1 และ 4 เท่านั้น

ตีความ

การขยายมิติชั่วคราวเปิดพื้นที่ให้ ReLU "เลือก" ว่าคุณลักษณะใดควรผ่าน
คล้ายการกางแผนที่ออกก่อนเลือกเส้นทาง แล้วจึงพับกลับให้มีขนาดเดิมเพื่อส่งต่อบล็อกถัดไป

ที่มาของไฮเปอร์พารามิเตอร์ จำนวนชั้น L และขนาด d_ff มาจากไหน

- **ที่มา** BERT-base และ ViT-B ใช้ $L = 12$ ชั้น และ $d_{ff} = 4d = 3{,}072$ ตามงานต้นฉบับของ Vaswani et al. (2017) ซึ่งใช้ $d = 512$, $d_{ff} = 2{,}048$
- **ผลเมื่อเปลี่ยน** ชั้นมากขึ้นเข้าใจความสัมพันธ์ที่ซับซ้อนขึ้น แต่ช้าและต้องใช้ข้อมูลมาก แบบจำลองขนาดเล็กอย่าง MiniLM ใช้ 6 ชั้น และ $d = 384$ จึงเร็วกว่าหลายเท่า
- **วิธีเลือก** สำหรับระบบผู้แนะนำ ให้เริ่มจากแบบจำลองขนาดเล็กที่ฝึกไว้แล้ว เพราะเวกเตอร์ของรายการคำนวณล่วงหน้าเพียงครั้งเดียว แล้วจึงลองแบบจำลองใหญ่ขึ้นถ้าตัวชี้วัดยังไม่พอ

โปรแกรม 4.9 หนึ่งบล็อกของ Transformer: Add & Norm และ FFN

```
import numpy as np

def layer_norm(r, eps=1e-12):
    return (r - r.mean()) / np.sqrt(r.var() + eps)

x = np.array([0.6, 0.2, -0.4])
att = np.array([0.4, 0.8, -0.6])           # ผลของ Attention
z = layer_norm(x + att)                     # Add & Norm 1
ffn = np.array([0.2929, -0.7071, 0.4142])   # ผลของ FFN
h = layer_norm(z + ffn)                     # Add & Norm 2
print('z =', np.round(z, 4))
print('h =', np.round(h, 4))

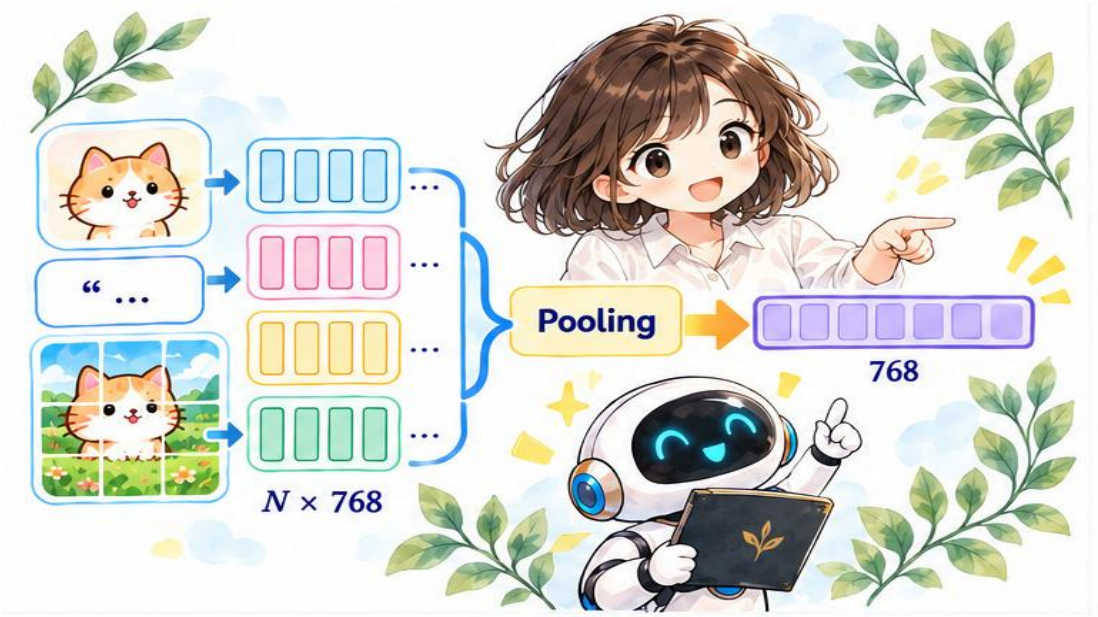
W1 = np.array([[1, 0], [0, 1], [1, 1], [1, -1]])
W2 = np.array([[0.2, 0, 0, 0.4], [0.5, 0, 0, -0.5]])
print('FFN(1,-1) =', W2 @ np.maximum(W1 @ np.array([1, -1]), 0))
```

ผลลัพธ์

```
z = [ 0.7071  0.7071 -1.4142]
h = [ 1.2247  0.     -1.2247]
FFN(1,-1) = [ 1.  -0.5]
```

4.4.6 Pooling และการฉาย: จากหลายเวกเตอร์สู่เวกเตอร์เดียว

ผลของ Transformer Encoder คือเวกเตอร์ **หนึ่งเส้นต่อหนึ่งโทเคน** แต่ระบบผู้แนะนำต้องการเวกเตอร์ **หนึ่งเส้นต่อหนึ่งรายการ** ภาพที่ 4.25 แสดงแนวคิดของ Pooling ซึ่งสรุปเวกเตอร์หลายเส้นเป็นเส้นเดียว วิธีที่นิยมมีสี่แบบตามสมการ (4.28)–(4.29)



ภาพที่ 4.25 แนวคิดของ Pooling: จากหลายเวกเตอร์สู่เวกเตอร์เดียว

$$\mathbf{v}_{mean} = \frac{1}{n}\sum_{t=1}^{n} \mathbf{h}_t\,, \qquad \mathbf{v}_{max}[j] = \max_t h_t[j] \tag{4.28}$$

โดยที่

- $\mathbf{h}_t$ เวกเตอร์ตามบริบทของโทเคนที่ t จาก n โทเคน
- j ลำดับมิติ

$$\mathbf{v}_{cls} = \mathbf{h}_{[CLS]}, \qquad \mathbf{v}_w = \sum_{t=1}^{n} a_t\ \mathbf{h}_t,\ \sum_t a_t = 1 \tag{4.29}$$

โดยที่

- $\mathbf{h}_{[CLS]}$ เวกเตอร์ของโทเคนพิเศษ [CLS] ที่วางไว้หน้าลำดับและถูกฝึกให้สรุปทั้งลำดับ
- a_t น้ำหนักของโทเคน t เช่น ค่า IDF หรือน้ำหนักที่เรียนรู้

ภาพที่ 4.26 เปรียบเทียบสี่วิธีด้วยเวกเตอร์โทเคนสี่ตัวจากเรื่องย่อของ i1 Toy Story 5 คือ toys, tablet, playtime และ friendship



ภาพที่ 4.26 Pooling สี่วิธีจากเวกเตอร์โทเคนของ Toy Story 5

ตัวอย่างการคำนวณ 4.21 Pooling และการฉายของเวกเตอร์โทเคนจาก Toy Story 5

โจทย์ จงหา Mean Pooling และ Max Pooling ของเวกเตอร์โทเคนสี่ตัวในภาพที่ 4.26 แล้วฉายผล Mean Pooling ลงสองมิติด้วย $\mathbf{W} = \begin{bmatrix} 1.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 1.0 \end{bmatrix}$, $\mathbf{b} = (0.05, -0.05)$ และปรับความยาวเป็น 1

สมการที่ใช้ สมการ (4.28) และ (4.30)

ข้อมูลที่ใช้ เวกเตอร์โทเคนจากภาพที่ 4.26 (ค่าสมมติ 4 มิติ)

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- โทเคนทั้งสี่ toys, tablet, playtime และ friendship พบจริงในเรื่องย่อ i1 Toy Story 5 (ตารางที่ 4.6): toys พบ 3 ครั้ง ส่วนคำอื่นพบคำละครั้ง
- เวกเตอร์ตามบริบทสี่มิติของแต่ละโทเคน (h1–h4 ในภาพที่ 4.26) และเมทริกซ์ฉาย W, ค่าเอนเอียง b เป็นค่าสมมติ

ขั้นที่ 1 Mean Pooling

- มิติที่ 1: $(0.1 + 0.5 + 0.3 + 0.0)/4 = 0.225$
- มิติที่ 2: $(0.4 + 0.2 + 0.8 + 0.1)/4 = 0.375$
- มิติที่ 3 และ 4 ได้ 0.300 และ 0.275 จึงได้ $\mathbf{v}_{mean} = (0.225, 0.375, 0.300, 0.275)$

ขั้นที่ 2 Max Pooling

- เลือกค่ามากสุดทีละมิติ: $\mathbf{v}_{max} = (0.5, 0.8, 0.6, 0.7)$

ขั้นที่ 3 ฉายเชิงเส้น

- $q_1 = 0.225 + 0.300 + 0.05 = 0.575$, $q_2 = 0.375 + 0.275 - 0.05 = 0.600$

ขั้นที่ 4 ปรับความยาว

- $\| \mathbf{q} \| = \sqrt{0.575^2 + 0.600^2} = 0.8310$
- $\mathbf{v} = (0.6919, 0.7220)$ ความยาวเท่ากับ 1

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

โทเคน	มิติ 1	มิติ 2	มิติ 3	มิติ 4
h1 toys	0.1	0.4	0.2	0.3
h2 tablet	0.5	0.2	0.1	0.0
h3 playtime	0.3	0.8	0.6	0.1
h4 friendship	0.0	0.1	0.3	0.7
ผลรวม	0.9	1.5	1.2	1.1
Mean = ผลรวม/4	0.225	0.375	0.300	0.275
Max	0.5	0.8	0.6	0.7

- ตัวอย่างการอ่านตาราง: มิติที่ 3 รวม 0.2 + 0.1 + 0.6 + 0.3 = 1.2 หารด้วย 4 ได้ 0.300 และค่ามากสุดคือ 0.6 จาก playtime
- Weighted Pooling (ภาพที่ 4.26 ช่องที่ 4) ใช้น้ำหนัก $\boldsymbol{\alpha}$ = (0.2, 0.1, 0.5, 0.2): มิติที่ 1 = 0.2×0.1 + 0.1×0.5 + 0.5×0.3 + 0.2×0 = 0.22 ได้ (0.22, 0.52, 0.41, 0.25)

ฉายและปรับความยาว

- แถวที่ 1 ของ W = (1, 0, 1, 0) เลือกมิติ 1 และ 3: q1 = 0.225 + 0.300 + 0.05 = 0.575
- แถวที่ 2 ของ W = (0, 1, 0, 1) เลือกมิติ 2 และ 4: q2 = 0.375 + 0.275 − 0.05 = 0.600
- $\|q\|$ = √(0.575² + 0.600²) = √(0.3306 + 0.3600) = √0.6906 = 0.8310 ได้ v = (0.575/0.8310, 0.600/0.8310) = (0.6919, 0.7220)

ตีความ

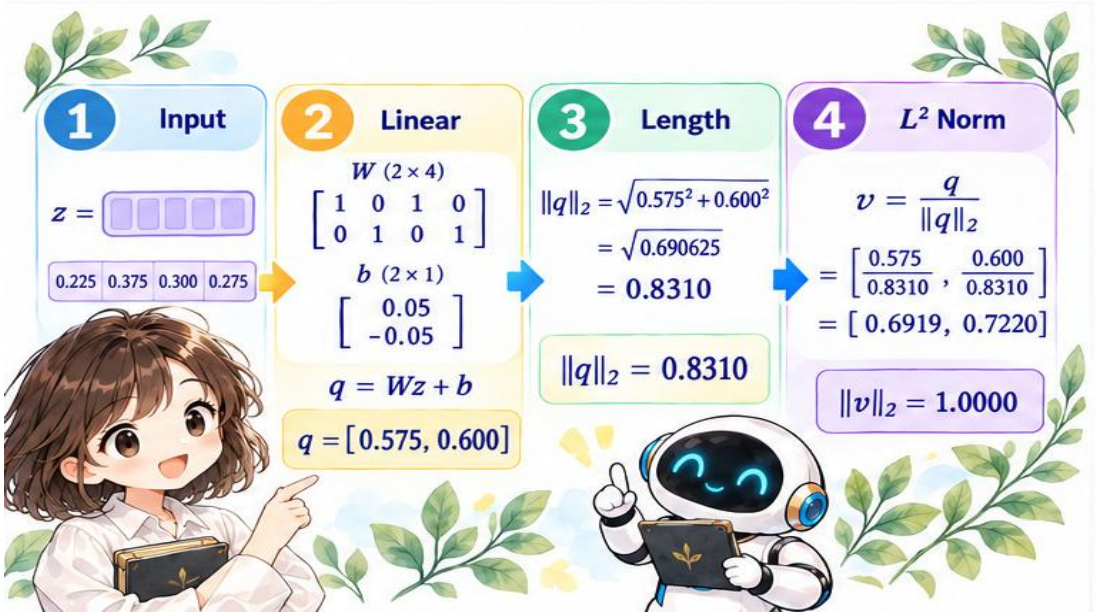
Mean Pooling ให้ทุกโทเคนมีส่วนเท่ากัน Max Pooling เก็บสัญญาณเด่นที่สุดของแต่ละมิติ ส่วนการฉายและปรับความยาวทำให้ได้เวกเตอร์ที่มีมิติตามต้องการและวัดความคล้ายด้วยโคไซน์ได้ทันที

$$\mathbf{q} = \mathbf{W}\mathbf{v} + \mathbf{b}, \qquad \hat{\mathbf{v}} = \frac{\mathbf{q}}{\|\mathbf{q}\|} \tag{4.30}$$

โดยที่

- $\mathbf{W}$ เมทริกซ์ฉายขนาด $d_{out} \times d$ ซึ่งเรียนรู้ได้
- $\mathbf{b}$ ค่าเอนเอียง
- $\hat{\mathbf{v}}$ เวกเตอร์ของรายการหลังปรับความยาวเป็น 1 (L^2 Normalization)

ภาพที่ 4.27 สรุปขั้นตอนการฉายและการปรับความยาวในตัวอย่างการคำนวณ 4.21 ตั้งแต่เวกเตอร์ที่ผ่าน Pooling จนได้เวกเตอร์ของรายการ



ภาพที่ 4.27 การฉายเชิงเส้นและการปรับ L^2 ของเวกเตอร์ข้อความ

แนวทางเลือกใช้ เลือกวิธี Pooling แบบใด

- **[CLS]** ใช้เมื่อแบบจำลองถูกฝึกให้ [CLS] สรุปทั้งลำดับ เช่น ViT และ BERT ที่ปรับมาสำหรับงานจำแนก
- **Mean Pooling** เป็นค่าเริ่มต้นที่ดีสำหรับเวกเตอร์ประโยค Sentence-BERT แสดงว่าให้ผลดีกว่า [CLS] ในงานวัดความคล้าย (Reimers and Gurevych, 2019)
- **Max Pooling** เหมาะเมื่อสนใจว่ามีคุณลักษณะเด่นปรากฏหรือไม่ เช่น Text CNN ในหัวข้อ 4.3.6
- **Weighted Pooling** ใช้เมื่อรู้ว่าโทเคนบางตัวสำคัญกว่า เช่น ถ่วงด้วย IDF ของบทที่ 3

4.4.7 การประยุกต์ Transformer กับข้อความ ภาพ เสียง และวิดีโอ

บล็อก Transformer ในหัวข้อ 4.4.2–4.4.6 เหมือนกันทุกสื่อ ส่วนที่ต่างกันคือ **วิธีแบ่งข้อมูลเป็นโทเคน** ซึ่งกำหนดจำนวนโทเคน N และต้นทุนของ Self-Attention ที่ต้องคำนวณทุกคู่ ดังสมการ (4.31) และ (4.32)

$$N_{img} = \frac{H}{P} \times \frac{W}{P}, \qquad N_{aud} = \left(\left\lfloor \frac{F-P}{s} \right\rfloor + 1\right) \times \left(\left\lfloor \frac{T-P}{s} \right\rfloor + 1\right), \qquad N_{vid} = \frac{T_v}{t} \times \frac{H}{P} \times \frac{W}{P} \quad (4.31)$$

โดยที่

- H, W ความสูงและความกว้างของภาพหรือเฟรม
- P ขนาดแพตช์ เช่น 16 พิกเซล
- F, T จำนวนแถบความถี่และจำนวนเฟรมเวลาของสเปกโทรแกรม
- s ระยะก้าวของแพตช์เสียง
- T_v, t จำนวนเฟรมวิดีโอ และความยาวตามเวลาของหนึ่งก้อน (Tubelet)

$$C_{attn} = (N+1)^2 \quad (4.32)$$

โดยที่

- $N+1$ จำนวนโทเคนรวมโทเคน [CLS]
- C_{attn} จำนวนคู่ที่ต้องคำนวณคะแนนต่อหนึ่งหัวต่อหนึ่งชั้น

ตารางที่ 4.14 สรุปเทคนิค Transformer ที่แนะนำสำหรับแต่ละสื่อพร้อมไลบรารี Python

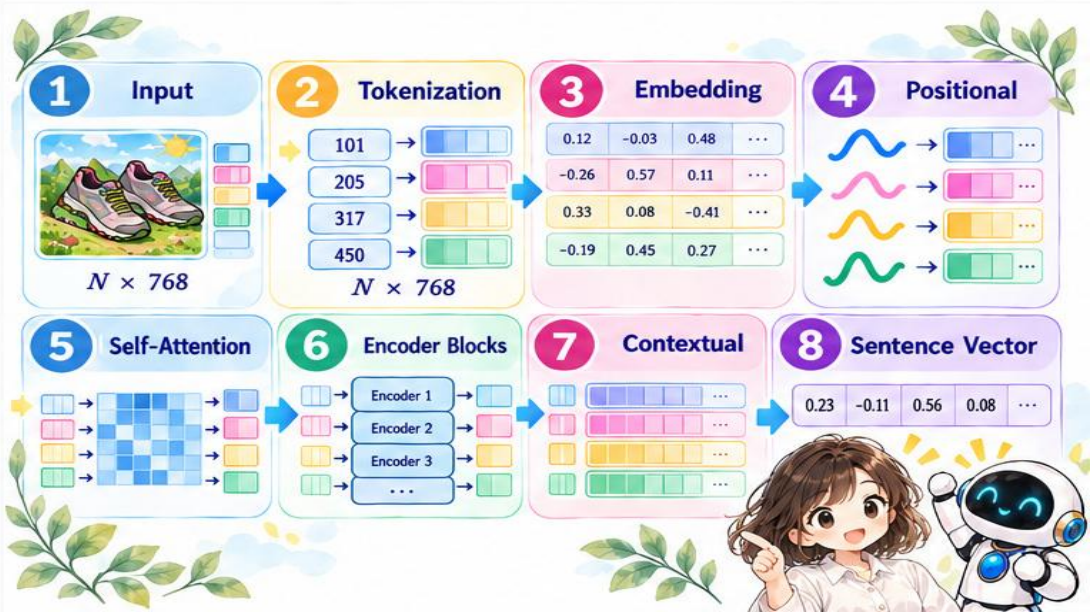
ตารางที่ 4.14 สรุปการประยุกต์ Transformer กับสื่อและไลบรารี Python

สื่อ	แบ่งเป็นโทเคน	แบบจำลองที่นิยม	เวกเตอร์ของรายการ	ไลบรารี Python
ข้อความ	คำย่อย (WordPiece, BPE)	BERT, Sentence-BERT, MiniLM, E5	Mean Pooling หรือ [CLS]	transformers, sentence-transformers
ภาพ	แพตช์ $P \times P$ ฉายเชิงเส้น	ViT, Swin, DINOv2, CLIP	[CLS]	transformers, timm, open_clip
เสียง	แพตช์ของ Log-Mel	AST, wav2vec 2.0, HuBERT, CLAP	[CLS] หรือ Mean Pooling	transformers (ASTModel), torchaudio
วิดีโอ	ก้อนเวลา × พื้นที่ (Tubelet)	ViViT, TimeSformer, VideoMAE, X-CLIP	Mean Pooling หรือ [CLS]	transformers (VideoMAEModel), decord

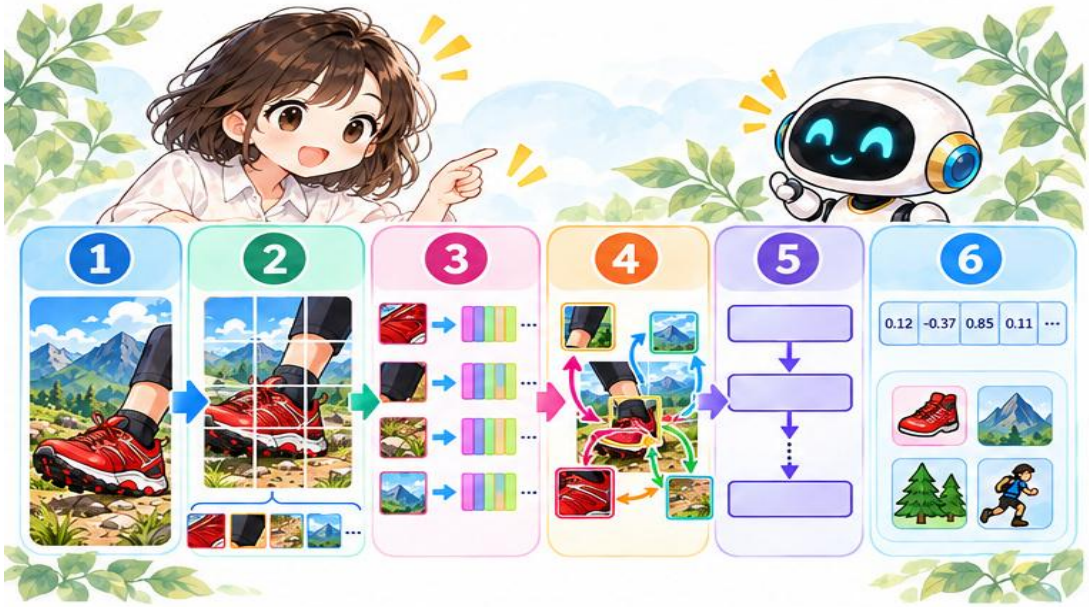
ที่มา: Devlin et al. (2019), Reimers and Gurevych (2019), Dosovitskiy et al. (2021), Gong et al. (2021), Bertasius et al. (2021), Arnab et al. (2021), Tong et al. (2022)

รายละเอียดโดยสังเขปของแต่ละสื่อมีดังนี้

- **ข้อความ** ตัดเรื่องย่อเป็นคำย่อย แล้วเปิดตารางฝังตามหัวข้อ 4.2.5 บวกตำแหน่ง ผ่านบล็อก 6–12 ชั้น คำเดียวกันในคนละเรื่องจึงได้เวกเตอร์ต่างกันตามบริบท (ภาพที่ 4.28)
- **ภาพ (Vision Transformer: ViT)** ตัดภาพเป็นแพตช์ 16 × 16 พิกเซล คลี่แต่ละแพตช์เป็นเวกเตอร์ $16 \times 16 \times 3 = 768$ ค่า แล้วฉายเชิงเส้นเป็นเวกเตอร์ D มิติ ซึ่งเรียกว่า Patch Embedding (ภาพที่ 4.29)
- **เสียง (Audio Spectrogram Transformer: AST)** ใช้ Log-Mel Spectrogram แบบเดียวกับ CNN ในหัวข้อ 4.3.6 แต่ตัดเป็นแพตช์ 16 × 16 ที่เลื่อนทีละ 10 ช่องให้ซ้อนกัน แล้วปฏิบัติต่อแพตช์เหมือนแพตช์ภาพ (ภาพที่ 4.30)
- **วิดีโอ** ตัดเป็นก้อนสามมิติ (Tubelet) เช่น 2 เฟรม × 16 × 16 พิกเซล เพื่อลดจำนวนโทเคน และใช้ **Divided Attention** คือทำ Attention ตามพื้นที่ภายในเฟรมและตามเวลาข้ามเฟรมแยกกัน (ภาพที่ 4.31)



ภาพที่ 4.28 Text Transformer Encoder แปดขั้น



ภาพที่ 4.29 ภาพรวมหกขั้นของ Image Transformer Encoder



ภาพที่ 4.30 Audio Transformer Encoder แปดขั้น



ภาพที่ 4.31 Video Transformer Encoder เจ็ดขั้น

ตัวอย่างการคำนวณ 4.22 จำนวนโทเคนและต้นทุน Attention ของสี่สื่อ

โจทย์ จงหาจำนวนโทเคนและจำนวนคู่ของ Attention ต่อหัวต่อชั้นของโปสเตอร์ คลิปเสียง และตัวอย่างภาพยนตร์ตามตารางที่ 4.8 แล้วเทียบกับเรื่องย่อ 60 โทเคน

สมการที่ใช้ สมการ (4.31) และ (4.32)

ข้อมูลที่ใช้ รูปแบบข้อมูลจากตารางที่ 4.8 ค่าแพตช์ตาม ViT-B/16 (Dosovitskiy et al., 2021) AST (Gong et al., 2021) และ ViViT (Arnab et al., 2021)

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- รูปแบบข้อมูลนำเข้า (เรื่องย่อ 60 โทเคน โปสเตอร์ 224 × 224 Log-Mel 128 × 1,024 และ 16 เฟรม) มาจากตารางที่ 4.8 ชุดเดียวกับตัวอย่างการคำนวณ 4.2 และ 4.13

- ขนาดแพตช์ 16 ของภาพ ระยะก้าว 10 ของเสียง และ Tubelet 2 × 16 × 16 ของวิดีโอ มาจาก ViT-B/16, AST และ ViViT ตามลำดับ

ขั้นที่ 1 ข้อความ

- $N = 60$ โทเคนรวม [CLS] เป็น 61 จำนวนคู่ $61^2 = 3{,}721$

ขั้นที่ 2 ภาพ (P = 16)

- $N = (224/16)^2 = 14^2 = 196$ แพตช์ รวม [CLS] เป็น 197
- จำนวนคู่ $197^2 = 38{,}809$
- แต่ละแพตช์คลี่ได้ $16 \times 16 \times 3 = 768$ ค่า ก่อนฉายเป็นเวกเตอร์ $D = 768$ มิติ

ขั้นที่ 3 เสียง (P = 16, s = 10)

- แกนความถี่: $\lfloor (128 - 16)/10 \rfloor + 1 = 12$ แกนเวลา: $\lfloor (1{,}024 - 16)/10 \rfloor + 1 = 101$
- $N = 12 \times 101 = 1{,}212$ แพตช์ ตรงกับค่าที่รายงานใน AST
- จำนวนคู่ $(1{,}212 + 1)^2 = 1{,}471{,}369$

ขั้นที่ 4 วิดีโอ (Tubelet 2 × 16 × 16)

- $N = (16/2) \times 14 \times 14 = 8 \times 196 = 1{,}568$ โทเคน
- Attention ทุกคู่ (Joint): $1{,}568^2 = 2{,}458{,}624$ คู่
- Divided Attention: แต่ละโทเคนเทียบ 196 โทเคนในเฟรมเดียวกัน และ 8 โทเคนตำแหน่งเดียวกันข้ามเวลา: $1{,}568 \times (196 + 8) = 319{,}872$ คู่

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

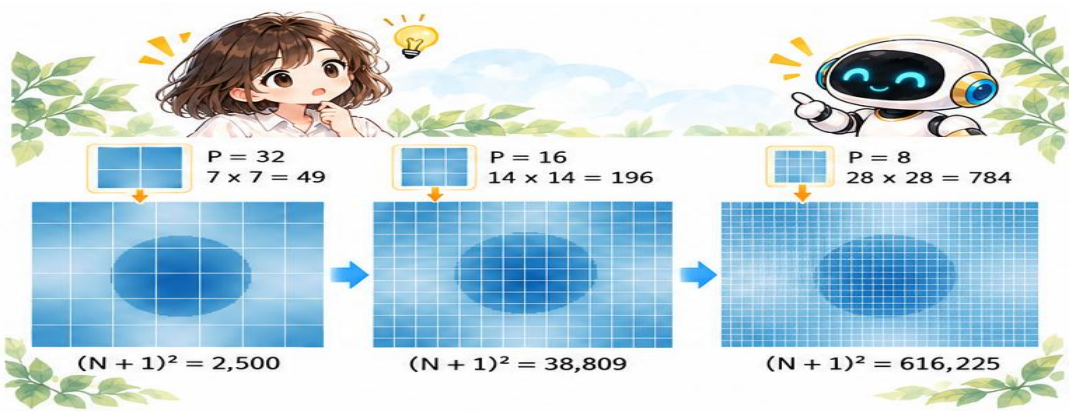
สื่อ	จำนวนโทเคน	+ [CLS]	คู่ Attention
ข้อความ	60	61	$61^2 = 3{,}721$
ภาพ	(224/16) × (224/16) = 14 × 14 = 196	197	$197^2 = 38{,}809$
เสียง	$\lfloor (128-16)/10 \rfloor + 1 = 12$ และ $\lfloor (1{,}024-16)/10 \rfloor + 1 = 101$ ⟶ 12 × 101 = 1,212	1,213	$1{,}213^2 = 1{,}471{,}369$
วิดีโอ (Joint)	(16/2) × 14 × 14 = 8 × 196 = 1,568	1,568	$1{,}568^2 = 2{,}458{,}624$
วิดีโอ (Divided)	1,568 โทเคน × (196 + 8) คู่ต่อโทเคน		319,872

- แกนความถี่ของเสียง: (128 − 16) = 112 หารด้วย 10 ได้ 11.2 ปัดลงเป็น 11 แล้วบวก 1 = 12 · แกนเวลา: (1,024 − 16) = 1,008 หารด้วย 10 ได้ 100.8 ปัดลงเป็น 100 บวก 1 = 101
- อัตราส่วน: 2,458,624 / 3,721 = 660.7 ≈ 661 เท่า และ 2,458,624 / 319,872 = 7.69 ≈ 7.7 เท่า

ตีความ

ต้นทุนของ Self-Attention โตเป็นกำลังสองของจำนวนโทเคน วิดีโอแบบคำนวณทุกคู่จึงแพงกว่าข้อความถึง 661 เท่า เทคนิคเฉพาะสื่อ เช่น แพตช์ขนาดใหญ่ Tubelet และ Divided Attention จึงมีไว้เพื่อลดจำนวนคู่ (Divided ลดได้ประมาณ 7.7 เท่า)

ภาพที่ 4.32 แสดงผลของขนาดแพตช์ต่อจำนวนแพตช์และจำนวนคู่ Attention ของภาพ 224 × 224 ซึ่งเป็นไฮเปอร์พารามิเตอร์สำคัญของ ViT



ภาพที่ 4.32 จำนวนแพตช์และคู่ Attention เมื่อเปลี่ยนขนาดแพตช์ P

ที่มาของไฮเปอร์พารามิเตอร์ ขนาดแพตช์ P และระยะก้าวของแพตช์เสียงมาจากไหน

- **ที่มา** ViT ทดลอง $P = 14,16,32$ และพบว่า 16 สมดุลระหว่างความละเอียดกับต้นทุน ชื่อ ViT-B/16 จึงหมายถึง Base ที่ใช้แพตช์ 16 (Dosovitskiy et al., 2021) AST ใช้แพตช์ 16 ระยะก้าว 10 ให้ซ้อนกัน 6 ช่องเพื่อไม่ให้เหตุการณ์เสียงถูกตัดขาดที่ขอบแพตช์ (Gong et al., 2021)
- **ผลเมื่อเปลี่ยน** P เล็กลงครึ่งหนึ่งทำให้แพตช์เพิ่มสี่เท่าและคู่ Attention เพิ่มประมาณสิบหกเท่า (ภาพที่ 4.32) ได้รายละเอียดมากขึ้นแต่ช้าลงมาก
- **วิธีเลือก** ใช้ค่าตามแบบจำลองที่ฝึกไว้แล้ว เพราะ Patch Embedding ถูกฝึกกับขนาดนั้น การเปลี่ยนขนาดภาพนำเข้าทำได้โดยประมาณค่าเวกเตอร์ตำแหน่งใหม่

ไลบรารี Python ที่ใช้ได้ แนวทางเรียกใช้ Transformer ที่ฝึกไว้แล้วด้วย Python

โค้ดต่อไปนี้เป็นแนวทางสำหรับระบบจริง ต้องติดตั้งไลบรารีและดาวน์โหลดน้ำหนักของแบบจำลองก่อน จึงไม่ได้รันในหนังสือ

```
# ข้อความ: Sentence-BERT ขนาดเล็ก ได้เวกเตอร์ 384 มิติ
from sentence_transformers import SentenceTransformer
st = SentenceTransformer("all-MiniLM-L6-v2")
v_txt = st.encode(synopses, normalize_embeddings=True)


# ภาพ: ViT-B/16 ใช้เวกเตอร์ของโทเคน [CLS]
from transformers import AutoImageProcessor, ViTModel
proc = AutoImageProcessor.from_pretrained("google/vit-base-patch16-224")
vit = ViTModel.from_pretrained("google/vit-base-patch16-224")
out = vit(**proc(images=poster, return_tensors="pt"))
v_img = out.last_hidden_state[:, 0]       # (1, 768)


# เสียง: Audio Spectrogram Transformer
from transformers import ASTFeatureExtractor, ASTModel
name = "MIT/ast-finetuned-audioset-10-10-0.4593"
ast = ASTModel.from_pretrained(name)
fx = ASTFeatureExtractor.from_pretrained(name)
v_aud = ast(**fx(wave, sampling_rate=16000,
                 return_tensors="pt")).pooler_output


# วิดีโอ: VideoMAE เฉลี่ยเวกเตอร์ของทุกโทเคน
from transformers import VideoMAEModel
vm = VideoMAEModel.from_pretrained("MCG-NJU/videomae-base")
v_vid = vm(pixel_values=clip).last_hidden_state.mean(1)
```

4.4.8 เวกเตอร์รายการจากเทคนิค Transformer และการเปรียบเทียบกับ CNN

หัวข้อนี้กำหนดเวกเตอร์ของภาพยนตร์ทั้ง 12 เรื่องที่ได้จากเทคนิค Transformer ในขนาดย่อสำหรับการสอน ตารางที่ 4.15 คือเวกเตอร์ข้อความตามบริบท 12 มิติ ใช้คลังแนวคิดเดียวกับตารางที่ 4.12 ผู้อ่านจึงเทียบสองตารางได้ทีละช่อง ส่วนตารางที่ 4.16 คือเวกเตอร์ภาพ เสียง และวิดีโอสามมิติ

ตารางที่ 4.15 เวกเตอร์ข้อความ 12 มิติจากเทคนิค Transformer

รหัส	tech	mind	spac	rom	crim	act	fam	mus	fant	toys	soc	amb
i1	.2145	.1287	.0429	·	.0858	.1287	.3377	·	.0858	.8889	·	·
i2	·	.9879	·	·	·	.0335	.1488	·	·	·	·	.0288
i3	.3544	.1688	·	·	·	.1181	.7852	·	.4642	·	·	·
i4	·	.1940	·	.8027	.3051	.0555	.4528	·	·	·	.1294	·
i5	·	·	·	.2809	·	.0551	.4312	.1926	.5777	·	.1926	.5694
i6	·	.0553	·	.6780	·	·	.7314	·	·	·	.0475	·
i7	·	.0330	·	·	.9065	.4121	.0769	.0384	·	·	·	·
i8	·	·	.0898	·	.0449	.9755	.0772	·	·	·	.1797	·
i9	·	.1355	·	·	.1734	.0758	·	·	.0379	·	.9473	.2167
i10	·	.2024	.4424	·	·	.5770	·	·	.1574	·	.6294	.0975
i11	.5949	.2511	.4850	.0511	.0595	.4591	·	·	.0595	·	.3569	·
i12	.2449	.2627	.8892	·	·	.0911	.2524	·	·	·	.0911	·

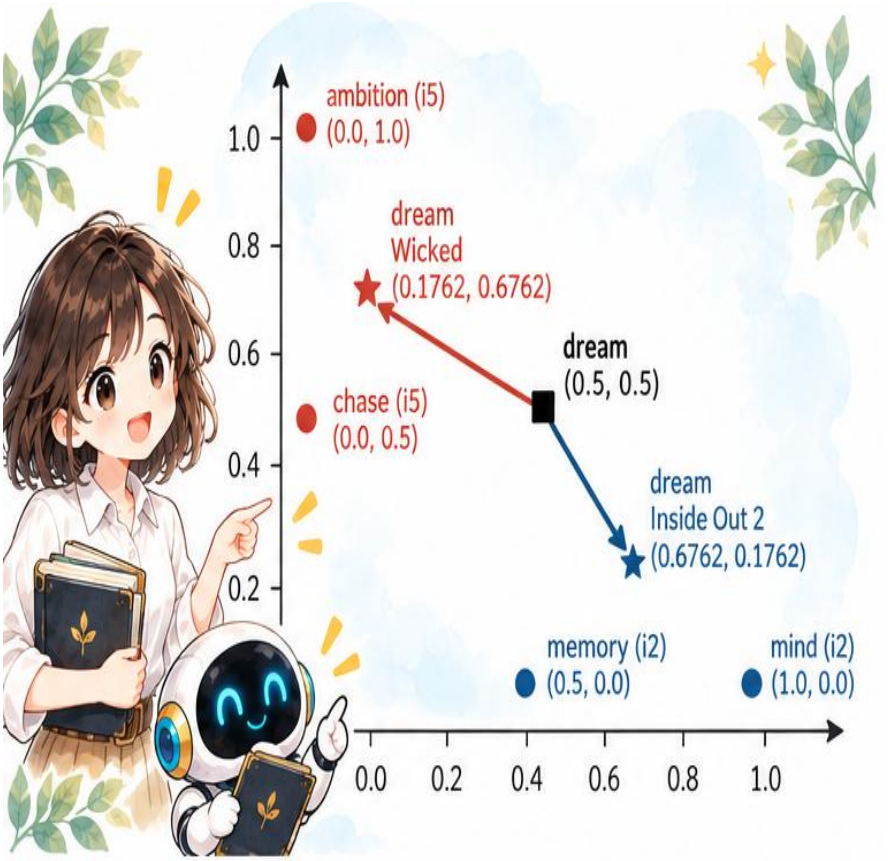
ที่มา: ค่าจำลองของ Text Transformer Encoder ปรับความยาวเป็น 1 แล้ว ละเลขศูนย์หน้าจุด จุด · แทนค่า 0

ตารางที่ 4.16 เวกเตอร์ภาพ เสียง และวิดีโอสามมิติจากเทคนิค Transformer

รหัส	I1	I2	I3	A1	A2	A3	V1	V2	V3
i1	0.9	0.6	0.9	0.5	0.7	0.1	0.6	0.5	1.0
i2	0.8	0.8	0.9	0.5	1.0	0.2	0.5	0.4	1.0
i3	0.7	0.5	0.8	0.3	0.8	0.4	0.4	0.3	1.0
i4	0.5	0.7	0.2	0.3	0.8	0.3	0.4	0.3	0.0
i5	0.7	0.6	0.5	0.4	1.0	0.1	0.3	0.4	0.0
i6	0.6	0.8	0.3	0.2	0.8	0.1	0.2	0.2	0.0
i7	0.2	0.3	0.1	0.9	0.2	0.8	0.9	0.9	0.0
i8	0.6	0.9	0.3	1.0	0.3	0.7	1.0	1.0	0.0
i9	0.4	0.3	0.1	0.3	0.5	0.6	0.3	0.4	0.0
i10	0.5	0.8	0.1	0.8	0.3	0.8	0.7	0.6	0.0
i11	0.3	0.2	0.1	0.6	0.3	0.7	0.6	0.5	0.0
i12	0.3	0.3	0.1	0.5	0.6	0.7	0.5	0.4	0.0

ที่มา: ค่าจำลองเพื่อการสอน (ไฟล์ mm4.py ในภาคผนวก 4ก) I = ภาพ, A = เสียง, V = วิดีโอ

ความต่างที่สำคัญที่สุดของสองเทคนิคเห็นได้จากคำว่า dream ซึ่งปรากฏทั้งใน i2 Inside Out 2 (ความฝันในจิตใจ) และ i5 Wicked (ความใฝ่ฝัน) เทคนิค CNN บนเวกเตอร์คำแบบ Static ให้ dream มีเวกเตอร์เดียวทั้งสองเรื่อง แต่ Self-Attention ดึง dream ไปคนละทิศตามคำรอบข้าง ดังภาพที่ 4.33



ภาพที่ 4.33 dream ก่อนและหลัง Self-Attention ใน i2 และ i5

ตัวอย่างการคำนวณ 4.23 บริบทเปลี่ยนความคล้ายของ Inside Out 2 กับ Wicked

โจทย์ จงเทียบโคไซน์ข้อความของ i2 Inside Out 2 กับ i5 Wicked จากเทคนิค CNN และ Transformer และอธิบายด้วยมิติ mind และ ambition

สมการที่ใช้ สมการ (4.1)

ข้อมูลที่ใช้ ตารางที่ 4.12 และ 4.15

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- i2 Inside Out 2 (Animation, Family) กับ i5 Wicked (Fantasy, Musical) ไม่มีแนวร่วมกันเลย (ตารางที่ 4.4) แต่ u6 ชอบทั้งสองเรื่อง (i2 = 5, i5 = 4 ในตารางที่ 4.5)
- เรื่องย่อในตารางที่ 4.6 มีคำว่า dream ทั้งสองเรื่อง: i2 พบ 4 ครั้ง ("each night a dream replays a memory, a dream of friendship and a dream of the future ... a frightening dream") ส่วน i5 พบ 2 ครั้ง ("chase a dream of meeting the wizard" และ "a dream of a better land")
- Static Embedding (ตารางที่ 4.12) ให้ dream เป็นค่าเฉลี่ยสองความหมาย (mind 0.5, ambition 0.5) ทั้งสองเรื่อง ส่วน Transformer (ตารางที่ 4.15) อ่านบริบท: ใน i2 อยู่กับ night, memory, frightening จึงเป็น "ความฝันขณะหลับ" (mind) ใน i5 อยู่กับ chase, wizard, better land จึงเป็น "ความใฝ่ฝัน" (ambition)
- คำว่า chase ใน i5 ("chase a dream") ก็ถูกอ่านเป็น "การไล่ตามความฝัน" (ambition) แทนการไล่ล่า (action) มิติ action ของ i5 จึงลดจาก .1189 เหลือ .0551

ขั้นที่ 1 ดูมิติ mind และ ambition

- เทคนิค CNN: i2 มี mind 0.9641, ambition 0.1679 ส่วน i5 มี mind 0.1189, ambition 0.4356
- เทคนิค Transformer: i2 มี mind 0.9879, ambition 0.0288 ส่วน i5 มี mind 0.0000, ambition 0.5694

ขั้นที่ 2 โคไซน์ของสองเทคนิค

- เทคนิค CNN: $\cos(i2, i5) = 0.2935$
- เทคนิค Transformer: $\cos(i2, i5) = 0.0824$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

เทคนิค CNN (Static Embedding) คูณทุกมิติที่มีค่า

มิติ	i2	i5	i2 × i5
mind	0.9641	0.1189	0.1146
romance	0.0266	0.3029	0.0081
action	0.0391	0.1189	0.0046
family	0.2001	0.4649	0.0930
music	0.0000	0.2076	0.0000
fantasy	0.0000	0.6229	0.0000
society	0.0000	0.2076	0.0000
ambition	0.1679	0.4356	0.0731
รวม			**0.2935**

เทคนิค Transformer

มิติ	i2	i5	i2 × i5
mind	0.9879	0.0000	0.0000
romance	0.0000	0.2809	0.0000
action	0.0335	0.0551	0.0018
family	0.1488	0.4312	0.0642
music	0.0000	0.1926	0.0000
fantasy	0.0000	0.5777	0.0000
society	0.0000	0.1926	0.0000
ambition	0.0288	0.5694	0.0164
รวม			**0.0824**

- ส่วนที่หายไปมากที่สุดคือ mind: จาก 0.9641 × 0.1189 = 0.1146 เหลือ 0.9879 × 0 = 0 เพราะ dream ใน i5 ไม่ถูกนับเป็น mind อีกต่อไป
- ส่วน ambition ลดจาก 0.1679 × 0.4356 = 0.0731 เหลือ 0.0288 × 0.5694 = 0.0164 เพราะ dream ใน i2 ไม่ถูกนับเป็น ambition

ตีความ

เมื่อใช้ Static ทั้งสองเรื่องได้ทั้ง mind และ ambition จาก dream ตัวเดียวกัน จึงดูคล้ายกันเกินจริง เมื่อใช้ Transformer dream ใน i2 ไปอยู่ที่ mind และใน i5 ไปอยู่ที่ ambition ความคล้ายจึงลดลง
นี่คือข้อได้เปรียบของ **เวกเตอร์ตามบริบท (Contextual Vector)** ซึ่งสำคัญกับผู้ใช้อย่าง u1 ที่ชอบ Inside Out 2 แต่ไม่ได้แสดงว่าชอบเรื่องความใฝ่ฝัน

ตารางที่ 4.17 เทียบเวกเตอร์ของสองเทคนิคทีละรายการ โดยวัดโคไซน์ระหว่างเวกเตอร์ CNN กับ Transformer ของเรื่องเดียวกันหลังเตรียมเวกเตอร์ตามหัวข้อ 4.5.1

ตารางที่ 4.17 โคไซน์ระหว่างเวกเตอร์ CNN กับ Transformer ของรายการเดียวกัน

สื่อ	ค่าเฉลี่ย 12 เรื่อง	ค่าต่ำสุด	เรื่องที่ต่างกันมากที่สุด
ข้อความ	0.9960	0.9799	i5 Wicked
ภาพ	0.9926	0.9895	i10 Dune: Part Two
เสียง	0.9745	0.9561	i3 The Wild Robot
วิดีโอ	0.9779	0.9541	i5 Wicked

ที่มา: ผู้เขียนคำนวณจากตารางที่ 4.12, 4.13, 4.15 และ 4.16

เวกเตอร์ของสองเทคนิคชี้ทิศใกล้กันเกือบทุกเรื่อง เพราะทั้งสองเทคนิคเห็นข้อมูลเดียวกัน ความต่างอยู่ที่รายละเอียดที่ต้องอาศัยบริบทกว้าง หัวข้อ 4.5.8 จะแสดงว่าความต่างเล็กน้อยระดับเวกเตอร์นี้ส่งผลต่อโปรไฟล์และอันดับมากน้อยเพียงใด

ตารางที่ 4.18 สรุปความต่างของ CNN และ Transformer ทั้งด้านแนวคิดและด้านการใช้งาน

ตารางที่ 4.18 เปรียบเทียบ CNN กับ Transformer

ประเด็น	CNN	Transformer
หน่วยประมวลผล	ตัวกรองเลื่อนบนหน้าต่างเฉพาะที่	Self-Attention ระหว่างทุกคู่โทเคน
ระยะมองเห็น	ขยายทีละชั้น $r_L = 1 + L(k-1)$	ทั้งลำดับตั้งแต่ชั้นแรก
ลำดับและตำแหน่ง	รู้โดยธรรมชาติจากการเลื่อนตัวกรอง	ต้องเพิ่มการฝังตำแหน่ง
ต้นทุนต่อชั้น	เชิงเส้นตามขนาดข้อมูล	กำลังสองของจำนวนโทเคน
ข้อมูลที่ต้องใช้ฝึก	น้อยถึงปานกลาง	มาก (มักฝึกล่วงหน้าบนข้อมูลขนาดใหญ่)
ความหมายตามบริบท	จำกัด (คำเดียวความหมายเดียว)	ดี (เวกเตอร์ขึ้นกับคำรอบข้าง)
ความสามารถข้ามสื่อ	แยกแบบจำลองตามสื่อ	มีแบบจำลองร่วม เช่น CLIP, CLAP

ที่มา: ผู้เขียนสรุปจากหัวข้อ 4.3 และ 4.4

แนวทางเลือกใช้ เลือก CNN หรือ Transformer

- **เลือก CNN** เมื่อข้อมูลฝึกน้อย ต้องการความเร็วหรือรันบนอุปกรณ์ขนาดเล็ก หรือคุณลักษณะสำคัญเป็นลวดลายเฉพาะที่
- **เลือก Transformer** เมื่อความหมายขึ้นกับบริบทกว้าง เช่น เรื่องย่อที่มีคำหลายความหมาย หรือต้องการเชื่อมหลายสื่อในปริภูมิร่วม
- **ในทางปฏิบัติ** ระบบผู้แนะนำส่วนใหญ่ใช้แบบจำลองที่ฝึกไว้แล้วสกัดเวกเตอร์ล่วงหน้า ความเร็วขณะให้บริการจึงไม่ขึ้นกับตัวเข้ารหัส ข้อได้เปรียบด้านต้นทุนของ CNN จึงสำคัญน้อยลง แต่ยังสำคัญเมื่อต้องสกัดเวกเตอร์ของรายการใหม่จำนวนมากทุกวัน

โปรแกรม 4.10 เทียบความคล้ายข้อความของสองเทคนิค

```
from mm4 import vectors

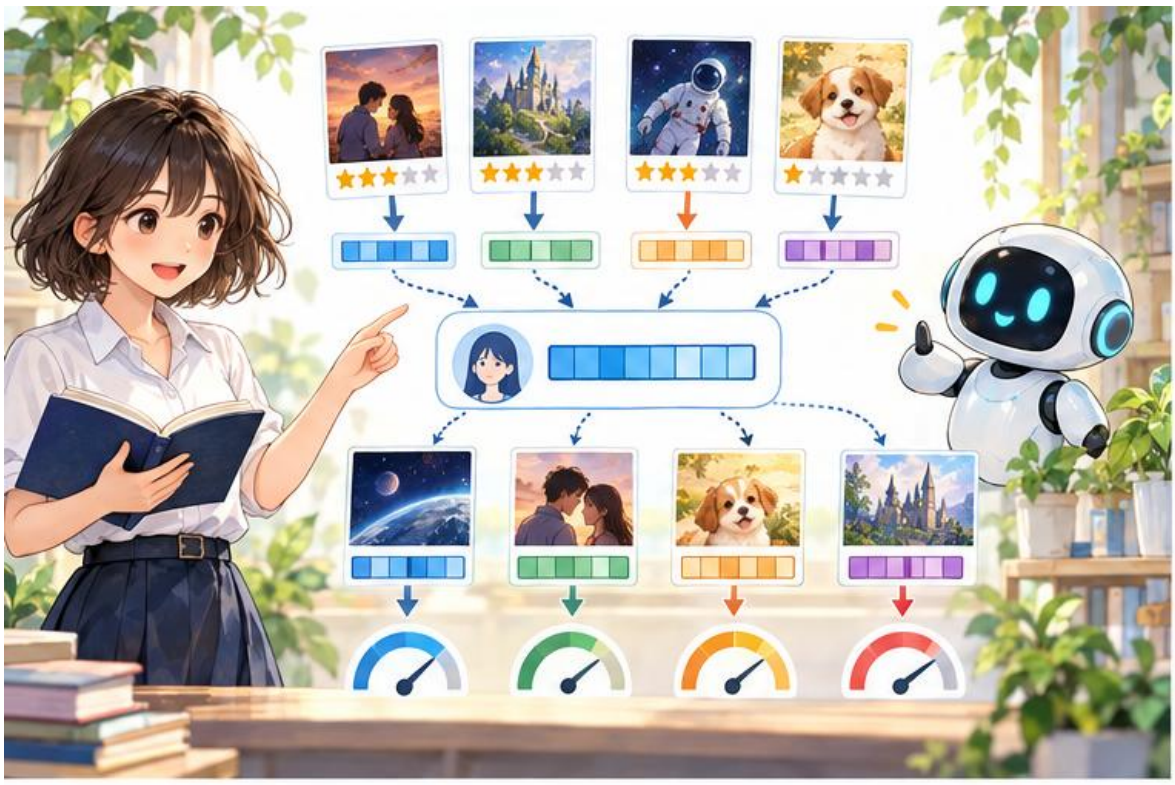
V = {t: vectors(t) for t in ['CNN', 'TF']}
for a, b in [('i2', 'i5'), ('i1', 'i2'), ('i7', 'i8')]:
    cnn = V['CNN']['T'][a] @ V['CNN']['T'][b]
    tf = V['TF']['T'][a] @ V['TF']['T'][b]
    print('%s-%s  CNN %.4f  TF %.4f' % (a, b, cnn, tf))
```

ผลลัพธ์

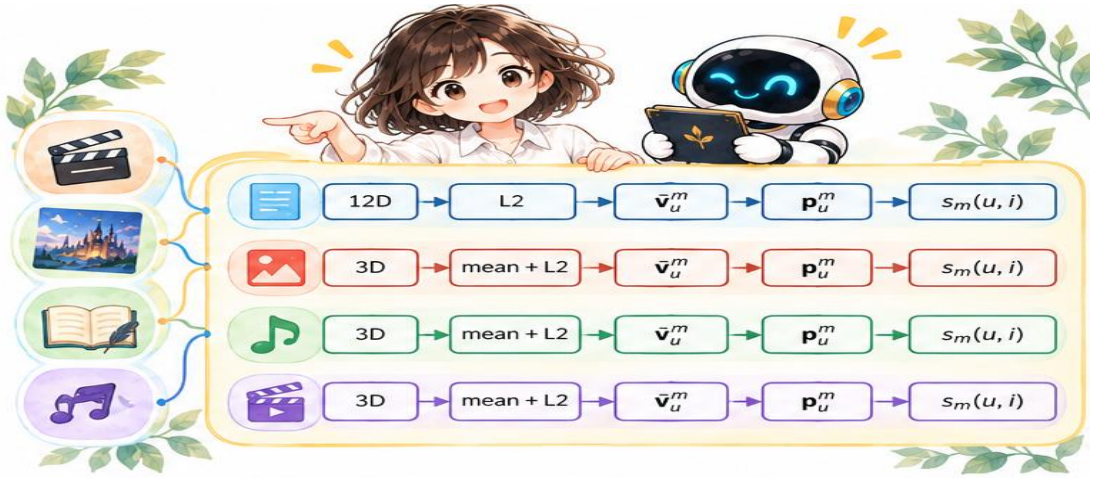
```
i2-i5  CNN 0.2935  TF 0.0824
i1-i2  CNN 0.2028  TF 0.1817
i7-i8  CNN 0.4493  TF 0.4486
```

4.5 การสร้างโปรไฟล์ผู้ใช้ (User Profile)

หัวข้อ 4.3 และ 4.4 ให้เวกเตอร์ของ **รายการ** มาแล้วสองชุด หัวข้อนี้เปลี่ยนเวกเตอร์ของรายการที่ผู้ใช้เคยให้คะแนนให้เป็นเวกเตอร์ของ **ผู้ใช้** เรียกว่า **โปรไฟล์ผู้ใช้ (User Profile)** แล้ววัดว่ารายการผู้สมัครแต่ละเรื่องใกล้โปรไฟล์เพียงใด ภาพที่ 4.34 แสดงว่าทุกสื่อใช้ขั้นตอนเดียวกัน ต่างกันเพียงขั้นเตรียมเวกเตอร์ ดังสรุปในภาพที่ 4.56 สรุปการรวมประวัติชอบ ไม่ชอบ เวลา และหลายจุดสนใจเป็นโปรไฟล์ผู้ใช้



ภาพที่ 4.56 วิธีสร้างโปรไฟล์ผู้ใช้จากเวกเตอร์รายการ

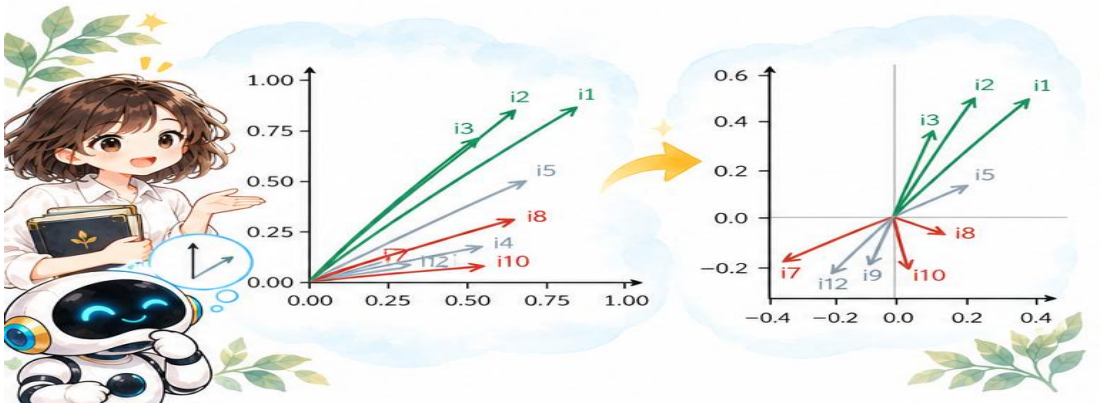


ภาพที่ 4.34 ขั้นตอนสร้างโปรไฟล์และให้คะแนนเดียวกันสำหรับทุกสื่อ

หัวข้อนี้เรียงวิธีสร้างโปรไฟล์จากง่ายไปยาก ทุกวิธีใช้กับเวกเตอร์ได้ทั้งจากเทคนิค CNN และ Transformer ตัวอย่างการคำนวณใช้เทคนิค Transformer เป็นหลัก แล้วสรุปผลของเทคนิค CNN ไว้ในตารางเปรียบเทียบของหัวข้อ 4.5.8

4.5.1 เตรียมเวกเตอร์และให้คะแนน

ค่าในตารางที่ 4.16 เป็นบวกทั้งหมด เวกเตอร์ทุกเส้นจึงชี้อยู่ในมุมบวกเดียวกัน และโคไซน์ระหว่างสองเรื่องใด ๆ จะสูงเกือบเสมอ ภาพที่ 4.35 ด้านซ้ายแสดงปัญหานี้ด้วยมิติที่ 1 และ 3 ของภาพส่วนด้านขวาแสดงว่าเมื่อลบค่าเฉลี่ยของคลังออก เวกเตอร์จะกระจายรอบจุดศูนย์กลาง และแยกกลุ่มแอนิเมชันออกจากกลุ่มแอ็กชันได้ชัด ขั้นเตรียมเวกเตอร์ของภาพ เสียง และวิดีโอจึงมีสองขั้นตามสมการ (4.33) และ (4.34)



ภาพที่ 4.35 เวกเตอร์ภาพก่อนและหลังลบค่าเฉลี่ยของคลัง

$$\boldsymbol{\mu}^{m} = \frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} \mathbf{x}_i^{m}, \qquad \tilde{\mathbf{x}}_i^{m} = \mathbf{x}_i^{m} - \boldsymbol{\mu}^{m} \tag{4.33}$$

โดยที่

- $\mathbf{x}_i^{m}$ เวกเตอร์ของรายการ i ในสื่อ m จากตารางที่ 4.13 หรือ 4.16
- $\mathcal{I}$ เซตของรายการทั้งคลัง ($|\mathcal{I}| = 12$)
- $\boldsymbol{\mu}^{m}$ เวกเตอร์ค่าเฉลี่ยของคลังในสื่อ m
- $\tilde{\mathbf{x}}_i^{m}$ เวกเตอร์หลังลบค่าเฉลี่ย (Mean Centering)

$$\mathbf{v}_i^{m} = \frac{\tilde{\mathbf{x}}_i^{m}}{\|\tilde{\mathbf{x}}_i^{m}\|} \tag{4.34}$$

โดยที่

- $\mathbf{v}_i^{m}$ เวกเตอร์ที่เตรียมแล้ว ความยาวเท่ากับ 1 ผลคูณจุดของสองเวกเตอร์จึงเท่ากับโคไซน์
- เวกเตอร์ข้อความในตารางที่ 4.12 และ 4.15 ปรับความยาวเป็น 1 แล้วและมีค่าศูนย์จำนวนมาก จึงใช้ตามเดิมโดยไม่ลบค่าเฉลี่ย

ตัวอย่างการคำนวณ 4.24 เตรียมเวกเตอร์ภาพของ Toy Story 5 และ Ballerina

โจทย์ จงเตรียมเวกเตอร์ภาพเทคนิค Transformer ของ i1 Toy Story 5 และ i7 Ballerina แล้วเทียบโคไซน์ก่อนและหลังเตรียม

สมการที่ใช้ สมการ (4.1), (4.33) และ (4.34)

ข้อมูลที่ใช้ เวกเตอร์ภาพจากตารางที่ 4.16

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- เวกเตอร์ภาพทั้ง 12 แถวมาจากตารางที่ 4.16 (เทคนิค Transformer) ค่าเฉลี่ยของคลังต้องใช้ทุกเรื่องในคลัง ไม่ใช่เฉพาะสองเรื่องที่เทียบ
- i1 Toy Story 5 เป็นโปสเตอร์แอนิเมชันสว่าง (l1 = 0.9, l3 = 0.9) ส่วน i7 Ballerina เป็นโปสเตอร์แอ็กชันโทนมืด (ทุกมิติ 0.1–0.3) ตามแนวในตารางที่ 4.4

ขั้นที่ 1 โคไซน์ก่อนเตรียม

- i1 = (0.9, 0.6, 0.9), i7 = (0.2, 0.3, 0.1)
- ผลคูณจุด $0.18 + 0.18 + 0.09 = 0.45$ ความยาว $\sqrt{1.98} = 1.4071$ และ $\sqrt{0.14} = 0.3742$
- โคไซน์ = $0.45/(1.4071 \times 0.3742) = 0.8547$ สูงแม้เป็นโปสเตอร์คนละแบบ

ขั้นที่ 2 หาค่าเฉลี่ยของคลัง

- มิติที่ 1: $(0.9 + 0.8 + \cdots + 0.3)/12 = 6.5/12 = 0.5417$
- มิติที่ 2 และ 3 ได้ 0.5667 และ 0.3667 จึงได้ $\boldsymbol{\mu}^I = (0.5417, 0.5667, 0.3667)$

ขั้นที่ 3 ลบค่าเฉลี่ยแล้วปรับความยาว

- $\tilde{\mathbf{x}}_{i1} = (0.3583, 0.0333, 0.5333)$ ความยาว 0.6434 จึงได้ $\mathbf{v}_{i1} = (0.5569, 0.0518, 0.8289)$
- $\tilde{\mathbf{x}}_{i7} = (-0.3417, -0.2667, -0.2667)$ ความยาว 0.5089 จึงได้ $\mathbf{v}_{i7} = (-0.6714, -0.5240, -0.5240)$

ขั้นที่ 4 โคไซน์หลังเตรียม

- $\mathbf{v}_{i1} \cdot \mathbf{v}_{i7} = -0.8355$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

หาค่าเฉลี่ยของคลังทีละมิติ (12 เรื่อง)

มิติ	ค่าของ i1 ถึง i12	ผลรวม	μ
I1	0.9, 0.8, 0.7, 0.5, 0.7, 0.6, 0.2, 0.6, 0.4, 0.5, 0.3, 0.3	6.5	0.5417
I2	0.6, 0.8, 0.5, 0.7, 0.6, 0.8, 0.3, 0.9, 0.3, 0.8, 0.2, 0.3	6.8	0.5667
I3	0.9, 0.9, 0.8, 0.2, 0.5, 0.3, 0.1, 0.3, 0.1, 0.1, 0.1, 0.1	4.4	0.3667

ลบค่าเฉลี่ยแล้วปรับความยาว

รายการ	x − μ	ผลรวมกำลังสอง	ความยาว	v = (x − μ)/‖x − μ‖
i1	(0.3583, 0.0333, 0.5333)	0.4140	0.6434	(0.5569, 0.0518, 0.8289)
i7	(−0.3417, −0.2667, −0.2667)	0.2590	0.5089	(−0.6714, −0.5240, −0.5240)

- ผลคูณจุด: (0.5569)(−0.6714) + (0.0518)(−0.5240) + (0.8289)(−0.5240) = −0.3739 − 0.0271 − 0.4344 = −0.8355
- i7 ต่ำกว่าค่าเฉลี่ยของคลังทุกมิติ (x − μ ติดลบทั้งหมด) ส่วน i1 สูงกว่าค่าเฉลี่ยทุกมิติ สองเวกเตอร์จึงชี้ไปคนละทาง

ตีความ

การลบค่าเฉลี่ยเปลี่ยนคำถามจาก "สองเรื่องมีค่ามากเหมือนกันหรือไม่" เป็น "สองเรื่อง **ต่างจากเรื่องทั่วไป** ไปทางเดียวกันหรือไม่" โคไซน์จึงลดจาก 0.8547 เป็น −0.8355 ซึ่งสะท้อนว่าโปสเตอร์สองเรื่องนี้ต่างกันจริง

เมื่อเตรียมเวกเตอร์แล้ว คะแนนของรายการผู้สมัคร i สำหรับผู้ใช้ u ในสื่อ m คือผลคูณจุดระหว่างโปรไฟล์กับเวกเตอร์ของรายการ ตามสมการ (4.35) ทุกวิธีในหัวข้อนี้ต่างกันเพียงวิธีสร้างโปรไฟล์ $\mathbf{p}_u^m$ เท่านั้น

$$s_m(u,i) = \mathbf{p}_u^m \cdot \mathbf{v}_i^m = \cos\left(\mathbf{p}_u^m, \mathbf{v}_i^m\right) \tag{4.35}$$

โดยที่

- $\mathbf{p}_u^m$ โปรไฟล์ของผู้ใช้ u ในสื่อ m ที่ปรับความยาวเป็น 1 แล้ว
- $s_m(u,i)$ คะแนนรายสื่อ อยู่ระหว่าง -1 ถึง 1

4.5.2 โปรไฟล์ค่าเฉลี่ยของเรื่องที่ชอบ

วิธีที่ง่ายที่สุดคือหาค่าเฉลี่ยของเวกเตอร์เรื่องที่ผู้ใช้ชอบ แล้วปรับความยาวเป็น 1 ดังสมการ (4.36) แนวคิดคือ "ผู้ใช้น่าจะชอบเรื่องที่อยู่ใกล้จุดศูนย์กลางของเรื่องที่เคยชอบ"

$$\mathbf{p}_u^m = \frac{\bar{\mathbf{p}}}{\|\bar{\mathbf{p}}\|}, \qquad \bar{\mathbf{p}} = \frac{1}{|L_u|}\sum_{i \in L_u} \mathbf{v}_i^m \tag{4.36}$$

โดยที่

- L_u เซตของเรื่องที่ผู้ใช้ u ชอบ (คะแนน 4–5)
- $\bar{\mathbf{p}}$ ค่าเฉลี่ยก่อนปรับความยาว

ตัวอย่างการคำนวณ 4.25 โปรไฟล์ข้อความค่าเฉลี่ยของ u1

โจทย์ u1 ชอบ i1 Toy Story 5, i2 Inside Out 2 และ i3 The Wild Robot
จงสร้างโปรไฟล์ข้อความด้วยเทคนิค Transformer แล้วหาคะแนนของ i5 Wicked และ Top-3

สมการที่ใช้ สมการ (4.36) และ (4.35)

ข้อมูลที่ใช้ เวกเตอร์ข้อความจากตารางที่ 4.15 และคะแนนของ u1 จากตารางที่ 4.5

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- คะแนนของ u1 มาจากตารางที่ 4.5: ชอบ i1 Toy Story 5 (5), i2 Inside Out 2 (5), i3 The Wild Robot (4) ไม่ชอบ i4 It Ends with Us (1), i6 Bridget Jones: Mad About the Boy (1) และยังไม่เคยให้คะแนนอีกเจ็ดเรื่อง
- เซตเรื่องที่ชอบ L_u1 = {i1, i2, i3} เพราะคะแนน 5, 5, 4 อยู่ในช่วง 4–5 ส่วน i4 และ i6 ได้ 1 จึงไม่ถูกนำมาเฉลี่ย (โปรไฟล์ค่าเฉลี่ยใช้เฉพาะเรื่องที่ชอบ)
- เวกเตอร์ข้อความมาจากตารางที่ 4.15 ซึ่งคำนวณจากเรื่องย่อในตารางที่ 4.6: i1 เด่นที่ toys (คำ toy/toys 5 ครั้ง) i2 เด่นที่ mind (mind, memory และ dream ที่หมายถึงความฝันขณะหลับ) i3 เด่นที่ family (child, family, home)
- รายการผู้สมัครคือเรื่องที่ u1 ยังไม่เคยให้คะแนน ได้จากการตัด 5 เรื่องที่ให้คะแนนแล้วออกจาก 12 เรื่องในตารางที่ 4.4 เหลือ i5, i7, i8, i9, i10, i11 และ i12

ขั้นที่ 1 หาค่าเฉลี่ยทีละมิติ

- technology: $(0.2145 + 0 + 0.3544)/3 = 0.1896$
- mind: $(0.1287 + 0.9879 + 0.1688)/3 = 0.4285$
- family: $(0.3377 + 0.1488 + 0.7852)/3 = 0.4239$ และ toys: $0.8889/3 = 0.2963$
- ทั้ง 12 มิติ: $\bar{\mathbf{p}} = (0.1896, 0.4285, 0.0143, 0.0000, 0.0286, 0.0934, 0.4239, 0.0000, 0.1833, 0.2963, 0.0000, 0.009$

ขั้นที่ 2 ปรับความยาว

- $\| \bar{\mathbf{p}} \| = 0.7283$
- $\mathbf{p}_{u1}^T = (0.2604, 0.5883, 0.0196, 0.0000, 0.0393, 0.1283, 0.5820, 0.0000, 0.2517, 0.4068, 0.0000, 0.013$

ขั้นที่ 3 คะแนนของ i5 Wicked

- ผลคูณจุดกับ i5 ได้ 0.4109 (มาจากมิติ family และ fantasy เป็นหลัก)

ขั้นที่ 4 จัดอันดับเจ็ดเรื่อง

- Top-3: i5 (0.4109), i12 (0.3943), i11 (0.3883)

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

ตารางคำนวณครบทั้ง 12 มิติ

- คอลัมน์ $\bar{p}$ = (i1 + i2 + i3)/3 คอลัมน์ p_u1 = $\bar{p}$ / $\|\bar{p}\|$ และคอลัมน์สุดท้ายคือผลคูณกับเวกเตอร์ของ i5 Wicked

มิติ	i1	i2	i3	$\bar{p}$	p_u1	i5	p_u1 × i5
tech	0.2145	0.0000	0.3544	0.1896	0.2604	0.0000	0.0000
mind	0.1287	0.9879	0.1688	0.4285	0.5883	0.0000	0.0000
space	0.0429	0.0000	0.0000	0.0143	0.0196	0.0000	0.0000
romance	0.0000	0.0000	0.0000	0.0000	0.0000	0.2809	0.0000
crime	0.0858	0.0000	0.0000	0.0286	0.0393	0.0000	0.0000
action	0.1287	0.0335	0.1181	0.0934	0.1283	0.0551	0.0071
family	0.3377	0.1488	0.7852	0.4239	0.5820	0.4312	0.2510
music	0.0000	0.0000	0.0000	0.0000	0.0000	0.1926	0.0000
fantasy	0.0858	0.0000	0.4642	0.1833	0.2517	0.5777	0.1454
toys	0.8889	0.0000	0.0000	0.2963	0.4068	0.0000	0.0000
society	0.0000	0.0000	0.0000	0.0000	0.0000	0.1926	0.0000
ambition	0.0000	0.0288	0.0000	0.0096	0.0132	0.5694	0.0075
รวม				**$\|\bar{p}\|$ = 0.7283**			**0.4109**

- $\|\bar{p}\|$ = √(0.1896² + 0.4285² + 0.0143² + 0.0286² + 0.0934² + 0.4239² + 0.1833² + 0.2963² + 0.0096²) = √0.5305 = 0.7283
- คะแนนของ i5 มาจาก family (0.2510) และ fantasy (0.1454) เป็นหลัก ส่วน action และ ambition ช่วยเพียงเล็กน้อย

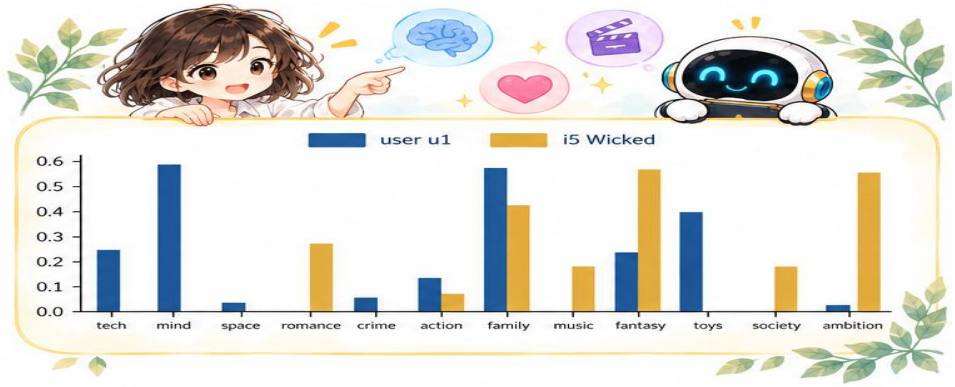
คะแนนของรายการผู้สมัครทั้งเจ็ดเรื่อง (ผลคูณจุดแบบเดียวกัน)

อันดับ	รายการผู้สมัคร	s(u1, i)	สองมิติที่ให้คะแนนมากที่สุด
1	i5 Wicked	0.4109	family 0.2510, fantasy 0.1454
2	i12 Project Hail Mary	0.3943	mind 0.1545, family 0.1469
3	i11 Mickey 17	0.3883	tech 0.1549, mind 0.1477
4	i10 Dune: Part Two	0.2427	mind 0.1191, action 0.0740
5	i8 Furiosa: A Mad Max Saga	0.1736	action 0.1251, family 0.0449
6	i7 Ballerina	0.1526	action 0.0529, family 0.0448
7	i9 Conclave	0.1086	mind 0.0797, action 0.0097

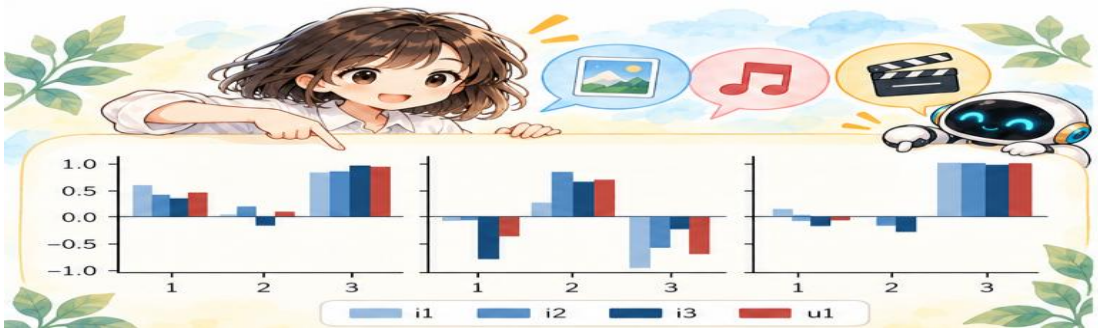
ตีความ

โปรไฟล์ข้อความของ u1 เด่นที่มิติ mind, family และ toys ตามเรื่องที่ชอบ i5 Wicked ได้อันดับ 1 เพราะเป็นเรื่องครอบครัวแฟนตาซี แต่คะแนนของ i12 และ i11 ใกล้กันมาก ข้อความเพียงอย่างเดียวจึงแยกสามเรื่องนี้ได้ไม่ชัด

ภาพที่ 4.36 เทียบโปรไฟล์ข้อความของ u1 กับเวกเตอร์ของ i5 Wicked ทีละมิติ ส่วนภาพที่ 4.37 แสดงโปรไฟล์ภาพ เสียง และวิดีโอของ u1 เทียบกับเวกเตอร์ของเรื่องที่ชอบทั้งสามหลังเตรียมเวกเตอร์ในทุกสื่อโปรไฟล์อยู่ "กลางกลุ่ม" ของเรื่องที่ชอบ



ภาพที่ 4.36 โปรไฟล์ข้อความของ u1 เทียบกับเวกเตอร์ของ i5 Wicked



ภาพที่ 4.37 โปรไฟล์ภาพ เสียง และวิดีโอของ u1 เทียบกับเรื่องที่ชอบ

ตารางที่ 4.19 รวมคะแนนรายสื่อของรายการผู้สมัครทั้งเจ็ดเรื่องจากโปรไฟล์ค่าเฉลี่ยของทั้งสองเทคนิค ตารางนี้เป็นข้อมูลนำเข้าของ Fusion ในหัวข้อ 4.6

ตารางที่ 4.19 คะแนนรายสื่อของ u1 จากโปรไฟล์ค่าเฉลี่ย (สองเทคนิค)

รหัส	เทคนิค Transformer				เทคนิค CNN			
	T	I	A	V	T	I	A	V
i5	0.4109	0.9180	0.9929	−0.6417	0.5504	0.9145	0.9954	−0.8430
i7	0.1526	−0.8154	−0.9609	−0.5177	0.1555	−0.8020	−0.9157	−0.6405
i8	0.1736	−0.0053	−0.8548	−0.4474	0.1871	0.0829	−0.7904	−0.5635
i9	0.1086	−0.8084	−0.3154	−0.6417	0.1200	−0.7527	−0.0276	−0.8430
i10	0.2427	−0.6555	−0.9799	−0.8390	0.2850	−0.5229	−0.9549	−0.9369

รหัส	เทคนิค Transformer				เทคนิค CNN			
	T	I	A	V	T	I	A	V
i11	0.3883	−0.7392	−0.9816	−0.9727	0.4170	−0.7019	−0.9844	−0.9925
i12	0.3943	−0.8268	−0.6487	−0.8781	0.4786	−0.7960	−0.6943	−0.9305

ที่มา: ผู้เขียนคำนวณด้วยสมการ (4.36) และ (4.35) จากเวกเตอร์ในหัวข้อ 4.3.7 และ 4.4.8

ตารางที่ 4.19 ชี้ประเด็นสำคัญสามข้อ

- **แต่ละสื่อเห็นต่างกัน** ภาพและเสียงให้ i5 Wicked สูงมาก (0.9180 และ 0.9929) ส่วนวิดีโอให้ i8 Furiosa สูงที่สุด
- **มาตราของคะแนนต่างกัน** ข้อความอยู่ในช่วงแคบประมาณ 0.1–0.4 แต่ภาพและเสียงกระจายตั้งแต่ −0.9 ถึง 1 การรวมคะแนนตรง ๆ จึงต้องระวัง (หัวข้อ 4.6.3)
- **สองเทคนิคให้อันดับใกล้กัน** แม้ค่าคะแนนต่างกันเล็กน้อย เช่น ข้อความของ i5 ได้ 0.4109 จาก Transformer และ 0.5504 จาก CNN

โปรแกรม 4.11 โปรไฟล์ค่าเฉลี่ยและคะแนนรายสื่อของ u1

```
import numpy as np
from mm4 import ITEMS, R, MODS, vectors, l2

def mean_profile(V, ratings):
    liked = [i for i, r in ratings.items() if r >= 4]
    return l2(np.mean([V[i] for i in liked], axis=0))

V = vectors('TF')
cand = [i for i in ITEMS if i not in R['u1']]
for m in MODS:
    p = mean_profile(V[m], R['u1'])
    s = {i: float(p @ V[m][i]) for i in cand}
    best = sorted(cand, key=lambda i: -s[i])[:3]
    print(m, [(i, round(s[i], 4)) for i in best])
```

ผลลัพธ์

```
T [('i5', 0.4109), ('i12', 0.3943), ('i11', 0.3883)]
I [('i5', 0.918), ('i8', -0.0053), ('i10', -0.6555)]
A [('i5', 0.9929), ('i9', -0.3154), ('i12', -0.6487)]
V [('i8', -0.4474), ('i7', -0.5177), ('i5', -0.6417)]
```

4.5.3 โปรไฟล์ถ่วงน้ำหนักด้วยคะแนน

โปรไฟล์ค่าเฉลี่ยทิ้งข้อมูลสองอย่าง คือระดับความชอบ (5 ชอบมากกว่า 4) และเรื่องที่ไม่ชอบ วิธีถ่วงน้ำหนักด้วยคะแนนแก้ทั้งสองข้อ โดยใช้ **คะแนนที่หักค่าเฉลี่ยของผู้ใช้** เป็นน้ำหนัก เรื่องที่ชอบได้น้ำหนักบวก เรื่องที่ไม่ชอบได้น้ำหนักลบ ดังสมการ (4.37)

$$\mathbf{p}_u^m \propto \sum_{i \in R_u} \left(r_{ui} - \bar{r}_u\right) \mathbf{v}_i^m, \qquad \bar{r}_u = \frac{1}{|R_u|} \sum_{i \in R_u} r_{ui} \tag{4.37}$$

โดยที่

- R_u เซตของเรื่องที่ผู้ใช้ u ให้คะแนนทั้งหมด
- r_{ui} คะแนน 1–5 ที่ผู้ใช้ u ให้เรื่อง i
- $\bar{r}_u$ ค่าเฉลี่ยคะแนนของผู้ใช้ u
- $\propto$ เท่ากันหลังปรับความยาวเป็น 1

ตัวอย่างการคำนวณ 4.26 โปรไฟล์ข้อความของ u1 ถ่วงด้วยคะแนน

โจทย์ จงสร้างโปรไฟล์ข้อความเทคนิค Transformer ของ u1 ด้วยสมการ (4.37) แล้วหา Top-3

สมการที่ใช้ สมการ (4.37) และ (4.35)

ข้อมูลที่ใช้ คะแนนของ u1 จากตารางที่ 4.5 เวกเตอร์จากตารางที่ 4.15

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- คะแนนของ u1 มาจากตารางที่ 4.5: ชอบ i1 Toy Story 5 (5), i2 Inside Out 2 (5), i3 The Wild Robot (4) ไม่ชอบ i4 It Ends with Us (1), i6 Bridget Jones: Mad About the Boy (1) และยังไม่เคยให้คะแนนอีกเจ็ดเรื่อง
- วิธีนี้ใช้ ทุกเรื่องที่ u1 ให้คะแนน (5 เรื่อง) ทั้งเรื่องที่ชอบและไม่ชอบ ต่างจากโปรไฟล์ค่าเฉลี่ยที่ใช้เพียงสามเรื่อง
- เวกเตอร์ข้อความมาจากตารางที่ 4.15 ซึ่งคำนวณจากเรื่องย่อในตารางที่ 4.6: i1 เด่นที่ toys (คำ toy/toys 5 ครั้ง) i2 เด่นที่ mind (mind, memory และ dream ที่หมายถึงความฝันขณะหลับ) i3 เด่นที่ family (child, family, home)
- เรื่องที่ไม่ชอบสองเรื่องเป็นแนว Drama, Romance (ตารางที่ 4.4) และเรื่องย่อมีคำ love หลายครั้ง (i4 สามครั้ง i6 สองครั้ง) จึงมีมิติ romance และ family สูงในตารางที่ 4.15

ขั้นที่ 1 หาค่าเฉลี่ยคะแนนและน้ำหนัก

- $\bar{r}_{u1} = (5 + 5 + 4 + 1 + 1)/5 = 3.2$
- น้ำหนัก: i1 = i2 = $5 - 3.2 = 1.8$, i3 = 0.8, i4 = i6 = $1 - 3.2 = -2.2$

ขั้นที่ 2 รวมเวกเตอร์ถ่วงน้ำหนัก (แสดงสามมิติที่เปลี่ยนมาก)

- romance: $-2.2(0.8027) - 2.2(0.6780) = -3.2577$ ติดลบเพราะสองเรื่องที่ไม่ชอบเป็นเรื่องรัก
- family: $1.8(0.3377) + 1.8(0.1488) + 0.8(0.7852) - 2.2(0.4528) - 2.2(0.7314) = -1.1015$
- mind: $1.8(0.1287) + 1.8(0.9879) + 0.8(0.1688) - 2.2(0.1940) - 2.2(0.0553) = 1.5964$

ขั้นที่ 3 ปรับความยาวแล้วจัดอันดับ

- Top-3: i11 (0.1533), i12 (0.0851), i10 (0.0828)

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

หาค่าเฉลี่ยคะแนนและน้ำหนักของแต่ละเรื่อง

- r̄_u1 = (5 + 5 + 4 + 1 + 1)/5 = 16/5 = 3.2
- น้ำหนัก r − r̄: i1 = 5 − 3.2 = 1.8, i2 = 1.8, i3 = 4 − 3.2 = 0.8, i4 = 1 − 3.2 = −2.2, i6 = −2.2 (ผลรวมน้ำหนักเป็น 0 เสมอ)

คูณเวกเตอร์ด้วยน้ำหนักแล้วรวมทีละมิติ

มิติ	1.8·i1	1.8·i2	0.8·i3	−2.2·i4	−2.2·i6	ผลรวม p	p/‖p‖
tech	0.3861	0.0000	0.2835	0.0000	0.0000	0.6696	0.1572
mind	0.2317	1.7782	0.1350	−0.4268	−0.1217	1.5964	0.3747
space	0.0772	0.0000	0.0000	0.0000	0.0000	0.0772	0.0181
romance	0.0000	0.0000	0.0000	−1.7659	−1.4916	−3.2577	−0.7645
crime	0.1544	0.0000	0.0000	−0.6712	0.0000	−0.5168	−0.1213
action	0.2317	0.0603	0.0945	−0.1221	0.0000	0.2643	0.0620

family	0.6079	0.2678	0.6282	−0.9962	−1.6091	−1.1015	−0.2585
music	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
fantasy	0.1544	0.0000	0.3714	0.0000	0.0000	0.5258	0.1234
toys	1.6000	0.0000	0.0000	0.0000	0.0000	1.6001	0.3755
society	0.0000	0.0000	0.0000	−0.2847	−0.1045	−0.3892	−0.0913
ambition	0.0000	0.0518	0.0000	0.0000	0.0000	0.0518	0.0122

- $\|p\|$ = 4.2610 จึงหารทุกมิติด้วยค่านี้ มิติ romance (−0.7645) และ family (−0.2585) ติดลบ เพราะเรื่องที่ไม่ชอบสองเรื่องมีสองมิตินี้สูง

คะแนนของรายการผู้สมัครทั้งเจ็ดเรื่อง

อันดับ	รายการผู้สมัคร	s(u1, i)	สองมิติที่ให้คะแนนบวกมากที่สุด
1	i11 Mickey 17	0.1533	mind 0.0941, tech 0.0935
2	i12 Project Hail Mary	0.0851	mind 0.0984, tech 0.0385
3	i10 Dune: Part Two	0.0828	mind 0.0758, action 0.0358
4	i8 Furiosa: A Mad Max Saga	0.0203	action 0.0605, space 0.0016
5	i9 Conclave	−0.0448	mind 0.0508, action 0.0047
6	i7 Ballerina	−0.0919	action 0.0256, mind 0.0124
7	i5 Wicked	−0.2622	fantasy 0.0713, ambition 0.0069

- i5 Wicked ได้ −0.2622 ติดลบ เพราะ family (0.4312) และ romance (0.2809) ของ i5 ไปคูณกับค่าลบของโปรไฟล์

ตีความ

น้ำหนักลบของเรื่องรักสองเรื่องกดมิติ family ให้ติดลบ (−1.1015) เพราะเรื่องที่ไม่ชอบมีค่า family สูงเช่นกัน i5 Wicked ซึ่งมี family สูงจึงตกอันดับ
ผลนี้เตือนว่าการผลักออกจากเรื่องที่ไม่ชอบอาจผลักมิติที่ผู้ใช้ชอบออกไปด้วย ถ้าเรื่องที่ชอบและไม่ชอบมีมิตินั้นร่วมกัน

4.5.4 Rocchio: ดึงเข้าหาเรื่องที่ชอบ ผลักออกจากเรื่องที่ไม่ชอบ

Rocchio มาจากงานค้นคืนสารสนเทศ (Rocchio, 1971) แยกบทบาทของเรื่องที่ชอบและไม่ชอบด้วยค่าสัมประสิทธิ์สองตัว ทำให้ควบคุมแรงผลักได้ ดังสมการ (4.38)

$$\mathbf{p}_u^m \propto \alpha \frac{1}{|L_u|} \sum_{i \in L_u} \mathbf{v}_i^m - \beta \frac{1}{|D_u|} \sum_{j \in D_u} \mathbf{v}_j^m \tag{4.38}$$

โดยที่

- D_u เซตของเรื่องที่ผู้ใช้ u ไม่ชอบ (คะแนน 1–2)
- α น้ำหนักของฝั่งเรื่องที่ชอบ บทนี้ใช้ 1
- β น้ำหนักของฝั่งเรื่องที่ไม่ชอบ ($\beta = 0$ คือโปรไฟล์ค่าเฉลี่ย)

ตัวอย่างการคำนวณ 4.27 โปรไฟล์ Rocchio ของ u1 เมื่อ $\beta = 0.5$

โจทย์ จงสร้างโปรไฟล์ข้อความ Rocchio ของ u1 เมื่อ $\alpha = 1, \beta = 0.5$ แล้วหาคะแนนของ i5 และ Top-3

สมการที่ใช้ สมการ (4.38) และ (4.35)

ข้อมูลที่ใช้ $\bar{\mathbf{p}}$ จากตัวอย่างการคำนวณ 4.25 และเวกเตอร์ i4, i6 จากตารางที่ 4.15

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- คะแนนของ u1 มาจากตารางที่ 4.5: ชอบ i1 Toy Story 5 (5), i2 Inside Out 2 (5), i3 The Wild Robot (4) ไม่ชอบ i4 It Ends with Us (1), i6 Bridget Jones: Mad About the Boy (1) และยังไม่เคยให้คะแนนอีกเจ็ดเรื่อง
- ฝั่งชอบ L_u1 = {i1, i2, i3} ใช้ $\bar{p}$ จากตัวอย่างการคำนวณ 4.25 ฝั่งไม่ชอบ D_u1 = {i4, i6} ซึ่งได้คะแนน 1 ทั้งคู่
- α = 1 และ β = 0.5 เป็นค่าที่เลือกเพื่อการสอน ผลของ β อื่นอยู่ในตารางที่ 4.20 และภาพที่ 4.38

ขั้นที่ 1 ค่าเฉลี่ยของเรื่องที่ไม่ชอบ

- romance: $(0.8027 + 0.6780)/2 = 0.7404$, family: $(0.4528 + 0.7314)/2 = 0.5921$

ขั้นที่ 2 ลบด้วยน้ำหนัก β

- family: $0.4239 - 0.5(0.5921) = 0.1278$
- romance: $0 - 0.5(0.7404) = -0.3702$

ขั้นที่ 3 ปรับความยาวและจัดอันดับ

- คะแนนของ i5 ลดจาก 0.4109 (ค่าเฉลี่ย) เป็น 0.0865
- Top-3: i11 (0.3282), i12 (0.2827), i10 (0.1901)

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

มิติ	$\bar{p}$ ชอบ	i4	i6	เฉลี่ยไม่ชอบ	$\bar{p}$ − 0.5 × ไม่ชอบ	ปรับ L^2
tech	0.1896	0.0000	0.0000	0.0000	0.1896	0.2809
mind	0.4285	0.1940	0.0553	0.1247	0.3661	0.5424
space	0.0143	0.0000	0.0000	0.0000	0.0143	0.0212
romance	0.0000	0.8027	0.6780	0.7404	−0.3702	−0.5484
crime	0.0286	0.3051	0.0000	0.1526	−0.0477	−0.0706
action	0.0934	0.0555	0.0000	0.0278	0.0796	0.1179
family	0.4239	0.4528	0.7314	0.5921	0.1278	0.1894
music	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
fantasy	0.1833	0.0000	0.0000	0.0000	0.1833	0.2716
toys	0.2963	0.0000	0.0000	0.0000	0.2963	0.4390
society	0.0000	0.1294	0.0475	0.0885	−0.0442	−0.0655
ambition	0.0096	0.0000	0.0000	0.0000	0.0096	0.0142

- ตัวอย่างมิติ family: 0.4239 − 0.5 × 0.5921 = 0.4239 − 0.2961 = 0.1278 ยังเป็นบวกแต่เล็กลงมาก

- ความยาวก่อนปรับ = 0.6750 หลังปรับ romance = −0.5484 เป็นมิติเดียวที่ติดลบชัดเจน

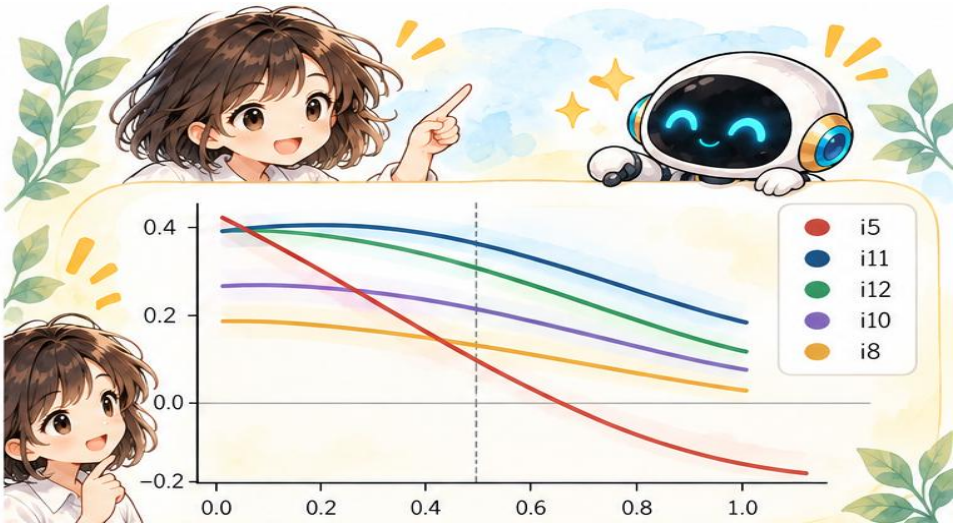
คะแนนของรายการผู้สมัครทั้งเจ็ดเรื่อง

อันดับ	รายการผู้สมัคร	s(u1, i)	สองมิติที่ให้คะแนนบวกมากที่สุด
1	i11 Mickey 17	0.3282	tech 0.1671, mind 0.1362
2	i12 Project Hail Mary	0.2827	mind 0.1425, tech 0.0688
3	i10 Dune: Part Two	0.1901	mind 0.1098, action 0.0680
4	i8 Furiosa: A Mad Max Saga	0.1166	action 0.1150, family 0.0146
5	i5 Wicked	0.0865	fantasy 0.1569, family 0.0817
6	i9 Conclave	0.0215	mind 0.0735, fantasy 0.0103
7	i7 Ballerina	0.0170	action 0.0486, mind 0.0179

ตีความ

β = 0.5 ผลักน้อยกว่าการถ่วงด้วยคะแนน มิติ family ยังเป็นบวก แต่ i5 Wicked ซึ่งมีมิติ romance ร่วมกับเรื่องที่ไม่ชอบหลุดจาก Top-3 ระบบจึงหันไปแนะนำแนววิทยาศาสตร์ i11 และ i12 แทน

ภาพที่ 4.38 แสดงคะแนนข้อความของห้าเรื่องเมื่อเปลี่ยน β จาก 0 ถึง 1 คะแนนของ i5 Wicked ลดลงเร็วที่สุดเพราะมีมิติ romance และสลับอันดับกับ i11 ที่ β ประมาณ 0.05 ค่า β จึงเป็นไฮเปอร์พารามิเตอร์ที่เปลี่ยนคำแนะนำได้จริง



ภาพที่ 4.38 คะแนนข้อความของ u1 เมื่อเปลี่ยน β ของ Rocchio

ที่มาของไฮเปอร์พารามิเตอร์ ค่า β ของ Rocchio มาจากไหน

- **ที่มา** งานค้นคืนสารสนเทศนิยม $\alpha = 1, \beta = 0.75$ หรือค่าใกล้เคียง (Manning et al., 2008) เพราะผลป้อนกลับเชิงบวกมักน่าเชื่อกว่าเชิงลบ
- **ผลเมื่อเปลี่ยน** ตารางที่ 4.20 แสดงว่าเมื่อใช้ข้อความอย่างเดียว $\beta = 0.25$ ให้ MRR ดีที่สุด และเมื่อรวมสี่สื่อ β ระหว่าง 0.25–0.5 ให้ HR@3 ดีขึ้น แต่ β สูงเกินทำให้ผลแย่ลงทั้งสองกรณี
- **วิธีเลือก** ทดลองหลายค่าบนชุดตรวจสอบ ห้ามเลือกจากชุดทดสอบ และระวังผู้ใช้ที่มีเรื่องไม่ชอบน้อยมาก ซึ่งค่าเฉลี่ยของฝั่งไม่ชอบจะไม่เสถียร

ตารางที่ 4.20 ผลของ β ต่อ HR@3 และ MRR (เทคนิค Transformer)

β	ข้อความอย่างเดียว		รวมสี่สื่อ (น้ำหนักตั้งต้น)	
	HR@3	MRR	HR@3	MRR
0.00	0.70	0.6044	0.75	0.7635
0.25	0.70	0.6794	0.80	0.7283
0.50	0.65	0.6453	0.80	0.7300
0.75	0.60	0.6440	0.80	0.6325
1.00	0.60	0.6153	0.70	0.6110

ที่มา: ผู้เขียนคำนวณด้วยโปรโตคอลของหัวข้อ 4.7.1 (20 กรณี)

4.5.5 โปรไฟล์ที่ลดน้ำหนักตามเวลา

รสนิยมของผู้ใช้เปลี่ยนได้ ระบบจึงควรให้เรื่องที่ให้คะแนนล่าสุดมีน้ำหนักมากกว่า วิธีที่นิยมคือการลดน้ำหนักแบบเอกซ์โพเนนเชียลตามอายุของคะแนน (Koren, 2010) ซึ่งแปลงวันที่เป็นน้ำหนักก่อน แล้วจึงเฉลี่ยแบบถ่วงน้ำหนัก ดังสมการ (4.39) และ (4.40)

$$w_i = 2^{-\Delta t_i / h} \tag{4.39}$$

โดยที่

- Δt_i อายุของคะแนน คือจำนวนวันจากวันที่ให้คะแนนรายการ i ถึงวันที่สร้างคำแนะนำ
- h ครึ่งชีวิต (Half-life) คะแนนที่อายุ h วันเหลือน้ำหนักครึ่งหนึ่ง

- w_i น้ำหนักตามเวลา อยู่ระหว่าง 0 ถึง 1

$$\mathbf{p}_u^m \propto \frac{\sum_{i \in L_u} w_i \mathbf{v}_i^m}{\sum_{i \in L_u} w_i} \tag{4.40}$$

โดยที่

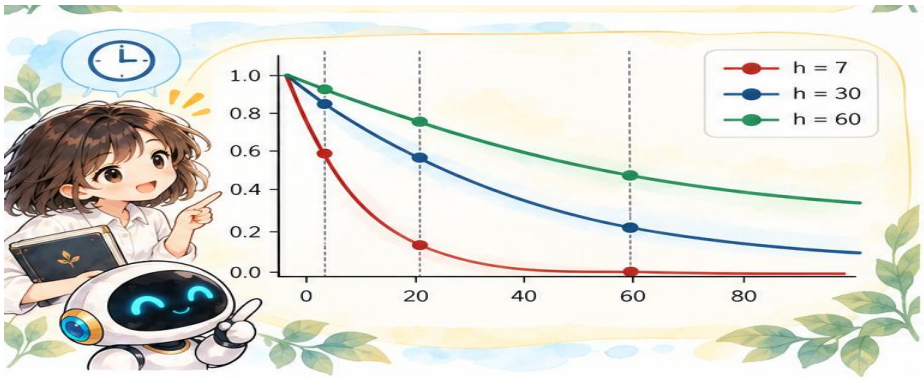
- $\sum_{i \in L_u} w_i$ ผลรวมน้ำหนัก ใช้หารเพื่อให้เป็นค่าเฉลี่ยถ่วงน้ำหนัก

ตารางที่ 4.21 แสดงวันที่สมมติที่ u1 ให้คะแนนเรื่องที่ชอบ และผลของการแปลงแต่ละขั้น เมื่อวันที่สร้างคำแนะนำคือ 1 ต.ค. 2569 และ $h = 30$ วัน ส่วนภาพที่ 4.39 แสดงเส้นน้ำหนักเมื่อครึ่งชีวิตต่างกัน

ตารางที่ 4.21 การแปลงวันที่ให้คะแนนของ u1 เป็นน้ำหนักตามเวลา

รหัส	ชื่อเรื่อง	วันที่ให้คะแนน	Δt (วัน)	Δt/h	w	สัดส่วน
i2	Inside Out 2	26 ก.ย. 2569	5	0.1667	0.8909	50.3%
i1	Toy Story 5	11 ก.ย. 2569	20	0.6667	0.6300	35.6%
i3	The Wild Robot	2 ส.ค. 2569	60	2.0000	0.2500	14.1%

ที่มา: วันที่เป็นค่าสมมติ วันที่สร้างคำแนะนำ 1 ต.ค. 2569 และ h = 30 วัน



ภาพที่ 4.39 น้ำหนักตามเวลาเมื่อครึ่งชีวิต h เท่ากับ 7, 30 และ 60 วัน

ตัวอย่างการคำนวณ 4.28 โปรไฟล์ข้อความของ u1 ที่ลดน้ำหนักตามเวลา

โจทย์ ใช้วันที่ในตารางที่ 4.21 และ $h = 30$ วัน จงหาน้ำหนัก โปรไฟล์ข้อความ และ Top-3 ของ u1

สมการที่ใช้ สมการ (4.39), (4.40) และ (4.35)

ข้อมูลที่ใช้ ตารางที่ 4.21 และเวกเตอร์จากตารางที่ 4.15

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- เรื่องที่ชอบของ u1 (i1, i2, i3) มาจากตารางที่ 4.5 ส่วนวันที่ให้คะแนนในตารางที่ 4.21 เป็นค่าสมมติ: i2 ล่าสุด (26 ก.ย. 2569) i1 (11 ก.ย.) และ i3 เก่าที่สุด (2 ส.ค.)
- อายุคะแนน Δt นับจากวันที่ให้คะแนนถึงวันสร้างคำแนะนำ 1 ต.ค. 2569: 26 ก.ย. ⟶ 5 วัน, 11 ก.ย. ⟶ 20 วัน, 2 ส.ค. ⟶ 60 วัน (สิงหาคมมี 31 วัน: 29 + 30 + 1)
- ครึ่งชีวิต h = 30 วันเป็นค่าที่เลือกเพื่อการสอน (ระดับเดือนตามแนวทางของ Koren, 2010)

ขั้นที่ 1 แปลงอายุเป็นน้ำหนัก

- i2: $2^{-5/30} = 0.8909$
- i1: $2^{-20/30} = 0.6300$

- i3: $2^{-60/30} = 2^{-2} = 0.2500$

ขั้นที่ 2 หาสัดส่วนอิทธิพล

- ผลรวม = 1.7709 จึงได้สัดส่วน i2 : i1 : i3 = 50.3% : 35.6% : 14.1%

ขั้นที่ 3 เฉลี่ยถ่วงน้ำหนัก (แสดงสามมิติ)

- mind: $(0.8909 \times 0.9879 + 0.6300 \times 0.1287 + 0.2500 \times 0.1688)/1.7709 = 0.5666$
- family: 0.3058 และ toys: 0.3162

ขั้นที่ 4 จัดอันดับ

- Top-3: i12 (0.3754), i11 (0.3633), i5 (0.2703)

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

รายการ	Δt	Δt/h	w = 2^(−Δt/h)	สัดส่วน = w/Σw
i2 Inside Out 2	5	0.1667	0.8909	50.3%
i1 Toy Story 5	20	0.6667	0.6300	35.6%
i3 The Wild Robot	60	2.0000	0.2500	14.1%
รวม			**1.7709**	**100%**

มิติ	w·i2	w·i1	w·i3	ผลรวม/Σw	ปรับ L²
tech	0.0000	0.1351	0.0886	0.1263	0.1708
mind	0.8801	0.0811	0.0422	0.5666	0.7658
space	0.0000	0.0270	0.0000	0.0153	0.0206
romance	0.0000	0.0000	0.0000	0.0000	0.0000
crime	0.0000	0.0541	0.0000	0.0305	0.0413
action	0.0298	0.0811	0.0295	0.0793	0.1072
family	0.1326	0.2127	0.1963	0.3058	0.4134
music	0.0000	0.0000	0.0000	0.0000	0.0000
fantasy	0.0000	0.0541	0.1161	0.0961	0.1298
toys	0.0000	0.5600	0.0000	0.3162	0.4274
society	0.0000	0.0000	0.0000	0.0000	0.0000
ambition	0.0257	0.0000	0.0000	0.0145	0.0196

- ตัวอย่างมิติ mind: (0.8909 × 0.9879 + 0.6300 × 0.1287 + 0.2500 × 0.1688)/1.7709 = (0.8801 + 0.0811 + 0.0422)/1.7709 = 0.5666

คะแนนของรายการผู้สมัครทั้งเจ็ดเรื่อง

อันดับ	รายการผู้สมัคร	s(u1, i)	สองมิติที่ให้คะแนนมากที่สุด
1	i12 Project Hail Mary	0.3754	mind 0.2012, family 0.1043

2	i11 Mickey 17	0.3633	mind 0.1923, tech 0.1016
3	i5 Wicked	0.2703	family 0.1782, fantasy 0.0750
4	i10 Dune: Part Two	0.2483	mind 0.1550, action 0.0618
5	i8 Furiosa: A Mad Max Saga	0.1402	action 0.1046, family 0.0319
6	i7 Ballerina	0.1386	action 0.0442, crime 0.0374
7	i9 Conclave	0.1282	mind 0.1038, action 0.0081

ตีความ

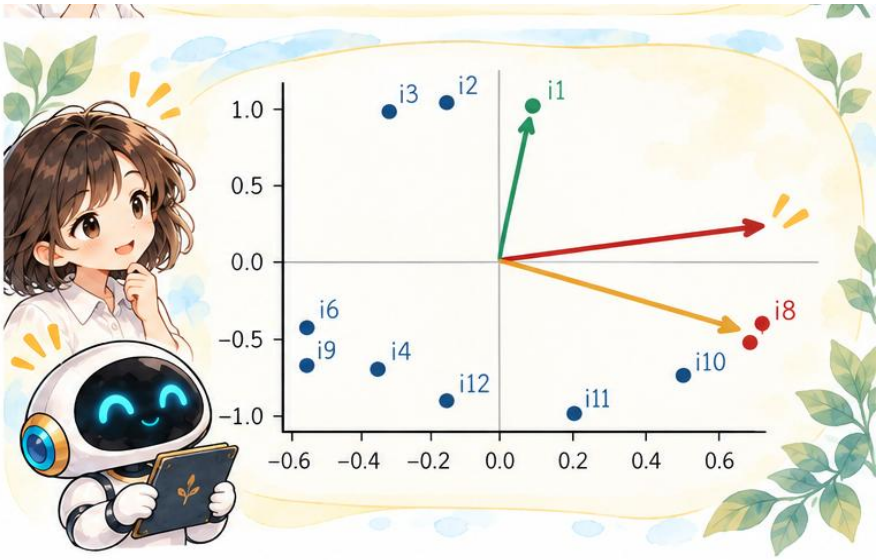
i2 Inside Out 2 ที่ให้คะแนนล่าสุดมีอิทธิพลครึ่งหนึ่งของโปรไฟล์ มิติ mind จึงเด่นขึ้น และ i12 Project Hail Mary ซึ่งมี mind สูงขึ้นมาอันดับ 1 แทน i5 Wicked

ที่มาของไฮเปอร์พารามิเตอร์ ครึ่งชีวิต h มาจากไหน

- **ที่มา** ไม่มีค่ามาตรฐานตายตัว ขึ้นกับความเร็วที่รสนิยมเปลี่ยนในโดเมนนั้น
  งานด้านข่าวใช้ระดับชั่วโมงถึงวัน ส่วนภาพยนตร์และสินค้าใช้ระดับสัปดาห์ถึงเดือน (Koren, 2010)
- **ผลเมื่อเปลี่ยน** h เล็กทำให้ระบบไวต่อพฤติกรรมล่าสุดแต่ลืมเร็ว h
  ใหญ่ทำให้โปรไฟล์เสถียรแต่ตามรสนิยมใหม่ไม่ทัน เมื่อ $h \to \infty$ จะได้โปรไฟล์ค่าเฉลี่ย
- **วิธีเลือก** แบ่งข้อมูลตามเวลา (บทที่ 2) แล้วทดลอง h เช่น 7, 14, 30, 60, 90 วัน
  เลือกค่าที่ตัวชี้วัดบนช่วงเวลาตรวจสอบดีที่สุด

4.5.6 ผู้ใช้ที่ชอบหลายแบบ: โปรไฟล์หลายจุดสนใจและโปรไฟล์แบบ Attention

ผู้ใช้หลายคนมีรสนิยมมากกว่าหนึ่งแบบ u4 ชอบทั้ง i1 Toy Story 5 (แอนิเมชันครอบครัว) และ i7 Ballerina กับ i8 Furiosa (แอ็กชันจังหวะเร็ว) ถ้าหาค่าเฉลี่ยของสามเรื่อง โปรไฟล์จะตกอยู่ "กลางทาง" ซึ่งอาจไม่ใกล้รสนิยมใดเลย ภาพที่ 4.40 แสดงปัญหานี้ในปริภูมิวิดีโอ โปรไฟล์ค่าเฉลี่ยชี้ไปทางแอ็กชันและเกือบตั้งฉากกับกลุ่มแอนิเมชัน



ภาพที่ 4.40 โปรไฟล์ค่าเฉลี่ยกับโปรไฟล์สองจุดสนใจของ u4 ในปริภูมิวิดีโอ

มีสองวิธีแก้ วิธีแรกคือ **โปรไฟล์หลายจุดสนใจ (Multi-interest Profile)** ซึ่งแบ่งเรื่องที่ชอบเป็นกลุ่ม แล้วให้คะแนนด้วยจุดสนใจที่ใกล้รายการนั้นที่สุด (Li et al., 2019) ดังสมการ (4.41) วิธีที่สองคือ **โปรไฟล์แบบ Attention** ซึ่งสร้างโปรไฟล์ใหม่สำหรับรายการผู้สมัครแต่ละเรื่อง โดยให้น้ำหนักเรื่องในประวัติที่คล้ายรายการนั้นมากกว่า (Zhou et al., 2018) ดังสมการ (4.42)

$$s_m(u,i) = \max_{k=1,\dots,K} \mathbf{p}_u^{m,k} \cdot \mathbf{v}_i^m \tag{4.41}$$

โดยที่

- K จำนวนจุดสนใจ
- $\mathbf{p}_u^{m,k}$ โปรไฟล์ค่าเฉลี่ยของเรื่องที่ชอบในกลุ่มที่ k (แบ่งกลุ่มด้วย k-means แบบโคไซน์)

$$\mathbf{p}_u^m(i) \propto \sum_{j \in L_u} \alpha_{ij}\, \mathbf{v}_j^m, \qquad \alpha_{ij} = \frac{\exp\left(\mathbf{v}_i^m \cdot \mathbf{v}_j^m / \tau\right)}{\sum_{l \in L_u} \exp\left(\mathbf{v}_i^m \cdot \mathbf{v}_l^m / \tau\right)} \tag{4.42}$$

โดยที่

- $\mathbf{p}_u^m(i)$ โปรไฟล์ที่สร้างเฉพาะสำหรับรายการผู้สมัคร i
- α_{ij} น้ำหนัก Attention ที่รายการ i ให้เรื่อง j ในประวัติ ซึ่งคือ softmax ของสมการ (4.22)
- τ อุณหภูมิ ยิ่งเล็กยิ่งเลือกเรื่องที่คล้ายที่สุดเรื่องเดียว บทนี้ใช้ 0.5

ตัวอย่างการคำนวณ 4.29 โปรไฟล์วิดีโอสองจุดสนใจของ u4

โจทย์ u4 ให้ i1 = 4, i7 = 5, i8 = 5 จงเทียบ Top-3 ของโปรไฟล์ค่าเฉลี่ยกับโปรไฟล์สองจุดสนใจ ($K = 2$) บนเวกเตอร์วิดีโอเทคนิค Transformer

สมการที่ใช้ สมการ (4.36), (4.35) และ (4.41)

ข้อมูลที่ใช้ คะแนนของ u4 จากตารางที่ 4.5 เวกเตอร์วิดีโอจากตารางที่ 4.16 เตรียมตามหัวข้อ 4.5.1

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- คะแนนของ u4 มาจากตารางที่ 4.5: ชอบ i1 Toy Story 5 (4), i7 Ballerina (5), i8 Furiosa (5) และไม่ชอบ i5 Wicked (1) เรื่องที่ชอบจึงมาจากสองแนวที่ต่างกันมาก คือแอนิเมชัน (i1) กับแอ็กชัน (i7, i8) ตามตารางที่ 4.4
- รายการผู้สมัครของ u4 คือแปดเรื่องที่ u4 ยังไม่เคยให้คะแนน ได้แก่ i2, i3, i4, i6, i9, i10, i11 และ i12

- เวกเตอร์วิดีโอ V1–V3 มาจากตารางที่ 4.16: แอนิเมชันสามเรื่องมี V3 = 1 ส่วนแอ็กชันมี V1, V2 สูง

(ตัดต่อเร็ว) แล้วเตรียมด้วยสมการ (4.33)–(4.34)

ขั้นที่ 1 เวกเตอร์ที่เตรียมแล้ว

- $\mathbf{v}_{i1} = (0.0885, 0.0111, 0.9960)$, $\mathbf{v}_{i7} = (0.6080, 0.6771, -0.4146)$, $\mathbf{v}_{i8} = (0.6358, 0.6926, -0.3406)$
- โคไซน์ i7–i8 = 0.9968 (กลุ่มเดียวกัน) แต่ i1–i7 = −0.3516 (ต่างกลุ่ม)

ขั้นที่ 2 โปรไฟล์ค่าเฉลี่ย

- $\mathbf{p} = (0.6890, 0.7140, 0.1245)$ คะแนนของ i2 Inside Out 2 เพียง 0.0066
- Top-3: i10 (0.5042), i11 (0.0801), i2 (0.0066)

ขั้นที่ 3 แบ่งกลุ่มเป็นสองจุดสนใจ

- จุดสนใจ 1 = {i1}: $\mathbf{p}^1 = (0.0885, 0.0111, 0.9960)$
- จุดสนใจ 2 = {i7, i8}: $\mathbf{p}^2 = (0.6224, 0.6854, -0.3779)$

ขั้นที่ 4 ให้คะแนนด้วยจุดสนใจที่ใกล้ที่สุด

- i2: $\max(0.9825, -0.4852) = 0.9825$
- i10: $\max(-0.7297, 0.8531) = 0.8531$
- Top-3: i2 (0.9825), i3 (0.9333), i10 (0.8531)

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

เตรียมเวกเตอร์วิดีโอ: ค่าเฉลี่ยของคลัง μV = (0.5333, 0.4917, 0.2500)

- μV มาจากผลรวมแต่ละมิติของ 12 เรื่องหารด้วย 12: V1 = 6.4/12, V2 = 5.9/12, V3 = 3/12 = 0.25 (มีเพียงสามเรื่องที่ V3 = 1)

รายการ	V (ตารางที่ 4.16)	V − μV	ความยาว	v (ยาว 1)
i1 Toy Story 5	(0.6, 0.5, 1.0)	(0.0667, 0.0083, 0.7500)	0.7530	(0.0885, 0.0111, 0.9960)
i7 Ballerina	(0.9, 0.9, 0.0)	(0.3667, 0.4083, −0.2500)	0.6031	(0.6080, 0.6771, −0.4146)
i8 Furiosa: A Mad Max Saga	(1.0, 1.0, 0.0)	(0.4667, 0.5083, −0.2500)	0.7339	(0.6358, 0.6926, −0.3406)
i2 Inside Out 2	(0.5, 0.4, 1.0)	(−0.0333, −0.0917, 0.7500)	0.7563	(−0.0441, −0.1212, 0.9916)
i3 The Wild Robot	(0.4, 0.3, 1.0)	(−0.1333, −0.1917, 0.7500)	0.7855	(−0.1697, −0.2440, 0.9548)
i4 It Ends with Us	(0.4, 0.3, 0.0)	(−0.1333, −0.1917, −0.2500)	0.3421	(−0.3898, −0.5603, −0.7308)
i6 Bridget Jones: Mad About the Boy	(0.2, 0.2, 0.0)	(−0.3333, −0.2917, −0.2500)	0.5086	(−0.6554, −0.5735, −0.4915)
i9 Conclave	(0.3, 0.4, 0.0)	(−0.2333, −0.0917, −0.2500)	0.3540	(−0.6591, −0.2589, −0.7061)
i10 Dune: Part Two	(0.7, 0.6, 0.0)	(0.1667, 0.1083, −0.2500)	0.3194	(0.5218, 0.3392, −0.7827)
i11 Mickey 17	(0.6, 0.5, 0.0)	(0.0667, 0.0083, −0.2500)	0.2589	(0.2575, 0.0322, −0.9657)
i12 Project Hail Mary	(0.5, 0.4, 0.0)	(−0.0333, −0.0917, −0.2500)	0.2684	(−0.1242, −0.3416, −0.9316)

โปรไฟล์ค่าเฉลี่ยเทียบกับโปรไฟล์สองจุดสนใจ

- โปรไฟล์ค่าเฉลี่ย = l2((v_i1 + v_i7 + v_i8)/3) = (0.6890, 0.7140, 0.1245)
- จุดสนใจ 1 = v_i1 = (0.0885, 0.0111, 0.9960) และจุดสนใจ 2 = l2((v_i7 + v_i8)/2) = (0.6224, 0.6854, −0.3779)

รายการผู้สมัคร	ค่าเฉลี่ย	จุดสนใจ 1	จุดสนใจ 2	max
i2 Inside Out 2	0.0066	0.9825	−0.4852	0.9825
i3 The Wild Robot	−0.1723	0.9333	−0.6337	0.9333
i4 It Ends with Us	−0.7596	−0.7686	−0.3505	−0.3505
i6 Bridget Jones: Mad About the Boy	−0.9222	−0.5539	−0.6152	−0.5539
i9 Conclave	−0.7269	−0.7645	−0.3208	−0.3208
i10 Dune: Part Two	0.5042	−0.7297	0.8531	0.8531
i11 Mickey 17	0.0801	−0.9387	0.5473	0.5473
i12 Project Hail Mary	−0.4455	−0.9427	0.0406	0.0406

- Top-3 ของโปรไฟล์ค่าเฉลี่ย: i10 (0.5042), i11 (0.0801), i2 (0.0066) · Top-3 ของสองจุดสนใจ: i2 (0.9825), i3 (0.9333), i10 (0.8531)

ตีความ

โปรไฟล์ค่าเฉลี่ยมองไม่เห็นความชอบแอนิเมชันของ u4 เลย ส่วนโปรไฟล์สองจุดสนใจแนะนำทั้งแอนิเมชัน (i2, i3) และแอ็กชัน (i10) จึงครอบคลุมรสนิยมทั้งสองด้าน

ตัวอย่างการคำนวณ 4.30 โปรไฟล์แบบ Attention ของ u4

โจทย์ จงสร้างโปรไฟล์วิดีโอแบบ Attention ของ u4 สำหรับรายการผู้สมัคร i2 Inside Out 2 และ i10 Dune: Part Two เมื่อ $\tau = 0.5$

สมการที่ใช้ สมการ (4.42) และ (4.35)

ข้อมูลที่ใช้ เวกเตอร์วิดีโอที่เตรียมแล้วจากตัวอย่างการคำนวณ 4.29

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- ใช้ประวัติของ u4 (i1, i7, i8 จากตารางที่ 4.5) และเวกเตอร์วิดีโอที่เตรียมแล้วจากตัวอย่างการคำนวณ 4.29 ซึ่งมาจากตารางที่ 4.16
- เลือกผู้สมัครสองเรื่องที่อยู่คนละแนว: i2 Inside Out 2 (แอนิเมชัน เหมือน i1) และ i10 Dune: Part Two (แอ็กชัน เหมือน i7 และ i8) ตามตารางที่ 4.4
- อุณหภูมิ $\boldsymbol{\tau}$ = 0.5 เป็นค่าที่เลือกเพื่อการสอน

ขั้นที่ 1 รายการ i2 ถามประวัติ

- โคไซน์กับ i1, i7, i8: (0.9825, −0.5200, −0.4497)
- หารด้วย $\tau = 0.5$ แล้ว softmax: $\alpha = (0.9037, 0.0448, 0.0515)$
- โปรไฟล์จึงเกือบเท่ากับ i1 คะแนนของ i2 = 0.9577

ขั้นที่ 2 รายการ i10 ถามประวัติ

- โคไซน์: (−0.7297, 0.8714, 0.8333) ได้ $\alpha = (0.0207, 0.5083, 0.4710)$
- โปรไฟล์เอียงไปทาง i7 และ i8 คะแนนของ i10 = 0.8438

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

ผู้สมัคร i2 Inside Out 2 ถามประวัติ

ประวัติ	cos(v_i2, v_h)	÷ τ	e^(·)	α
i1	0.9825	1.9649	7.1342	0.9037
i7	−0.5200	−1.0399	0.3535	0.0448
i8	−0.4497	−0.8995	0.4068	0.0515
รวม			**7.8945**	**1.0000**

- โปรไฟล์ = l2(0.9037 v_i1 + 0.0448 v_i7 + 0.0515 v_i8) = (0.1593, 0.0865, 0.9834) · คะแนน = โปรไฟล์ · v_i2 = 0.9577

ผู้สมัคร i10 Dune: Part Two ถามประวัติ

ประวัติ	cos(v_i10, v_h)	÷ τ	e^(·)	α
i1	−0.7297	−1.4593	0.2324	0.0207
i7	0.8714	1.7428	5.7135	0.5083
i8	0.8333	1.6666	5.2944	0.4710
รวม			**11.2402**	**1.0000**

- โปรไฟล์ = l2(0.0207 v_i1 + 0.5083 v_i7 + 0.4710 v_i8) = (0.6278, 0.6898, −0.3606) · คะแนน = โปรไฟล์ · v_i10 = 0.8438

ตีความ

โปรไฟล์แบบ Attention ไม่ต้องกำหนดจำนวนจุดสนใจล่วงหน้า
แต่ต้องคำนวณโปรไฟล์ใหม่ทุกครั้งที่ให้คะแนนรายการผู้สมัคร
จึงใช้การคำนวณมากกว่าโปรไฟล์ที่คำนวณไว้ล่วงหน้า

ที่มาของไฮเปอร์พารามิเตอร์ จำนวนจุดสนใจ K และอุณหภูมิ τ มาจากไหน

- K ระบบขนาดใหญ่อย่าง MIND ใช้ 4–8 จุดสนใจ (Li et al., 2019) สำหรับผู้ใช้ที่มีประวัติน้อย $K = 2$ ก็เพียงพอ และควรกำหนดให้แต่ละกลุ่มมีเรื่องอย่างน้อยหนึ่งเรื่อง
- τ ค่าเล็ก เช่น 0.1 ทำให้โปรไฟล์แบบ Attention ใกล้เคียงการเลือกเรื่องที่คล้ายที่สุดเรื่องเดียว ค่าใหญ่ เช่น 10 ทำให้ใกล้โปรไฟล์ค่าเฉลี่ย บทนี้ใช้ 0.5 เพื่อให้เห็นทั้งสองด้าน
- **วิธีเลือก** ทดลองบนชุดตรวจสอบเช่นเดียวกับ β และ h

4.5.7 ตัวเข้ารหัสลำดับผู้ใช้ด้วย CNN และ Transformer

ทุกวิธีที่ผ่านมามองประวัติของผู้ใช้เป็น **เซต** ที่ไม่มีลำดับ (ยกเว้นการลดตามเวลา) แต่ลำดับมีความหมาย ผู้ใช้ที่เพิ่งดูแอ็กชันสองเรื่องติดกันน่าจะอยากดูแอ็กชันอีก แนวคิดของหัวข้อ 4.3 และ 4.4 จึงถูกนำมาใช้กับ **ลำดับของรายการ** แทนลำดับของคำ

- **แบบ CNN (Caser)** เรียงเวกเตอร์ของ L รายการล่าสุดเป็นตาราง $L \times d$ แล้วใช้ตัวกรองแนวตั้งซึ่งถ่วงน้ำหนักรายการตามตำแหน่งเวลา และตัวกรองแนวนอนซึ่งจับลวดลายของรายการติดกัน (Tang and Wang, 2018)
- **แบบ Transformer (SASRec)** ใช้ Self-Attention บนลำดับรายการ พร้อม **หน้ากากเชิงเหตุ (Causal Mask)** ให้ตำแหน่ง t มองได้เฉพาะรายการที่ตำแหน่ง $\leq t$ แล้วใช้เวกเตอร์ของตำแหน่งสุดท้ายเป็นโปรไฟล์สำหรับทายรายการถัดไป (Kang and McAuley, 2018)

$$\mathbf{p}_u^{Caser} = \sum_{t=1}^{L} c_t \, \mathbf{v}_{(t)}, \qquad \mathbf{p}_u^{SAS} = \sum_{t=1}^{L} \alpha_{L,t} \, \mathbf{v}_{(t)}, \quad \alpha_{L,t} = \mathrm{softmax}_t\left(\frac{\mathbf{v}_{(L)} \cdot \mathbf{v}_{(t)}}{\sqrt{d}}\right) \tag{4.43}$$

โดยที่

- $\mathbf{v}_{(t)}$ เวกเตอร์ของรายการลำดับที่ t ในประวัติ เรียงจากเก่าไปใหม่
- c_t น้ำหนักของตัวกรองแนวตั้งที่ตำแหน่ง t ซึ่งเรียนรู้จากผู้ใช้ทุกคน
- $\alpha_{L,t}$ น้ำหนัก Attention ของตำแหน่งสุดท้ายต่อตำแหน่ง t (หน้ากากเชิงเหตุทำให้ $t \leq L$ เท่านั้น)
- d จำนวนมิติของเวกเตอร์

ตัวอย่างการคำนวณ 4.31 โปรไฟล์จากลำดับของ u4 แบบ Caser และ SASRec

โจทย์ สมมติว่า u4 ดู i1 ก่อน แล้วจึงดู i7 และ i8 ตามลำดับ จงหาโปรไฟล์วิดีโอแบบ Caser ($c = (0.2,0.3,0.5)$) และแบบ SASRec แล้วหา Top-3

สมการที่ใช้ สมการ (4.43) และ (4.35)

ข้อมูลที่ใช้ เวกเตอร์วิดีโอที่เตรียมแล้วจากตัวอย่างการคำนวณ 4.29 ลำดับเวลาเป็นค่าสมมติ

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- รายการที่ u4 ชอบ (i1, i7, i8) มาจากตารางที่ 4.5 แต่ลำดับเวลา i1 ⟶ i7 ⟶ i8 เป็นค่าสมมติ เพราะตารางคะแนนไม่ได้เก็บลำดับ
- เวกเตอร์วิดีโอที่เตรียมแล้วมาจากตัวอย่างการคำนวณ 4.29 น้ำหนักตามตำแหน่ง c = (0.2, 0.3, 0.5) เป็นค่าสมมติแทนน้ำหนักที่ Caser เรียนรู้ และ d = 3 คือจำนวนมิติของเวกเตอร์วิดีโอ

ขั้นที่ 1 แบบ Caser: ถ่วงตามตำแหน่งเวลา

- $\mathbf{p} = 0.2\mathbf{v}_{i1} + 0.3\mathbf{v}_{i7} + 0.5\mathbf{v}_{i8} = (0.5180, 0.5516, -0.0955)$
- Top-3: i10 (0.6977), i11 (0.3191), i12 (−0.2148)

ขั้นที่ 2 แบบ SASRec: ตำแหน่งสุดท้าย (i8) ถามทุกตำแหน่ง

- ผลคูณจุดหารด้วย $\sqrt{3}$: (−0.1589, 0.5755, 0.5774)
- softmax: $\alpha = (0.1933, 0.4030, 0.4037)$
- $\mathbf{p} = (0.5188, 0.5546, -0.1120)$
- Top-3: i10 (0.7119), i11 (0.3382), i12 (−0.1948)

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

Caser: ถ่วงตามตำแหน่งเวลา

ตำแหน่ง	รายการ	c_t	c_t × v_t
1	i1	0.2	(0.0177, 0.0022, 0.1992)
2	i7	0.3	(0.1824, 0.2031, −0.1244)
3	i8	0.5	(0.3179, 0.3463, −0.1703)
รวม		**1.0**	**(0.5180, 0.5516, −0.0955)**

SASRec: ตำแหน่งสุดท้าย (i8) ถามทุกตำแหน่ง

ตำแหน่ง	v_i8 · v_t	÷ √3	**α**	**α** × v_t
i1	−0.2753	−0.1589	0.1933	(0.0171, 0.0021, 0.1926)
i7	0.9968	0.5755	0.4030	(0.2450, 0.2728, −0.1670)
i8	1.0000	0.5774	0.4037	(0.2567, 0.2796, −0.1375)
รวม			**1.0000**	**(0.5188, 0.5546, −0.1120)**

คะแนนของผู้สมัครแปดเรื่องหลังปรับโปรไฟล์ให้ยาว 1

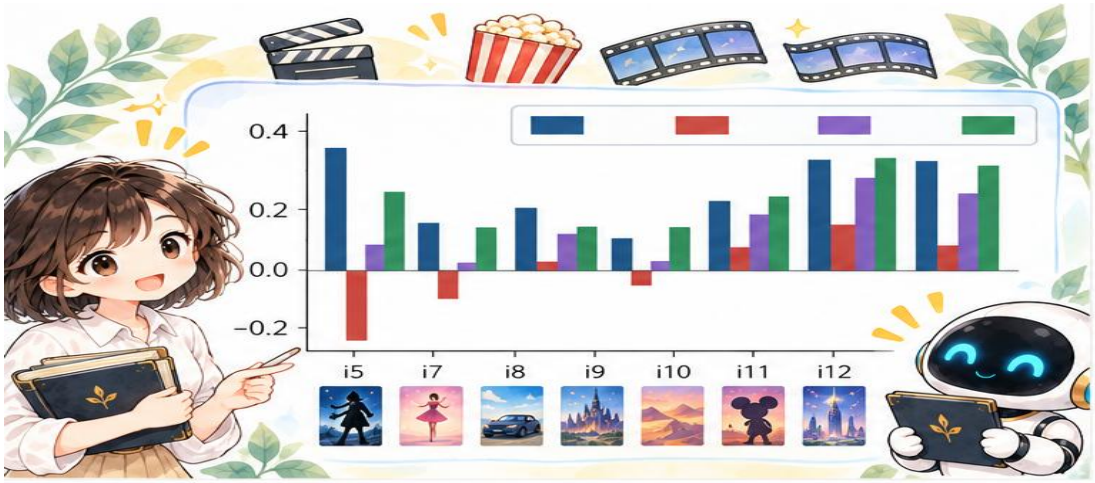
ผู้สมัคร	Caser	SASRec
i2 Inside Out 2	−0.2417	−0.2620
i3 The Wild Robot	−0.4113	−0.4303
i4 It Ends with Us	−0.5785	−0.5616
i6 Bridget Jones: Mad About the Boy	−0.7983	−0.7855
i9 Conclave	−0.5465	−0.5294
i10 Dune: Part Two	0.6977	0.7119
i11 Mickey 17	0.3191	0.3382
i12 Project Hail Mary	−0.2148	−0.1948

ตีความ

ทั้งสองแบบให้ความสำคัญกับเรื่องล่าสุด (แอ็กชัน) จึงแนะนำ i10 เป็นอันดับแรก ต่างจากโปรไฟล์หลายจุดสนใจที่ยังเก็บความชอบแอนิเมชันไว้ ในระบบจริงน้ำหนัก c_t และเมทริกซ์ฉายของ SASRec ได้จากการฝึกบนลำดับของผู้ใช้จำนวนมาก ซึ่งต้องใช้ข้อมูลมากกว่าที่ MovieMini-12 มี

4.5.8 เปรียบเทียบโปรไฟล์จากเทคนิค CNN และ Transformer

ระบบหลายรูปแบบเก็บโปรไฟล์แยกตามสื่อ คือ $\{\mathbf{p}_u^T, \mathbf{p}_u^I, \mathbf{p}_u^A, \mathbf{p}_u^V\}$ ซึ่งเรียกรวมว่า **โปรไฟล์ผู้ใช้หลายสื่อ (Multimodal User Profile)** แล้วจึงรวมคะแนนรายสื่อด้วย Fusion ในหัวข้อ 4.6 ภาพที่ 4.41 เทียบคะแนนข้อความของ u1 จากสี่วิธีสร้างโปรไฟล์บนเทคนิค Transformer



ภาพที่ 4.41 คะแนนข้อความของ u1 จากสี่วิธีสร้างโปรไฟล์

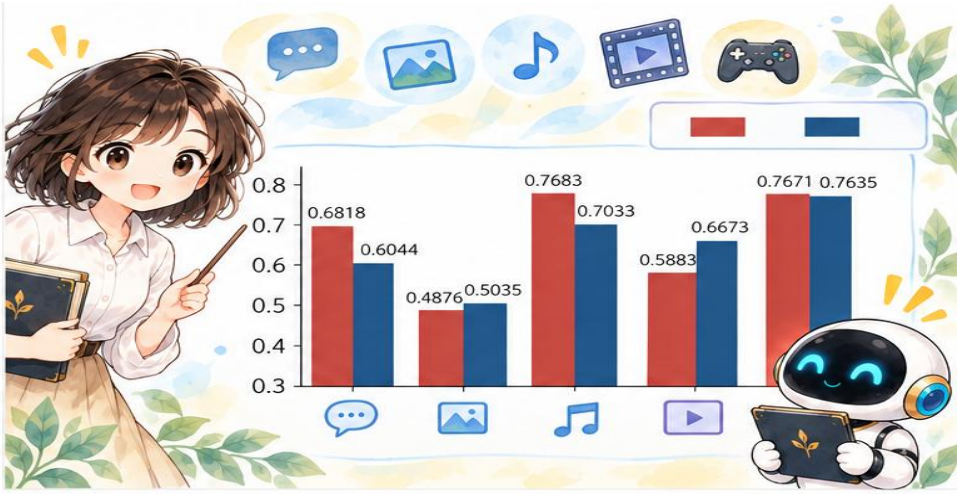
ตารางที่ 4.22 เทียบผลของสองเทคนิคต่อผู้ใช้ u1 คอลัมน์ที่สองคือโคไซน์ระหว่างโปรไฟล์ CNN กับโปรไฟล์ Transformer ของสื่อที่ต่างกันมากที่สุด ส่วนสองคอลัมน์สุดท้ายคือ Top-3 หลังรวมสี่สื่อด้วยน้ำหนักตั้งต้นของหัวข้อ 4.6.2

ตารางที่ 4.22 โปรไฟล์และ Top-3 ของ u1 จากสองเทคนิค

วิธีสร้างโปรไฟล์	โคไซน์โปรไฟล์ CNN–TF (ต่ำสุด)	Top-3 เทคนิค CNN	Top-3 เทคนิค Transformer
ค่าเฉลี่ยเรื่องที่ชอบ	0.9781	i5, i8, i9	i5, i8, i9
ถ่วงด้วยคะแนน	0.9825	i8, i7, i5	i8, i7, i5
Rocchio β = 0.5	0.9738	i5, i8, i7	i5, i8, i7
ลดตามเวลา h = 30	0.9837	i5, i8, i9	i5, i8, i12

ที่มา: ผู้เขียนคำนวณ น้ำหนักรวมสื่อ (T, I, A, V) = (0.4, 0.2, 0.2, 0.2)

ภาพที่ 4.42 สรุปผลการประเมินแบบกันออกทีละรายการของโปรไฟล์ค่าเฉลี่ย แยกตามสื่อและเทคนิค เทคนิค CNN ดีกว่าในข้อความและเสียง ส่วน Transformer ดีกว่าในภาพและวิดีโอ แต่เมื่อรวมสี่สื่อแล้วผลต่างกันเพียงเล็กน้อย



ภาพที่ 4.42 MRR ของโปรไฟล์ค่าเฉลี่ยจากสองเทคนิค

แนวทางเลือกใช้ เลือกวิธีสร้างโปรไฟล์อย่างไร

- **เริ่มจากโปรไฟล์ค่าเฉลี่ย** เพราะไม่มีไฮเปอร์พารามิเตอร์ อธิบายง่าย และในบทนี้ให้ MRR ดีที่สุดเมื่อรวมสี่สื่อ
- **ใช้ Rocchio** เมื่อผู้ใช้มีเรื่องที่ไม่ชอบชัดเจนและต้องการลดการแนะนำแนวที่ผู้ใช้ปฏิเสธ แต่ควรเลือก β ด้วยชุดตรวจสอบ
- **ใช้การลดตามเวลา** เมื่อรสนิยมเปลี่ยนเร็วหรือประวัติยาวหลายปี
- **ใช้โปรไฟล์หลายจุดสนใจหรือแบบ Attention** เมื่อผู้ใช้ชอบหลายแนวที่ต่างกันมาก
- **ใช้ตัวเข้ารหัสลำดับ (Caser, SASRec)** เมื่อมีประวัติเป็นลำดับยาวและมีผู้ใช้จำนวนมากพอให้ฝึก

อธิบายแบบเข้าใจง่าย โปรไฟล์คือ "ภาพเฉลี่ย" ของรสนิยม

ถ้านำโปสเตอร์ของทุกเรื่องที่เราชอบมาซ้อนกันแบบโปร่งแสง ภาพที่ได้คือโปรไฟล์ค่าเฉลี่ย เรื่องใหม่ที่ "ซ้อนทับได้พอดี" ย่อมน่าแนะนำ

Rocchio คือการซ้อนโปสเตอร์ที่ไม่ชอบแบบกลับสีลงไปด้วย การลดตามเวลาคือการทำให้โปสเตอร์เก่าจางลง และโปรไฟล์หลายจุดสนใจคือการแยกกองโปสเตอร์ตามแนวก่อนซ้อน

โปรแกรม 4.12 เทียบโปรไฟล์สี่แบบของสองเทคนิค

```
import numpy as np
from mm4 import ITEMS, R, vectors, l2


DAYS = {'i2': 5, 'i1': 20, 'i3': 60}        # อายุคะแนนของ u1


def profile(V, r, kind, beta=0.5, h=30):
    like = [i for i, x in r.items() if x >= 4]
    dis = [i for i, x in r.items() if x <= 2]
    if kind == 'mean':
        p = np.mean([V[i] for i in like], axis=0)
    elif kind == 'rw':
        rbar = np.mean(list(r.values()))
        p = sum((x - rbar) * V[i] for i, x in r.items())
    elif kind == 'rocchio':
        p = np.mean([V[i] for i in like], axis=0) \
            - beta * np.mean([V[i] for i in dis], axis=0)
    else:
        w = {i: 2 ** (-DAYS[i] / h) for i in like}
        p = sum(w[i] * V[i] for i in like) / sum(w.values())
    return l2(p)

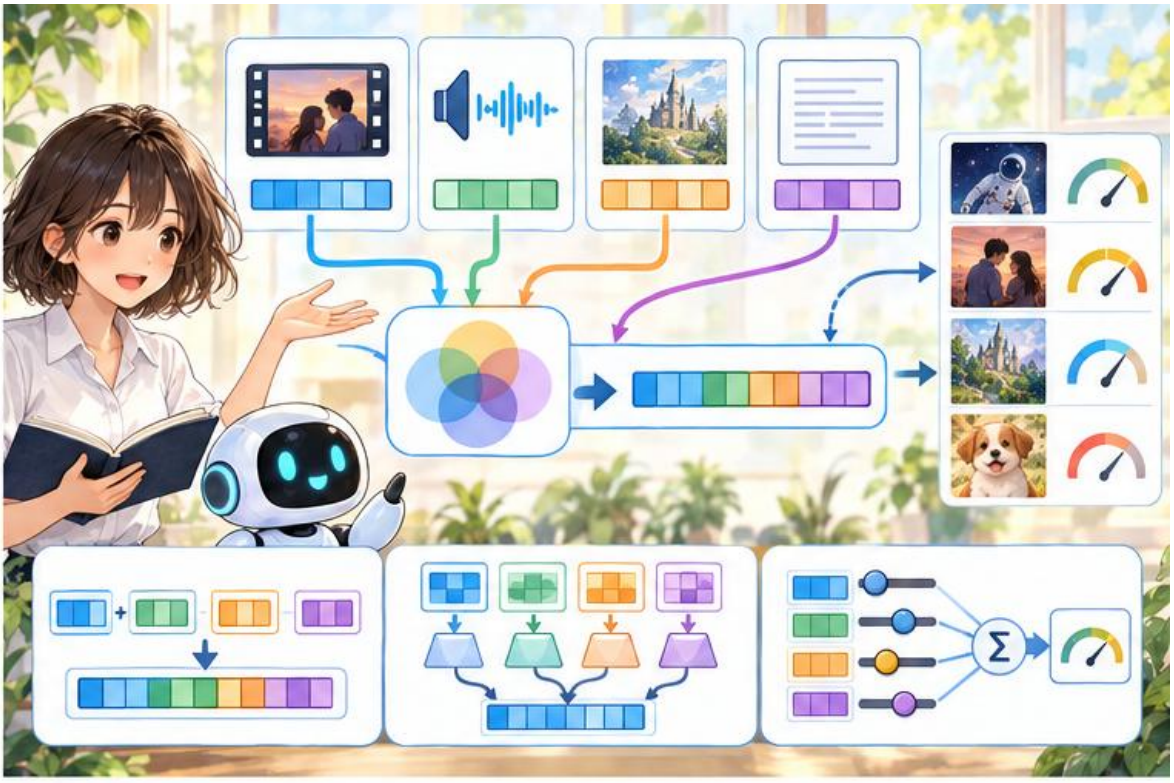

cand = [i for i in ITEMS if i not in R['u1']]
for tech in ['CNN', 'TF']:
    V = vectors(tech)['T']
    for kind in ['mean', 'rw', 'rocchio', 'decay']:
        p = profile(V, R['u1'], kind)
        s = {i: p @ V[i] for i in cand}
        print(tech, kind, sorted(cand, key=lambda i: -s[i])[:3])
```

ผลลัพธ์

```
CNN mean ['i5', 'i12', 'i11']
CNN rw ['i11', 'i12', 'i10']
CNN rocchio ['i12', 'i11', 'i10']
CNN decay ['i12', 'i5', 'i11']
TF mean ['i5', 'i12', 'i11']
TF rw ['i11', 'i12', 'i10']
TF rocchio ['i11', 'i12', 'i10']
TF decay ['i12', 'i11', 'i5']
```

4.6 Multimodal Fusion: รวมหลักฐานจากหลายสื่อ

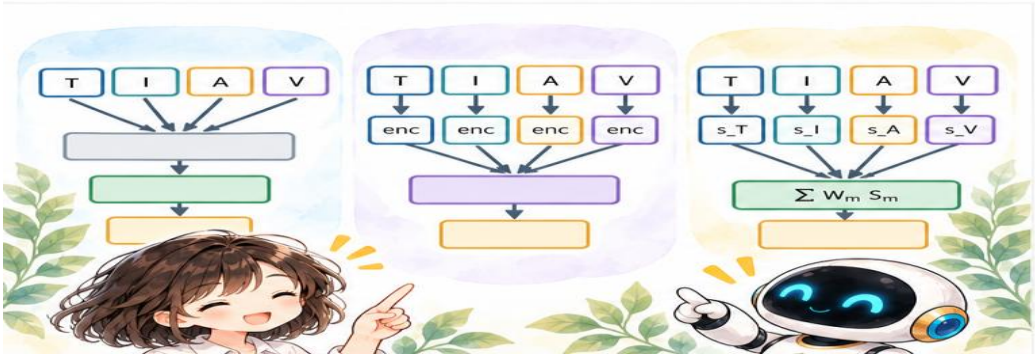
หัวข้อ 4.5 ให้คะแนนของรายการผู้สมัครแยกตามสื่อ ตารางที่ 4.19 แสดงว่าแต่ละสื่อเห็นต่างกันและมาตราของคะแนนต่างกัน **Multimodal Fusion** คือการรวมหลักฐานเหล่านี้เป็นการตัดสินใจเดียว หัวข้อนี้อธิบายเทคนิค Fusion จากง่ายไปยาก ทุกเทคนิคมีตัวอย่างการคำนวณกับ u1 โดยใช้คะแนนจากเทคนิค Transformer ในตารางที่ 4.19 ดังสรุปในภาพที่ 4.57 เปรียบเทียบการรวมระดับต้น ระดับกลาง และระดับปลาย รวมถึงการจัดแนวข้ามสื่อ



ภาพที่ 4.57 ระดับและวิธีรวมหลักฐานหลายสื่อ

4.6.1 สามระดับของการรวม

ภาพที่ 4.43 แสดงสามจุดที่รวมสื่อได้ ซึ่งต่างกันที่ "รวมก่อนหรือหลังให้คะแนน"



ภาพที่ 4.43 Early, Intermediate และ Late Fusion

- **Early Fusion** ต่อเวกเตอร์ของทุกสื่อเป็นเวกเตอร์ยาวเส้นเดียวก่อนสร้างโปรไฟล์และให้คะแนน (หัวข้อ 4.6.6)
- **Intermediate Fusion** ให้แต่ละสื่อผ่านตัวเข้ารหัสของตนก่อน แล้วรวมตรงกลางด้วยกลไกที่ปรับตามผู้ใช้หรือรายการ เช่น Gate หรือ Attention (หัวข้อ 4.6.7)

- **Late Fusion** ให้คะแนนแยกตามสื่อจนจบ แล้วจึงรวมคะแนนหรืออันดับ (หัวข้อ 4.6.2–4.6.5) ซึ่งง่ายที่สุดและเป็นจุดเริ่มต้นที่แนะนำ

อธิบายแบบเข้าใจง่าย คณะกรรมการสี่คนตัดสินภาพยนตร์

ให้ผู้เชี่ยวชาญสี่คนดูภาพยนตร์คนละด้าน คนหนึ่งอ่านเรื่องย่อ คนหนึ่งดูโปสเตอร์ คนหนึ่งฟังเพลง และคนหนึ่งดูตัวอย่างภาพยนตร์

Late Fusion คือให้แต่ละคนให้คะแนนแล้วนำมาถ่วงน้ำหนัก **Early Fusion** คือให้ทั้งสี่คนนั่งดูทุกอย่างพร้อมกันแล้วสรุปเป็นความเห็นเดียว ส่วน **Intermediate Fusion** คือให้ประธานกรรมการตัดสินว่าสำหรับผู้ชมคนนี้ควรเชื่อกรรมการคนใดมากกว่า

4.6.2 Late Fusion แบบถ่วงน้ำหนัก

Late Fusion ที่ง่ายที่สุดคือผลรวมถ่วงน้ำหนักของคะแนนรายสื่อ ดังสมการ (4.44)

$$s(u,i) = \sum_{m\in\{T,I,A,V\}} w_m \; s_m(u,i), \qquad \sum_m w_m = 1,\; w_m \geq 0 \tag{4.44}$$

โดยที่

- $s_m(u,i)$ คะแนนรายสื่อจากสมการ (4.35)
- w_m น้ำหนักของสื่อ m
- $s(u,i)$ คะแนนรวมที่ใช้จัดอันดับ

ตัวอย่างการคำนวณ 4.32 Late Fusion ของ u1 ด้วยน้ำหนักตั้งต้น

โจทย์ ใช้น้ำหนักตั้งต้น $(w_T, w_I, w_A, w_V) = (0.4, 0.2, 0.2, 0.2)$ จงหาคะแนนรวมของ i5 Wicked และ i8 Furiosa แล้วหา Top-3 ของ u1

สมการที่ใช้ สมการ (4.44)

ข้อมูลที่ใช้ คะแนนรายสื่อเทคนิค Transformer จากตารางที่ 4.19

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- คะแนนรายสื่อของ u1 มาจากตารางที่ 4.19 ซึ่งคำนวณด้วยโปรไฟล์ค่าเฉลี่ยของเรื่องที่ u1 ชอบ (i1, i2, i3 จากตารางที่ 4.5) กับเวกเตอร์เทคนิค Transformer ของสี่สื่อ (ข้อความจากตารางที่ 4.15 ภาพ เสียง วิดีโอจากตารางที่ 4.16 หลังเตรียมตามหัวข้อ 4.5.1)
- รายการผู้สมัครเจ็ดเรื่อง (i5, i7, i8, i9, i10, i11, i12) คือเรื่องที่ u1 ยังไม่เคยให้คะแนนในตารางที่ 4.5
- น้ำหนักตั้งต้น (0.4, 0.2, 0.2, 0.2) ผู้เขียนกำหนดให้ข้อความเป็นสองเท่าของสื่ออื่น เพราะเป็นหลักฐานที่ระบบในบทที่ 3 ทดสอบแล้ว (กล่องที่มาของไฮเปอร์พารามิเตอร์ท้ายตัวอย่างนี้)

ขั้นที่ 1 คะแนนรวมของ i5

$$s(u1, i5) = 0.4(0.4109) + 0.2(0.9180) + 0.2(0.9929) + 0.2(-0.6417) = 0.4182$$

ขั้นที่ 2 คะแนนรวมของ i8

$$s(u1, i8) = 0.4(0.1736) + 0.2(-0.0053) + 0.2(-0.8548) + 0.2(-0.4474) = -0.1921$$

ขั้นที่ 3 จัดอันดับเจ็ดเรื่อง

- Top-3: i5 (0.4182), i8 (−0.1921), i9 (−0.3097)

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

คำนวณครบทั้งเจ็ดเรื่อง (เรียงตามคะแนนรวม)

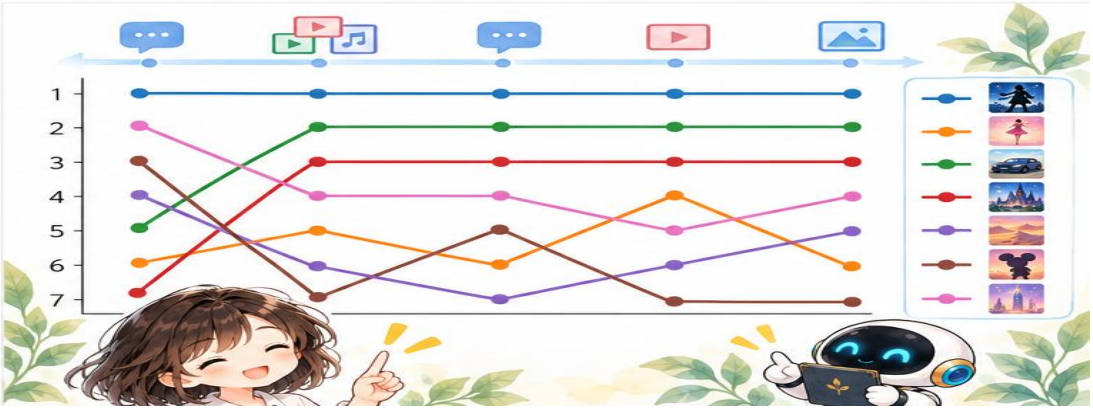
รายการผู้สมัคร	T	I	A	V	0.4T + 0.2I + 0.2A + 0.2V	s
i5 Wicked	0.4109	0.9180	0.9929	−0.6417	0.1644 + 0.1836 + 0.1986 − 0.1283	0.4182
i8 Furiosa: A Mad Max Saga	0.1736	−0.0053	−0.8548	−0.4474	0.0694 − 0.0011 − 0.1710 − 0.0895	−0.1921
i9 Conclave	0.1086	−0.8084	−0.3154	−0.6417	0.0435 − 0.1617 − 0.0631 − 0.1283	−0.3097
i12 Project Hail Mary	0.3943	−0.8268	−0.6487	−0.8781	0.1577 − 0.1654 − 0.1297 − 0.1756	−0.3130
i11 Mickey 17	0.3883	−0.7392	−0.9816	−0.9727	0.1553 − 0.1478 − 0.1963 − 0.1945	−0.3834
i7 Ballerina	0.1526	−0.8154	−0.9609	−0.5177	0.0611 − 0.1631 − 0.1922 − 0.1035	−0.3978
i10 Dune: Part Two	0.2427	−0.6555	−0.9799	−0.8390	0.0971 − 0.1311 − 0.1960 − 0.1678	−0.3978

- i7 กับ i10 ได้คะแนนรวมเท่ากันที่ทศนิยมสี่ตำแหน่ง (−0.3978) ตามข้อตกลงของบท คะแนนเสมอกันเรียงตามเลขรหัส i7 จึงอยู่ก่อน i10
- i8 ได้อันดับ 2 ทั้งที่คะแนนข้อความเพียง 0.1736 เพราะคะแนนภาพ (−0.0053) และวิดีโอ (−0.4474) ติดลบน้อยกว่าเรื่องอื่นมาก

ตีความ

i5 Wicked นำห่างเพราะได้คะแนนสูงจากสามสื่อ ส่วน i8 Furiosa ขึ้นมาอันดับ 2 เพราะวิดีโอและภาพ ทั้งที่ข้อความให้ i8 อยู่อันดับ 5 นี่คือผลของการฟังหลายสื่อ

ภาพที่ 4.44 แสดงอันดับของเจ็ดเรื่องภายใต้น้ำหนักห้าชุด อันดับ 1 คงที่ แต่อันดับ 2–7 เปลี่ยนไปตามน้ำหนัก โดยเฉพาะ i12 และ i11 ซึ่งข้อความชอบแต่สื่ออื่นไม่ชอบ



ภาพที่ 4.44 อันดับของเจ็ดเรื่องของ u1 ภายใต้น้ำหนักห้าชุด

ที่มาของไฮเปอร์พารามิเตอร์ น้ำหนักตั้งต้น (0.4, 0.2, 0.2, 0.2) มาจากไหน

- **ที่มา** ผู้เขียนตั้งเป็นค่าเริ่มต้นตามหลักว่าข้อความเป็นหลักฐานที่ระบบในบทที่ 3 พิสูจน์แล้ว จึงให้น้ำหนักเป็นสองเท่าของสื่ออื่น ไม่ได้มาจากการค้นหา
- **ผลเมื่อเปลี่ยน** ภาพที่ 4.44 แสดงว่าน้ำหนักเปลี่ยนอันดับกลางรายการได้มาก
- **วิธีเลือก** เลือกจากข้อมูลด้วย Grid Search บนชุดตรวจสอบ (หัวข้อ 4.6.5) หรือให้ระบบปรับตามผู้ใช้ด้วย Gate (หัวข้อ 4.6.7)

4.6.3 ปรับมาตราคะแนนก่อนรวม: z-score และ Min-Max

คะแนนข้อความของ u1 อยู่ในช่วงแคบประมาณ 0.1–0.4 แต่คะแนนภาพและเสียงกระจายตั้งแต่ −0.9 ถึง 1 สื่อที่มีช่วงกว้างจึง "เสียงดัง" กว่าในผลรวม แม้น้ำหนักจะน้อยกว่า วิธีแก้คือปรับมาตราคะแนนของแต่ละสื่อก่อนรวม ดังสมการ (4.45) และ (4.46)

$$z_m(u,i) = \frac{s_m(u,i) - \mu_m}{\sigma_m} \tag{4.45}$$

โดยที่

- μ_m, σ_m ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานของคะแนนสื่อ m บนรายการผู้สมัครของผู้ใช้คนนั้น
- $z_m(u,i)$ คะแนนมาตรฐาน บอกว่ารายการอยู่เหนือหรือต่ำกว่าค่าเฉลี่ยกี่เท่าของ σ_m

$$n_m(u,i) = \frac{s_m(u,i) - \min_j s_m(u,j)}{\max_j s_m(u,j) - \min_j s_m(u,j)} \tag{4.46}$$

โดยที่

- $n_m(u,i)$ คะแนนที่ปรับให้อยู่ในช่วง 0 ถึง 1

ตัวอย่างการคำนวณ 4.33 Late Fusion หลังแปลงเป็น z-score

โจทย์ จงแปลงคะแนนข้อความของ u1 เป็น z-score แล้วหา Top-3 หลังรวมด้วยน้ำหนักตั้งต้น

สมการที่ใช้ สมการ (4.45) และ (4.44)

ข้อมูลที่ใช้ คะแนนรายสื่อเทคนิค Transformer จากตารางที่ 4.19

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- คะแนนรายสื่อของ u1 มาจากตารางที่ 4.19 ซึ่งคำนวณด้วยโปรไฟล์ค่าเฉลี่ยของเรื่องที่ u1 ชอบ (i1, i2, i3 จากตารางที่ 4.5) กับเวกเตอร์เทคนิค Transformer ของสี่สื่อ (ข้อความจากตารางที่ 4.15 ภาพ เสียง วิดีโอจากตารางที่ 4.16 หลังเตรียมตามหัวข้อ 4.5.1)
- รายการผู้สมัครเจ็ดเรื่อง (i5, i7, i8, i9, i10, i11, i12) คือเรื่องที่ u1 ยังไม่เคยให้คะแนนในตารางที่ 4.5
- ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานคำนวณจาก เจ็ดผู้สมัครของ u1 เท่านั้น (หารด้วย n = 7 ตาม numpy.std) ไม่ได้ใช้ทั้งคลัง

ขั้นที่ 1 สถิติของคะแนนข้อความ

- ค่าเฉลี่ยของเจ็ดเรื่อง $\mu_T = 0.2673$
- ส่วนเบี่ยงเบนมาตรฐาน $\sigma_T = 0.1190$ (หารด้วย 7)

ขั้นที่ 2 แปลงคะแนนของ i5 และ i12

- i5: $(0.4109 - 0.2673)/0.1190 = 1.2070$
- i12: $(0.3943 - 0.2673)/0.1190 = 1.0676$

ขั้นที่ 3 ทำเหมือนกันทุกสื่อแล้วรวม

- z ของ i12 ในสี่สื่อ: (1.0676, −0.6702, −0.1707, −0.9544)
- คะแนนรวมของ i12: **0.0680**
- Top-3: i5 (1.4538), i12 (0.0680), i8 (0.0100)

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

สถิติของคะแนนข้อความทีละขั้น

- ผลรวม = 0.4109 + 0.1526 + 0.1736 + 0.1086 + 0.2427 + 0.3883 + 0.3943 = 1.8712 · ค่าเฉลี่ย **μ**T = 1.8712/7 = 0.2673
- ส่วนเบี่ยงเบนกำลังสอง: 0.0206, 0.0132, 0.0088, 0.0252, 0.0006, 0.0146, 0.0161 รวม 0.0991 หารด้วย 7 ได้ความแปรปรวน 0.0142 จึงได้ **σ**T = 0.1190

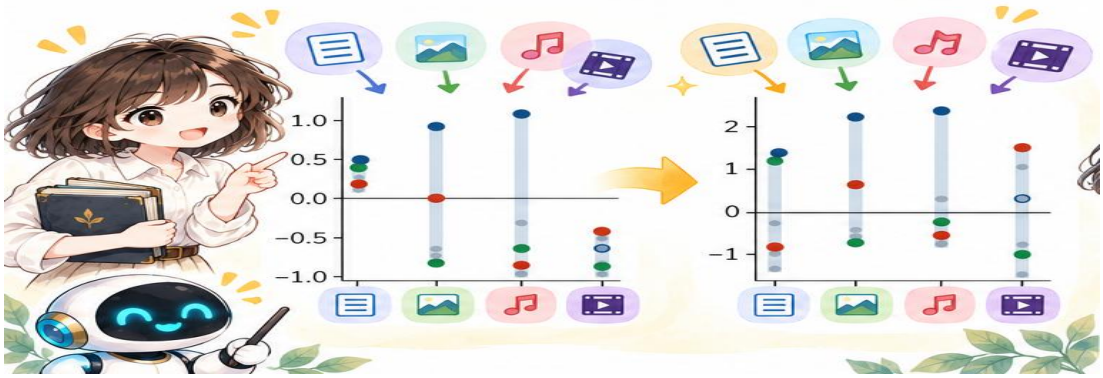
สื่อ	**μ**	**σ**	i5	i7	i8	i9	i10	i11	i12
ข้อความ	0.2673	0.1190	1.21	−0.96	−0.79	−1.33	−0.21	1.02	1.07
ภาพ	−0.4190	0.6085	2.20	−0.65	0.68	−0.64	−0.39	−0.53	−0.67
เสียง	−0.5355	0.6630	2.31	−0.64	−0.48	0.33	−0.67	−0.67	−0.17
วิดีโอ	−0.7055	0.1809	0.35	1.04	1.43	0.35	−0.74	−1.48	−0.95
รวม 0.4/0.2/0.2/0.2			**1.45**	**−0.44**	**0.01**	**−0.52**	**−0.44**	**−0.13**	**0.07**

- ตัวอย่าง i12: 0.4(1.0676) + 0.2(−0.6702) + 0.2(−0.1707) + 0.2(−0.9544) = 0.4270 − 0.1340 − 0.0341 − 0.1909 = 0.0680
- อันดับหลังรวม: i5 (1.4538), i12 (0.0680), i8 (0.0100), i11 (−0.1285), i7 (−0.4365), i10 (−0.4423), i9 (−0.5244)

ตีความ

หลังปรับมาตรา ความต่างเล็กน้อยของคะแนนข้อความถูกขยายเป็นหลายเท่าของ **σ**
ข้อความจึงมีเสียงเท่ากับสื่ออื่นจริง และ i12 Project Hail Mary ซึ่งข้อความชอบมาก ขยับขึ้นมาอันดับ 2

ภาพที่ 4.45 เทียบการกระจายของคะแนนก่อนและหลังแปลงเป็น z-score ด้านซ้ายข้อความมีช่วงแคบมาก ด้านขวาทุกสื่อมีช่วงใกล้กัน



ภาพที่ 4.45 คะแนนรายสื่อของ u1 ก่อนและหลังแปลงเป็น z-score

ข้อควรระวัง z-score กับรายการผู้สมัครน้อย

- สถิติ μ_m และ σ_m คำนวณจากรายการผู้สมัครของผู้ใช้คนเดียว ถ้ามีผู้สมัครเพียงไม่กี่เรื่อง ค่าเหล่านี้ไม่เสถียร
- ถ้าทุกเรื่องมีคะแนนเท่ากันในสื่อหนึ่ง $\sigma_m = 0$ ต้องกำหนดให้คะแนนมาตรฐานเป็น 0 เพื่อไม่ให้หารด้วยศูนย์
- Min-Max ไวต่อค่าผิดปกติเพียงเรื่องเดียว เพราะใช้เฉพาะค่าสูงสุดและต่ำสุด

4.6.4 รวมด้วยอันดับ: Borda Count และ Reciprocal Rank Fusion

อีกวิธีหนึ่งในการหลีกเลี่ยงปัญหามาตราคือทิ้งค่าคะแนนไปเลย แล้วรวม **อันดับ** แทน วิธีที่นิยมมีสองวิธีตามสมการ (4.47) และ (4.48)

$$B(i) = \sum_m \left(n - \text{rank}_m(i)\right) \tag{4.47}$$

โดยที่

- n จำนวนรายการผู้สมัคร
- $\text{rank}_m(i)$ อันดับของรายการ i ในสื่อ m เริ่มจาก 1

$$\text{RRF}(i) = \sum_m \frac{1}{k + \text{rank}_m(i)} \tag{4.48}$$

โดยที่

- k ค่าคงที่ที่ลดอิทธิพลของอันดับต้น ๆ Cormack et al. (2009) ใช้ $k = 60$

ตัวอย่างการคำนวณ 4.34 Borda และ RRF ของ u1

โจทย์ จงหาอันดับรายสื่อของ i5 และ i8 แล้วคำนวณคะแนน Borda และ RRF ($k = 60$)

สมการที่ใช้ สมการ (4.47) และ (4.48)

ข้อมูลที่ใช้ อันดับจากคะแนนในตารางที่ 4.19

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- คะแนนรายสื่อของ u1 มาจากตารางที่ 4.19 ซึ่งคำนวณด้วยโปรไฟล์ค่าเฉลี่ยของเรื่องที่ u1 ชอบ (i1, i2, i3 จากตารางที่ 4.5) กับเวกเตอร์เทคนิค Transformer ของสี่สื่อ (ข้อความจากตารางที่ 4.15 ภาพ เสียง วิดีโอจากตารางที่ 4.16 หลังเตรียมตามหัวข้อ 4.5.1)
- อันดับรายสื่อได้จากการเรียงคะแนนในตารางที่ 4.19 จากมากไปน้อยแยกทีละคอลัมน์ ถ้าเสมอกันเรียงตามเลขรหัส
- n = 7 คือจำนวนรายการผู้สมัครของ u1 และ k = 60 ตาม Cormack et al. (2009)

ขั้นที่ 1 อันดับรายสื่อ

- ข้อความ: i5 > i12 > i11 > i10 > i8 > i7 > i9
- ภาพ: i5 > i8 > i10 > i11 > i9 > i7 > i12
- เสียง: i5 > i9 > i12 > i8 > i7 > i10 > i11
- วิดีโอ: i8 > i7 > i5 > i9 > i10 > i12 > i11

ขั้นที่ 2 Borda ($n = 7$)

- i5 อันดับ (1, 1, 1, 3): $6 + 6 + 6 + 4 = 22$
- i8 อันดับ (5, 2, 4, 1): $2 + 5 + 3 + 6 = 16$

ขั้นที่ 3 RRF

- i5: $3/61 + 1/63 = 0.0651$
- i8: $1/65 + 1/62 + 1/64 + 1/61 = 0.0635$
- Top-3 ของ RRF: i5, i8, i12

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

อันดับรายสื่อและคะแนนของทั้งเจ็ดเรื่อง

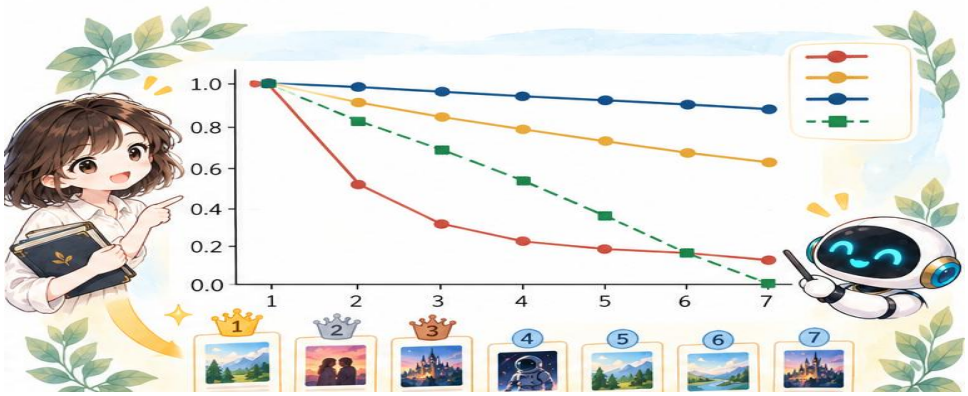
รายการผู้สมัคร	อันดับ T	I	A	V	Borda Σ(7 − rank)	RRF Σ 1/(60 + rank)
i5 Wicked	1	1	1	3	22	0.0651
i8 Furiosa: A Mad Max Saga	5	2	4	1	16	0.0635
i12 Project Hail Mary	2	7	3	6	10	0.0621
i9 Conclave	7	5	2	4	10	0.0621
i10 Dune: Part Two	4	3	6	5	10	0.0620
i7 Ballerina	6	6	5	2	9	0.0618
i11 Mickey 17	3	4	7	7	7	0.0613

- ตัวอย่าง RRF ของ i12: อันดับ (2, 7, 3, 6) จึงได้ 1/62 + 1/67 + 1/63 + 1/66 = 0.0161 + 0.0149 + 0.0159 + 0.0152 = 0.0621
- Top-3 ของ Borda: i5 (22), i8 (16), i9 (10) · Top-3 ของ RRF: i5, i8, i12

ตีความ

ทั้งสองวิธีไม่สนว่าคะแนนห่างกันเท่าใด สนเพียงลำดับ i8 ขึ้นอันดับ 2 เพราะอยู่อันดับต้นในภาพและวิดีโอ RRF ด้วย $k = 60$ ทำให้คะแนนของทุกเรื่องใกล้กันมาก (0.0613 ถึง 0.0651)
อันดับต้นจึงมีอิทธิพลไม่มากกว่าอันดับท้ายนัก

ภาพที่ 4.46 เทียบแต้มที่แต่ละอันดับได้รับ (หารด้วยแต้มของอันดับ 1) ใน RRF สามค่า k กับ Borda ค่า k เล็กให้รางวัลอันดับ 1 สูงมาก ส่วน $k = 60$ เกือบแบนราบ



ภาพที่ 4.46 แต้มของแต่ละอันดับใน RRF (k = 0, 10, 60) และ Borda

ที่มาของไฮเปอร์พารามิเตอร์ ค่า k = 60 ของ RRF มาจากไหน

- **ที่มา** Cormack et al. (2009) ทดลองกับการรวมผลค้นหาจากระบบจำนวนมาก และพบว่า $k = 60$ ให้ผลดีสม่ำเสมอ ค่านี้จึงเป็นค่าตั้งต้นในไลบรารีส่วนใหญ่
- **ผลเมื่อเปลี่ยน** ใน MovieMini-12 ซึ่งมีผู้สมัครเพียง 7–9 เรื่อง $k = 0$ ให้ผลดีกว่า (ตารางที่ 4.23) เพราะงานนี้ต้องการให้รางวัลเรื่องที่สื่อใดสื่อหนึ่งมั่นใจมาก
- **วิธีเลือก** $k = 60$ เหมาะกับรายการยาวหลายร้อยรายการ ถ้ารายการสั้นควรลองค่าเล็กลงบนชุดตรวจสอบ

4.6.5 เลือกน้ำหนักจากข้อมูลด้วย Grid Search

แทนที่จะกำหนดน้ำหนักเอง เราค้นหาน้ำหนักที่ให้ตัวชี้วัดดีที่สุดได้ โดยไล่ทุกชุดน้ำหนักบนตาราง (Grid) ที่ก้าวทีละ 0.1 และรวมกันเท่ากับ 1 จำนวนชุดทั้งหมดคำนวณได้จากสมการ (4.49)

$$N_{grid} = \binom{G+M-1}{M-1} \tag{4.49}$$

โดยที่

- G จำนวนก้าวของน้ำหนัก เช่น $1/0.1 = 10$
- M จำนวนสื่อ เท่ากับ 4
- N_{grid} จำนวนชุดน้ำหนักที่ไม่ติดลบและรวมกันได้ 1

ตัวอย่างการคำนวณ 4.35 Grid Search และกับดักของการเลือกน้ำหนักบนข้อมูลชุดเดียวกัน

โจทย์ จงนับจำนวนชุดน้ำหนัก แล้วเทียบผลของการเลือกน้ำหนักบนผู้ใช้ u1–u4 กับผลบนผู้ใช้ u5–u8

สมการที่ใช้ สมการ (4.49) และ (4.44)

ข้อมูลที่ใช้ ผลการประเมินแบบกันออกทีละรายการ (หัวข้อ 4.7.1) เทคนิค Transformer

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- กรณีทดสอบมาจากตารางที่ 4.5: ทุกเรื่องที่ผู้ใช้ให้ 4–5 ถูกซ่อนหนึ่งครั้ง u1–u4 มีเรื่องที่ซอบคนละ 3 เรื่อง (12 กรณี) และ u5–u8 คนละ 2 เรื่อง (8 กรณี) รวม 20 กรณี
- คะแนนรายสื่อของแต่ละกรณีคำนวณใหม่จากประวัติที่เหลือ ด้วยโปรไฟล์ค่าเฉลี่ยและเวกเตอร์เทคนิค Transformer (ตารางที่ 4.15 และ 4.16)

ขั้นที่ 1 นับชุดน้ำหนัก

- $\binom{10+4-1}{3} = \binom{13}{3} = 286$ ชุด

ขั้นที่ 2 ค้นหาบนผู้ใช้ทั้งแปดคน

- ชุดที่ดีที่สุดคือ (0.60, 0.10, 0.20, 0.10) ได้ MRR 0.7755 เทียบกับน้ำหนักตั้งต้น 0.7635
- มี 20 ชุดจาก 286 ชุดที่ดีกว่าน้ำหนักตั้งต้น

ขั้นที่ 3 ค้นหาบน u1–u4 แล้วทดสอบบน u5–u8

- ชุดที่ดีที่สุดบน u1–u4 คือ (0.00, 0.00, 0.10, 0.90) ได้ MRR 0.9333 บนชุดที่ใช้เลือก
- เมื่อนำไปใช้กับ u5–u8 ได้ MRR เพียง 0.5192 ต่ำกว่าน้ำหนักตั้งต้น (0.6095)

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

นับชุดน้ำหนักด้วยหลัก "ดาวกับไม้กั้น"

- น้ำหนักแต่ละตัวเป็นจำนวนเท่าของ 0.1 รวมกัน 10 ส่วน แจก 10 ส่วนให้ 4 สื่อ เท่ากับเลือกตำแหน่งไม้กั้น 3 อันจาก 13 ตำแหน่ง: C(13, 3) = 13 × 12 × 11 / (3 × 2 × 1) = 1,716/6 = 286

ห้าชุดน้ำหนักที่ดีที่สุดบนผู้ใช้ทั้งแปดคน

อันดับ	(wT, wI, wA, wV)	MRR
1	(0.60, 0.10, 0.20, 0.10)	0.7755
2	(0.50, 0.10, 0.20, 0.20)	0.7738
3	(0.10, 0.10, 0.20, 0.60)	0.7713
4	(0.20, 0.10, 0.20, 0.50)	0.7713
5	(0.30, 0.10, 0.20, 0.40)	0.7713
–	ตั้งต้น (0.40, 0.20, 0.20, 0.20)	0.7635

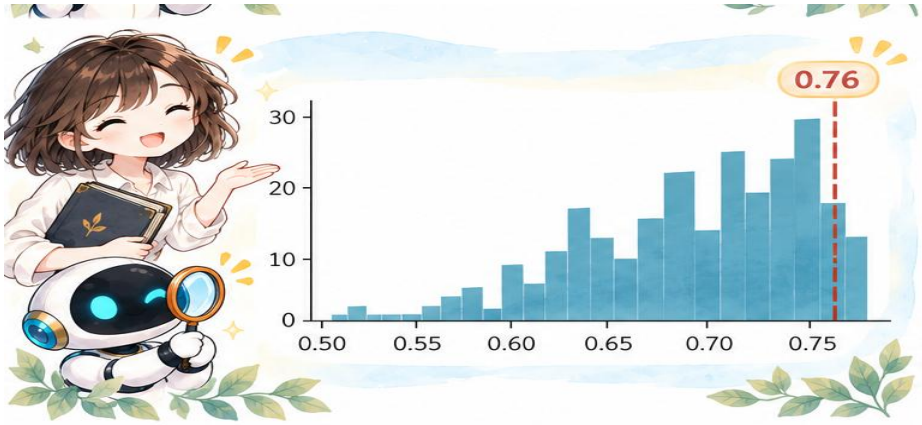
แยกชุดเลือกกับชุดทดสอบ

- บน u1–u4 (12 กรณี) มี 3 ชุดที่ได้ MRR สูงสุดเท่ากัน (0.9333) ชุดแรกตามลำดับการไล่คือ (0.00, 0.00, 0.10, 0.90)
- น้ำหนักตั้งต้นบน u1–u4 ได้ MRR 0.8661 ส่วนบน u5–u8 (8 กรณี) ได้ 0.6095
- นำ (0.00, 0.00, 0.10, 0.90) ไปใช้กับ u5–u8 ได้ MRR 0.5192 ต่ำกว่าน้ำหนักตั้งต้น 0.6095

ตีความ

การเลือกน้ำหนักบนข้อมูลที่ใช้วัดผลให้ตัวเลขที่ดูดีเกินจริง (Overfitting) ข้อมูลขนาดเล็กอย่าง MovieMini-12 ทำให้ปัญหานี้รุนแรงเป็นพิเศษ จึงต้องแยกชุดเลือกน้ำหนักออกจากชุดทดสอบเสมอตามบทที่ 2

ภาพที่ 4.47 แสดงการกระจายของ MRR จาก 286 ชุดน้ำหนัก น้ำหนักตั้งต้นอยู่ค่อนไปทางดี แต่มีชุดที่ดีกว่าอยู่ 20 ชุด



ภาพที่ 4.47 การกระจายของ MRR จาก 286 ชุดน้ำหนัก

4.6.6 Early Fusion: ต่อเวกเตอร์ก่อนให้คะแนน

Early Fusion ต่อเวกเตอร์ของสี่สื่อเป็นเส้นเดียว แล้วสร้างโปรไฟล์และให้คะแนนในปริภูมิที่ต่อแล้ว ถ้าคูณเวกเตอร์ของแต่ละสื่อด้วย $\sqrt{w_m}$ ก่อนต่อ ผลคูณจุดของเวกเตอร์ยาวจะเท่ากับ Late Fusion พอดี ดังสมการ (4.50)

$$\mathbf{x}_i = \left[\sqrt{w_T}\mathbf{v}_i^T \;\|\; \sqrt{w_I}\mathbf{v}_i^I \;\|\; \sqrt{w_A}\mathbf{v}_i^A \;\|\; \sqrt{w_V}\mathbf{v}_i^V\right], \qquad \mathbf{q}_u \cdot \mathbf{x}_i = \sum_m w_m \, \mathbf{p}_u^m \cdot \mathbf{v}_i^m \tag{4.50}$$

โดยที่

- $\mathbf{x}_i$ เวกเตอร์ต่อของรายการ i ยาว $12 + 3 + 3 + 3 = 21$ มิติ
- $\mathbf{q}_u$ โปรไฟล์ต่อของผู้ใช้ สร้างแบบเดียวกันจาก $\mathbf{p}_u^m$

ตัวอย่างการคำนวณ 4.36 Early Fusion ของ i5 เท่ากับ Late Fusion

โจทย์ จงต่อเวกเตอร์ของ i5 และโปรไฟล์ของ u1 ตามสมการ (4.50) ด้วยน้ำหนักตั้งต้น แล้วตรวจว่าผลคูณจุดเท่ากับคะแนน Late Fusion

สมการที่ใช้ สมการ (4.50) และ (4.44)

ข้อมูลที่ใช้ โปรไฟล์จากตัวอย่างการคำนวณ 4.25 และคะแนนจากตัวอย่างการคำนวณ 4.32

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- โปรไฟล์ของ u1 ทั้งสี่สื่อมาจากตัวอย่างการคำนวณ 4.25 (ข้อความ) และภาพที่ 4.37 (ภาพ เสียง วิดีโอ) ซึ่งเฉลี่ยจาก i1, i2, i3 ที่ u1 ชอบ (ตารางที่ 4.5)
- เวกเตอร์ของ i5 Wicked มาจากตารางที่ 4.15 (12 มิติ) และตารางที่ 4.16 (สามสื่อ สื่อละ 3 มิติ) หลังเตรียมตามหัวข้อ 4.5.1 รวมเป็น 12 + 3 + 3 + 3 = 21 มิติ

ขั้นที่ 1 ตัวคูณของแต่ละสื่อ

- $\sqrt{0.4} = 0.6325$ สำหรับข้อความ และ $\sqrt{0.2} = 0.4472$ สำหรับสื่ออื่น

ขั้นที่ 2 ผลคูณจุดของเวกเตอร์ 21 มิติ

- ส่วนข้อความ: $0.4 \times 0.4109 = 0.1644$
- ส่วนภาพ เสียง วิดีโอ: $0.2 \times [0.9180 + 0.9929 + (-0.6417)] = 0.2538$
- รวม: 0.4182 เท่ากับตัวอย่างการคำนวณ 4.32

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

ส่วนของเวกเตอร์	ตัวคูณ √w	p_u1 · v_i5 (โคไซน์รายสื่อ)	(√w)² × โคไซน์
ข้อความ มิติ 1–12	0.6325	0.4109	0.1644
ภาพ มิติ 13–15	0.4472	0.9180	0.1836
เสียง มิติ 16–18	0.4472	0.9929	0.1986
วิดีโอ มิติ 19–21	0.4472	−0.6417	−0.1283
รวม 21 มิติ			**0.4182**

- เหตุที่ได้ค่าเท่ากับ Late Fusion: ในแต่ละส่วน ทั้งโปรไฟล์และรายการถูกคูณด้วย √w เหมือนกัน ผลคูณจุดของส่วนนั้นจึงถูกคูณด้วย √w × √w = w พอดี
- ตัวอย่างส่วนภาพ: $(0.4472)^2 \times 0.9180 = 0.2 \times 0.9180 = 0.1836$

ตีความ

Early Fusion แบบต่อเวกเตอร์ตรง ๆ จึงไม่ได้ให้อะไรมากกว่า Late Fusion ถ้าตัวให้คะแนนเป็นผลคูณจุด ข้อได้เปรียบของ Early Fusion จะเกิดเมื่อนำเวกเตอร์ต่อไปผ่านแบบจำลองที่เรียนรู้ได้ เช่น โครงข่ายประสาทหลายชั้น ซึ่งจับปฏิสัมพันธ์ข้ามสื่อได้ แต่ต้องการข้อมูลฝึกมาก

4.6.7 Intermediate Fusion ด้วย Gate: น้ำหนักเฉพาะผู้ใช้

ผู้ใช้แต่ละคนเชื่อสื่อต่างกัน บางคนเลือกภาพยนตร์จากโปสเตอร์ บางคนเลือกจากเพลง **Gate** คือกลไกที่ให้น้ำหนักสื่อเปลี่ยนตามผู้ใช้ วิธีที่ไม่ต้องฝึกคือวัด **ความสม่ำเสมอ** ของเรื่องที่ผู้ใช้ชอบในแต่ละสื่อ ถ้าเรื่องที่ชอบคล้ายกันมากในสื่อใด แปลว่าสื่อนั้นอธิบายรสนิยมของผู้ใช้ได้ดี แล้วแปลงเป็นน้ำหนักด้วย softmax ดังสมการ (4.51)

$$g_m(u) = \frac{2}{|L_u|(|L_u|-1)} \sum_{i<j \in L_u} \mathbf{v}_i^m \cdot \mathbf{v}_j^m, \qquad w_m(u) = \frac{\exp(g_m(u)/\tau)}{\sum_k \exp(g_k(u)/\tau)} \tag{4.51}$$

โดยที่

- $g_m(u)$ ค่าเฉลี่ยโคไซน์ของทุกคู่เรื่องที่ชอบในสื่อ m
- τ อุณหภูมิของ Gate บทนี้ใช้ 0.5
- $w_m(u)$ น้ำหนักของสื่อ m สำหรับผู้ใช้ u

ถ้าผู้ใช้มีเรื่องที่ชอบน้อยกว่าสองเรื่อง จะหาค่าเฉลี่ยของคู่ไม่ได้ ระบบจึงให้น้ำหนักเท่ากันทุกสื่อ (0.25) ในกรณีนั้น

ตัวอย่างการคำนวณ 4.37 น้ำหนัก Gate ของ u1

โจทย์ จงหาความสม่ำเสมอ น้ำหนัก Gate ($\tau = 0.5$) และ Top-3 ของ u1

สมการที่ใช้ สมการ (4.51) และ (4.44)

ข้อมูลที่ใช้ เวกเตอร์ของ i1, i2, i3 ที่เตรียมแล้ว (ตารางที่ 4.15 และ 4.16)

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- เรื่องที่ u1 ชอบ (i1, i2, i3) มาจากตารางที่ 4.5 ทั้งสามเรื่องเป็นแอนิเมชัน (ตารางที่ 4.4) จึงมีสามคู่ให้เทียบ คือ i1–i2, i1–i3, i2–i3
- เวกเตอร์ของแต่ละสื่อมาจากตารางที่ 4.15 และ 4.16 หลังเตรียมตามหัวข้อ 4.5.1 ส่วนคะแนนรายสื่อของผู้สมัครมาจากตารางที่ 4.19
- อุณหภูมิ τ = 0.5 เป็นค่าที่เลือกเพื่อการสอน

ขั้นที่ 1 ความสม่ำเสมอของภาพ

- คู่ i1–i2: 0.9390, i1–i3: 0.9523, i2–i3: 0.8638
- $g_I = 0.9184$

ขั้นที่ 2 ทุกสื่อ

- $g = (g_T, g_I, g_A, g_V) = (0.2957, 0.9184, 0.6042, 0.9665)$

ขั้นที่ 3 softmax หลังหารด้วย τ

- $e^{g_m/0.5}$ = (1.8066, 6.2765, 3.3479, 6.9107)
- $w(u1) = (0.0985, 0.3422, 0.1825, 0.3768)$

ขั้นที่ 4 รวมคะแนน

- Top-3: i5 (0.2941), i8 (−0.3093), i9 (−0.5653)

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

ความสม่ำเสมอของเรื่องที่ชอบในแต่ละสื่อ

สื่อ	i1–i2	i1–i3	i2–i3	g = ค่าเฉลี่ย	g/τ	e^(g/τ)	w
ข้อความ	0.1817	0.4179	0.2875	0.2957	0.5915	1.8066	0.0985
ภาพ	0.9390	0.9523	0.8638	0.9184	1.8368	6.2765	0.3422
เสียง	0.7412	0.3948	0.6765	0.6042	1.2083	3.3479	0.1825
วิดีโอ	0.9825	0.9333	0.9839	0.9665	1.9331	6.9107	0.3768

รวม						18.3418	1.0000

- แถวข้อความใช้โคไซน์ข้อความ (TF) ของคู่ที่ชอบ เช่น i1–i2 = 0.1817 ซึ่งต่ำ เพราะเรื่องย่อของ Toy Story 5 กับ Inside Out 2 ใช้คำต่างกันมาก (toys กับ mind)

รวมคะแนนด้วยน้ำหนัก Gate

รายการผู้สมัคร	T	I	A	V	s (Gate)
i5 Wicked	0.4109	0.9180	0.9929	−0.6417	0.2941
i8 Furiosa: A Mad Max Saga	0.1736	−0.0053	−0.8548	−0.4474	−0.3093
i9 Conclave	0.1086	−0.8084	−0.3154	−0.6417	−0.5653
i7 Ballerina	0.1526	−0.8154	−0.9609	−0.5177	−0.6345
i12 Project Hail Mary	0.3943	−0.8268	−0.6487	−0.8781	−0.6933
i10 Dune: Part Two	0.2427	−0.6555	−0.9799	−0.8390	−0.6954
i11 Mickey 17	0.3883	−0.7392	−0.9816	−0.9727	−0.7603

ตีความ

สามเรื่องที่ u1 ชอบเป็นแอนิเมชันทั้งหมด จึงคล้ายกันมากในภาพและวิดีโอ แต่เรื่องย่อต่างกัน Gate จึงลดน้ำหนักข้อความเหลือ 0.0985 ข้อควรระวังคือความสม่ำเสมอสูงไม่ได้แปลว่าสื่อนั้น **แยกแยะ** เรื่องใหม่ได้ดี เพราะรายการผู้สมัครทั้งเจ็ดไม่มีแอนิเมชันเลย (ดูบทวิเคราะห์หัวข้อ 4.10)

4.6.8 เมื่อบางสื่อหายไป

ในระบบจริง รายการใหม่มักมีข้อมูลไม่ครบ เช่น i12 Project Hail Mary (2026) อาจยังไม่มีตัวอย่างภาพยนตร์ ถ้าแทนคะแนนวิดีโอด้วย 0 รายการนั้นจะเสียเปรียบหรือได้เปรียบโดยไม่ตั้งใจ วิธีที่ปลอดภัยกว่าคือ **กระจายน้ำหนักใหม่** ให้สื่อที่มีอยู่ ดังสมการ (4.52)

$$s(u,i) = \frac{\sum_{m \in A_i} w_m \, s_m(u,i)}{\sum_{m \in A_i} w_m} \tag{4.52}$$

โดยที่

- A_i เซตของสื่อที่รายการ i มีอยู่

ตัวอย่างการคำนวณ 4.38 i12 ไม่มีตัวอย่างภาพยนตร์

โจทย์ ถ้า i12 ไม่มีวิดีโอ จงเทียบคะแนนรวมของ i12 สามแบบ คือมีครบ แทนด้วย 0 และกระจายน้ำหนักใหม่

สมการที่ใช้ สมการ (4.44) และ (4.52)

ข้อมูลที่ใช้ คะแนนของ i12 จากตารางที่ 4.19

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- i12 Project Hail Mary ฉายปี 2026 (ตารางที่ 4.4) เป็นเรื่องใหม่ล่าสุดในคลัง จึงสมมติว่ายังไม่มีตัวอย่างภาพยนตร์
- คะแนนรายสื่อของ i12 มาจากตารางที่ 4.19: T = 0.3943, I = −0.8268, A = −0.6487, V = −0.8781 (ค่า V ใช้เฉพาะกรณีมีครบ)

ขั้นที่ 1 มีครบสี่สื่อ

- -0.3130 (ตัวอย่างการคำนวณ 4.32)

ขั้นที่ 2 แทนคะแนนวิดีโอด้วย 0

- $0.4(0.3943) + 0.2(-0.8268) + 0.2(-0.6487) + 0.2(0) = -0.1373$

ขั้นที่ 3 กระจายน้ำหนักใหม่

- $-0.1373/0.8 = -0.1717$ ซึ่งเท่ากับใช้น้ำหนัก (0.5, 0.25, 0.25) กับสามสื่อที่เหลือ

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

กรณี	การคำนวณ	คะแนน
มีครบสี่สื่อ	0.4(0.3943) + 0.2(−0.8268) + 0.2(−0.6487) + 0.2(−0.8781) = 0.1577 − 0.1654 − 0.1297 − 0.1756	−0.3130
แทนวิดีโอด้วย 0	0.1577 − 0.1654 − 0.1297 + 0 = −0.1373	−0.1373
กระจายน้ำหนักใหม่	−0.1373 / (0.4 + 0.2 + 0.2) = −0.1373 / 0.8	−0.1717

- น้ำหนักที่กระจายใหม่: 0.4/0.8 = 0.5, 0.2/0.8 = 0.25 และ 0.2/0.8 = 0.25 รวมกันได้ 1
- ผลต่ออันดับ: เมื่อมีครบ i12 อยู่อันดับ 4 (−0.3130) ถ้าแทนด้วย 0 จะได้ −0.1373 ซึ่งสูงกว่า i8 (−0.1921) และขึ้นเป็นอันดับ 2 ส่วนการกระจายน้ำหนักใหม่ได้ −0.1717 ซึ่งยังสูงกว่า i8 เล็กน้อย

ตีความ

การแทนด้วย 0 ทำให้ i12 ได้ประโยชน์จากการ "ไม่มีข้อมูล" เพราะคะแนนวิดีโอจริงติดลบ ส่วนการกระจายน้ำหนักใหม่ให้คะแนนที่ตีความได้ว่า "ประเมินจากหลักฐานที่มี" ซึ่งยุติธรรมกว่า ระบบควรบันทึกด้วยว่าคะแนนของรายการนี้มาจากสื่อใดบ้าง

4.6.9 จัดแนวข้ามสื่อแบบ CLIP

เทคนิคข้างต้นเทียบรายการภายในสื่อเดียวกัน แล้วจึงรวมคะแนน แต่บางงานต้องเทียบ **ข้ามสื่อ** โดยตรง เช่น ให้ผู้ใช้พิมพ์ "แอนิเมชันอบอุ่นเรื่องครอบครัว" แล้วค้นหาโปสเตอร์ การเทียบแบบนี้ต้องให้ภาพและข้อความอยู่ใน **ปริภูมิร่วม (Shared Embedding Space)** เดียวกัน CLIP สร้างปริภูมิร่วมด้วยการฝึกตัวเข้ารหัสภาพและข้อความพร้อมกัน บนคู่ภาพกับคำบรรยาย 400 ล้านคู่ ด้วย **การเรียนรู้แบบเปรียบต่าง (Contrastive Learning)** (Radford et al., 2021) ดังสมการ (4.53) และ (4.54)

$$p_{ij} = \frac{\exp(s_{ij}/\tau)}{\sum_{k=1}^{B} \exp(s_{ik}/\tau)}, \qquad s_{ij} = \cos\left(f_{img}(x_i), f_{txt}(t_j)\right) \tag{4.53}$$

โดยที่

- x_i, t_j ภาพที่ i และข้อความที่ j ในชุดฝึก คู่ (x_i, t_i) คือคู่ที่ถูกต้อง
- f_{img}, f_{txt} ตัวเข้ารหัสภาพ (ViT) และข้อความ (Transformer) ส่งออกเวกเตอร์ขนาดเท่ากัน
- B ขนาดชุด
- τ อุณหภูมิ
- p_{ij} ความน่าจะเป็นที่ภาพ i คู่กับข้อความ j

$$\mathcal{L}_{CLIP} = \frac{1}{2}\left(-\frac{1}{B}\sum_i \ln p_{ii} - \frac{1}{B}\sum_j \ln p_{jj}^{txt}\right) \tag{4.54}$$

โดยที่

- p_{jj}^{txt} ความน่าจะเป็นของคู่ถูกเมื่อทำ softmax ตามคอลัมน์ (ให้ข้อความเลือกภาพ)
- $\mathcal{L}_{CLIP}$ ค่าสูญเสียแบบสมมาตรที่ใช้ฝึกตัวเข้ารหัสทั้งสอง

ตัวอย่างการคำนวณ 4.39 ค่าสูญเสียของ CLIP บนชุดสามคู่

โจทย์ ชุดฝึกมีโปสเตอร์และเรื่องย่อของ i1, i8 และ i12 โคไซน์ภาพ–ข้อความเป็นดังภาพที่ 4.48 ด้านซ้าย จงหาความน่าจะเป็นของแถวแรกและค่าสูญเสียที่ $\tau = 0.07$

สมการที่ใช้ สมการ (4.53) และ (4.54)

ข้อมูลที่ใช้ โคไซน์สมมติ แถว i1 = (0.31, 0.12, 0.05), i8 = (0.10, 0.28, 0.09), i12 = (0.04, 0.11, 0.30)

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- สามคู่ภาพ–ข้อความเลือกจากสามแนวที่ต่างกันในตารางที่ 4.4: i1 Toy Story 5 (แอนิเมชัน) i8 Furiosa (แอ็กชัน) และ i12 Project Hail Mary (ดราม่าวิทยาศาสตร์)
  โปสเตอร์คู่กับเรื่องย่อของเรื่องเดียวกันในตารางที่ 4.6
- โคไซน์ภาพ–ข้อความทั้งเก้าค่าเป็นค่าสมมติ (ภาพที่ 4.48) โดยให้คู่ที่ถูก (แนวทแยง) สูงกว่าคู่ผิดเล็กน้อย เหมือนตัวเข้ารหัสที่เพิ่งเริ่มฝึก
- τ = 0.07 คือค่าเริ่มต้นของ CLIP

ขั้นที่ 1 หารด้วย τ

- แถว i1: $(0.31, 0.12, 0.05)/0.07 = (4.4286, 1.7143, 0.7143)$

ขั้นที่ 2 softmax ตามแถว

- $e^{4.4286} = 83.81$, $e^{1.7143} = 5.55$, $e^{0.7143} = 2.04$
- $p_{1\cdot} = (0.9169, 0.0607, 0.0223)$

ขั้นที่ 3 ค่าสูญเสียสองทิศ

- ทิศภาพ→ข้อความ: คู่ถูก (0.9169, 0.8751, 0.9169) ได้ $\mathcal{L}_{img} = 0.1023$
- ทิศข้อความ→ภาพ: คู่ถูก (0.9338, 0.8404, 0.9277) ได้ $\mathcal{L}_{txt} = 0.1058$
- $\mathcal{L}_{CLIP} = (0.1023 + 0.1058)/2 = 0.1040$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

หารด้วย τ แล้วยกกำลัง e ครบทั้งตาราง

ภาพ \ ข้อความ	i1	i8	i12	ผลรวมแถว
i1	e^(0.31/0.07) = 83.81	e^(0.12/0.07) = 5.55	e^(0.05/0.07) = 2.04	91.41
i8	e^(0.10/0.07) = 4.17	e^(0.28/0.07) = 54.60	e^(0.09/0.07) = 3.62	62.39
i12	e^(0.04/0.07) = 1.77	e^(0.11/0.07) = 4.81	e^(0.30/0.07) = 72.65	79.24
ผลรวมคอลัมน์	89.76	64.96	78.31	

คู่ถูก	p (ตามแถว: ภาพเลือกข้อความ)	p (ตามคอลัมน์: ข้อความเลือกภาพ)
i1	83.81 / 91.41 = 0.9169	83.81 / 89.76 = 0.9338
i8	54.60 / 62.39 = 0.8751	54.60 / 64.96 = 0.8404
i12	72.65 / 79.24 = 0.9169	72.65 / 78.31 = 0.9277

- L_img = −(ln 0.9169 + ln 0.8751 + ln 0.9169)/3 = 0.1023 และ L_txt = −(ln 0.9338 + ln 0.8404 + ln 0.9277)/3 = 0.1058
- L_CLIP = (0.1023 + 0.1058)/2 = 0.1040

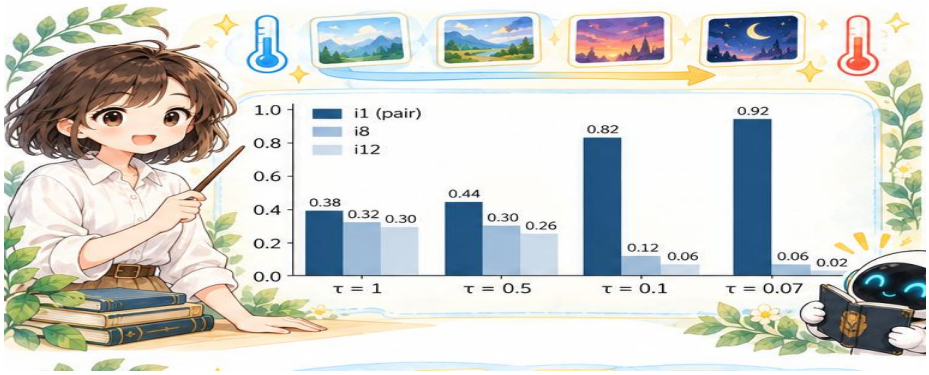
ตีความ

แม้โคไซน์ 0.31 กับ 0.12 ต่างกันไม่มาก แต่หลังหารด้วย 0.07 ความน่าจะเป็นของคู่ถูกสูงถึง 0.9169
การฝึกจะผลักให้คู่ถูกมีโคไซน์สูงขึ้นอีกและคู่ผิดต่ำลง เมื่อฝึกเสร็จ
ระบบผู้แนะนำนำโปรไฟล์ข้อความของผู้ใช้ไปค้นโปสเตอร์ได้โดยตรง



ภาพที่ 4.48 โคไซน์ภาพ–ข้อความและความน่าจะเป็นของ CLIP

ภาพที่ 4.48 ด้านขวาแสดงความน่าจะเป็นหลัง softmax ตามแถว เส้นทแยงมุมคือคู่ที่ถูกต้อง ส่วนภาพที่ 4.49 แสดงความน่าจะเป็นของแถว i1 เมื่อเปลี่ยนอุณหภูมิ



ภาพที่ 4.49 ความน่าจะเป็นของแถว i1 เมื่อเปลี่ยนอุณหภูมิ τ

ที่มาของไฮเปอร์พารามิเตอร์ อุณหภูมิ τ = 0.07 ของ CLIP มาจากไหน

- **ที่มา** CLIP ตั้งค่าเริ่มต้น $\tau = 0.07$ ตามงานเรียนรู้แบบเปรียบต่างก่อนหน้า แล้วให้ τ เป็นพารามิเตอร์ที่เรียนต่อระหว่างฝึก โดยจำกัดไม่ให้ $1/\tau$ เกิน 100 (Radford et al., 2021)
- **ผลเมื่อเปลี่ยน** $\tau = 1$ แยกคู่ถูกกับคู่ผิดแทบไม่ออก (ภาพที่ 4.49) ส่วน τ เล็กเกินทำให้ softmax แหลมจนเกรเดียนต์ของคู่ผิดเกือบเป็นศูนย์
- **ขนาดชุด B** ยิ่งใหญ่ยิ่งมีคู่ผิดให้เปรียบเทียบมาก CLIP ใช้ $B = 32{,}768$ ระบบขนาดเล็กควรใช้ชุดใหญ่ที่สุดที่หน่วยความจำรองรับ

ไลบรารี Python ที่ใช้ได้ แนวทางใช้ CLIP ค้นโปสเตอร์ด้วยข้อความ

โค้ดต่อไปนี้เป็นแนวทางสำหรับระบบจริง ต้องติดตั้ง open_clip และดาวน์โหลดน้ำหนักก่อน จึงไม่ได้รันในหนังสือ

```
import open_clip, torch
model, _, prep = open_clip.create_model_and_transforms(
    "ViT-B-32", pretrained="laion2b_s34b_b79k")
tok = open_clip.get_tokenizer("ViT-B-32")
img = model.encode_image(torch.stack([prep(p) for p in posters]))
txt = model.encode_text(tok(["a warm family animation"]))
img = img / img.norm(dim=-1, keepdim=True)
txt = txt / txt.norm(dim=-1, keepdim=True)
scores = (txt @ img.T).squeeze()        # โคไซน์ข้ามสื่อ
```

4.6.10 รวมสื่อกับพฤติกรรมผู้ใช้: VBPR

เทคนิคทั้งหมดในบทนี้ใช้เฉพาะเนื้อหาของรายการ **VBPR (Visual Bayesian Personalized Ranking)** (He and McAuley, 2016) รวมเนื้อหาเข้ากับ การกรองแบบร่วมมือ (บทที่ 5) โดยเพิ่มพจน์ของคุณลักษณะภาพเข้าไปในแบบจำลองแยกตัวประกอบ ดังสมการ (4.55)

$$\hat{x}_{ui} = \beta_i + \boldsymbol{\gamma}_u \cdot \boldsymbol{\gamma}_i + \boldsymbol{\theta}_u \cdot (\mathbf{E}\,\mathbf{f}_i) \tag{4.55}$$

โดยที่

- β_i ความนิยมพื้นฐานของรายการ i
- $\boldsymbol{\gamma}_u, \boldsymbol{\gamma}_i$ เวกเตอร์แฝงของผู้ใช้และรายการ เรียนรู้จากพฤติกรรม (แบบเดียวกับบทที่ 5)
- $\mathbf{f}_i$ เวกเตอร์ภาพของรายการจาก CNN หรือ Transformer (ไม่ต้องเรียนรู้)
- $\mathbf{E}$ เมทริกซ์ฉายคุณลักษณะภาพลงมิติแฝง เรียนรู้ร่วมกับผู้ใช้ทุกคน
- $\boldsymbol{\theta}_u$ รสนิยมด้านภาพของผู้ใช้ u

$$\mathcal{L}_{BPR} = -\sum_{(u,i,j)} \ln \sigma\left(\hat{x}_{ui} - \hat{x}_{uj}\right), \qquad \sigma(z) = \frac{1}{1+e^{-z}} \tag{4.56}$$

โดยที่

- (u, i, j) สามสิ่ง: ผู้ใช้ u เรื่องที่ชอบ i และเรื่องที่ไม่ชอบหรือยังไม่เคยดู j
- σ ฟังก์ชันซิกมอยด์
- $\mathcal{L}_{BPR}$ ค่าสูญเสียแบบจัดอันดับเป็นคู่ (Rendle et al., 2009)

ตัวอย่างการคำนวณ 4.40 คะแนน VBPR และค่าสูญเสีย BPR ของ u1

โจทย์ ให้ $\mathbf{E} = \begin{bmatrix} 0.5 & 0 & 0.5 \\ 0 & 1 & 0 \end{bmatrix}$, $\boldsymbol{\theta}_{u1} = (1.0, 0.2)$, $\boldsymbol{\gamma}_{u1} = (0.3, 0.1)$, $\boldsymbol{\gamma}_{i5} = (0.2, 0.4)$, $\boldsymbol{\gamma}_{i7} = (-0.1, 0.3)$, $\beta_{i5} = 0.1$, $\beta_{i7} = 0.2$ จงหาคะแนนของ i5 และ i7 ค่าสูญเสีย BPR ของคู่ (u1, i5, i7) และคะแนนของ i12 ซึ่งเป็นรายการใหม่

สมการที่ใช้ สมการ (4.55) และ (4.56)

ข้อมูลที่ใช้ เวกเตอร์ภาพที่เตรียมแล้วของเทคนิค Transformer (หัวข้อ 4.5.1) พารามิเตอร์อื่นเป็นค่าสมมติ

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- เวกเตอร์ภาพ f_i ของ i5 Wicked, i7 Ballerina และ i12 Project Hail Mary มาจากตารางที่ 4.16 หลังลบค่าเฉลี่ยของคลัง $\boldsymbol{\mu}$I = (0.5417, 0.5667, 0.3667) และปรับความยาว (หัวข้อ 4.5.1)
- คู่ (u1, i5, i7) เลือกเพราะ i5 อยู่อันดับ 1 ของ u1 ในทุกวิธีที่ผ่านมา ส่วน i7 อยู่ท้ายตาราง (ตารางที่ 4.19) i12 เป็นเรื่องฉายปี 2026 (ตารางที่ 4.4) จึงใช้แทนรายการใหม่ที่ยังไม่มีพฤติกรรม

- พารามิเตอร์ E, θ_u1, γ, β เป็นค่าสมมติ ในระบบจริงเรียนรู้ด้วยค่าสูญเสีย BPR จากพฤติกรรมของผู้ใช้ทุกคน

ขั้นที่ 1 ฉายเวกเตอร์ภาพ

- $\mathbf{f}_{i5} = (0.7552, 0.1590, 0.6359)$ ได้ $\mathbf{Ef}_{i5} = (0.6956, 0.1590)$
- $\mathbf{f}_{i7} = (-0.6714, -0.5240, -0.5240)$ ได้ $\mathbf{Ef}_{i7} = (-0.5977, -0.5240)$

ขั้นที่ 2 คะแนนของ i5

- พจน์พฤติกรรม $\boldsymbol{\gamma}_{u1} \cdot \boldsymbol{\gamma}_{i5} = 0.06 + 0.04 = 0.1000$
- พจน์ภาพ $\boldsymbol{\theta}_{u1} \cdot (\mathbf{Ef}_{i5}) = 0.7274$
- $\hat{x}_{u1,i5} = 0.1 + 0.1000 + 0.7274 = 0.9274$

ขั้นที่ 3 คะแนนของ i7 และค่าสูญเสีย

- $\hat{x}_{u1,i7} = 0.2 + 0.0000 + (-0.7025) = -0.5025$
- ผลต่าง 1.4299 ได้ $\sigma = 0.8069$ และ $-\ln\sigma = 0.2146$

ขั้นที่ 4 รายการใหม่ i12

- ยังไม่มีพฤติกรรม จึง $\beta_{i12} = 0$, $\boldsymbol{\gamma}_{i12} = \mathbf{0}$ เหลือเพียงพจน์ภาพ = -0.6865

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

รายการ	ภาพ (ตาราง 4.16)	x − μI	f_i (ยาว 1)	E f_i	θ_u1 · E f_i
i5 Wicked	(0.7, 0.6, 0.5)	(0.1583, 0.0333, 0.1333)	(0.7552, 0.1590, 0.6359)	(0.6956, 0.1590)	0.7274
i7 Ballerina	(0.2, 0.3, 0.1)	(−0.3417, −0.2667, −0.2667)	(−0.6714, −0.5240, −0.5240)	(−0.5977, −0.5240)	−0.7025
i12 Project Hail Mary	(0.3, 0.3, 0.1)	(−0.2417, −0.2667, −0.2667)	(−0.5395, −0.5954, −0.5954)	(−0.5674, −0.5954)	−0.6865

- E f_i: แถวที่ 1 ของ E = (0.5, 0, 0.5) เฉลี่ยมิติ 1 กับ 3 แถวที่ 2 = (0, 1, 0) หยิบมิติ 2 เช่น i5: (0.5 × 0.7552 + 0.5 × 0.6359, 0.1590) = (0.6956, 0.1590)
- θ_u1 · E f_i5 = 1.0 × 0.6956 + 0.2 × 0.1590 = 0.6956 + 0.0318 = 0.7274
- x_u1,i5 = 0.1 + (0.3 × 0.2 + 0.1 × 0.4) + 0.7274 = 0.1 + 0.1000 + 0.7274 = 0.9274
- x_u1,i7 = 0.2 + (0.3 × (−0.1) + 0.1 × 0.3) + (−0.7025) = 0.2 + 0.0000 − 0.7025 = −0.5025
- ผลต่าง 0.9274 − (−0.5025) = 1.4299 ⟶ σ(1.4299) = 1/(1 + e^(−1.4299)) = 1/(1 + 0.2393) = 0.8069 ⟶ −ln 0.8069 = 0.2146

ตีความ

ค่าสูญเสียต่ำ (0.2146) แปลว่าแบบจำลองจัด i5 เหนือ i7 ถูกต้องแล้ว ส่วนรายการใหม่ i12 ยังได้คะแนนจากพจน์ภาพ แม้ไม่มีใครเคยดู นี่คือวิธีที่ VBPR แก้ปัญหา **รายการเริ่มต้นเย็น (Cold Start)** ซึ่งการกรองแบบร่วมมือล้วนทำไม่ได้

4.6.11 แนวทางเลือกเทคนิค Fusion

ตารางที่ 4.23 เทียบเทคนิค Fusion ทั้งหมดด้วยการประเมินแบบกันออกทีละรายการของหัวข้อ 4.7 บนสองเทคนิคของตัวเข้ารหัส

ตารางที่ 4.23 ผลของเทคนิค Fusion (กันออกทีละรายการ 20 กรณี)

เทคนิค Fusion	เทคนิค CNN		เทคนิค Transformer	
	HR@3	MRR	HR@3	MRR
Late Fusion น้ำหนักตั้งต้น	0.75	0.7671	0.75	0.7635
z-score	0.80	0.7506	0.75	0.7384
Min-Max	0.80	0.7256	0.75	0.7410
Borda	0.80	0.7546	0.75	0.7463
RRF k = 60	0.80	0.7535	0.80	0.7521
RRF k = 0	0.85	0.8185	0.85	0.7921
Gate τ = 0.5	0.75	0.7392	0.75	0.7175

ที่มา: ผู้เขียนคำนวณ โปรไฟล์ค่าเฉลี่ยทุกระบบ

แนวทางเลือกใช้ เลือกเทคนิค Fusion อย่างไร

- **เริ่มจาก Late Fusion** เพราะง่าย อธิบายได้ และเพิ่มหรือถอดสื่อได้ทันที
- **ปรับมาตราหรือใช้อันดับ** เมื่อคะแนนของแต่ละสื่อมีช่วงต่างกันมาก หรือมาจากตัวเข้ารหัสต่างชนิด
- **เลือกน้ำหนักด้วยชุดตรวจสอบ** อย่าเลือกบนชุดทดสอบ (ตัวอย่างการคำนวณ 4.35)
- **ใช้ Gate หรือ Attention** เมื่อผู้ใช้มีความหลากหลายสูงและมีข้อมูลพอให้ประเมินว่าใครเชื่อสื่อใด
- **ใช้ CLIP** เมื่อต้องค้นข้ามสื่อ และ **ใช้ VBPR** เมื่อมีพฤติกรรมมากพอและต้องรับมือรายการใหม่

โปรแกรม 4.13 Late Fusion, z-score และ RRF ของ u1

```
import numpy as np
from mm4 import ITEMS, R, MODS, vectors, l2


V = vectors('TF')
r = R['u1']
cand = [i for i in ITEMS if i not in r]
like = [i for i, x in r.items() if x >= 4]
S = {}
for m in MODS:
    p = l2(np.mean([V[m][i] for i in like], axis=0))
    S[m] = {i: p @ V[m][i] for i in cand}


def rank(s):
    return sorted(s, key=lambda i: (-round(s[i], 12), int(i[1:])))


W = {'T': 0.4, 'I': 0.2, 'A': 0.2, 'V': 0.2}
late = {i: sum(W[m] * S[m][i] for m in MODS) for i in cand}
z = {}
for m in MODS:
    v = np.array([S[m][i] for i in cand])
    z[m] = {i: (S[m][i] - v.mean()) / v.std() for i in cand}
zf = {i: sum(W[m] * z[m][i] for m in MODS) for i in cand}
```

```
rrf = {i: sum(1 / (60 + rank(S[m]).index(i) + 1) for m in MODS)
       for i in cand}
for name, s in [('late', late), ('z-score', zf), ('RRF', rrf)]:
    print('%-8s' % name, rank(s)[:3])
```

ผลลัพธ์

```
late     ['i5', 'i8', 'i9']
z-score  ['i5', 'i12', 'i8']
RRF      ['i5', 'i8', 'i12']
```

4.7 การประเมินผล

หัวข้อนี้นำตัวชี้วัดของบทที่ 2 มาวัดว่าการเพิ่มสื่อและการเลือกเทคนิคช่วยระบบจริงหรือไม่ โดยใช้โปรโตคอลเดียวกับบทที่ 3 ผลจึงเทียบกับบทที่ 3 ได้โดยตรง ดังสรุปในภาพที่ 4.58 แสดงการซ่อนคำตอบ การจัดอันดับ การวัดผล และการถอดสื่อทีละชนิด



ภาพที่ 4.58 โปรโตคอลและตัวชี้วัดการประเมินผล

4.7.1 โปรโตคอลซ่อนคำตอบทีละเรื่อง

MovieMini-12 มีผู้ใช้เพียง 8 คน จึงใช้การประเมินแบบ **กันออกทีละรายการ (Leave-One-Out)** ทุกเรื่องที่ผู้ใช้ชอบจะถูกซ่อนหนึ่งครั้ง รวม 20 กรณี ขั้นตอนของแต่ละกรณีมีดังนี้

1. ซ่อนเรื่องที่ชอบหนึ่งเรื่อง h ออกจากประวัติของผู้ใช้
2. สร้างโปรไฟล์จากประวัติที่เหลือ
3. ให้คะแนนรายการที่ผู้ใช้ยังไม่เคยให้คะแนนรวมกับ h แล้วจัดอันดับ
4. บันทึกอันดับของ h แล้วคำนวณตัวชี้วัดตามสมการ (4.57)

$$\mathrm{HR@}K = \frac{1}{N}\sum_{c=1}^{N} \mathbb{1}\left[r_c \le K\right], \quad \mathrm{MRR} = \frac{1}{N}\sum_{c=1}^{N} \frac{1}{r_c}, \quad \mathrm{nDCG@}K = \frac{1}{N}\sum_{c=1}^{N} \frac{\mathbb{1}[r_c \le K]}{\log_2(r_c+1)} \tag{4.57}$$

โดยที่

- N จำนวนกรณี เท่ากับ 20
- r_c อันดับของเรื่องที่ซ่อนในกรณีที่ c
- $\mathbb{1}[\cdot]$ มีค่า 1 เมื่อเงื่อนไขเป็นจริง และ 0 เมื่อเป็นเท็จ
- K ขนาดรายการแนะนำ บทนี้ใช้ 3

4.7.2 ตัวอย่างการคำนวณของผู้ใช้สองคน

ตัวอย่างการคำนวณ 4.41 ซ่อน i2 ของ u1 และซ่อน i1 ของ u4

โจทย์ ใช้โปรไฟล์ค่าเฉลี่ยและ Late Fusion น้ำหนักตั้งต้นของเทคนิค Transformer จงหาอันดับของเรื่องที่ซ่อนในสองกรณี และค่าของตัวชี้วัดแต่ละกรณี

สมการที่ใช้ สมการ (4.36), (4.44) และ (4.57)

ข้อมูลที่ใช้ คะแนนของผู้ใช้จากตารางที่ 4.5

ที่มาของข้อมูล (ข้อมูลแต่ละตัวมาจากไหน)

- ทั้งสองกรณีมาจากตารางที่ 4.5: u1 ชอบ i1, i2, i3 และไม่ชอบ i4, i6 ส่วน u4 ชอบ i1, i7, i8 และไม่ชอบ i5 ต้องตัดเรื่องที่ซ่อนออกจากประวัติก่อนสร้างโปรไฟล์ทุกครั้ง
- ผู้สมัครของแต่ละกรณี = เรื่องที่ผู้ใช้ยังไม่เคยให้คะแนน รวมกับเรื่องที่ซ่อน: u1 มี 7 + 1 = 8 เรื่อง และ u4 มี 8 + 1 = 9 เรื่อง
- เวกเตอร์ทั้งสี่สื่อมาจากตารางที่ 4.15 และ 4.16 (เทคนิค Transformer) ใช้น้ำหนักตั้งต้น (0.4, 0.2, 0.2, 0.2)

ขั้นที่ 1 u1 ซ่อน i2 Inside Out 2

- โปรไฟล์สร้างจาก i1 และ i3 ผู้สมัครคือเจ็ดเรื่องเดิมรวม i2 เป็นแปดเรื่อง
- สามอันดับแรก: i2 (0.6637), i5 (0.4350), i8 (−0.2203)
- i2 อยู่อันดับ 1 จึงได้ HR@3 = 1, 1/r = 1.0000, nDCG@3 = 1.0000

ขั้นที่ 2 u4 ซ่อน i1 Toy Story 5

- โปรไฟล์สร้างจาก i7 และ i8 ซึ่งเป็นแอ็กชันทั้งคู่
- สามอันดับแรก: i10 (0.7706), i11 (0.5290), i12 (0.2421)
- i1 อยู่อันดับ 7 จึงได้ HR@3 = 0, 1/r = 0.1429, nDCG@3 = 0

ขั้นที่ 3 ค่าเฉลี่ยของสองกรณี

- HR@3 = $(1 + 0)/2 = 0.5$, MRR = $(1.0000 + 0.1429)/2 = 0.5714$

ขั้นตอนโดยละเอียด: ที่มาของตัวเลขทีละขั้น

u1 ซ่อน i2 Inside Out 2

- ประวัติที่เหลือ: i1 (5), i3 (4), i4 (1), i6 (1) · เรื่องที่ชอบที่ใช้สร้างโปรไฟล์: i1, i3 · ผู้สมัคร 8 เรื่อง = เรื่องที่ u1 ยังไม่ให้คะแนน 7 เรื่อง + i2

อันดับ	รายการ	T	I	A	V	s
1	i2 Inside Out 2 ⟵ ซ่อน	0.2786	0.9124	0.8488	1.0000	0.6637
2	i5 Wicked	0.4843	0.9013	0.9480	−0.6427	0.4350
3	i8 Furiosa: A Mad Max Saga	0.1990	−0.1399	−0.9136	−0.4462	−0.2203
4	i9 Conclave	0.0563	−0.7181	−0.1603	−0.6427	−0.2817
5	i12 Project Hail Mary	0.3335	−0.7472	−0.6586	−0.8786	−0.3235
6	i11 Mickey 17	0.3474	−0.6424	−0.9214	−0.9726	−0.3683
7	i7 Ballerina	0.1637	−0.7468	−0.9768	−0.5166	−0.3826

8	i10 Dune: Part Two	0.1830	−0.7483	−0.9970	−0.8383	−0.4435

- i2 อยู่อันดับ 1: HR@3 = 1, 1/r = 1.0000, nDCG@3 = 1.0000

u4 ซ่อน i1 Toy Story 5

- ประวัติที่เหลือ: i7 (5), i8 (5), i5 (1) · เรื่องที่ชอบที่ใช้สร้างโปรไฟล์: i7, i8 · ผู้สมัคร 9 เรื่อง = เรื่องที่ u4 ยังไม่ให้คะแนน 8 เรื่อง + i1

อันดับ	รายการ	T	I	A	V	s
1	i10 Dune: Part Two	0.5641	0.9012	0.9708	0.8531	0.7706
2	i11 Mickey 17	0.4757	0.2985	0.8480	0.5473	0.5290
3	i12 Project Hail Mary	0.1587	0.4430	0.4096	0.0406	0.2421
4	i9 Conclave	0.2614	0.3655	−0.0897	−0.3208	0.0955
5	i4 It Ends with Us	0.2742	0.9349	−0.9999	−0.3505	0.0266
6	i6 Bridget Jones: Mad About the Boy	0.0723	0.4972	−0.9528	−0.6152	−0.1852
7	i1 Toy Story 5 ⟵ ซ่อน	0.1882	−0.8678	−0.6338	−0.3137	−0.2878
8	i2 Inside Out 2	0.0599	−0.6547	−0.7649	−0.4852	−0.3570
9	i3 The Wild Robot	0.1706	−0.9181	−0.9593	−0.6337	−0.4340

- i1 อยู่อันดับ 7: HR@3 = 0, 1/r = 0.1429, nDCG@3 = 0.0000

ตีความ

กรณีของ u4 แสดงจุดอ่อนของโปรไฟล์ค่าเฉลี่ย เมื่อซ่อนเรื่องแอนิเมชันเพียงเรื่องเดียว ประวัติที่เหลือไม่มีร่องรอยของรสนิยมนี้เลย ไม่มีเทคนิค Fusion ใดกู้กลับมาได้ ต้องใช้ข้อมูลประเภทอื่น เช่น พฤติกรรมของผู้ใช้คนอื่นในบทที่ 5

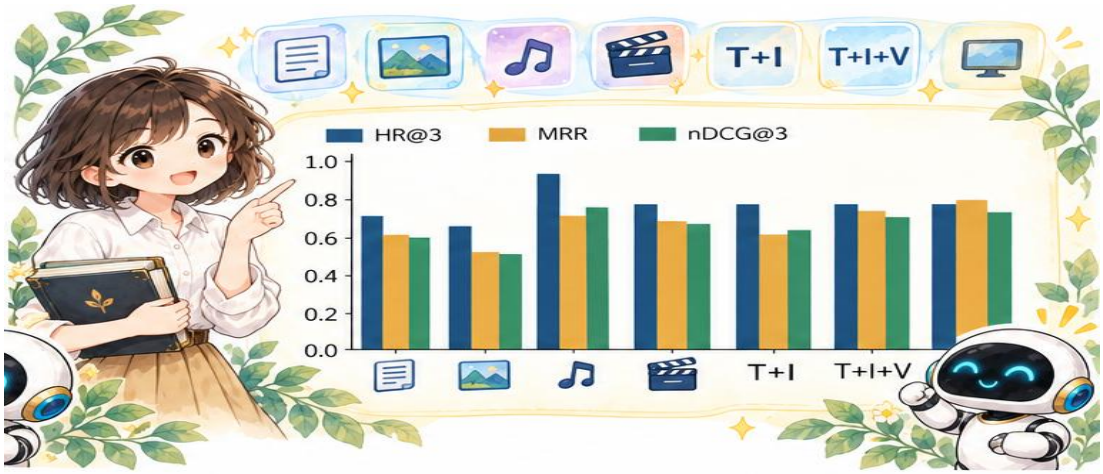
4.7.3 ผลของสื่อเดี่ยว การรวมสื่อ และสองเทคนิค

ตารางที่ 4.24 แสดงผลของระบบเจ็ดแบบบนสองเทคนิค และภาพที่ 4.50 แสดงผลของเทคนิค Transformer เป็นกราฟ

ตารางที่ 4.24 ผลของสื่อเดี่ยวและการรวมสื่อบนสองเทคนิค (20 กรณี)

ระบบ	น้ำหนัก (T, I, A, V)	เทคนิค CNN			เทคนิค Transformer		
		HR@3	MRR	nDCG@3	HR@3	MRR	nDCG@3
ข้อความ	(1.00, 0.00, 0.00, 0.00)	0.65	0.6818	0.6250	0.70	0.6044	0.5946
ภาพ	(0.00, 1.00, 0.00, 0.00)	0.65	0.4876	0.4827	0.65	0.5035	0.5012
เสียง	(0.00, 0.00, 1.00, 0.00)	0.95	0.7683	0.8077	0.90	0.7033	0.7393
วิดีโอ	(0.00, 0.00, 0.00, 1.00)	0.60	0.5883	0.5381	0.75	0.6673	0.6577
T+I	(0.50, 0.50, 0.00, 0.00)	0.75	0.6094	0.6196	0.75	0.6082	0.6196
T+I+V	(0.40, 0.20, 0.00, 0.40)	0.75	0.6969	0.6815	0.75	0.7247	0.7000
สี่สื่อ	(0.40, 0.20, 0.20, 0.20)	0.75	0.7671	0.7250	0.75	0.7635	0.7250

ที่มา: ผู้เขียนคำนวณ โปรไฟล์ค่าเฉลี่ย Late Fusion



ภาพที่ 4.50 HR@3, MRR และ nDCG@3 ของระบบเจ็ดแบบ (เทคนิค Transformer)

ข้อสังเกตจากตารางที่ 4.24 มีสามข้อ

- **เสียงเป็นสื่อเดี่ยวที่ดีที่สุด** ในชุดข้อมูลนี้ (HR@3 = 0.90 บนเทคนิค Transformer) ดีกว่าข้อความซึ่งเป็นหลักฐานเดียวของบทที่ 3
- **การรวมสี่สื่อให้ MRR สูงสุด** (0.7635) แม้ HR@3 ไม่สูงกว่าเสียงอย่างเดียว แปลว่าการรวมช่วยดันเรื่องที่ถูกขึ้นอันดับ 1 บ่อยขึ้น
- **สองเทคนิคให้ผลรวมใกล้กันมาก** ความต่างของ MRR หลังรวมสี่สื่อน้อยกว่า 0.01 ทั้งที่ระดับสื่อเดี่ยวต่างกันได้ถึง 0.08

4.7.4 ถอดสื่อทีละชนิด

การทดลอง **ถอดสื่อ (Ablation)** ตอบคำถามว่าสื่อใด "จำเป็น" ต่อระบบรวม โดยถอดสื่อหนึ่งออกแล้วกระจายน้ำหนักให้สื่อที่เหลือตามสมการ (4.52) ภาพที่ 4.51 แสดงผลบนเทคนิค Transformer



ภาพที่ 4.51 HR@3 และ MRR เมื่อถอดสื่อทีละชนิด

- **ถอดวิดีโอทำให้ MRR ลดมากที่สุด** (0.7635 → 0.6796) แม้วิดีโอเดี่ยวไม่ใช่สื่อที่ดีที่สุด แปลว่าวิดีโอให้ข้อมูลที่ **ไม่ซ้ำ** กับสื่ออื่น
- **ถอดข้อความแล้ว HR@3 เพิ่มขึ้น** (0.80) แต่ MRR ลดลง (0.7130)

ข้อความจึงช่วยจัดลำดับอันดับต้นมากกว่าช่วยให้ติด Top-3

- **ถอดภาพแทบไม่เปลี่ยนผล** (MRR 0.7471) เพราะข้อมูลภาพซ้ำกับวิดีโอมาก

ข้อควรระวัง อ่านผลบนชุดข้อมูลขนาดเล็กอย่างระมัดระวัง

- ทั้งหมดมีเพียง 20 กรณี การเปลี่ยนอันดับของเรื่องเดียวทำให้ HR@3 เปลี่ยน 0.05 ความต่างเล็กกว่านี้จึงไม่มีความหมายทางสถิติ
- ควรรายงานช่วงความเชื่อมั่นด้วย Bootstrap ตามบทที่ 2 ก่อนสรุปว่าระบบใดดีกว่า
- ผลของบทนี้ใช้เวกเตอร์เพื่อการสอน จึงแสดง **วิธีวิเคราะห์** ไม่ใช่ข้อสรุปว่าสื่อใดดีที่สุดในโลกจริง

โปรแกรม 4.14 ประเมินแบบกันออกทีละรายการ

```
import math
import numpy as np
from mm4 import ITEMS, R, MODS, vectors, l2

W = {'T': 0.4, 'I': 0.2, 'A': 0.2, 'V': 0.2}

def scores(V, r, cand, w):
    like = [i for i, x in r.items() if x >= 4]
    s = {i: 0.0 for i in cand}
    for m in MODS:
        p = l2(np.mean([V[m][i] for i in like], axis=0))
        for i in cand:
            s[i] += w[m] * (p @ V[m][i])
    return s

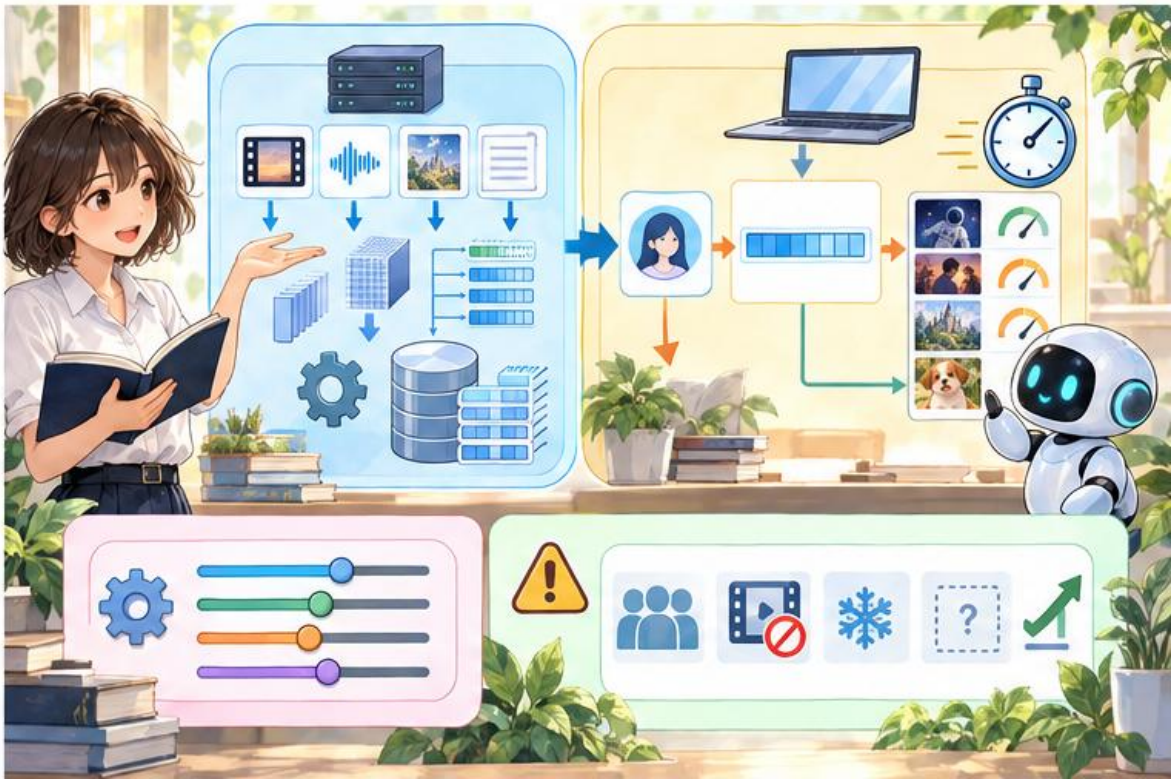
for tech in ['CNN', 'TF']:
    V = vectors(tech)
    ranks = []
    for u, r in R.items():
        for h in [i for i, x in r.items() if x >= 4]:
            train = {i: x for i, x in r.items() if i != h}
            cand = [i for i in ITEMS if i not in r] + [h]
            s = scores(V, train, cand, W)
            order = sorted(cand, key=lambda i: (-round(s[i], 12),
                                                int(i[1:])))
            ranks.append(order.index(h) + 1)
    hr = np.mean([x <= 3 for x in ranks])
    mrr = np.mean([1 / x for x in ranks])
    nd = np.mean([1 / math.log2(x + 1) if x <= 3 else 0
                  for x in ranks])
    print(tech, 'HR@3 %.4f MRR %.4f nDCG@3 %.4f' % (hr, mrr, nd))
```

ผลลัพธ์

```
CNN HR@3 0.7500 MRR 0.7671 nDCG@3 0.7250
TF HR@3 0.7500 MRR 0.7635 nDCG@3 0.7250
```

4.8 ประกอบเป็นระบบจริง ไฮเปอร์พารามิเตอร์ และข้อจำกัด

หัวข้อ 4.1–4.7 อธิบายแต่ละขั้นแยกกัน หัวข้อนี้ประกอบทุกขั้นเป็นระบบที่ใช้งานได้จริง สรุปไฮเปอร์พารามิเตอร์ทั้งบท และชี้ข้อจำกัดที่ผู้อ่านต้องรู้ก่อนนำไปใช้ ดังสรุปในภาพที่ 4.59 แยกงานออฟไลน์ออกจากงานออนไลน์ พร้อมไฮเปอร์พารามิเตอร์และข้อจำกัดสำคัญ



ภาพที่ 4.59 การประกอบระบบจริงและข้อจำกัด

4.8.1 งานออฟไลน์และงานออนไลน์

ระบบหลายรูปแบบแบ่งงานเป็นสองส่วนตามเวลาที่ทำ ส่วนที่ใช้การคำนวณหนัก เช่น การสกัดเวกเตอร์ด้วย CNN หรือ Transformer ทำล่วงหน้าแบบ **ออฟไลน์** ส่วนที่ต้องตอบผู้ใช้ทันทีทำแบบ **ออนไลน์** ซึ่งต้องเสร็จภายในเสี้ยววินาที ตารางที่ 4.25 สรุปงานของทั้งสองส่วน

ตารางที่ 4.25 งานออฟไลน์และงานออนไลน์ของระบบผู้แนะนำหลายรูปแบบ

ส่วน	งาน	หัวข้อ	ไลบรารีที่ใช้ได้
ออฟไลน์	สกัดเวกเตอร์ของทุกรายการทุกสื่อด้วยตัวเข้ารหัสที่ฝึกไว้แล้ว	4.3, 4.4	torchvision, transformers, sentence-transformers
ออฟไลน์	ลบค่าเฉลี่ยของคลัง ปรับความยาว และสร้างดัชนีค้นหาเพื่อนบ้านใกล้ที่สุด	4.5.1	numpy, faiss
ออฟไลน์	สร้างหรือปรับโปรไฟล์ผู้ใช้ตามรอบเวลา และเลือกน้ำหนัก Fusion บนชุดตรวจสอบ	4.5, 4.6.5	numpy, pandas
ออนไลน์	ดึงรายการผู้สมัครจากดัชนีของแต่ละสื่อ	4.5	faiss
ออนไลน์	คำนวณคะแนนรายสื่อ รวมด้วย Fusion ตัดเรื่องที่ดูแล้ว แล้วส่ง Top-K	4.6	numpy

ส่วน	งาน	หัวข้อ	ไลบรารีที่ใช้ได้
ออนไลน์	บันทึกผลตอบรับเพื่อปรับโปรไฟล์และประเมินผลรอบถัดไป	4.7	ระบบบันทึกข้อมูล

ขั้นตอนวิธี 4.2 แนะนำภาพยนตร์ด้วยระบบหลายรูปแบบ

1. (ออฟไลน์) สำหรับทุกรายการ i และทุกสื่อ m คำนวณ $\mathbf{x}_i^m = f_\theta^m\left(\text{input}_i^m\right)$ ด้วย CNN หรือ Transformer (ขั้นตอนวิธี 4.1)
2. (ออฟไลน์) เตรียมเวกเตอร์ตามสมการ (4.33) และ (4.34) แล้วสร้างดัชนีของแต่ละสื่อ
3. (ออฟไลน์) สร้างโปรไฟล์ $\mathbf{p}_u^m$ ของผู้ใช้ทุกคนด้วยวิธีในหัวข้อ 4.5 และเลือกน้ำหนัก w_m บนชุดตรวจสอบ
4. (ออนไลน์) เมื่อผู้ใช้ u เปิดหน้าแนะนำ ดึงรายการผู้สมัครจากดัชนีด้วย $\mathbf{p}_u^m$ ของทุกสื่อ แล้วรวมเป็นเซตเดียว
5. (ออนไลน์) คำนวณ $s_m(u,i)$ ตามสมการ (4.35) ปรับมาตรา แล้วรวมด้วยสมการ (4.44) หรือ RRF กระจายน้ำหนักใหม่เมื่อสื่อหาย
6. (ออนไลน์) ตัดรายการที่ผู้ใช้เคยดู จัดอันดับ และส่ง Top-K พร้อมบันทึกผลตอบรับ

4.8.2 สรุปไฮเปอร์พารามิเตอร์ของบท

ตารางที่ 4.26 รวมไฮเปอร์พารามิเตอร์ทั้งหมดของบท พร้อมค่าที่นิยม ที่มา ค่าที่ใช้ในบทนี้ และหัวข้อที่อธิบาย ผู้อ่านใช้ตารางนี้เป็นรายการตรวจสอบเมื่อสร้างระบบของตนเองได้

ตารางที่ 4.26 สรุปไฮเปอร์พารามิเตอร์ของบท

ไฮเปอร์พารามิเตอร์	ค่าที่นิยมและที่มา	ในบทนี้	หัวข้อ
อัตราการเรียนรู้ η	10^{-3} (Adam), 2×10^{-5} (ปรับ BERT)	0.5 (ตัวอย่าง)	4.2.4
ขนาดเวกเตอร์ d	384–2,048 ตามตัวเข้ารหัส	12 / 3	4.2.5
ขนาดตัวกรอง k	3 (VGG), 7 (ชั้นแรก ResNet), 3–5 คำ (Text CNN)	2–3	4.3.2
การเติมขอบ p, ระยะก้าว s	$p=(k-1)/2$, $s=1$ หรือ 2	0–1, 1–2	4.3.2
Pooling	2×2 ระยะก้าว 2, GAP ที่ปลายโครงข่าย	2, GAP	4.3.3
หน้าต่างเสียง / ระยะเลื่อน / แถบ Mel	25 ms / 10 ms / 128 แถบ	ตามค่านิยม	4.3.6
จำนวนเฟรมวิดีโอ	8–32 เฟรม	16	4.3.6
จำนวนหัว h, มิติ d	12 หัว, 768 มิติ (BERT-base, ViT-B)	2 หัว, 2 มิติ	4.4.3
จำนวนชั้น L, d_ff	12 ชั้น, $4d$	1 บล็อก	4.4.5
ขนาดแพตช์ P	16 (ViT, AST ระยะก้าว 10), Tubelet $2\times16\times16$	ตามค่านิยม	4.4.7
β ของ Rocchio	0.75 (งานค้นคืน)	0.5	4.5.4
ครึ่งชีวิต h	สัปดาห์ถึงเดือน ตามโดเมน	30 วัน	4.5.5
จำนวนจุดสนใจ K, อุณหภูมิ τ	K = 4–8 (MIND)	K = 2, τ = 0.5	4.5.6
น้ำหนัก Fusion	เลือกบนชุดตรวจสอบ	(0.4, 0.2, 0.2, 0.2)	4.6.2
k ของ RRF	60 (Cormack et al., 2009)	60 และ 0	4.6.4

ไฮเปอร์พารามิเตอร์	ค่าที่นิยมและที่มา	ในบทนี้	หัวข้อ
อุณหภูมิ Gate	–	0.5	4.6.7
อุณหภูมิ CLIP, ขนาดชุด B	0.07 เรียนรู้ต่อได้, 32,768	0.07, 3	4.6.9
ขนาด Top-K	5–20 ตามหน้าจอ	3	4.7

4.8.3 ข้อจำกัดและทางต่อไป

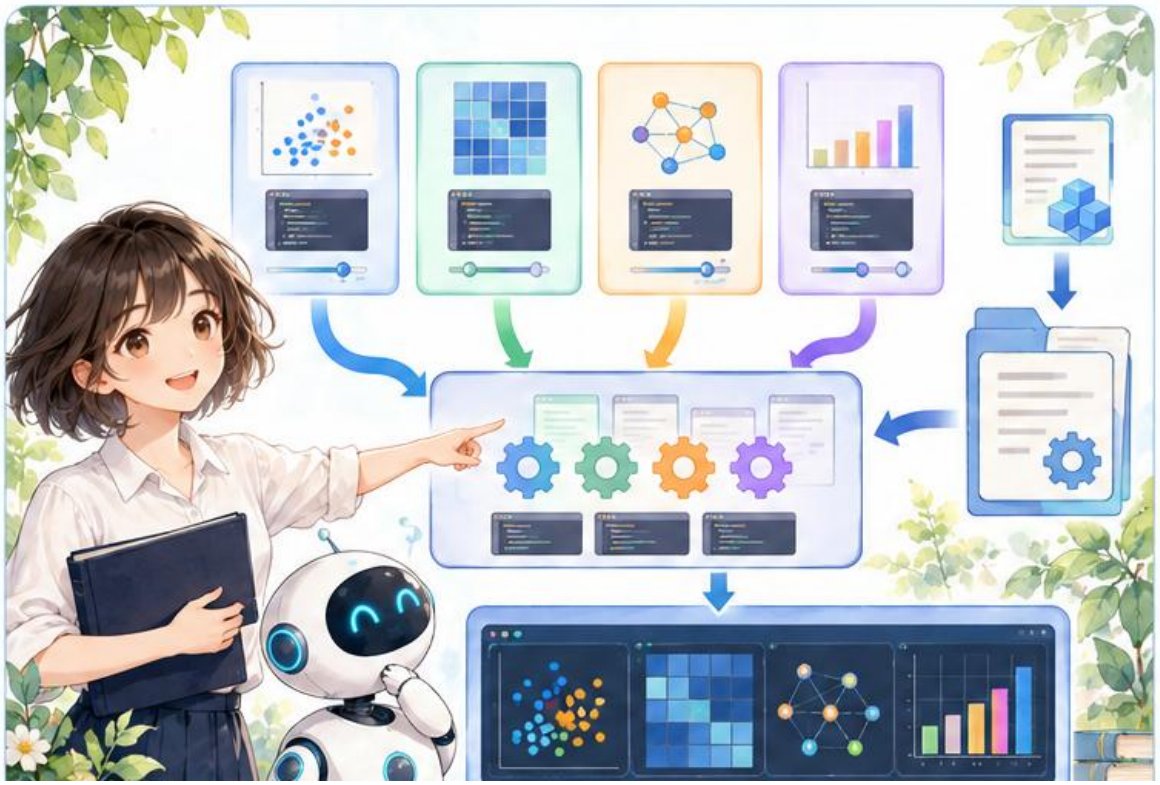
ระบบในบทนี้มีข้อจำกัดที่ควรรู้ ดังนี้

- **ข้อมูลเพื่อการสอน** เวกเตอร์ของบทนี้เป็นค่าจำลองขนาดย่อ ผลเชิงตัวเลขจึงแสดงวิธีคิด ไม่ใช่ข้อสรุปเกี่ยวกับภาพยนตร์จริง
- **ต้นทุนการสกัดเวกเตอร์** วิดีโอหนึ่งเรื่องมีตัวเลขนำเข้าหลายล้านค่า การสกัดเวกเตอร์ของคลังขนาดใหญ่ต้องใช้ GPU และเวลามาก
- **ตัวเข้ารหัสสำเร็จรูปอาจไม่ตรงโดเมน** แบบจำลองที่ฝึกบนภาพทั่วไปอาจไม่เข้าใจลักษณะของโปสเตอร์ภาพยนตร์ไทยหรือภาษาไทยในเรื่องย่อ
- **ความเป็นธรรมและอคติ** การให้น้ำหนักภาพมากอาจทำให้ภาพยนตร์ทุนน้อยที่โปสเตอร์ไม่สวยถูกแนะนำน้อยลง
- **เนื้อหาอย่างเดียวไม่พอ** กรณีของ u4 ในตัวอย่างการคำนวณ 4.41 แสดงว่าเมื่อประวัติไม่มีร่องรอยของรสนิยม ระบบต้องอาศัยพฤติกรรมของผู้ใช้คนอื่น ซึ่งเป็นเรื่องของบทที่ 5

ทางต่อไปที่สำคัญคือ **แบบจำลองพื้นฐานหลายรูปแบบ (Multimodal Foundation Models)** ซึ่งรับข้อความ ภาพ เสียง และวิดีโอในแบบจำลองเดียว และ **แบบจำลองภาษาขนาดใหญ่ (Large Language Models)** ที่ใช้อธิบายเหตุผลของคำแนะนำ ทั้งสองแนวทางยังใช้แนวคิดของบทนี้ครบ คือสกัดเวกเตอร์ สร้างโปรไฟล์ และรวมหลักฐาน

4.9 แนวทางโปรแกรมประกอบหัวข้อและ Lab ท้ายบท

ทุกหัวข้อของบทนี้มีโปรแกรมขนาดเล็กประกอบเพื่อให้ผู้อ่านตรวจผลการคำนวณด้วยมือและทดลองเปลี่ยนค่าเองได้ โปรแกรมทั้งหมดใช้เพียง NumPy และเรียกข้อมูลกลางจากไฟล์ mm4.py ในภาคผนวก 4ก ส่วนที่ 1 จากนั้น Lab ท้ายบทนำโปรแกรมย่อยทั้งหมดมารวมเป็นระบบเดียว ดังสรุปในภาพที่ 4.60 แสดงลำดับงานจากการเตรียมข้อมูลหลายสื่อไปสู่การทดลองและประเมินผล



ภาพที่ 4.60 สายงานโปรแกรมและ Lab ท้ายบท

4.9.1 แผนที่โปรแกรม

ตารางที่ 4.27 แสดงโปรแกรมประกอบหัวข้อทั้งหมดและสิ่งที่ผู้อ่านควรสังเกต

ตารางที่ 4.27 แผนที่โปรแกรมประกอบหัวข้อ

โปรแกรม	ชื่อ	หัวข้อ	สิ่งที่ควรสังเกต
โปรแกรม 4.1	โหลดข้อมูลกลางของ MovieMini-12	4.1.3	รายการผู้สมัครของ u1 และมิติของแต่ละสื่อ
โปรแกรม 4.2	นิวรอนหนึ่งตัวกับ ReLU	4.2.3	ReLU ตัดค่าลบของ i7 เป็น 0
โปรแกรม 4.3	การลดตามเกรเดียนต์ของน้ำหนักหนึ่งตัว	4.2.4	$\eta = 1.5$ ทำให้ค่าสูญเสียเพิ่มขึ้น
โปรแกรม 4.4	เปิดแถวของตารางฝัง	4.2.5	x @ E เท่ากับการเปิดแถว
โปรแกรม 4.5	คอนโวลูชันหนึ่งมิติ การเติมขอบ และระยะก้าว	4.3.2	ความยาวผลลัพธ์ตาม p และ s
โปรแกรม 4.6	คอนโวลูชันสองมิติ ReLU และ Max Pooling	4.3.4	ค่าสูงอยู่ตรงขอบแนวตั้ง
โปรแกรม 4.7	โคไซน์ของเรื่องที่ u1 ชอบจากเทคนิค CNN	4.3.7	ภาพและวิดีโอของแอนิเมชันคล้ายกันมาก
โปรแกรม 4.8	Self-Attention ของสามโทเคน	4.4.2	น้ำหนักเปลี่ยนเมื่อเมทริกซ์ฉายเปลี่ยน

โปรแกรม	ชื่อ	หัวข้อ	สิ่งที่ควรสังเกต
โปรแกรม 4.9	หนึ่งบล็อกของ Transformer: Add & Norm และ FFN	4.4.5	ค่าเฉลี่ย 0 หลัง Layer Normalization
โปรแกรม 4.10	เทียบความคล้ายข้อความของสองเทคนิค	4.4.8	บริบทลดความคล้าย i2–i5
โปรแกรม 4.11	โปรไฟล์ค่าเฉลี่ยและคะแนนรายสื่อของ u1	4.5.2	แต่ละสื่อเห็น Top-3 ต่างกัน
โปรแกรม 4.12	เทียบโปรไฟล์สี่แบบของสองเทคนิค	4.5.8	Rocchio และการถ่วงคะแนนผลัก i5 ออก
โปรแกรม 4.13	Late Fusion, z-score และ RRF ของ u1	4.6.11	z-score ดัน i12 ขึ้นอันดับ 2
โปรแกรม 4.14	ประเมินแบบกันออกทีละรายการ	4.7.4	สองเทคนิคให้ผลรวมใกล้กัน

ประเด็นสำคัญ การพิมพ์โปรแกรมให้ทำงานถูกต้อง

- บันทึก mm4.py และโปรแกรมทุกตัวไว้ในโฟลเดอร์เดียวกัน แล้วรันด้วย python ชื่อไฟล์.py
- Python ใช้การย่อหน้าบอกขอบเขตของคำสั่ง ให้ย่อหน้าด้วยช่องว่างสี่ช่องเสมอ ห้ามผสมกับแท็บ
- บรรทัดที่ลงท้ายด้วย \ คือบรรทัดเดียวกับบรรทัดถัดไป
- ผลลัพธ์ในกรอบ "ผลลัพธ์" ของทุกโปรแกรมได้จากการรันจริง ผู้อ่านจึงใช้ตรวจโปรแกรมของตนเองได้

4.9.2 Lab 4 ระบบผู้แนะนำหลายรูปแบบ

Lab 4 รวมโปรแกรมของบทเป็นไฟล์เดียวชื่อ lab4.py (ภาคผนวก 4ก ส่วนที่ 2) แบ่งเป็นสี่ตอนตามลำดับของภาพที่ 4.1 ดังตารางที่ 4.28

ตารางที่ 4.28 โครงสร้างของ Lab 4

ตอน	งาน	ผลที่ต้องได้	หัวข้อ
1	เตรียมเวกเตอร์ของสี่สื่อจากสองเทคนิค	โคไซน์ภาพ i1–i7 ก่อนและหลังเตรียม	4.5.1
2	สร้างโปรไฟล์สี่แบบของ u1 บนสองเทคนิค	Top-3 ข้อความแปดชุด	4.5
3	รวมสี่สื่อด้วย Late, z-score, RRF และ Gate	Top-3 ของ u1 สี่ชุด	4.6
4	ประเมินแบบกันออกทีละรายการ ถอดสื่อ และเทียบสองเทคนิค	HR@3, MRR, nDCG@3	4.7

ผลลัพธ์ของ Lab 4 เมื่อรันทั้งไฟล์แสดงในกรอบด้านล่าง ผู้อ่านควรได้ตัวเลขตรงกันทุกหลัก

โปรแกรม ผลลัพธ์ของ Lab 4

```
ตอนที่ 1: cos ภาพ i1-i7 ก่อนและหลังเตรียมเวกเตอร์
   CNN ก่อน 0.9275  หลัง -0.8218
   TF  ก่อน 0.8547  หลัง -0.8355
ตอนที่ 2: Top-3 ข้อความของ u1 จากโปรไฟล์สี่แบบ
   CNN mean     ['i5', 'i12', 'i11']
   CNN rw       ['i11', 'i12', 'i10']
   CNN rocchio  ['i12', 'i11', 'i10']
   CNN decay    ['i12', 'i5', 'i11']
   TF  mean     ['i5', 'i12', 'i11']
   TF  rw       ['i11', 'i12', 'i10']
```

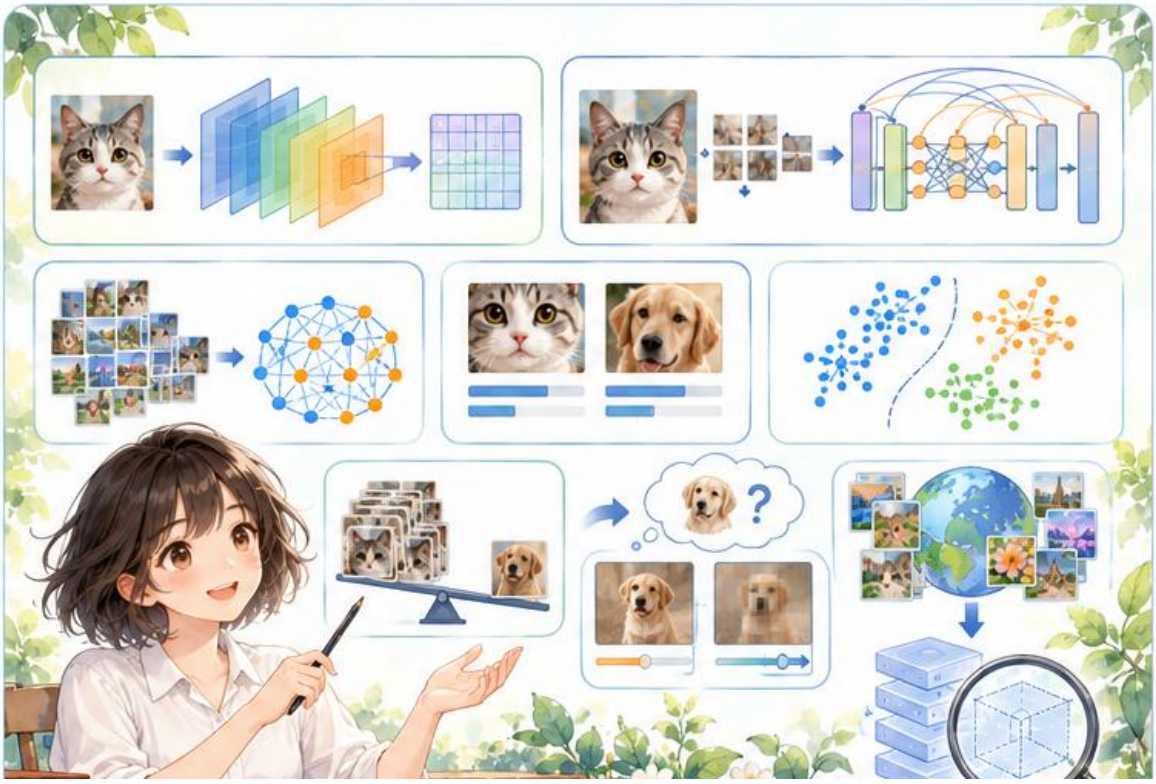
```
   TF  rocchio  ['i11', 'i12', 'i10']
   TF  decay    ['i12', 'i11', 'i5']
ตอนที่ 3: Top-3 ของ u1 หลังรวมสี่สื่อ (เทคนิค Transformer)
   late     [('i5', 0.4182), ('i8', -0.1921), ('i9', -0.3097)]
   z-score  [('i5', 1.4538), ('i12', 0.068), ('i8', 0.01)]
   RRF      [('i5', 0.0651), ('i8', 0.0635), ('i12', 0.0621)]
   gate     [('i5', 0.2941), ('i8', -0.3093), ('i9', -0.5653)]
ตอนที่ 4: LOO 20 กรณี (HR@3, MRR, nDCG@3)
   CNN T      0.6500 0.6818 0.6250
   CNN I      0.6500 0.4876 0.4827
   CNN A      0.9500 0.7683 0.8077
   CNN V      0.6000 0.5883 0.5381
   CNN late   0.7500 0.7671 0.7250
   TF  T      0.7000 0.6044 0.5946
   TF  I      0.6500 0.5035 0.5012
   TF  A      0.9000 0.7033 0.7393
   TF  V      0.7500 0.6673 0.6577
   TF  late   0.7500 0.7635 0.7250
   ถอด T     0.8000 0.7130 0.7065
   ถอด I     0.7500 0.7471 0.7131
   ถอด A     0.7000 0.6955 0.6565
   ถอด V     0.7500 0.6796 0.6631
   z-score    0.7500 0.7384 0.7131
   RRF        0.8000 0.7521 0.7381
   gate       0.7500 0.7175 0.6881
```

งานต่อยอดที่แนะนำสำหรับ Lab นี้มีดังนี้

- เพิ่มโปรไฟล์หลายจุดสนใจของหัวข้อ 4.5.6 แล้วเทียบผลกับโปรไฟล์ค่าเฉลี่ย
- แยกผู้ใช้เป็นชุดเลือกน้ำหนักและชุดทดสอบ แล้วทำ Grid Search ตามตัวอย่างการคำนวณ 4.35
- คำนวณช่วงความเชื่อมั่นแบบ Bootstrap ของ MRR ตามบทที่ 2 เพื่อดูว่าความต่างระหว่าง CNN กับ Transformer มีนัยสำคัญหรือไม่
- ถ้ามี GPU ให้แทนเวกเตอร์ในไฟล์ mm4.py ด้วยเวกเตอร์จริงจากกรอบไลบรารีในหัวข้อ 4.3.6 และ 4.4.7

4.10 บทวิเคราะห์ของผู้เขียน

หัวข้อนี้เป็นความเห็นเชิงวิเคราะห์ของผู้เขียนจากผลของบท โดยเชื่อมกับงานวิจัยที่เกี่ยวข้อง จุดประสงค์คือชี้ให้เห็นว่าตัวเลขในบทบอกอะไร และไม่ได้บอกอะไร เพื่อให้ผู้อ่านนำแนวคิดไปใช้อย่างมีวิจารณญาณ ดังสรุปในภาพที่ 4.61 สรุปข้อแลกเปลี่ยนระหว่างตัวเข้ารหัส สื่อที่จำเป็น โปรไฟล์ และวิธีรวมข้อมูล



ภาพที่ 4.61 ประเด็นวิเคราะห์ในการออกแบบระบบหลายสื่อ

4.10.1 การสกัดคุณลักษณะไม่ได้หายไป แต่ย้ายที่

เรามักเล่าวิวัฒนาการในหัวข้อ 4.2.2 ว่า "เครื่องเรียนรู้คุณลักษณะเองแทนมนุษย์" ผู้เขียนเห็นว่าคำอธิบายนี้ไม่ครบ การออกแบบของมนุษย์ไม่ได้หายไป แต่ย้ายจาก **สูตรของคุณลักษณะ** ไปอยู่ใน **โครงสร้างของแบบจำลอง** และ **การเลือกข้อมูล** CNN ฝังสมมติฐานว่าลวดลายสำคัญอยู่ใกล้กันและเกิดได้ทุกตำแหน่ง (หัวข้อ 4.3.1) ส่วน Transformer ฝังสมมติฐานน้อยกว่า จึงต้องการข้อมูลมากกว่ามากเพื่อเรียนรู้สิ่งที่ CNN "รู้มาก่อน" (Dosovitskiy et al., 2021)

ผลต่อผู้พัฒนาระบบผู้แนะนำคือ คำถามที่ควรถามไม่ใช่ "เทคนิคใดใหม่กว่า" แต่เป็น "สมมติฐานของเทคนิคใดตรงกับข้อมูลของเรา และเรามีข้อมูลพอสำหรับเทคนิคที่สมมติน้อยหรือไม่"

4.10.2 CNN หรือ Transformer: ต่างกันน้อยกว่าที่คาดในระดับระบบ

ในระดับสื่อเดี่ยว สองเทคนิคให้ MRR ต่างกันได้ถึง 0.08 (ภาพที่ 4.42) แต่หลังรวมสี่สื่อ MRR ต่างกันเพียง 0.0037 (0.7671 เทียบกับ 0.7635) ผู้เขียนตีความว่าความต่างระดับสื่อเกิดในทิศทางที่ต่างกัน CNN ได้เปรียบในข้อความและเสียง Transformer ได้เปรียบในภาพและวิดีโอ Fusion จึงหักล้างความต่างเหล่านั้นจนเหลือน้อย

บทเรียนนี้สอดคล้องกับข้อสังเกตในวงการว่าการเปลี่ยนตัวเข้ารหัสให้ดีขึ้นมักให้ผลน้อยกว่าที่คาดเมื่อวัดที่ปลายทางของระบบ ก่อนลงทุนเปลี่ยนตัวเข้ารหัสทั้งระบบ จึงควรวัดผลที่ระดับระบบรวมก่อนเสมอ

4.10.3 สื่อที่เก่งที่สุดไม่ใช่สื่อที่จำเป็นที่สุด

เสียงเป็นสื่อเดี่ยวที่ดีที่สุด (HR@3 = 0.90) แต่การถอดเสียงออกจากระบบรวมไม่ได้ทำให้ MRR ลดมากที่สุด สื่อที่ถอดแล้วเสียหายที่สุดคือวิดีโอ (MRR ลดเหลือ 0.6796) ผู้เขียนเห็นว่าความต่างนี้มาจากคำถามสองข้อที่ต่างกัน "สื่อนี้ทายถูกเองได้ดีเพียงใด" กับ "สื่อนี้ให้ข้อมูลที่สื่ออื่นไม่มีเพียงใด" การเลือกว่าจะลงทุนเก็บข้อมูลสื่อใดเพิ่มจึงควรใช้การทดลองถอดสื่อ (หัวข้อ 4.7.4) ไม่ใช่ผลของสื่อเดี่ยว

4.10.4 ความสม่ำเสมอไม่เท่ากับความสามารถในการแยกแยะ

Gate ในหัวข้อ 4.6.7 ให้น้ำหนักสื่อที่เรื่องที่ชอบ "คล้ายกัน" มาก แนวคิดนี้ฟังดูสมเหตุสมผล แต่ผลรวมกลับแย่ลง (MRR 0.7175 เทียบกับ 0.7635) เหตุผลคือ u1 ชอบแอนิเมชันสามเรื่องภาพและวิดีโอจึงสม่ำเสมอมาก แต่รายการผู้สมัครไม่มีแอนิเมชันเลย สื่อที่สม่ำเสมอที่สุดจึงแยกแยะผู้สมัครได้แย่ที่สุด ผู้เขียนเสนอว่า Gate ที่ดีควรดูทั้งความสม่ำเสมอของประวัติและการกระจายของคะแนนผู้สมัคร หรือเรียนรู้จากข้อมูลโดยตรงเมื่อมีผู้ใช้มากพอ

4.10.5 เรื่องที่ไม่ชอบเป็นดาบสองคม

การใช้เรื่องที่ไม่ชอบดูเหมือนเพิ่มข้อมูล แต่ผลของบทนี้แสดงว่าการถ่วงด้วยคะแนนทำให้ MRR ของระบบรวมลดจาก 0.7635 เหลือ 0.5688 ส่วน Rocchio ($\beta = 0.5$) เพิ่ม HR@3 เป็น 0.80 แต่ลด MRR เหลือ 0.7300 สาเหตุคือเรื่องที่ไม่ชอบมักมีบางมิติร่วมกับเรื่องที่ชอบ (เช่น family ในตัวอย่างการคำนวณ 4.26) การผลักออกจึงผลักสิ่งที่ผู้ใช้ชอบออกไปด้วย ผู้เขียนแนะนำให้ใช้เรื่องที่ไม่ชอบเป็น **ตัวกรอง** หรือ **ตัวลดอันดับ** ที่เบากว่า มากกว่าใช้เป็นแรงผลักเต็มที่

4.10.6 ตัวเข้ารหัสสำเร็จรูปคือสมมติฐานที่มองไม่เห็น

ทุกตัวเลขของบทเริ่มจากเวกเตอร์ของตัวเข้ารหัส ถ้าตัวเข้ารหัสฝึกบนข้อมูลที่ต่างจากโดเมนของเรา เช่น ภาพถ่ายทั่วไปแทนโปสเตอร์ หรือภาษาอังกฤษแทนภาษาไทย ความคล้ายที่วัดได้อาจสะท้อนสิ่งที่ตัวเข้ารหัสเคยเห็นมากกว่าสิ่งที่ผู้ชมรู้สึก ปัญหานี้มองไม่เห็นจากตัวชี้วัดใด เพราะทุกขั้นหลังจากนั้นทำงานถูกต้องตามสูตร ผู้เขียนจึงเสนอให้ตรวจเวกเตอร์ด้วยตาก่อนเสมอ เช่น ดูว่าเพื่อนบ้านใกล้ที่สุดของภาพยนตร์ที่รู้จักดีสมเหตุสมผลหรือไม่

มุมมองของผู้เขียน แนวทางที่ผู้เขียนแนะนำสำหรับระบบหลายรูปแบบระยะแรก

- เริ่มจาก **ข้อความ + สื่อที่มีครบทุกรายการ** ด้วยตัวเข้ารหัสที่ฝึกไว้แล้ว และ Late Fusion แบบปรับมาตรา
- วัดผลด้วยการถอดสื่อทีละชนิดก่อนตัดสินใจเพิ่มสื่อใหม่ (หัวข้อ 4.10.3)
- เลือกน้ำหนักและไฮเปอร์พารามิเตอร์บนชุดตรวจสอบที่แยกจากชุดทดสอบเสมอ (ตัวอย่างการคำนวณ 4.35)
- เก็บเวกเตอร์ของทั้ง CNN และ Transformer ไว้ถ้าทำได้ เพราะความต่างของสองเทคนิคเป็นข้อมูลที่เสริมกัน (หัวข้อ 4.10.2)
- เมื่อมีพฤติกรรมผู้ใช้มากพอ ให้รวมเนื้อหากับการกรองแบบร่วมมือด้วยแนวคิดอย่าง VBPR ซึ่งต่อยอดสู่บทที่ 5

สรุปท้ายบท

- แต่ละสื่อเห็นความคล้ายของภาพยนตร์คนละด้าน ระบบหลายรูปแบบจึงจับรสนิยมได้ครบกว่าระบบที่ใช้ข้อความอย่างเดียว (หัวข้อ 4.1)
- การสกัดคุณลักษณะพัฒนาจากการออกแบบด้วยมือ สู่ CNN และ Transformer หน่วยพื้นฐานร่วมคือนิวรอน การลดตามเกรเดียนต์ และการฝัง (หัวข้อ 4.2)
- CNN ใช้ตัวกรองเลื่อนบนหน้าต่างเฉพาะที่ ใช้น้ำหนักร่วมกัน และซ้อนชั้นเพื่อขยายระยะมองเห็น ประยุกต์กับทุกสื่อโดยจัดข้อมูลเป็นตารางที่เหมาะสม (หัวข้อ 4.3)
- Transformer ให้ทุกโทเคนปรึกษากันด้วย Self-Attention หลายหัว เพิ่มการฝังตำแหน่ง และซ้อนบล็อกที่มี Add & Norm กับ FFN ประยุกต์กับทุกสื่อโดยเปลี่ยนวิธีแบ่งโทเคน (หัวข้อ 4.4)
- โปรไฟล์ผู้ใช้สร้างได้หลายวิธี ตั้งแต่ค่าเฉลี่ย ถ่วงคะแนน Rocchio ลดตามเวลา หลายจุดสนใจ แบบ Attention จนถึงตัวเข้ารหัสลำดับ ต้องเตรียมเวกเตอร์ด้วยการลบค่าเฉลี่ยและปรับความยาวก่อน (หัวข้อ 4.5)
- Fusion รวมคะแนนรายสื่อได้ทั้งแบบ Late, Early และ Intermediate ต้องระวังมาตราของคะแนน และต้องเลือกน้ำหนักบนชุดตรวจสอบ (หัวข้อ 4.6)
- การประเมินแบบกันออกทีละรายการแสดงว่าการรวมสี่สื่อให้ MRR สูงสุด สองเทคนิคให้ผลรวมใกล้กัน และวิดีโอเป็นสื่อที่จำเป็นที่สุดเมื่อวัดด้วยการถอดสื่อ (หัวข้อ 4.7)

แบบฝึกหัดท้ายบท

แบบฝึกหัดแบ่งเป็นสามระดับ ระดับที่ 1 คำนวณด้วยมือ ระดับที่ 2 เขียนโปรแกรม และระดับที่ 3 วิเคราะห์และออกแบบ เฉลยอยู่ในเอกสารเฉลยแบบฝึกหัดแยกต่างหาก

ระดับที่ 1 คำนวณด้วยมือ

1. นิวรอนมีน้ำหนัก $\mathbf{w} = (0.4, -0.2, 0.7)$ และ $b = -0.5$ จงหา z และค่าส่งออกหลัง ReLU เมื่อป้อนเวกเตอร์ภาพของ i5 Wicked จากตารางที่ 4.16
2. แถวพิกเซล $\mathbf{x} = (0.1, 0.4, 0.8, 0.8, 0.3)$ ตัวกรอง $(-1, 0, 1)$ เติมขอบ $p = 1$ ระยะก้าว $s = 1$ จงหาความยาวผลลัพธ์ด้วยสมการ (4.13) และค่าส่งออกทุกตำแหน่ง
3. ชั้นคอนโวลูชันรับแผนที่ $112 \times 112 \times 64$ ใช้ตัวกรอง 3×3 จำนวน 128 ตัว $p = 1, s = 2$ จงหาขนาดผลลัพธ์และจำนวนพารามิเตอร์
4. ซ้อนตัวกรอง 3×3 ระยะก้าว 1 จำนวน 5 ชั้น จงหาระยะมองเห็น และจำนวนชั้นที่ต้องใช้เพื่อให้ระยะมองเห็นอย่างน้อย 15 พิกเซล
5. จงหา Max Pooling และ Average Pooling ขนาด 2 ระยะก้าว 2 ของ $(0.2, 0.9, 0.4, 0.1, 0.6, 0.5)$ และหา Global Average Pooling
6. ใช้โทเคน toy, friend, screen ในตัวอย่างการคำนวณ 4.15 จงหาน้ำหนัก Attention และเวกเตอร์ตามบริบทของ screen เมื่อ $\mathbf{W} = \mathbf{I}$
7. จงหาเวกเตอร์ตำแหน่งของ $pos = 3$ เมื่อ $d = 4$ ด้วยสมการ (4.24)
8. จงคำนวณ Layer Normalization ของ $\mathbf{r} = (2, 0, -1)$ เมื่อ $\gamma = 1, \beta = 0$
9. ภาพขนาด 384×384 ใช้แพตช์ $P = 16$ จงหาจำนวนแพตช์และจำนวนคู่ Attention รวม [CLS] และถ้า Log-Mel มีขนาด 128×512 ใช้แพตช์ 16 ระยะก้าว 10 จงหาจำนวนแพตช์เสียง
10. u6 ชอบ i2 Inside Out 2 และ i5 Wicked จงสร้างโปรไฟล์ข้อความค่าเฉลี่ยเทคนิค Transformer แล้วหาคะแนนของ i3 และ i11
11. จงหาน้ำหนักตามเวลาของ u1 เมื่อครึ่งชีวิต $h = 60$ วัน และเทียบสัดส่วนอิทธิพลกับกรณี $h = 30$ ในตารางที่ 4.21
12. ใช้คะแนนในตารางที่ 4.19 (เทคนิค Transformer) จงหาคะแนน Late Fusion ของ i8 และ i12 เมื่อใช้น้ำหนักเท่ากันทุกสื่อ
13. จงหาคะแนน RRF ของ i5 และ i8 เมื่อ $k = 0$ จากอันดับรายสื่อในตัวอย่างการคำนวณ 4.34
14. ถ้า i11 ไม่มีคลิปเสียง จงหาคะแนนรวมของ i11 ด้วยการกระจายน้ำหนักใหม่ (เทคนิค Transformer น้ำหนักตั้งต้น)
15. ใช้โคไซน์ในตัวอย่างการคำนวณ 4.39 จงหาความน่าจะเป็นของแถวโปสเตอร์ i8 เมื่อ $\tau = 0.1$
16. การประเมินสี่กรณีได้อันดับของเรื่องที่ซ่อนเป็น 1, 3, 5 และ 2 จงหา HR@3, MRR และ nDCG@3

ระดับที่ 2 เขียนโปรแกรม

1. แก้โปรแกรม 4.6 ให้ใช้ตัวกรองขอบแนวนอน $\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$ และระยะก้าว 2 แล้วอธิบายผลที่ได้
2. เขียนฟังก์ชัน Multi-Head Attention สองหัวด้วย NumPy ตามตัวอย่างการคำนวณ 4.17 แล้วพิมพ์ผลของทั้งสามโทเคน
3. คำนวณโปรไฟล์ค่าเฉลี่ยของผู้ใช้ทั้งแปดคนบนสองเทคนิค แล้วหาผู้ใช้และสื่อที่โปรไฟล์ CNN กับ Transformer ต่างกันมากที่สุด

4. ทำ Grid Search ของน้ำหนัก Fusion โดยเลือกบนผู้ใช้ u5–u8 แล้วทดสอบบน u1–u4 เทียบกับตัวอย่างการคำนวณ 4.35
5. เขียนโปรไฟล์แบบ Attention ตามสมการ (4.42) สำหรับทุกสื่อ แล้วประเมินแบบกันออกทีละรายการเทียบกับโปรไฟล์ค่าเฉลี่ย
6. เพิ่มฟังก์ชัน Bootstrap ใน Lab 4 เพื่อหาช่วงความเชื่อมั่น 95% ของผลต่าง MRR ระหว่างเทคนิค CNN กับ Transformer

ระดับที่ 3 วิเคราะห์และออกแบบ

1. ออกแบบระบบผู้แนะนำหลายรูปแบบสำหรับบริการสตรีมมิงภาพยนตร์ไทย ที่มีภาพยนตร์ใหม่เข้ามาทุกสัปดาห์และหลายเรื่องยังไม่มีตัวอย่างภาพยนตร์ ระบุตัวเข้ารหัสของแต่ละสื่อ วิธีสร้างโปรไฟล์ เทคนิค Fusion และวิธีประเมินผล พร้อมเหตุผล
2. จากผลในหัวข้อ 4.7 เสียงเป็นสื่อเดี่ยวที่ดีที่สุด แต่การถอดวิดีโอทำให้ระบบเสียหายมากที่สุด จงอธิบายด้วยแนวคิด "ข้อมูลที่ไม่ซ้ำกัน" และเสนอการทดลองเพิ่มเติมเพื่อยืนยัน
3. ทีมของท่านมีงบประมาณจำกัดและข้อมูลฝึกน้อย จงวิเคราะห์ว่าควรเลือก CNN หรือ Transformer สำหรับแต่ละสื่อ โดยอ้างอิงตารางที่ 4.18 และผลในหัวข้อ 4.10.2
4. จงวิเคราะห์ความเสี่ยงด้านความเป็นธรรมเมื่อระบบให้น้ำหนักภาพโปสเตอร์สูง และเสนอวิธีตรวจวัดและลดอคติดังกล่าว

อภิธานศัพท์

ตารางที่ 4.29 รวบรวมคำศัพท์สำคัญของบทพร้อมคำภาษาอังกฤษและความหมายโดยย่อ เรียงตามลำดับที่ปรากฏในบท

ตารางที่ 4.29 อภิธานศัพท์ของบท

คำศัพท์	ภาษาอังกฤษ	ความหมายโดยย่อ
รูปแบบข้อมูล	Modality	ชนิดของสัญญาณ เช่น ข้อความ ภาพ เสียง วิดีโอ
การสกัดคุณลักษณะ	Feature Extraction	การแปลงข้อมูลนำเข้าเป็นเวกเตอร์สั้นที่เก็บสาระสำคัญ
ตัวบรรยาย	Descriptor	เวกเตอร์คุณลักษณะที่ได้จากสูตรที่ออกแบบด้วยมือ
ตัวเข้ารหัส	Encoder	ฟังก์ชันที่มีพารามิเตอร์ แปลงข้อมูลนำเข้าเป็นเวกเตอร์
เวกเตอร์ฝัง	Embedding	เวกเตอร์ความหมายยาวคงที่ของรายการหรือโทเคน
ตารางฝัง	Embedding Matrix	ตารางที่แต่ละแถวคือเวกเตอร์ของสัญลักษณ์หนึ่งตัว
นิวรอน	Neuron	หน่วยที่คำนวณผลรวมถ่วงน้ำหนักแล้วผ่านฟังก์ชันกระตุ้น
ฟังก์ชันกระตุ้น	Activation Function	ฟังก์ชันไม่เชิงเส้น เช่น ReLU, GELU
การลดตามเกรเดียนต์	Gradient Descent	การปรับพารามิเตอร์สวนทิศความชันของค่าสูญเสีย
อัตราการเรียนรู้	Learning Rate	ขนาดก้าวของการลดตามเกรเดียนต์
คอนโวลูชัน	Convolution	การเลื่อนตัวกรองคูณและบวกบนหน้าต่างเฉพาะที่
ตัวกรอง	Filter / Kernel	ชุดน้ำหนักขนาดเล็กที่ตรวจจับลวดลายหนึ่งแบบ
การเติมขอบ	Padding	การเติมศูนย์รอบข้อมูลก่อนคอนโวลูชัน
ระยะก้าว	Stride	จำนวนช่องที่ตัวกรองเลื่อนแต่ละครั้ง
ระยะมองเห็น	Receptive Field	บริเวณของข้อมูลนำเข้าที่มีผลต่อนิวรอนหนึ่งตัว
แผนที่คุณลักษณะ	Feature Map	ผลลัพธ์ของชั้นคอนโวลูชัน
ทางลัด	Residual Connection	การบวกข้อมูลนำเข้ากลับเข้ากับผลลัพธ์ของบล็อก
การจัดความสนใจตนเอง	Self-Attention	กลไกที่ทุกโทเคนดึงข้อมูลจากโทเคนอื่นตามน้ำหนัก
หลายหัว	Multi-Head Attention	Attention หลายชุดขนานกัน แต่ละชุดมองความสัมพันธ์ต่างแบบ
การฝังตำแหน่ง	Positional Encoding	เวกเตอร์ที่บอกลำดับของโทเคน
เครือข่ายป้อนไปข้างหน้า	Feed-Forward Network	ชั้นขยายแล้วย่อมิติที่ทำงานกับโทเคนทีละตัว
แพตช์	Patch	ชิ้นสี่เหลี่ยมของภาพหรือสเปกโทรแกรมที่ใช้เป็นโทเคน
ก้อนเวลา–พื้นที่	Tubelet	ชิ้นสามมิติของวิดีโอที่ใช้เป็นโทเคน
โปรไฟล์ผู้ใช้	User Profile	เวกเตอร์แทนรสนิยมของผู้ใช้
การลบค่าเฉลี่ย	Mean Centering	การลบเวกเตอร์ค่าเฉลี่ยของคลังออกจากทุกเวกเตอร์
โปรไฟล์หลายจุดสนใจ	Multi-interest Profile	โปรไฟล์หลายเวกเตอร์ แต่ละเวกเตอร์แทนรสนิยมหนึ่งแบบ
การรวมหลายรูปแบบ	Multimodal Fusion	การรวมหลักฐานจากหลายสื่อเป็นการตัดสินใจเดียว
ปริภูมิร่วม	Shared Embedding Space	ปริภูมิที่เวกเตอร์ต่างสื่อเทียบกันได้โดยตรง
การเรียนรู้แบบเปรียบต่าง	Contrastive Learning	การฝึกให้คู่ถูกใกล้กันและคู่ผิดห่างกัน
การถอดสื่อ	Ablation	การทดลองนำส่วนประกอบออกทีละส่วนเพื่อวัดความจำเป็น

ภาคผนวก 4ก โปรแกรม mm4.py และ Lab 4 ฉบับเต็ม

ภาคผนวกนี้รวบรวมโปรแกรมฉบับเต็มสองไฟล์ ผู้อ่านคัดลอกไปใช้ได้ทันที ทั้งสองไฟล์ต้องอยู่ในโฟลเดอร์เดียวกัน

ส่วนที่ 1 ไฟล์ mm4.py: ข้อมูลกลางของบท

โปรแกรม mm4.py

```
"""mm4.py: ข้อมูล MovieMini-12 ของบทที่ 4 (ใช้ร่วมทุกโปรแกรม)"""
import numpy as np


ITEMS = ['i%d' % k for k in range(1, 13)]
MODS = ['T', 'I', 'A', 'V']          # ข้อความ ภาพ เสียง วิดีโอ
R = {'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
     'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
     'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
     'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
     'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
     'u6': {'i2': 5, 'i5': 4, 'i11': 3},
     'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
     'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3}}


# ข้อความ 12 มิติ จากเทคนิค Transformer (ตารางเวกเตอร์ข้อความ)
TEXT_TF = """
i1 .2145 .1287 .0429 0 .0858 .1287 .3377 0 .0858 .8889 0 0
i2 0 .9879 0 0 0 .0335 .1488 0 0 0 0 .0288
i3 .3544 .1688 0 0 0 .1181 .7852 0 .4642 0 0 0
i4 0 .1940 0 .8027 .3051 .0555 .4528 0 0 0 .1294 0
i5 0 0 0 .2809 0 .0551 .4312 .1926 .5777 0 .1926 .5694
i6 0 .0553 0 .6780 0 0 .7314 0 0 0 .0475 0
i7 0 .0330 0 0 .9065 .4121 .0769 .0384 0 0 0 0
i8 0 0 .0898 0 .0449 .9755 .0772 0 0 0 .1797 0
i9 0 .1355 0 0 .1734 .0758 0 0 .0379 0 .9473 .2167
i10 0 .2024 .4424 0 0 .5770 0 0 .1574 0 .6294 .0975
i11 .5949 .2511 .4850 .0511 .0595 .4591 0 0 .0595 0 .3569 0
i12 .2449 .2627 .8892 0 0 .0911 .2524 0 0 0 .0911 0
"""
# ข้อความ 12 มิติ จากเทคนิค CNN (Static Embedding ของบทที่ 3)
TEXT_CNN = """
i1 .2212 .1327 .0822 0 .0885 .1327 .3482 0 .0885 .8787 0 0
i2 0 .9641 0 .0266 0 .0391 .2001 0 0 0 0 .1679
i3 .3654 .1741 .0523 0 0 .1218 .7835 0 .4525 0 0 0
i4 0 .1940 0 .8027 .3051 .0555 .4528 0 0 0 .1294 0
i5 0 .1189 0 .3029 0 .1189 .4649 .2076 .6229 0 .2076 .4356
i6 0 .1369 0 .6710 0 0 .7270 0 0 0 .0498 0
i7 0 .0330 0 0 .9065 .4121 .0769 .0384 0 0 0 0
i8 0 0 .0933 0 .0466 .9728 .0801 0 0 0 .1865 .0401
i9 0 .1355 0 0 .1734 .0758 0 0 .0379 0 .9473 .2167
i10 0 .2055 .4034 0 0 .5857 .0229 0 .1826 0 .6390 .0990
i11 .6062 .2146 .4942 .0727 .0606 .4418 .0467 0 .0606 0 .3637 0
i12 .2656 .2486 .8730 .0181 0 .1215 .2690 0 0 .0912 .0987 0
"""
# ภาพ มิติ 1-3 | เสียง มิติ 1-3 | วิดีโอ มิติ 1-3
MEDIA_TF = """
i1 .9 .6 .9 .5 .7 .1 .6 .5 1
i2 .8 .8 .9 .5 1 .2 .5 .4 1
i3 .7 .5 .8 .3 .8 .4 .4 .3 1
i4 .5 .7 .2 .3 .8 .3 .4 .3 0
i5 .7 .6 .5 .4 1 .1 .3 .4 0
i6 .6 .8 .3 .2 .8 .1 .2 .2 0
i7 .2 .3 .1 .9 .2 .8 .9 .9 0
i8 .6 .9 .3 1 .3 .7 1 1 0
i9 .4 .3 .1 .3 .5 .6 .3 .4 0
i10 .5 .8 .1 .8 .3 .8 .7 .6 0
i11 .3 .2 .1 .6 .3 .7 .6 .5 0
```

```
i12 .3 .3 .1 .5 .6 .7 .5 .4 0
"""
MEDIA_CNN = """
i1 .9 .6 .77 .5 .65 .19 .57 .5 1
i2 .8 .8 .77 .5 .8 .26 .52 .42 1
i3 .7 .5 .69 .3 .7 .41 .47 .35 1
i4 .5 .7 .24 .3 .7 .34 .47 .35 0
i5 .7 .6 .47 .4 .8 .19 .42 .42 0
i6 .6 .8 .32 .2 .7 .19 .37 .27 0
i7 .2 .3 .17 .9 .4 .71 .72 .8 0
i8 .6 .9 .32 1 .45 .64 .77 .87 0
i9 .4 .3 .17 .3 .55 .56 .42 .42 0
i10 .5 .8 .17 .8 .45 .71 .62 .57 0
i11 .3 .2 .17 .6 .45 .64 .57 .5 0
i12 .3 .3 .17 .5 .6 .64 .52 .42 0
"""



def parse(block):
    table = {}
    for line in block.strip().splitlines():
        parts = line.split()
        table[parts[0]] = np.array(parts[1:], dtype=float)
    return table



def l2(v):
    n = np.linalg.norm(v)
    return v / n if n > 0 else v



def load(tech='TF'):
    # คืนเวกเตอร์ดั้งเดิมของสี่สื่อ {สื่อ: {รายการ: เวกเตอร์}}
    t = parse(TEXT_TF if tech == 'TF' else TEXT_CNN)
    md = parse(MEDIA_TF if tech == 'TF' else MEDIA_CNN)
    return {'T': t,
            'I': {i: md[i][0:3] for i in ITEMS},
            'A': {i: md[i][3:6] for i in ITEMS},
            'V': {i: md[i][6:9] for i in ITEMS}}



def prepare(table, center):
    # ลบค่าเฉลี่ยของคลัง (เฉพาะภาพ เสียง วิดีโอ) แล้วปรับความยาวเป็น 1
    mu = np.mean(list(table.values()), axis=0) if center else 0.0
    return {i: l2(v - mu) for i, v in table.items()}



def vectors(tech='TF'):
    raw = load(tech)
    return {m: prepare(raw[m], center=(m != 'T')) for m in MODS}
```

ส่วนที่ 2 ไฟล์ lab4.py: Lab 4 ระบบผู้แนะนำหลายรูปแบบ

โปรแกรม lab4.py

```
"""Lab 4: ระบบผู้แนะนำหลายรูปแบบบน MovieMini-12 (CNN และ Transformer)
ตอนที่ 1 เตรียมเวกเตอร์ของสี่สื่อจากสองเทคนิค
```

```
ตอนที่ 2 โปรไฟล์ผู้ใช้สี่แบบและคะแนนรายสื่อ
ตอนที่ 3 Fusion: Late, z-score, RRF และ Gate
ตอนที่ 4 ประเมินผลแบบกันออกทีละรายการ ถอดสื่อ และเทียบสองเทคนิค
ต้องมีไฟล์ mm4.py (ภาคผนวก 4ก ส่วนที่ 1) อยู่ในโฟลเดอร์เดียวกัน"""
import math
import numpy as np
from mm4 import ITEMS, R, MODS, load, vectors, l2

W0 = {'T': 0.4, 'I': 0.2, 'A': 0.2, 'V': 0.2}
DAYS = {'u1': {'i2': 5, 'i1': 20, 'i3': 60}}      # อายุคะแนน (วัน)



# ---------------------------------------------------------------
# ตอนที่ 1  เตรียมเวกเตอร์
# ---------------------------------------------------------------
def part1():
    print('ตอนที่ 1: cos ภาพ i1-i7 ก่อนและหลังเตรียมเวกเตอร์')
    for tech in ['CNN', 'TF']:
        raw = load(tech)['I']
        V = vectors(tech)['I']
        before = l2(raw['i1']) @ l2(raw['i7'])
        print('   %-3s ก่อน %.4f  หลัง %.4f'
              % (tech, before, V['i1'] @ V['i7']))



# ---------------------------------------------------------------
# ตอนที่ 2  โปรไฟล์ผู้ใช้
# ---------------------------------------------------------------
def liked(r):
    return [i for i, x in r.items() if x >= 4]



def profile(V, r, kind='mean', beta=0.5, h=30, days=None):
    like = liked(r)
    if kind == 'mean':
        p = np.mean([V[i] for i in like], axis=0)
    elif kind == 'rw':
        rbar = np.mean(list(r.values()))
        p = sum((x - rbar) * V[i] for i, x in r.items())
    elif kind == 'rocchio':
        dis = [i for i, x in r.items() if x <= 2]
        p = np.mean([V[i] for i in like], axis=0)
        if dis:
            p = p - beta * np.mean([V[i] for i in dis], axis=0)
    else:                                          # 'decay'
        w = {i: 2 ** (-days[i] / h) for i in like}
        p = sum(w[i] * V[i] for i in like) / sum(w.values())
    return l2(p)



def mod_scores(V, r, cand, kind='mean', **kw):
    S = {}
    for m in MODS:
        p = profile(V[m], r, kind, **kw)
        S[m] = {i: float(p @ V[m][i]) for i in cand}
    return S



def rank(s):
    return sorted(s, key=lambda i: (-round(s[i], 12), int(i[1:])))
```

```
def part2():
    print('ตอนที่ 2: Top-3 ข้อความของ u1 จากโปรไฟล์สี่แบบ')
    cand = [i for i in ITEMS if i not in R['u1']]
    for tech in ['CNN', 'TF']:
        V = vectors(tech)
        for kind in ['mean', 'rw', 'rocchio', 'decay']:
            S = mod_scores(V, R['u1'], cand, kind, days=DAYS['u1'])
            print('   %-3s %-8s' % (tech, kind), rank(S['T'])[:3])



# ---------------------------------------------------------------
# ตอนที่ 3  Fusion
# ---------------------------------------------------------------
def late(S, w=W0):
    return {i: sum(w[m] * S[m][i] for m in MODS) for i in S['T']}



def zfuse(S, w=W0):
    out = {i: 0.0 for i in S['T']}
    for m in MODS:
        v = np.array(list(S[m].values()))
        for i in out:
            out[i] += w[m] * float((S[m][i] - v.mean()) / v.std())
    return out



def rrf(S, k=60):
    out = {i: 0.0 for i in S['T']}
    for m in MODS:
        for pos, i in enumerate(rank(S[m]), start=1):
            out[i] += 1 / (k + pos)
    return out



def gate_weights(V, r, tau=0.5):
    like = liked(r)
    if len(like) < 2:                       # ข้อมูลน้อย ให้น้ำหนักเท่ากัน
        return {m: 0.25 for m in MODS}
    g = {}
    for m in MODS:
        pairs = [(a, b) for k, a in enumerate(like) for b in like[k + 1:]]
        g[m] = np.mean([V[m][a] @ V[m][b] for a, b in pairs])
    e = {m: math.exp(g[m] / tau) for m in MODS}
    return {m: e[m] / sum(e.values()) for m in MODS}



def part3():
    print('ตอนที่ 3: Top-3 ของ u1 หลังรวมสี่สื่อ (เทคนิค Transformer)')
    V = vectors('TF')
    cand = [i for i in ITEMS if i not in R['u1']]
    S = mod_scores(V, R['u1'], cand)
    gw = gate_weights(V, R['u1'])
    for name, s in [('late', late(S)), ('z-score', zfuse(S)),
                    ('RRF', rrf(S)), ('gate', late(S, gw))]:
        print('   %-8s' % name, [(i, round(s[i], 4)) for i in rank(s)[:3]])



# ---------------------------------------------------------------
# ตอนที่ 4  ประเมินผล
# ---------------------------------------------------------------
def loo(V, fuse, use_gate=False, K=3):
    ranks = []
```

```
    for u, r in R.items():
        for h in liked(r):
            train = {i: x for i, x in r.items() if i != h}
            cand = [i for i in ITEMS if i not in r] + [h]
            S = mod_scores(V, train, cand)
            s = late(S, gate_weights(V, train)) if use_gate else fuse(S)
            ranks.append(rank(s).index(h) + 1)
    hr = np.mean([x <= K for x in ranks])
    mrr = np.mean([1 / x for x in ranks])
    nd = np.mean([1 / math.log2(x + 1) if x <= K else 0 for x in ranks])
    return hr, mrr, nd



def part4():
    print('ตอนที่ 4: LOO 20 กรณี (HR@3, MRR, nDCG@3)')
    for tech in ['CNN', 'TF']:
        V = vectors(tech)
        for m in MODS:
            w = {k: float(k == m) for k in MODS}
            print('   %-3s %-6s %.4f %.4f %.4f'
                  % ((tech, m) + loo(V, lambda S, w=w: late(S, w))))
        print('   %-3s %-6s %.4f %.4f %.4f'
              % ((tech, 'late') + loo(V, late)))
    V = vectors('TF')
    for m in MODS:
        w = {k: (0.0 if k == m else W0[k] / (1 - W0[m])) for k in MODS}
        print('   ถอด %s     %.4f %.4f %.4f'
              % ((m,) + loo(V, lambda S, w=w: late(S, w))))
    for name, f in [('z-score', zfuse), ('RRF', rrf)]:
        print('   %-10s %.4f %.4f %.4f' % ((name,) + loo(V, f)))
    print('   %-10s %.4f %.4f %.4f' % (('gate',) + loo(V, None, True)))



if __name__ == '__main__':
    part1()
    part2()
    part3()
    part4()
```